industrial-engineering-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `industrial_engineering_ai999x.about()` for the same answer.

Every capability industrial-engineering-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import industrial_engineering_ai999x

result = industrial_engineering_ai999x.solve('ie_eoq', demand=1200, order_cost=100,
holding_cost=6)
result.show()
# Q = 200, orders_per_year = 6, cycle_time = 0.166667, ordering_cost = 600,
holding_cost_per_year = 600, total_cost = 1200
```

By topic

topic	capabilities
Break-even analysis	Break-even point, Multiproduct break-even (sales-mix weighted)
Engineering economy / depreciation	Depreciation schedule (straight line, declining balance, sum of years' digits, MACRS)
Engineering economy / income taxes	After-tax cash flow with depreciation and recapture
Facility location / single facility	Centre-of-gravity location, Minimum weighted-distance location (Weber problem), Minimum rectilinear-distance location (weighted median)
Facility location / site selection	Factor-rating method for site selection, Locational cost-volume (break-even) analysis
Forecasting / causal methods	Associative forecast by simple linear regression
Forecasting / forecast accuracy	Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal, Tracking signal over time
Forecasting / seasonality	Seasonal indices (average method)
Forecasting / time series methods	Moving average forecast, Weighted moving average forecast, Simple exponential smoothing forecast, Holt's linear trend (double exponential smoothing), Linear trend projection (least squares)
Inventory / classification	ABC inventory classification
Inventory / continuous review	(R, Q) reorder point for a target fill rate, Service level of a reorder point

topic	capabilities
Inventory / EOQ	Economic order quantity (EOQ), Annual inventory cost of an order quantity, EOQ with planned backorders, EOQ with all-units quantity discounts, Economic production quantity (EPQ)
Inventory / lean	Number of kanbans
Inventory / newsvendor	Newsvendor order quantity (normal demand), Newsvendor order quantity (uniform demand), Newsvendor order quantity (Poisson demand), Newsvendor order quantity (discrete demand table)
Inventory / performance	Inventory turnover and days of supply
Inventory / periodic review	Periodic review order-up-to level
Inventory / reorder point	Reorder point (constant demand)
Inventory / safety stock	Safety stock and reorder point (normal demand), Safety stock with variable demand and lead time
Learning curves / estimation	Learning rate from two unit times, Fit a learning curve to data (log-log regression)
Learning curves / unit model	Learning curve: time of the n-th unit, Learning curve: total and average time of N units
Learning curves / Wright model	Cumulative-average learning curve (Wright)
Line balancing / cycle time	Takt (cycle) time and minimum stations
Line balancing / heuristics	Line balancing by the largest candidate rule, Line balancing by ranked positional weight
Line balancing / measures	Line balance efficiency, balance delay and idle time
Project management / earned value	Earned value analysis (CV, SV, CPI, SPI, EAC)
Queueing / Little's law	Little's law
Queueing / M/G/1 queues	M/D/1 queue (constant service time), M/G/1 queue (Pollaczek-Khinchine)
Queueing / Markovian queues	M/M/1 queue, M/M/c queue (Erlang C), M/M/1/K queue (finite capacity)
Queueing / staffing	Servers needed for a service level (Erlang C), Cost-optimal number of servers
Reliability / availability	Inherent availability from MTBF and MTTR
Reliability / life data	Weibull fit by median-rank regression, Confidence bounds on MTBF (chi-square)
Reliability / life models	Exponential (constant failure rate) reliability, Weibull reliability, hazard and MTTF, Failure rate and MTBF of a series system, MTTF of identical components in parallel
Reliability / systems	Reliability of components in series, Reliability of components in parallel, K-out-of-n system reliability, Series system of parallel stages, Parallel components needed for a target reliability, Cold standby redundancy
Scheduling / flow shop	Johnson's rule (two machines), Johnson's rule extended to three machines, Permutation flow shop makespan
Scheduling / single machine	Shortest processing time (SPT) sequence, Earliest due date (EDD) sequence, Sequencing by a dispatching rule (FCFS, SPT, LPT, EDD, CR), Minimise the number of late jobs (Moore-Hodgson)
Statistical process control / acceptance sampling	Single sampling plan: OC, AOQ and ATI
Statistical process control / advanced charts	EWMA chart control limits, Tabular CUSUM chart
Statistical process control / attribute charts	P chart control limits, Np chart control limits, C chart control limits, U chart control limits, P chart from sample counts

topic	capabilities
Statistical process control / capability	Process capability Cp and Cpk, Process performance Pp and Ppk from data, Taguchi capability index Cpm, Fraction nonconforming (normal process)
Statistical process control / chart performance	Average run length of a Shewhart X-bar chart
Statistical process control / six sigma	DPMO and sigma level, Rolled throughput yield
Statistical process control / variables charts	X-bar and R chart control limits, X-bar and S chart control limits, Process standard deviation from the average range
Work measurement / productivity	Overall equipment effectiveness (OEE), Capacity utilisation and efficiency, Single-factor and multifactor productivity
Work measurement / time study	Standard time from a time study, Number of cycles to time in a time study
Work measurement / work sampling	Work sampling sample size

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
ie_eoq	Economic order quantity (EOQ)	demand, order_cost, holding_cost	Q, orders_per_year, cycle_time, ordering_cost, holding_cost_per_year, total_cost
ie_inventory_total_cost	Annual inventory cost of an order quantity	demand, order_cost, holding_cost, order_quantity, unit_cost	total_cost, ordering_cost, holding_cost_per_year, purchase_cost, average_inventory
ie_eoq_backorders	EOQ with planned backorders	demand, order_cost, holding_cost, backorder_cost	Q, max_backorder, max_inventory, total_cost, orders_per_year, fraction_backordered_time
ie_eoq_quantity_discount	EOQ with all-units quantity discounts	demand, order_cost, holding_rate, break_quantities, unit_prices	Q, unit_price, total_cost, tier, ordering_cost, holding_cost_per_year, purchase_cost
ie_epq	Economic production quantity (EPQ)	demand, production_rate, order_cost, holding_cost	Q, max_inventory, production_run_time, cycle_time, runs_per_year, total_cost
ie_reorder_point	Reorder point (constant demand)	demand_rate, lead_time, safety_stock	reorder_point, lead_time_demand
ie_safety_stock	Safety stock and reorder point (normal demand)	mean_demand, sd_demand, lead_time, service_level	safety_stock, reorder_point, z, sd_lead_time_demand, lead_time_demand
ie_safety_stock_variable_lead_time	Safety stock with variable demand and lead time	mean_demand, sd_demand, mean_lead_time, sd_lead_time, service_level	safety_stock, reorder_point, sd_lead_time_demand, z
ie_newsvendor_normal	Newsvendor order quantity (normal demand)	price, cost, mean_demand, sd_demand, salvage	Q, critical_ratio, z, expected_sales, expected_leftover, expected_shortage, expected_profit
ie_newsvendor_uniform	Newsvendor order quantity	price, cost, low, high, salvage	Q, critical_ratio, expected_sales, expected_leftover, expected_profit

id	operation	inputs	outputs
	(uniform demand)		
ie_newsvendor_poisson	Newsvendor order quantity (Poisson demand)	price, cost, mean_demand, salvage	Q, critical_ratio, expected_sales, expected_leftover, expected_profit, stockout_probability
ie_newsvendor_discrete	Newsvendor order quantity (discrete demand table)	price, cost, demands, probabilities, salvage	Q, critical_ratio, expected_sales, expected_profit
ie_rq_fill_rate	(R, Q) reorder point for a target fill rate	mean_ltd, sd_ltd, order_quantity, fill_rate	reorder_point, safety_stock, z, expected_shortage_per_cycle, cycle_service_level
ie_service_level_from_reorder_point	Service level of a reorder point	mean_ltd, sd_ltd, reorder_point, order_quantity	cycle_service_level, z, safety_stock, expected_shortage_per_cycle, fill_rate
ie_order_up_to_level	Periodic review order-up-to level	mean_demand, sd_demand, review_period, lead_time, service_level, inventory_position	order_up_to_level, safety_stock, z, sd_protection_demand, order_quantity
ie_abc_classification	ABC inventory classification	annual_values, a_share, b_share	classes, count_a, count_b, count_c, share_a, share_b
ie_inventory_turnover	Inventory turnover and days of supply	cost_of_goods_sold, average_inventory, days_per_year	turnover, days_of_supply, weeks_of_supply
ie_kanban_count	Number of kanbans	demand_rate, lead_time, container_size, safety_factor	kanbans, exact, max_inventory
ie_xbar_r_limits	X-bar and R chart control limits	grand_mean, mean_range, subgroup_size	xbar_ucl, xbar_lcl, center, r_ucl, r_lcl, A2, D3, D4
ie_xbar_s_limits	X-bar and S chart control limits	grand_mean, mean_sd, subgroup_size	xbar_ucl, xbar_lcl, center, s_ucl, s_lcl, A3, B3, B4
ie_sigma_from_range	Process standard deviation from the average range	mean_range, subgroup_size	sigma, d2, natural_tolerance
ie_p_chart_limits	p chart control limits	p_bar, sample_size	ucl, lcl, center, sigma_p
ie_np_chart_limits	np chart control limits	p_bar, sample_size	ucl, lcl, center, sigma_np
ie_c_chart_limits	c chart control limits	c_bar	ucl, lcl, center, sigma_c
ie_u_chart_limits	u chart control limits	u_bar, units	ucl, lcl, center, sigma_u

id	operation	inputs	outputs
ie_p_chart_from_samples	p chart from sample counts	defectives, sample_size	p_bar, ucl, lcl, out_of_control, count_out
ie_process_capability	Process capability Cp and Cpk	usl, lsl, mean, sd	cpk, cp, cpu, cpl
ie_process_performance	Process performance Pp and Ppk from data	data, usl, lsl	ppk, pp, mean, sd
ie_cpm_taguchi	Taguchi capability index Cpm	usl, lsl, mean, sd, target	cpm, cp, tau
ie_fraction_nonconforming	Fraction nonconforming (normal process)	mean, sd, lsl, usl	fraction_out, below_lsl, above_usl, ppm, yield
ie_arl_shewhart	Average run length of a Shewhart X-bar chart	shift, subgroup_size, limit_sigma	arl, beta, detection_probability, arl_in_control
ie_ewma_limits	EWMA chart control limits	target, sigma, smoothing, limit_sigma, sample_number	ucl, lcl, ucl_steady, lcl_steady, sd_ewma
ie_tabular_cusum	Tabular CUSUM chart	data, target, k, h	first_signal, c_plus, c_minus, final_c_plus, final_c_minus
ie_dpmo_sigma_level	DPMO and sigma level	defects, units, opportunities	dpmo, dpu, dpo, yield, sigma_level
ie_rolled_throughput_yield	Rolled throughput yield	yields	rty, normalized_yield, total_dpu
ie_acceptance_sampling	Single sampling plan: OC, AOQ and ATI	sample_size, acceptance_number, fraction_defective, lot_size	probability_accept, aoq, ati, probability_reject
ie_series_reliability	Reliability of components in series	reliabilities	reliability, unreliability
ie_parallel_reliability	Reliability of components in parallel	reliabilities	reliability, unreliability
ie_k_out_of_n_reliability	k-out-of-n system reliability	component_reliability, k, n	reliability, unreliability
ie_series_parallel_reliability	Series system of parallel stages	stage_reliabilities	reliability, unreliability
ie_exponential_reliability	Exponential (constant failure rate) reliability	failure_rate, time	reliability, unreliability, mtbf, hazard_rate
ie_availability	Inherent availability from	mtbf, mttr, hours_per_year	availability, unavailability, downtime_per_year, failure_rate, repair_rate

id	operation	inputs	outputs
	MTBF and MTTR		
ie_weibull_reliability	Weibull reliability, hazard and MTTF	shape, scale, time, location	reliability, unreliability, hazard_rate, mttf, b10_life
ie_weibull_median_rank_regression	Weibull fit by median-rank regression	failure_times	shape, scale, r_squared
ie_redundancy_needed	Parallel components needed for a target reliability	target_reliability, component_reliability	components, achieved_reliability
ie_system_failure_rate	Failure rate and MTBF of a series system	failure_rates, time	system_failure_rate, mtbf, reliability
ie_parallel_mttf	MTTF of identical components in parallel	failure_rate, n	mttf, single_mttf, improvement_factor
ie_standby_redundancy	Cold standby redundancy	failure_rate, time, units	reliability, mttf
ie_mtbf_confidence	Confidence bounds on MTBF (chi-square)	total_time, failures, confidence, test	mtbf, lower_one_sided, lower_two_sided, upper_two_sided
ie_mm1_queue	M/M/1 queue	arrival_rate, service_rate	utilization, L, Lq, W, Wq, p0
ie_mmc_queue	M/M/c queue (Erlang C)	arrival_rate, service_rate, servers	utilization, p_wait, Lq, Wq, W, L, p0
ie_md1_queue	M/D/1 queue (constant service time)	arrival_rate, service_rate	utilization, Lq, Wq, W, L
ie_mg1_queue	M/G/1 queue (Pollaczek-Khinchine)	arrival_rate, mean_service_time, sd_service_time	utilization, Lq, Wq, W, L
ie_mm1k_queue	M/M/1/K queue (finite capacity)	arrival_rate, service_rate, capacity	p0, p_block, L, Lq, effective_arrival_rate, W, Wq, utilization
ie_erlang_c_staffing	Servers needed for a service level (Erlang C)	arrival_rate, service_rate, target_time, target_level	servers, service_level, p_wait, utilization, mean_wait
ie_queue_server_cost	Cost-optimal number of servers	arrival_rate, service_rate, server_cost_rate, waiting_cost_rate, max_servers	servers, total_cost, L
ie_littles_law	Little's law	arrival_rate, number_in_system, time_in_system	arrival_rate, number_in_system, time_in_system
ie_spt_sequence	Shortest processing time	processing_times, due_dates	sequence, mean_flow_time, makespan,

id	operation	inputs	outputs
	(SPT) sequence		average_jobs_in_system, utilization, total_tardiness, mean_tardiness, max_tardiness, max_lateness, tardy_jobs
ie_edd_sequence	Earliest due date (EDD) sequence	processing_times, due_dates	sequence, mean_flow_time, makespan, average_jobs_in_system, utilization, total_tardiness, mean_tardiness, max_tardiness, max_lateness, tardy_jobs
ie_dispatching_rule	Sequencing by a dispatching rule (FCFS, SPT, LPT, EDD, CR)	processing_times, due_dates, rule	sequence, mean_flow_time, makespan, average_jobs_in_system, utilization, total_tardiness, mean_tardiness, max_tardiness, max_lateness, tardy_jobs
ie_johnson_two_machine	Johnson's rule (two machines)	machine1_times, machine2_times	sequence, makespan, idle_time_machine2
ie_johnson_three_machine	Johnson's rule extended to three machines	machine1_times, machine2_times, machine3_times	sequence, makespan
ie_moore_hodgson	Minimise the number of late jobs (Moore-Hodgson)	processing_times, due_dates	number_late, late_jobs, on_time_jobs, sequence
ie_flow_shop_makespan	Permutation flow shop makespan	times	makespan, completion_times, idle_time_last_machine
ie_moving_average_forecast	Moving average forecast	data, periods	forecast, forecasts
ie_weighted_moving_average	Weighted moving average forecast	data, weights	forecast, weight_sum
ie_exponential_smoothing	Simple exponential smoothing forecast	data, alpha, initial_forecast	forecast, forecasts
ie_holt_trend	Holt's linear trend (double exponential smoothing)	data, alpha, beta, initial_level, initial_trend, horizon	forecast, level, trend
ie_forecast_errors	Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal	actual, forecast	mad, mse, rmse, mape, bias, running_sum_of_errors, tracking_signal
ie_tracking_signal	Tracking signal over time	actual, forecast, limit	final_signal, signals, first_out_of_control
ie_linear_trend_forecast	Linear trend projection (least squares)	data, horizon	forecast, intercept, slope

id	operation	inputs	outputs
ie_seasonal_index	Seasonal indices (average method)	data, next_year_total	indices, season_averages, overall_average, forecasts
ie_regression_forecast	Associative forecast by simple linear regression	x, y, x_new	intercept, slope, r, r_squared, standard_error, forecast
ie_takt_time	Takt (cycle) time and minimum stations	available_time, demand, total_task_time	cycle_time, output_rate, minimum_stations, exact_stations
ie_line_efficiency	Line balance efficiency, balance delay and idle time	task_times, cycle_time, stations	efficiency, balance_delay, idle_time, total_task_time, minimum_stations
ie_largest_candidate_rule	Line balancing by the largest candidate rule	task_times, precedence, cycle_time	stations, assignment, station_times, efficiency, idle_time, minimum_stations
ie_ranked_positional_weight	Line balancing by ranked positional weight	task_times, precedence, cycle_time	stations, assignment, station_times, efficiency, idle_time, minimum_stations, positional_weights
ie_learning_curve_unit	Learning curve: time of the n-th unit	first_unit_time, learning_rate, unit	unit_time, exponent
ie_learning_curve_total	Learning curve: total and average time of N units	first_unit_time, learning_rate, units	total_time, average_time, last_unit_time
ie_learning_rate_from_two_units	Learning rate from two unit times	unit1, time1, unit2, time2	learning_rate, exponent, first_unit_time
ie_learning_curve_fit	Fit a learning curve to data (log-log regression)	units, times	learning_rate, exponent, first_unit_time, r_squared
ie_learning_curve_cumulative_average	Cumulative-average learning curve (Wright)	first_unit_time, learning_rate, units	cumulative_average_time, total_time, unit_time
ie_center_of_gravity	Centre-of-gravity location	x, y, weights	x, y, total_weight
ie_weber_location	Minimum weighted-distance location (Weber problem)	x, y, weights	x, y, total_weighted_distance
ie_rectilinear_location	Minimum rectilinear-distance location (weighted median)	x, y, weights	x, y, total_weighted_distance

id	operation	inputs	outputs
ie_factor_rating	Factor-rating method for site selection	weights, scores	weighted_scores, best_site, best_score
ie_location_cost_volume	Locational cost-volume (break-even) analysis	fixed_costs, variable_costs, volume	total_costs, best_site, best_cost
ie_break_even	Break-even point	fixed_cost, price, variable_cost, target_profit	break_even_units, break_even_revenue, contribution_margin, contribution_margin_ratio, units_for_target_profit
ie_multiproduct_break_even	Multiproduct break-even (sales-mix weighted)	fixed_cost, prices, variable_costs, sales_mix	break_even_revenue, weighted_contribution_ratio, revenue_by_product, units_by_product
ie_standard_time	Standard time from a time study	observed_times, rating, allowance, basis	standard_time, normal_time, mean_observed_time
ie_time_study_sample_size	Number of cycles to time in a time study	mean, sd, accuracy, confidence	sample_size, exact, z
ie_work_sampling_sample_size	Work sampling sample size	proportion, accuracy, confidence	sample_size, exact, z
ie_oee	Overall equipment effectiveness (OEE)	planned_time, downtime, ideal_cycle_time, total_count, good_count	oee, availability, performance, quality, run_time
ie_capacity_utilization	Capacity utilisation and efficiency	design_capacity, effective_capacity, actual_output	utilization, efficiency, effective_share
ie_depreciation_schedule	Depreciation schedule (straight line, declining balance, sum of years' digits, MACRS)	cost, life, method, salvage, factor, year	depreciation, book_value, total_depreciation, depreciation_in_year, book_value_end_of_year, accumulated_depreciation
ie_after_tax_cash_flow	After-tax cash flow with depreciation and recapture	before_tax_cash_flows, depreciation, tax_rate, investment, sale_price	taxable_income, taxes, after_tax_cash_flows, total_tax, book_value_end, gain_on_sale, tax_on_sale, cash_flows_with_investment
ie_earned_value	Earned value analysis (CV, SV, CPI, SPI, EAC)	planned_value, earned_value, actual_cost, budget_at_completion	cost_variance, schedule_variance, cpi, spi, eac, etc, vac, eac_atypical, eac_cpi_spi, tcpi, percent_complete

id	operation	inputs	outputs
ie_productivity	Single-factor and multifactor productivity	output, input_amounts, previous_output, previous_inputs	partial_productivities, multifactor_productivity, previous_multifactor_productivity, change_percent, partial_change_percent

Economic order quantity (EOQ), `ie_eoq`

The order quantity that minimises annual ordering plus holding cost for constant demand: Q* = sqrt(2DS/H).

$$Q^* = \sqrt{\frac{2DS}{H}}, \quad TC^* = \sqrt{2DSH}$$

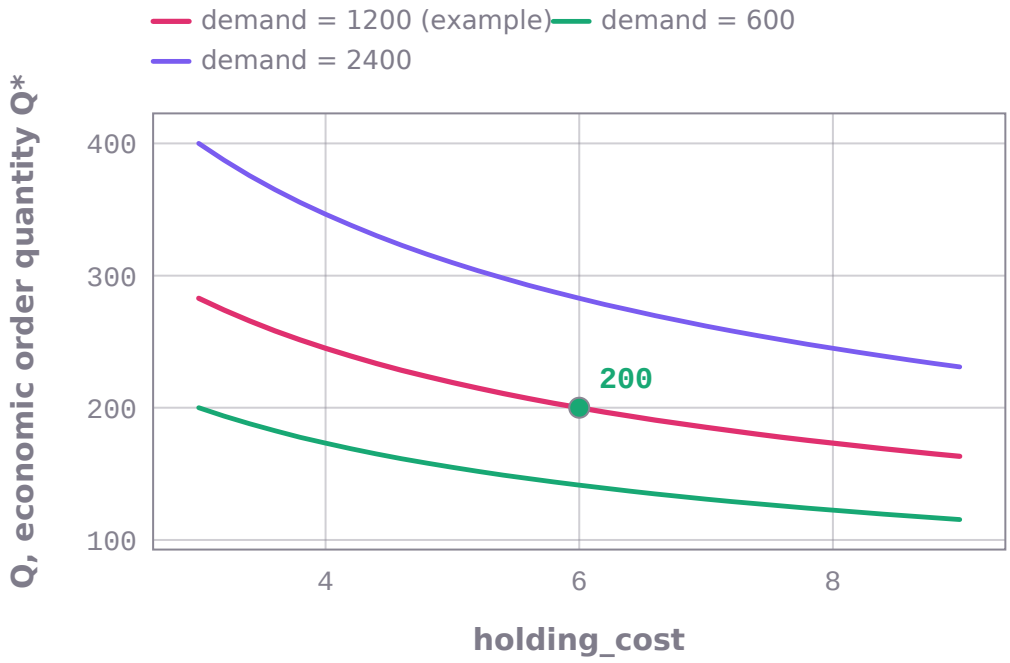
input	meaning	condition
`demand`	demand per year (units)	
`order_cost`	fixed cost per order or setup	
`holding_cost`	holding cost per unit per year	

output	meaning
`Q`	economic order quantity Q*
`orders_per_year`	orders per year D/Q*
`cycle_time`	time between orders Q*/D (years)
`ordering_cost`	annual ordering cost DS/Q*
`holding_cost_per_year`	annual holding cost HQ*/2
`total_cost`	annual ordering plus holding cost

- **Method:** Q* = sqrt(2 D S / H); $TC = D$ S / Q + H $Q/2 = sqrt$(2 D S H)
- **Assumes:** demand > 0; order_cost > 0; holding_cost > 0 (per unit per year)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (economic order quantity Q*) against demand (demand per year (units)), with holding_cost = 6, order_cost = 100. 31 points computed by industrial-engineering-ai999x; marked: demand = 1200, Q = 200.



Q (economic order quantity Q*) against holding_cost (holding cost per unit per year) from 3 to 9. One curve for each value of demand (demand per year (units)): 1200, 600, 2400. Other inputs as in the worked example: order_cost = 100. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, holding_cost = 6, Q = 200.

```
industrial_engineering_ai999x.solve('ie_eoq', demand=1200, order_cost=100, holding_cost=6)
# -> Q = 200, orders_per_year = 6, cycle_time = 0.166667, ordering_cost = 600,
holding_cost_per_year = 600, total_cost = 1200   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_eoq', demand=8000, order_cost=45, holding_cost=2.5)
# -> Q = 536.656, orders_per_year = 14.9071, cycle_time = 0.067082, ordering_cost = 670.82,
holding_cost_per_year = 670.82, total_cost = 1341.64   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Economic order quantity with annual demand = 1200, ordering cost = 100, carrying cost = 6.	SOLVED	Q = 200, orders_per_year = 6, cycle_time = 0.166667, ordering_cost = 600, holding_cost_per_year = 600, total_cost = 1200
Economic order quantity with annual demand = 8000, ordering cost = 45, carrying cost = 2.5.	SOLVED	Q = 536.656, orders_per_year = 14.9071, cycle_time = 0.067082, ordering_cost = 670.82, holding_cost_per_year = 670.82, total_cost = 1341.64
Economic order quantity with annual demand = 1200, ordering cost = 100.	NEEDS_INPUT	ie_eoq needs holding_cost (holding cost per unit per year). Example: solve('ie_eoq', demand=1200, order_cost=100, holdin
Economic order quantity with annual demand = 1200, ordering cost = 100, carrying cost = 6, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Economic order quantity with annual demand = 1200, ordering cost = 100, carrying cost = 0.	INVALID_INPUT	Economic order quantity (EOQ): holding_cost must be positive.

Annual inventory cost of an order quantity, `ie_inventory_total_cost`

Annual ordering, holding and purchase cost of ordering a given quantity Q each time (constant demand).

$$TC(Q) = \frac{D}{Q}S + \frac{Q}{2}H + Dc$$

input	meaning	condition
`demand`	demand per year (units)	
`order_cost`	fixed cost per order or setup	
`holding_cost`	holding cost per unit per year	
`order_quantity`	quantity ordered each time	
`unit_cost`	purchase price per unit (optional, default 0)	

output	meaning
`total_cost`	annual total cost $DS/Q + HQ/2 + Dc$
`ordering_cost`	annual ordering cost
`holding_cost_per_year`	annual holding cost
`purchase_cost`	annual purchase cost
`average_inventory`	average inventory $Q/2$

- **Method:** $TC = D$ S / Q + H $Q/2 + D$ c
- **Assumes:** demand, order_cost, holding_cost, order_quantity > 0; unit_cost >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



total_cost (annual total cost DS/Q + HQ/2 + Dc) against order_quantity (quantity ordered each time) from 30 to 3000 on a logarithmic axis. One curve for each value of holding_cost (holding cost per unit per year): 6, 3, 12. Other inputs as in the worked example: demand = 1200, order_cost = 100. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, order_quantity = 300, total_cost = 1300.

```
industrial_engineering_ai999x.solve('ie_inventory_total_cost', demand=1200, order_cost=100,
holding_cost=6, order_quantity=300)
# -> total_cost = 1300, ordering_cost = 400, holding_cost_per_year = 900, purchase_cost =
0, average_inventory = 150   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_inventory_total_cost', demand=5000, order_cost=60,
holding_cost=4, order_quantity=400, unit_cost=12)
# -> total_cost = 61550, ordering_cost = 750, holding_cost_per_year = 800, purchase_cost =
60000, average_inventory = 200   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Total annual inventory cost with annual demand = 1200, ordering cost = 100, carrying cost = 6, lot size = 300.	SOLVED	total_cost = 1300, ordering_cost = 400, holding_cost_per_year = 900, purchase_cost = 0, average_inventory = 150
Total annual inventory cost with annual demand = 5000, ordering cost = 60, carrying cost = 4, lot size = 400, price per unit = 12.	SOLVED	total_cost = 61550, ordering_cost = 750, holding_cost_per_year = 800, purchase_cost = 60000, average_inventory = 200
Total annual inventory cost with annual demand = 1200, ordering cost = 100, carrying cost = 6.	NEEDS_INPUT	ie_inventory_total_cost needs order_quantity (quantity ordered each time). Example: solve('ie_inventory_total_cost', dem
Total annual inventory cost with annual demand = 1200, ordering cost = 100, carrying cost = 6, lot size = 300, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Total annual inventory cost with annual demand = 1200, ordering cost = 100, carrying cost = 6, lot size = -5.	INVALID_INPUT	Annual inventory cost of an order quantity: order_quantity must be positive.

EOQ with planned backorders, `ie_eoq_backorders`

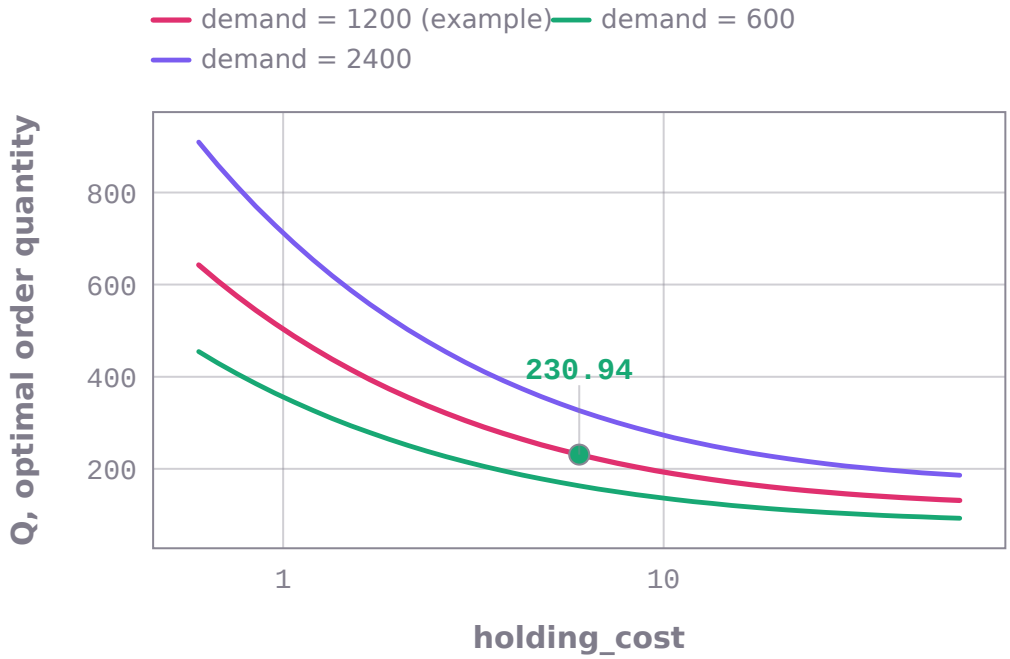
Order quantity and maximum backorder level minimising ordering, holding and backorder cost when shortages are allowed and fully backordered.

$$Q^* = \sqrt{\frac{2DS}{H} \cdot \frac{H+B}{B}}, \quad b^* = Q^* \frac{H}{H+B}$$

input	meaning	condition
`demand`	demand per year (units)	
`order_cost`	fixed cost per order or setup	
`holding_cost`	holding cost per unit per year	
`backorder_cost`	backorder (shortage) cost per unit per year	

output	meaning
`Q`	optimal order quantity
`max_backorder`	maximum backorder level b*
`max_inventory`	maximum inventory Q* - b*
`total_cost`	minimum annual cost
`orders_per_year`	orders per year
`fraction_backordered_time`	fraction of time out of stock H/(H+B)

- **Method:** Q* = sqrt($2DS/H$ (H+B)/B), b* = $QH/(H+B)$, TC* = sqrt(2DSH B/(H+B))
- **Assumes:** all inputs > 0; every shortage is backordered
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (optimal order quantity) against holding_cost (holding cost per unit per year) from 0.6 to 60 on a logarithmic axis. One curve for each value of demand (demand per year (units)): 1200, 600, 2400. Other inputs as in the worked example: order_cost = 100, backorder_cost = 18. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, holding_cost = 6, Q = 230.94.

```
industrial_engineering_ai999x.solve('ie_eoq_backorders', demand=1200, order_cost=100,
holding_cost=6, backorder_cost=18)
# -> Q = 230.94, max_backorder = 57.735, max_inventory = 173.205, total_cost = 1039.23,
orders_per_year = 5.19615, fraction_backordered_time = 0.25   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_eoq_backorders', demand=20000, order_cost=75,
holding_cost=3, backorder_cost=12)
# -> Q = 1118.03, max_backorder = 223.607, max_inventory = 894.427, total_cost = 2683.28,
orders_per_year = 17.8885, fraction_backordered_time = 0.2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
EOQ with planned backorders with annual demand = 1200, ordering cost = 100, carrying cost = 6, shortage cost = 18.	SOLVED	Q = 230.94, max_backorder = 57.735, max_inventory = 173.205, total_cost = 1039.23, orders_per_year = 5.19615, fraction_backordered_time = 0.25
EOQ with planned backorders with annual demand = 20000, ordering cost = 75, carrying cost = 3, shortage cost = 12.	SOLVED	Q = 1118.03, max_backorder = 223.607, max_inventory = 894.427, total_cost = 2683.28, orders_per_year = 17.8885, fraction_backordered_time = 0.2
EOQ with planned backorders with annual demand = 1200, ordering cost = 100, carrying cost = 6.	NEEDS_INPUT	ie_eoq_backorders needs backorder_cost (backorder (shortage) cost per unit per year). Example: solve('ie_eoq_backorders'
EOQ with planned backorders with annual demand = 1200, ordering cost = 100, carrying cost	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
= 6, shortage cost = 18, colour = 3.		
EOQ with planned backorders with annual demand = 1200, ordering cost = 100, carrying cost = 6, shortage cost = 0.	INVALID_INPUT	EOQ with planned backorders: backorder_cost must be positive.

EOQ with all-units quantity discounts, `ie_eoq_quantity_discount`

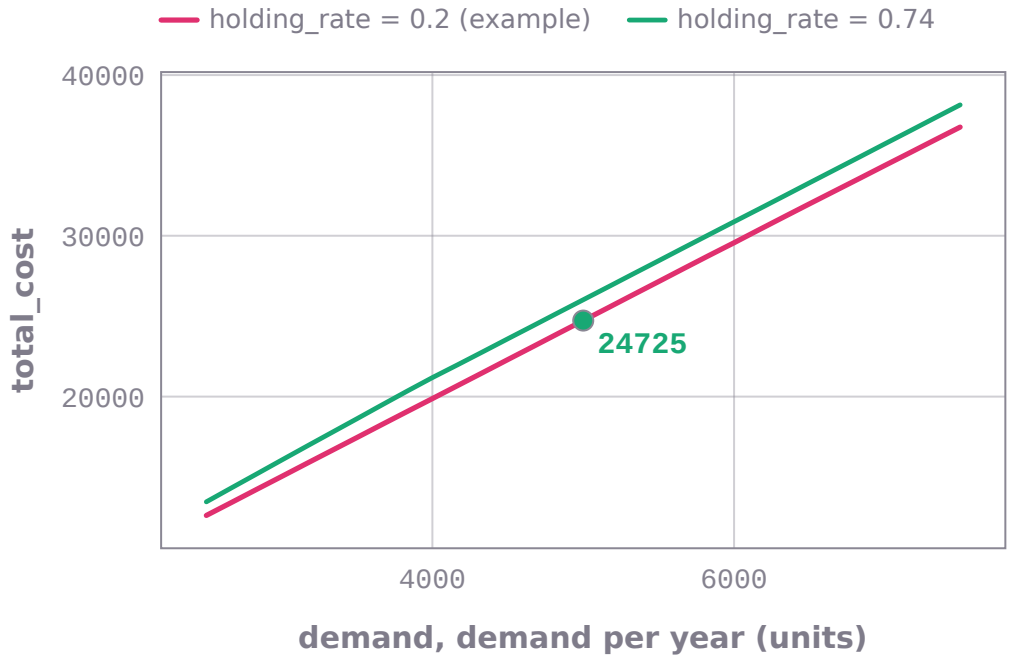
Best order quantity when the unit price falls at quantity breaks (all-units discount) and holding cost is a fraction of the unit price.

$$TC_k(Q) = \frac{D}{Q}S + \frac{Q}{2}\,i\,p_k + Dp_k$$

input	meaning	condition
`demand`	demand per year (units)	
`order_cost`	fixed cost per order or setup	
`holding_rate`	annual holding cost as a fraction of the unit price (decimal; 20% = 0.2)	
`break_quantities`	minimum quantity of each price tier, starting with 0	
`unit_prices`	unit price of each tier (falling)	

output	meaning
`Q`	best order quantity
`unit_price`	unit price paid
`total_cost`	annual ordering + holding + purchase cost
`tier`	price tier chosen (1 = first)
`ordering_cost`	annual ordering cost
`holding_cost_per_year`	annual holding cost
`purchase_cost`	annual purchase cost

- **Method:** for each tier: EOQ at its price, raised to the tier's minimum if below it (skipped if above the next break); the lowest total cost D S/Q + i p $Q/2 + D$ p wins
- **Assumes:** break_quantities start at 0 and increase; unit_prices fall tier by tier; 0 < holding_rate < 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



total_cost (annual ordering + holding + purchase cost) against demand (demand per year (units)) from 2500 to 7500. One curve for each value of holding_rate (annual holding cost as a fraction of the unit price (decimal; 20% = 0.2)): 0.2, 0.74. Other inputs as in the worked example: order_cost = 49, break_quantities = [0, 1000, 2000], unit_prices = [5.0, 4.8, 4.75]. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, demand = 5000, total_cost = 24725.

```
industrial_engineering_ai999x.solve('ie_eoq_quantity_discount', demand=5000, order_cost=49,
holding_rate=0.2, break_quantities=[0, 1000, 2000], unit_prices=[5.0, 4.8, 4.75])
# -> Q = 1000, unit_price = 4.8, total_cost = 24725, tier = 2, ordering_cost = 245,
holding_cost_per_year = 480, purchase_cost = 24000   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_eoq_quantity_discount', demand=1200,
order_cost=100, holding_rate=0.25, break_quantities=[0, 300, 600], unit_prices=[20, 19,
18.5])
# -> Q = 600, unit_price = 18.5, total_cost = 23787.5, tier = 3, ordering_cost = 200,
holding_cost_per_year = 1387.5, purchase_cost = 22200   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
EOQ with quantity discounts with annual demand = 5000, ordering cost = 49, holding cost rate = 0.2, breaks = [0, 1000, 2000], prices = [5, 4.8, 4.75].	SOLVED	Q = 1000, unit_price = 4.8, total_cost = 24725, tier = 2, ordering_cost = 245, holding_cost_per_year = 480, purchase_cost = 24000
EOQ with quantity discounts with annual demand = 1200, ordering cost = 100, holding cost rate = 0.25, breaks = [0, 300, 600], prices = [20, 19, 18.5].	SOLVED	Q = 600, unit_price = 18.5, total_cost = 23787.5, tier = 3, ordering_cost = 200, holding_cost_per_year = 1387.5, purchase_cost = 22200
EOQ with quantity discounts with annual demand = 5000, ordering cost = 49, holding cost rate = 0.2, breaks = [0, 1000, 2000].	NEEDS_INPUT	ie_eoq_quantity_discount needs unit_prices (unit price of each tier (falling)). Example: solve('ie_eoq_quantity_discount

question	outcome	result
EOQ with quantity discounts with annual demand = 5000, ordering cost = 49, holding cost rate = 0.2, breaks = [0, 1000, 2000], prices = [5, 4.8, 4.75], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
EOQ with quantity discounts with annual demand = 5000, ordering cost = 49, holding cost rate = 0.2, breaks = [0, 1000, 2000], prices = [5, 5.2, 4.75].	INVALID_INPUT	EOQ with all-units quantity discounts: unit prices must fall as the break quantity rises.

Economic production quantity (EPQ), `ie_epq`

Production lot size minimising setup plus holding cost when stock builds up gradually at a finite production rate.

$$Q^* = \sqrt{\frac{2DS}{H\,(1 - D/P)}}$$

input	meaning	condition
`demand`	demand per year (units)	
`production_rate`	production rate per year (same units as demand)	
`order_cost`	fixed cost per order or setup	
`holding_cost`	holding cost per unit per year	

output	meaning
`Q`	economic production quantity
`max_inventory`	maximum inventory Q(1 - D/P)
`production_run_time`	length of a production run Q/P (years)
`cycle_time`	cycle Q/D (years)
`runs_per_year`	production runs per year
`total_cost`	annual setup plus holding cost

- **Method:** Q* = sqrt(2DS / (H $(1 - D/P)$)), I_max = Q*(1 - D/P)
- **Assumes:** all inputs > 0; production_rate > demand
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (economic production quantity) against demand (demand per year (units)) from 1200 to 120000 on a logarithmic axis. One curve for each value of order_cost (fixed cost per order or setup): 400, 800. Other inputs as in the worked example: production_rate = 48000, holding_cost = 3. 33 points per curve computed by industrial-engineering-ai999x; marked: the worked example, demand = 12000, Q = 2065.59.

```
industrial_engineering_ai999x.solve('ie_epq', demand=12000, production_rate=48000,
order_cost=400, holding_cost=3)
# -> Q = 2065.59, max_inventory = 1549.19, production_run_time = 0.0430331, cycle_time =
0.172133, runs_per_year = 5.80948, total_cost = 4647.58   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_epq', demand=1000, production_rate=4000,
order_cost=10, holding_cost=0.5)
# -> Q = 230.94, max_inventory = 173.205, production_run_time = 0.057735, cycle_time =
0.23094, runs_per_year = 4.33013, total_cost = 86.6025   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Economic production quantity with annual demand = 12000, annual production rate = 48000, ordering cost = 400, carrying cost = 3.	SOLVED	Q = 2065.59, max_inventory = 1549.19, production_run_time = 0.0430331, cycle_time = 0.172133, runs_per_year = 5.80948, total_cost = 4647.58
Economic production quantity with annual demand = 1000, annual production rate = 4000, ordering cost = 10, carrying cost = 0.5.	SOLVED	Q = 230.94, max_inventory = 173.205, production_run_time = 0.057735, cycle_time = 0.23094, runs_per_year = 4.33013, total_cost = 86.6025
Economic production quantity with annual demand = 12000, annual production rate = 48000, ordering cost = 400.	NEEDS_INPUT	ie_epq needs holding_cost (holding cost per unit per year). Example: solve('ie_epq', demand=12000, production_rate=48000
Economic production quantity with annual demand = 12000, annual production rate = 48000, ordering	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
cost = 400, carrying cost = 3, colour = 3.		
Economic production quantity with annual demand = 12000, annual production rate = 9000, ordering cost = 400, carrying cost = 3.	INVALID_INPUT	Economic production quantity (EPQ): the production rate must exceed the demand rate.

Reorder point (constant demand), `ie_reorder_point`

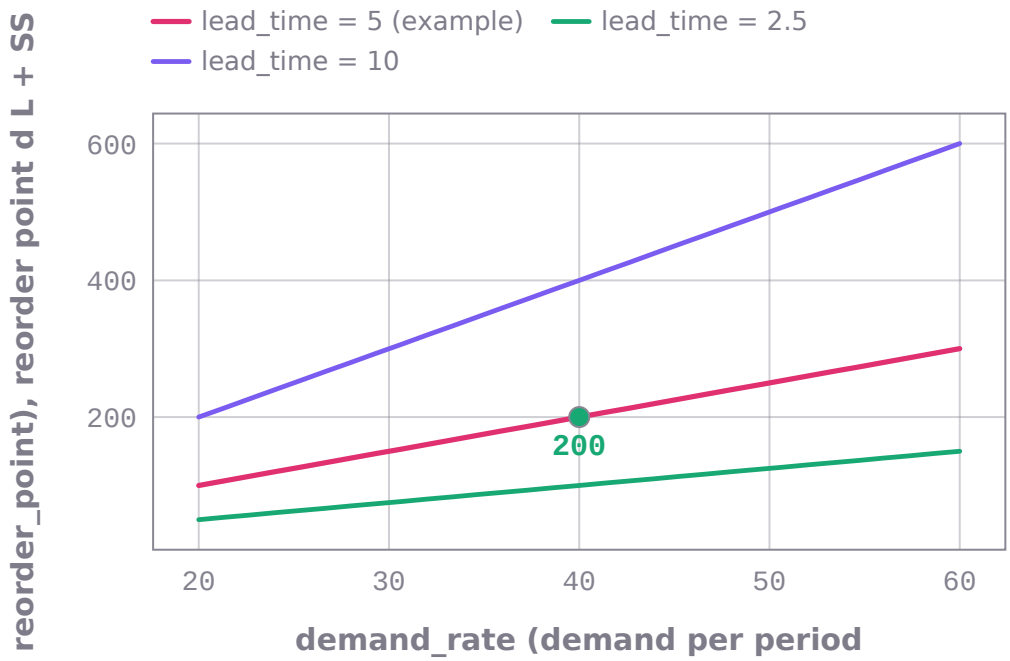
Inventory level at which to reorder: demand during the lead time plus any safety stock.

$$ROP = d\,L + SS$$

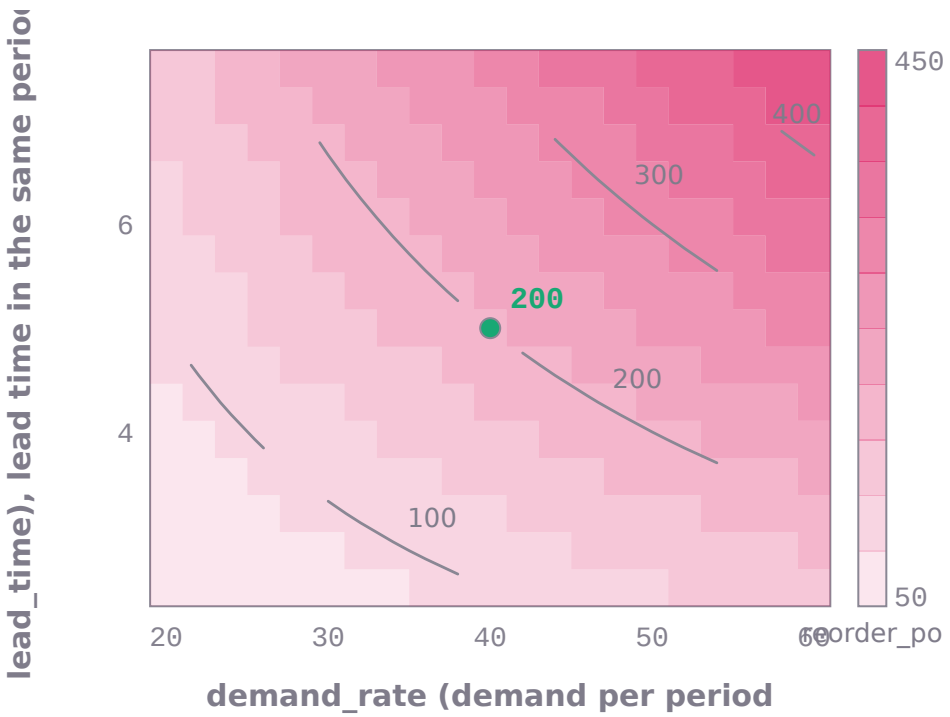
input	meaning	condition
`demand_rate`	demand per period (per day, say)	
`lead_time`	lead time in the same periods	
`safety_stock`	safety stock (optional, default 0)	

output	meaning
`reorder_point`	reorder point d L + SS
`lead_time_demand`	demand during the lead time d L

- **Method:** ROP = d L + SS
- **Assumes:** demand_rate > 0; lead_time >= 0; safety_stock >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reorder_point (reorder point d L + SS) against demand_rate (demand per period (per day, say)) from 20 to 60. One curve for each value of lead_time (lead time in the same periods): 5, 2.5, 10. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, demand_rate = 40, reorder_point = 200.



reorder_point (reorder point d L + SS) over demand_rate (demand per period (per day, say)) from 20 to 60 and lead_time (lead time in the same periods) from 2.5 to 7.5: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 50 to dark 450); iso-lines at reorder_point = 100, 200, 300, 400. Marked: the worked example, demand_rate = 40, lead_time = 5, reorder_point = 200.

```
industrial_engineering_ai999x.solve('ie_reorder_point', demand_rate=40, lead_time=5)
# -> reorder_point = 200, lead_time_demand = 200   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_reorder_point', demand_rate=120, lead_time=3.5,
safety_stock=60)
# -> reorder_point = 480, lead_time_demand = 420   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reorder point with daily demand = 40, lead time in days = 5.	SOLVED	reorder_point = 200, lead_time_demand = 200
Reorder point with daily demand = 120, lead time in days = 3.5, buffer stock = 60.	SOLVED	reorder_point = 480, lead_time_demand = 420
Reorder point with daily demand = 40.	NEEDS_INPUT	ie_reorder_point needs lead_time (lead time in the same periods). Example: solve('ie_reorder_point', demand_rate=40, lea
Reorder point with daily demand = 40, lead time in days = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Reorder point with daily demand = 40, lead time in days = -2.	INVALID_INPUT	Reorder point (constant demand): lead_time must not be negative.

Safety stock and reorder point (normal demand), `ie_safety_stock`

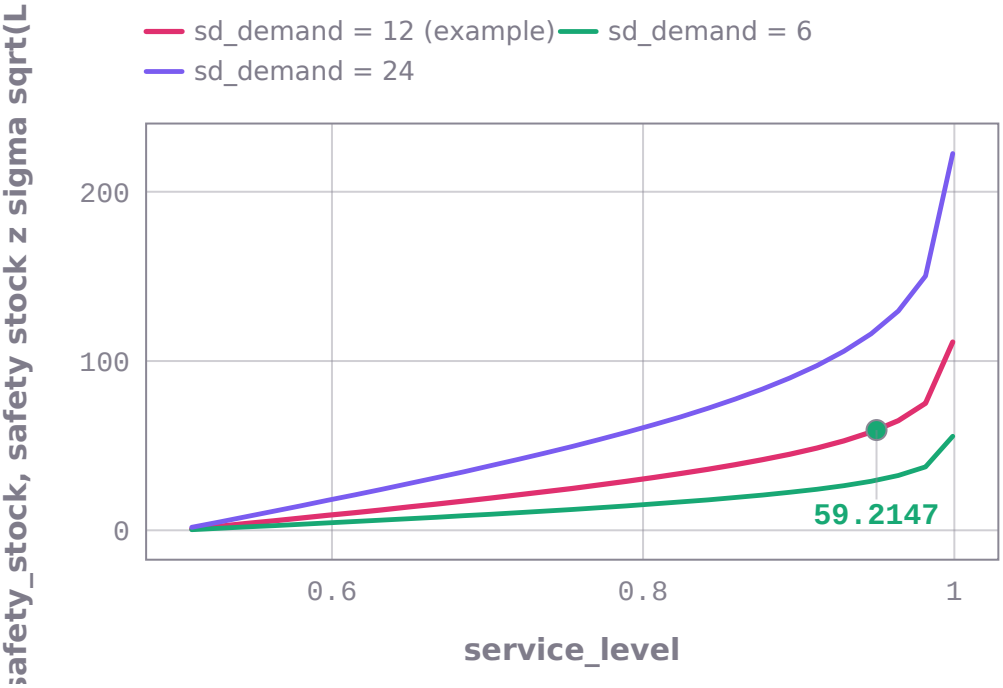
Safety stock and reorder point for a cycle service level when demand per period is normal and the lead time is fixed.

$$SS = z\sigma_d\sqrt{L}, \quad ROP = \bar{d}L + SS$$

input	meaning	condition
`mean_demand`	mean demand per period (per day, say)	
`sd_demand`	standard deviation of demand per period	
`lead_time`	lead time in periods	
`service_level`	cycle service level: probability of no stock-out per cycle (decimal)	

output	meaning
`safety_stock`	safety stock z sigma sqrt(L)
`reorder_point`	reorder point d L + SS
`z`	service factor z
`sd_lead_time_demand`	standard deviation of lead-time demand
`lead_time_demand`	mean lead-time demand

- **Method:** $sigma_L = sigma_d$ sqrt(L), $z = Phi^{-1}$(CSL), $SS = z$ sigma_L, ROP = d L + SS
- **Assumes:** mean_demand, sd_demand, lead_time > 0; 0.5 < service_level < 1; demand per period independent and normal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



safety_stock (safety stock z sigma sqrt(L)) against service_level (cycle service level: probability of no stock-out per cycle (decimal)) from 0.475 to 0.999. One curve for each value of sd_demand (standard deviation of demand per period): 12, 6, 24. Other inputs as in the worked example: mean_demand = 50, lead_time = 9. 29 points per curve computed by industrial-engineering-ai999x; marked: the worked example, service_level = 0.95, safety_stock = 59.2147.

```
industrial_engineering_ai999x.solve('ie_safety_stock', mean_demand=50, sd_demand=12,
lead_time=9, service_level=0.95)
# -> safety_stock = 59.2147, reorder_point = 509.215, z = 1.64485, sd_lead_time_demand =
36, lead_time_demand = 450   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_safety_stock', mean_demand=200, sd_demand=40,
lead_time=4, service_level=0.99)
# -> safety_stock = 186.108, reorder_point = 986.108, z = 2.32635, sd_lead_time_demand =
80, lead_time_demand = 800   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Safety stock with average daily demand = 50, standard deviation of daily demand = 12, lead time in days = 9, cycle service level = 0.95.	SOLVED	safety_stock = 59.2147, reorder_point = 509.215, z = 1.64485, sd_lead_time_demand = 36, lead_time_demand = 450
Safety stock with average daily demand = 200, standard deviation of daily demand = 40, lead time in days = 4, cycle service level = 0.99.	SOLVED	safety_stock = 186.108, reorder_point = 986.108, z = 2.32635, sd_lead_time_demand = 80, lead_time_demand = 800
Safety stock with average daily demand = 50, standard deviation of daily demand = 12, lead time in days = 9.	NEEDS_INPUT	ie_safety_stock needs service_level (cycle service level: probability of no stock-out per cycle (decimal)). Example: sol
Safety stock with average daily demand = 50, standard deviation of daily demand = 12, lead time in days = 9, cycle service level = 0.95, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Safety stock with average daily demand = 50, standard deviation of daily demand = 12, lead time in days = 9, cycle service level = 1.2.	INVALID_INPUT	Safety stock and reorder point (normal demand): service_level must lie strictly between 0.5 and 1 (write 95% or 0.95).

Safety stock with variable demand and lead time, `ie_safety_stock_variable_lead_time`

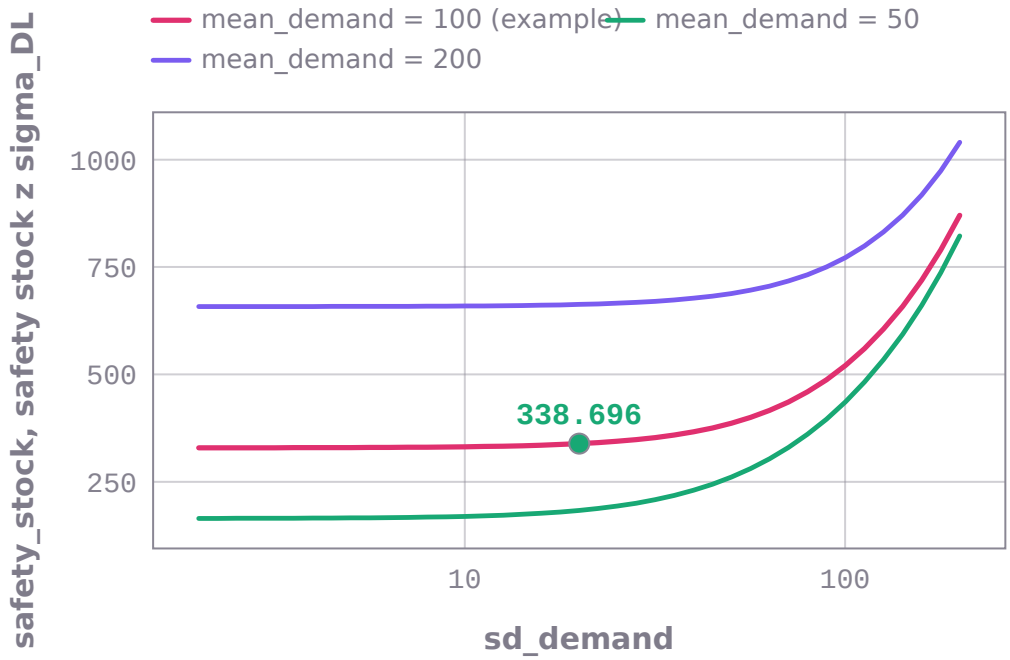
Safety stock and reorder point when both the demand per period and the lead time vary.

$$\sigma_{DL} = \sqrt{\bar{L}\,\sigma_d^2 + \bar{d}^{\,2}\sigma_L^2}, \quad SS = z\,\sigma_{DL}$$

input	meaning	condition
`mean_demand`	mean demand per period (per day, say)	
`sd_demand`	standard deviation of demand per period	
`mean_lead_time`	mean lead time in periods	
`sd_lead_time`	standard deviation of the lead time	
`service_level`	cycle service level: probability of no stock-out per cycle (decimal)	

output	meaning
`safety_stock`	safety stock z sigma_DL
`reorder_point`	reorder point d L + SS
`sd_lead_time_demand`	$sigma_D L = sqrt$(L $sigma_d^2 + d^2$ sigma_L^2)
`z`	service factor

- **Method:** $sigma_D L = sqrt$(L $sigma_d^2 + d^2$ sigma_L^2), $SS = z$ sigma_DL
- **Assumes:** mean_demand, mean_lead_time > 0; standard deviations >= 0, not both 0; 0.5 < service_level < 1; demand and lead time independent; lead-time demand treated as normal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



safety_stock (safety stock z sigma_DL) against sd_demand (standard deviation of demand per period) from 2 to 200 on a logarithmic axis. One curve for each value of mean_demand (mean demand per period (per day, say)): 100, 50, 200. Other inputs as in the worked example: mean_lead_time = 6, sd_lead_time = 2, service_level = 0.95. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, sd_demand = 20, safety_stock = 338.696.

```
industrial_engineering_ai999x.solve('ie_safety_stock_variable_lead_time', mean_demand=100,
sd_demand=20, mean_lead_time=6, sd_lead_time=2, service_level=0.95)
# -> safety_stock = 338.696, reorder_point = 938.696, sd_lead_time_demand = 205.913, z =
1.64485   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_safety_stock_variable_lead_time', mean_demand=40,
sd_demand=10, mean_lead_time=12, sd_lead_time=1.5, service_level=0.98)
# -> safety_stock = 142.288, reorder_point = 622.288, sd_lead_time_demand = 69.282, z =
2.05375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Safety stock with variable lead time with average daily demand = 100, standard deviation of daily demand = 20, average	SOLVED	safety_stock = 338.696, reorder_point = 938.696, sd_lead_time_demand = 205.913, z = 1.64485

question	outcome	result
lead time = 6, standard deviation of lead time = 2, cycle service level = 0.95.		
Safety stock with variable lead time with average daily demand = 40, standard deviation of daily demand = 10, average lead time = 12, standard deviation of lead time = 1.5, cycle service level = 0.98.	SOLVED	safety_stock = 142.288, reorder_point = 622.288, sd_lead_time_demand = 69.282, z = 2.05375
Safety stock with variable lead time with average daily demand = 100, standard deviation of daily demand = 20, average lead time = 6, standard deviation of lead time = 2.	NEEDS_INPUT	ie_safety_stock_variable_lead_time needs service_level (cycle service level: probability of no stock-out per cycle (deci
Safety stock with variable lead time with average daily demand = 100, standard deviation of daily demand = 20, average lead time = 6, standard deviation of lead time = 2, cycle service level = 0.95, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Safety stock with variable lead time with average daily demand = 100, standard deviation of daily demand = 20, average lead time = 6, standard deviation of lead time = -2, cycle service level = 0.95.	INVALID_INPUT	Safety stock with variable demand and lead time: sd_lead_time must not be negative.

Newsvendor order quantity (normal demand), `ie_newsvendor_normal`

Single-period order quantity that maximises expected profit when demand is normal: the critical ratio Cu/(Cu + Co).

$$F(Q^*) = \frac{C_u}{C_u + C_o}, \quad Q^* = \mu + z\sigma$$

input	meaning	condition
`price`	selling price per unit	
`cost`	purchase cost per unit	
`mean_demand`	mean demand per period (per day, say)	
`sd_demand`	standard deviation of demand per period	
`salvage`	salvage value per unsold unit (optional, default 0)	

output	meaning
`Q`	optimal order quantity mu + z sigma
`critical_ratio`	critical ratio Cu/(Cu+Co)
`z`	z of the critical ratio
`expected_sales`	expected units sold
`expected_leftover`	expected units left over
`expected_shortage`	expected unmet demand

output	meaning
`expected_profit`	expected profit

- **Method:** $Cu = p - c$, $Co = c - v$, $CR = Cu/(Cu + Co)$, $Q = mu + Phi^{-1}$(CR) sigma; E[sales] = mu - sigma L(z)
- **Assumes:** salvage < cost < price; sd_demand > 0; demand normal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (optimal order quantity mu + z sigma) against price (selling price per unit) from 5 to 15. One curve for each value of mean_demand (mean demand per period (per day, say)): 500, 250, 1000. Other inputs as in the worked example: cost = 6, sd_demand = 100, salvage = 2. 27 points per curve computed by industrial-engineering-ai999x; marked: the worked example, price = 10, Q = 500.

```
industrial_engineering_ai999x.solve('ie_newsvendor_normal', price=10, cost=6,
mean_demand=500, sd_demand=100, salvage=2)
# -> Q = 500, critical_ratio = 0.5, z = 0, expected_sales = 460.106, expected_leftover =
39.8942, expected_shortage = 39.8942, expected_profit = 1680.85   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_newsvendor_normal', price=5, cost=1.5,
mean_demand=1000, sd_demand=250)
# -> Q = 1131.1, critical_ratio = 0.7, z = 0.524401, expected_sales = 952.407,
expected_leftover = 178.693, expected_shortage = 47.5931, expected_profit = 3065.38
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newsvendor order quantity with selling price = 10, unit cost = 6, average daily demand = 500, standard deviation of daily demand = 100, salvage value = 2.	SOLVED	Q = 500, critical_ratio = 0.5, z = 0, expected_sales = 460.106, expected_leftover = 39.8942, expected_shortage = 39.8942, expected_profit = 1680.85

question	outcome	result
Newsvendor order quantity with selling price = 5, unit cost = 1.5, average daily demand = 1000, standard deviation of daily demand = 250.	SOLVED	Q = 1131.1, critical_ratio = 0.7, z = 0.524401, expected_sales = 952.407, expected_leftover = 178.693, expected_shortage = 47.5931, expected_profit = 3065.38
Newsvendor order quantity with selling price = 10, unit cost = 6, average daily demand = 500, salvage value = 2.	NEEDS_INPUT	ie_newsvendor_normal needs sd_demand (standard deviation of demand per period). Example: solve('ie_newsvendor_normal', p
Newsvendor order quantity with selling price = 10, unit cost = 6, average daily demand = 500, standard deviation of daily demand = 100, salvage value = 2, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Newsvendor order quantity with selling price = 5, unit cost = 6, average daily demand = 500, standard deviation of daily demand = 100.	INVALID_INPUT	Newsvendor order quantity (normal demand): the newsvendor needs salvage < cost < price.

Newsvendor order quantity (uniform demand), `ie_newsvendor_uniform`

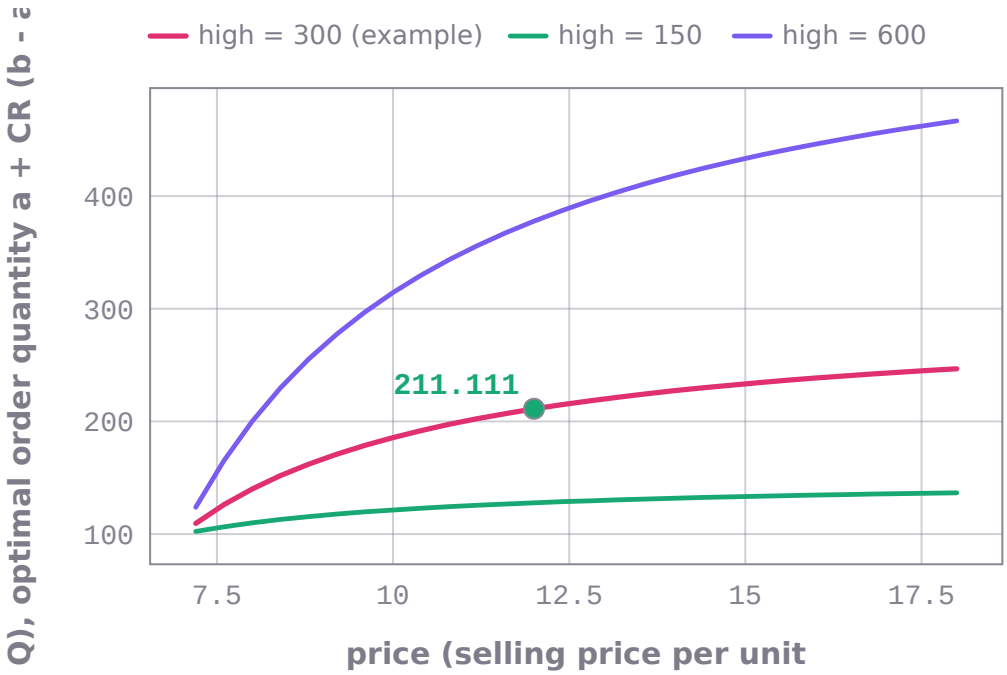
Single-period order quantity and expected profit when demand is uniform between two bounds.

$$Q^* = a + \frac{C_u}{C_u + C_o}(b - a)$$

input	meaning	condition
`price`	selling price per unit	
`cost`	purchase cost per unit	
`low`	lowest possible demand	
`high`	highest possible demand	
`salvage`	salvage value per unsold unit (optional, default 0)	

output	meaning
`Q`	optimal order quantity a + CR (b - a)
`critical_ratio`	critical ratio
`expected_sales`	expected units sold
`expected_leftover`	expected units left over
`expected_profit`	expected profit

- **Method:** $Q = a + CR$ (b - a), E[sales] = Q - (Q - a)^2 / (2 (b - a))
- **Assumes:** salvage < cost < price; high > low; demand uniform on [low, high]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Q (optimal order quantity a + CR (b - a)) against price (selling price per unit) from 6 to 18. One curve for each value of high (highest possible demand): 300, 150, 600. Other inputs as in the worked example: cost = 7, low = 100, salvage = 3. 28 points per curve computed by industrial-engineering-ai999x; marked: the worked example, price = 12, Q = 211.111.

```
industrial_engineering_ai999x.solve('ie_newsvendor_uniform', price=12, cost=7, low=100,
high=300, salvage=3)
# -> Q = 211.111, critical_ratio = 0.555556, expected_sales = 180.247, expected_leftover =
30.8642, expected_profit = 777.778   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_newsvendor_uniform', price=8, cost=5, low=0,
high=1000)
# -> Q = 375, critical_ratio = 0.375, expected_sales = 304.688, expected_leftover =
70.3125, expected_profit = 562.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newsvendor with uniform demand with selling price = 12, unit cost = 7, minimum demand = 100, maximum demand = 300, salvage value = 3.	SOLVED	Q = 211.111, critical_ratio = 0.555556, expected_sales = 180.247, expected_leftover = 30.8642, expected_profit = 777.778
Newsvendor with uniform demand with selling price = 8, unit cost = 5, minimum demand = 0, maximum demand = 1000.	SOLVED	Q = 375, critical_ratio = 0.375, expected_sales = 304.688, expected_leftover = 70.3125, expected_profit = 562.5
Newsvendor with uniform demand with selling price = 12, unit cost = 7, minimum demand = 100, salvage value = 3.	NEEDS_INPUT	ie_newsvendor_uniform needs high (highest possible demand). Example: solve('ie_newsvendor_uniform', price=12, cost=7, lo
Newsvendor with uniform demand with selling price = 12, unit cost = 7, minimum demand = 100, maximum demand = 300, salvage value = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Newsvendor with uniform demand with selling price = 12, unit cost = 7, minimum demand = 300, maximum demand = 100.	INVALID_INPUT	Newsvendor order quantity (uniform demand): high must exceed low.

Newsvendor order quantity (Poisson demand), `ie_newsvendor_poisson`

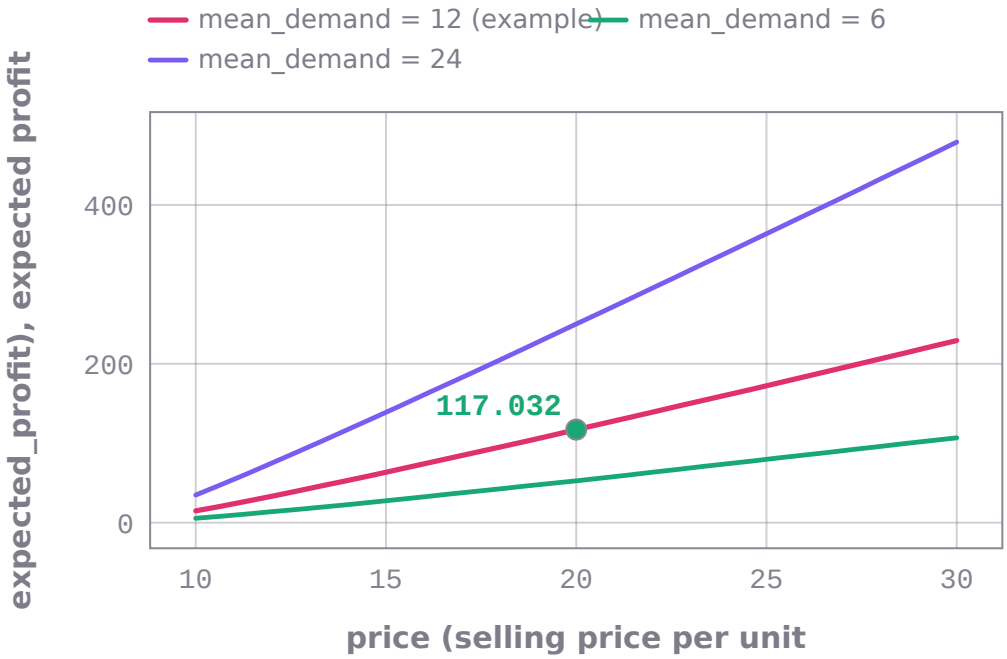
Whole-unit single-period order quantity when demand is Poisson: the smallest Q with F(Q) >= critical ratio.

$$Q^* = \min\{Q : F(Q) \geq \frac{C_u}{C_u+C_o}\}$$

input	meaning	condition
`price`	selling price per unit	
`cost`	purchase cost per unit	
`mean_demand`	mean (Poisson) demand	
`salvage`	salvage value per unsold unit (optional, default 0)	

output	meaning
`Q`	optimal whole order quantity
`critical_ratio`	critical ratio
`expected_sales`	expected units sold
`expected_leftover`	expected units left over
`expected_profit`	expected profit
`stockout_probability`	probability demand exceeds Q

- **Method:** smallest Q with P(D <= Q) >= Cu/(Cu+Co); E[leftover] = sum_{$k < Q$} (Q - k) P(D = k)
- **Assumes:** salvage < cost < price; 0 < mean_demand <= 500; demand Poisson
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



expected_profit (expected profit) against price (selling price per unit) from 10 to 30. One curve for each value of mean_demand (mean (Poisson) demand): 12, 6, 24. Other inputs as in the worked example: cost = 8. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, price = 20, expected_profit = 117.032.

```
industrial_engineering_ai999x.solve('ie_newsvendor_poisson', price=20, cost=8,
mean_demand=12)
# -> Q = 13, critical_ratio = 0.6, expected_sales = 11.0516, expected_leftover = 1.94838,
expected_profit = 117.032, stockout_probability = 0.318464   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_newsvendor_poisson', price=50, cost=30,
mean_demand=40, salvage=10)
# -> Q = 40, critical_ratio = 0.5, expected_sales = 37.4821, expected_leftover = 2.51788,
expected_profit = 699.285, stockout_probability = 0.458082   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newsvendor with Poisson demand with selling price = 20, unit cost = 8, average demand = 12.	SOLVED	Q = 13, critical_ratio = 0.6, expected_sales = 11.0516, expected_leftover = 1.94838, expected_profit = 117.032, stockout_probability = 0.318464
Newsvendor with Poisson demand with selling price = 50, unit cost = 30, average demand = 40, salvage value = 10.	SOLVED	Q = 40, critical_ratio = 0.5, expected_sales = 37.4821, expected_leftover = 2.51788, expected_profit = 699.285, stockout_probability = 0.458082
Newsvendor with Poisson demand with selling price = 20, unit cost = 8.	NEEDS_INPUT	ie_newsvendor_poisson needs mean_demand (mean (Poisson) demand). Example: solve('ie_newsvendor_poisson', price=20, cost=
Newsvendor with Poisson demand with selling price = 20, unit cost = 8, average demand = 12, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Newsvendor with Poisson demand with selling price = 20,	INVALID_INPUT	Newsvendor order quantity (Poisson demand): mean_demand must be positive.

question	outcome	result
unit cost = 8, average demand = -3.		

Newsvendor order quantity (discrete demand table), `ie_newsvendor_discrete`

Best single-period order quantity among the listed demand values, given their probabilities.

$$Q^* = \min\{d_k : F(d_k) \geq \frac{C_u}{C_u + C_o}\}$$

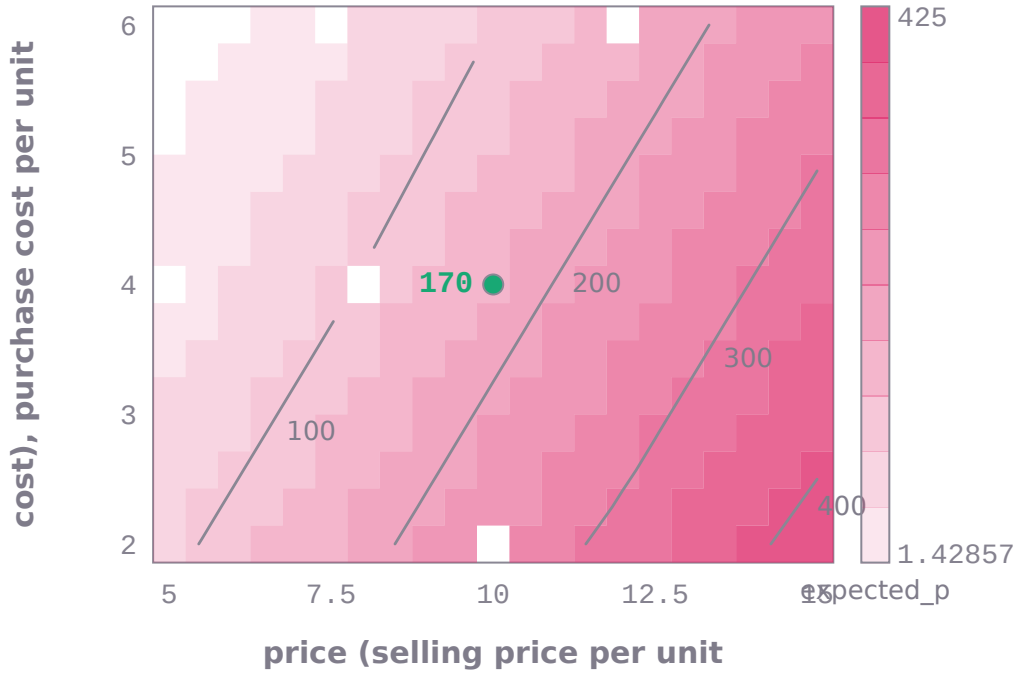
input	meaning	condition
`price`	selling price per unit	
`cost`	purchase cost per unit	
`demands`	possible demand values (increasing)	
`probabilities`	probability of each demand value (sum 1)	
`salvage`	salvage value per unsold unit (optional, default 0)	

output	meaning
`Q`	optimal order quantity (one of the demand values)
`critical_ratio`	critical ratio
`expected_sales`	expected units sold
`expected_profit`	expected profit

- **Method:** smallest demand value whose cumulative probability exceeds Cu/(Cu+Co); expected profit sum p_k profit(Q, d_k)
- **Assumes:** salvage < cost < price; demands increasing; probabilities add to 1; refused when two quantities are equally good
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



expected_profit (expected profit) against price (selling price per unit) from 5 to 15. One curve for each value of cost (purchase cost per unit): 4, 2, 8. Other inputs as in the worked example: demands = [20, 30, 40, 50], probabilities = [0.2, 0.3, 0.3, 0.2]. 29 points per curve computed by industrial-engineering-ai999x; marked: the worked example, price = 10, expected_profit = 170.



expected_profit (expected profit) over price (selling price per unit) from 5 to 15 and cost (purchase cost per unit) from 2 to 6: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 1.42857 to dark 425); iso-lines at expected_profit = 100, 200, 300, 400; blank cells: inputs the capability refuses. Other inputs as in the worked example: demands = [20, 30, 40, 50], probabilities = [0.2, 0.3, 0.3, 0.2]. Marked: the worked example, price = 10, cost = 4, expected_profit = 170.

```
industrial_engineering_ai999x.solve('ie_newsvendor_discrete', price=10, cost=4, demands=
[20, 30, 40, 50], probabilities=[0.2, 0.3, 0.3, 0.2])
# -> Q = 40, critical_ratio = 0.6, expected_sales = 33, expected_profit = 170   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_newsvendor_discrete', price=6, cost=3.5, demands=
[100, 200, 300], probabilities=[0.25, 0.5, 0.25], salvage=1)
# -> Q = 200, critical_ratio = 0.5, expected_sales = 175, expected_profit = 375   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Newsvendor with discrete demand with selling price = 10, unit cost = 4, demand values = [20, 30, 40, 50], probs = [0.2, 0.3, 0.3, 0.2].	SOLVED	Q = 40, critical_ratio = 0.6, expected_sales = 33, expected_profit = 170
Newsvendor with discrete demand with selling price = 6, unit cost = 3.5, demand values = [100, 200, 300], probs = [0.25, 0.5, 0.25], salvage value = 1.	SOLVED	Q = 200, critical_ratio = 0.5, expected_sales = 175, expected_profit = 375
Newsvendor with discrete demand with selling price = 10, unit cost = 4, demand values = [20, 30, 40, 50].	NEEDS_INPUT	ie_newsvendor_discrete needs probabilities (probability of each demand value (sum 1)). Example: solve('ie_newsvendor_dis
Newsvendor with discrete demand with selling price = 10, unit cost = 4, demand values = [20, 30, 40, 50], probs = [0.2, 0.3, 0.3, 0.2], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Newsvendor with discrete demand with selling price = 10, unit cost = 4, demand values = [20, 30, 40, 50], probs = [0.2, 0.3, 0.3, 0.3].	INVALID_INPUT	Newsvendor order quantity (discrete demand table): the probabilities must add up to 1.

(R, Q) reorder point for a target fill rate, `ie_rq_fill_rate`

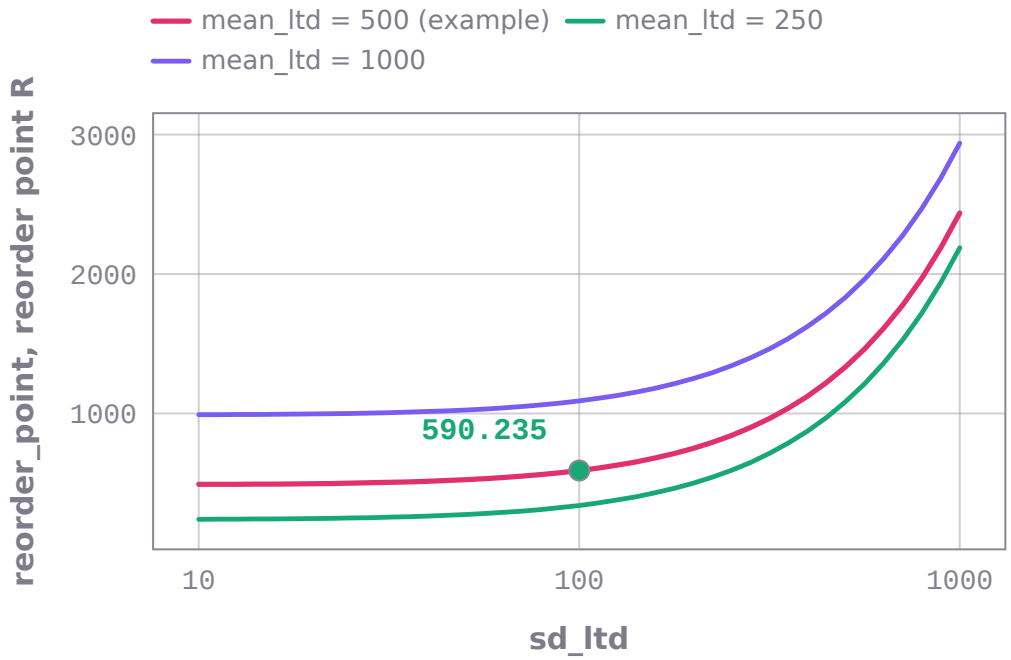
Reorder point of a continuous-review (R, Q) policy that meets a fill rate (fraction of demand met from stock) with normal lead-time demand.

$$\sigma_L \, L(z) = (1-\beta)\, Q, \quad R = \mu_L + z\,\sigma_L$$

input	meaning	condition
`mean_ltd`	mean demand during the lead time	
`sd_ltd`	standard deviation of lead-time demand	
`order_quantity`	order quantity Q	
`fill_rate`	target fill rate (decimal)	

output	meaning
`reorder_point`	reorder point R
`safety_stock`	safety stock R - mu
`z`	service factor
`expected_shortage_per_cycle`	expected units short per cycle $(1-\beta)$ Q
`cycle_service_level`	probability of no stock-out per cycle

- **Method:** solve sigma L(z) = (1 - beta) Q for z (L the standard normal loss function); $R = mu + z$ sigma
- **Assumes:** sd_ltd, order_quantity > 0; 0 < fill_rate < 1; lead-time demand normal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reorder_point (reorder point R) against sd_ltd (standard deviation of lead-time demand) from 10 to 1000 on a logarithmic axis. One curve for each value of mean_ltd (mean demand during the lead time): 500, 250, 1000. Other inputs as in the worked example: order_quantity = 1000, fill_rate = 0.99. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, sd_ltd = 100, reorder_point = 590.235.

```
industrial_engineering_ai999x.solve('ie_rq_fill_rate', mean_ltd=500, sd_ltd=100,
order_quantity=1000, fill_rate=0.99)
# -> reorder_point = 590.235, safety_stock = 90.2346, z = 0.902346,
expected_shortage_per_cycle = 10, cycle_service_level = 0.816564   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_rq_fill_rate', mean_ltd=200, sd_ltd=40,
order_quantity=150, fill_rate=0.95)
# -> reorder_point = 221.361, safety_stock = 21.3612, z = 0.534029,
expected_shortage_per_cycle = 7.5, cycle_service_level = 0.703339   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Reorder point for a fill rate with mean lead time demand = 500, standard deviation of lead time demand = 100, lot size = 1000, beta = 0.99.	SOLVED	reorder_point = 590.235, safety_stock = 90.2346, z = 0.902346, expected_shortage_per_cycle = 10, cycle_service_level = 0.816564
Reorder point for a fill rate with mean lead time demand = 200, standard deviation of lead time demand = 40, lot size = 150, beta = 0.95.	SOLVED	reorder_point = 221.361, safety_stock = 21.3612, z = 0.534029, expected_shortage_per_cycle = 7.5, cycle_service_level = 0.703339
Reorder point for a fill rate with mean lead time demand = 500, standard	NEEDS_INPUT	ie_rq_fill_rate needs fill_rate (target fill rate (decimal)). Example: solve('ie_rq_fill_rate', mean_ltd=500,

question	outcome	result
deviation of lead time demand = 100, lot size = 1000.		sd_ltd=100
Reorder point for a fill rate with mean lead time demand = 500, standard deviation of lead time demand = 100, lot size = 1000, beta = 0.99, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Reorder point for a fill rate with mean lead time demand = 500, standard deviation of lead time demand = 100, lot size = 1000, beta = 1.5.	INVALID_INPUT	(R, Q) reorder point for a target fill rate: fill_rate must lie strictly between 0 and 1 (write 95% or 0.95).

Service level of a reorder point, `ie_service_level_from_reorder_point`

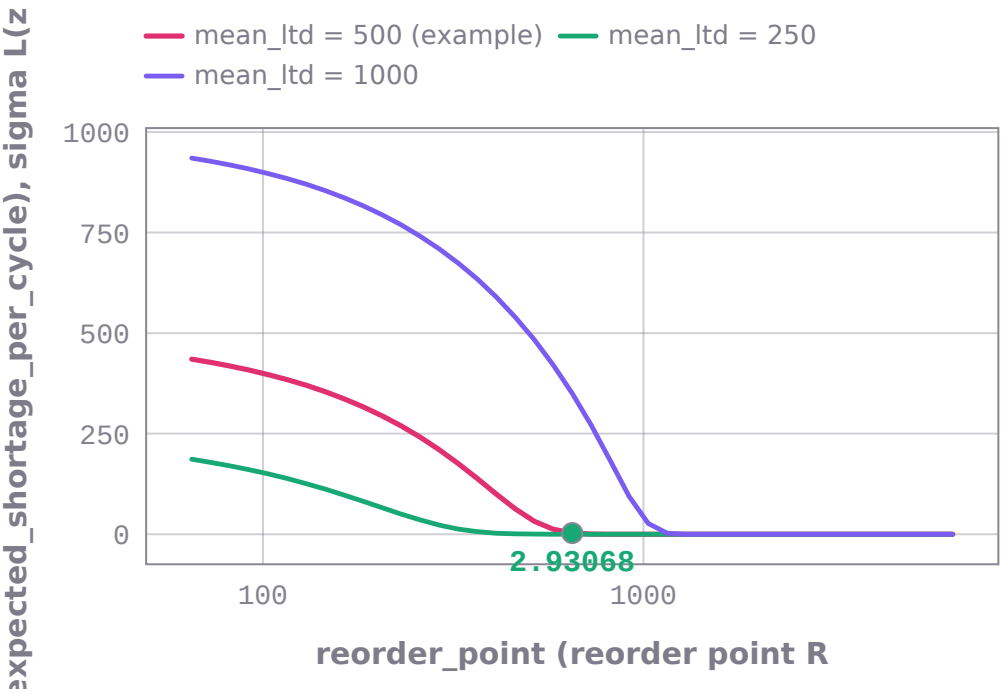
Cycle service level, expected shortage per cycle and fill rate achieved by a given reorder point with normal lead-time demand.

$$CSL = \Phi(z), \quad ESC = \sigma_L \, L(z), \quad \beta = 1 - \frac{ESC}{Q}$$

input	meaning	condition
`mean_ltd`	mean demand during the lead time	
`sd_ltd`	standard deviation of lead-time demand	
`reorder_point`	reorder point R	
`order_quantity`	order quantity Q (optional; gives the fill rate)	

output	meaning
`cycle_service_level`	P(no stock-out in a cycle) = Phi(z)
`z`	service factor (R - mu)/sigma
`safety_stock`	safety stock R - mu
`expected_shortage_per_cycle`	sigma L(z)
`fill_rate`	fill rate 1 - ESC/Q (when Q is given)

- **Method:** $z = (R - mu)/\sigma$, CSL = Phi(z), ESC = sigma L(z), fill rate = 1 - ESC/Q
- **Assumes:** sd_ltd > 0; order_quantity > 0 when given
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



expected_shortage_per_cycle (sigma L(z)) against reorder_point (reorder point R) from 65 to 6500 on a logarithmic axis. One curve for each value of mean_ltd (mean demand during the lead time): 500, 250, 1000. Other inputs as in the worked example: sd_ltd = 100, order_quantity = 1000. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, reorder_point = 650, expected_shortage_per_cycle = 2.93068.

```
industrial_engineering_ai999x.solve('ie_service_level_from_reorder_point', mean_ltd=500,
sd_ltd=100, reorder_point=650, order_quantity=1000)
# -> cycle_service_level = 0.933193, z = 1.5, safety_stock = 150,
expected_shortage_per_cycle = 2.93068, fill_rate = 0.997069   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_service_level_from_reorder_point', mean_ltd=240,
sd_ltd=30, reorder_point=260)
# -> cycle_service_level = 0.747507, z = 0.666667, safety_stock = 20,
expected_shortage_per_cycle = 4.53359   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Service level of a reorder point with mean lead time demand = 500, standard deviation of lead time demand = 100, reorder point = 650, lot size = 1000.	SOLVED	cycle_service_level = 0.933193, z = 1.5, safety_stock = 150, expected_shortage_per_cycle = 2.93068, fill_rate = 0.997069
Service level of a reorder point with mean lead time demand = 240, standard deviation of lead time demand = 30, reorder point = 260.	SOLVED	cycle_service_level = 0.747507, z = 0.666667, safety_stock = 20, expected_shortage_per_cycle = 4.53359
Service level of a reorder point with mean lead time demand = 500, standard deviation of lead time demand = 100, lot size = 1000.	NEEDS_INPUT	ie_service_level_from_reorder_point needs reorder_point (reorder point R). Example: solve('ie_service_level_from_reorder
Service level of a reorder point with mean lead time demand = 500, standard deviation of lead time	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
demand = 100, reorder point = 650, lot size = 1000, colour = 3.		
Service level of a reorder point with mean lead time demand = 500, standard deviation of lead time demand = 0, reorder point = 650.	INVALID_INPUT	Service level of a reorder point: sd_ltd must be positive.

Periodic review order-up-to level, `ie_order_up_to_level`

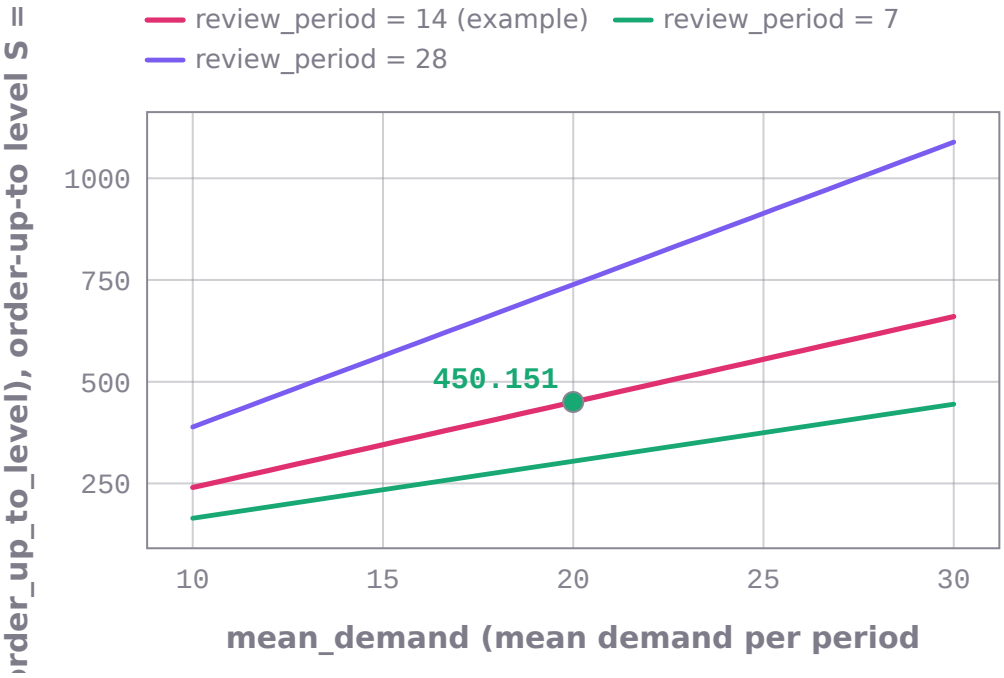
Order-up-to level (and order size) of a periodic-review policy that protects against demand over the review period plus the lead time.

$$S = \bar{d}(T + L) + z\sigma_d\sqrt{T + L}$$

input	meaning	condition
`mean_demand`	mean demand per period (per day, say)	
`sd_demand`	standard deviation of demand per period	
`review_period`	periods between reviews	
`lead_time`	lead time in periods	
`service_level`	cycle service level: probability of no stock-out per cycle (decimal)	
`inventory_position`	current inventory position: on hand + on order - backorders (optional)	

output	meaning
`order_up_to_level`	order-up-to level $S = \bar{d}$(T+L) + z sigma sqrt(T+L)
`safety_stock`	safety stock
`z`	service factor
`sd_protection_demand`	standard deviation of demand over T + L
`order_quantity`	order size S - inventory position (when given)

- **Method:** $S = \bar{d}$ (T + L) + z sigma_d sqrt(T + L); order S - IP
- **Assumes:** mean_demand, sd_demand, review_period > 0; lead_time >= 0; 0.5 < service_level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



order_up_to_level (order-up-to level S = d(T+L) + z sigma sqrt(T+L)) against mean_demand (mean demand per period (per day, say)) from 10 to 30. One curve for each value of review_period (periods between reviews): 14, 7, 28. Other inputs as in the worked example: sd_demand = 4, lead_time = 7, service_level = 0.95, inventory_position = 150. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, mean_demand = 20, order_up_to_level = 450.151.

```
industrial_engineering_ai999x.solve('ie_order_up_to_level', mean_demand=20, sd_demand=4,
review_period=14, lead_time=7, service_level=0.95, inventory_position=150)
# -> order_up_to_level = 450.151, safety_stock = 30.1507, z = 1.64485, sd_protection_demand
= 18.3303, order_quantity = 300.151   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_order_up_to_level', mean_demand=100, sd_demand=25,
review_period=7, lead_time=3, service_level=0.98)
# -> order_up_to_level = 1162.36, safety_stock = 162.363, z = 2.05375, sd_protection_demand
= 79.0569   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Order-up-to level with average daily demand = 20, standard deviation of daily demand = 4, review interval = 14, lead time in days = 7, cycle service level = 0.95, on hand = 150.	SOLVED	order_up_to_level = 450.151, safety_stock = 30.1507, z = 1.64485, sd_protection_demand = 18.3303, order_quantity = 300.151
Order-up-to level with average daily demand = 100, standard deviation of daily demand = 25, review interval = 7, lead time in days = 3, cycle service level = 0.98.	SOLVED	order_up_to_level = 1162.36, safety_stock = 162.363, z = 2.05375, sd_protection_demand = 79.0569
Order-up-to level with average daily demand = 20, standard deviation of daily demand = 4, review interval = 14, lead time in days = 7, on hand = 150.	NEEDS_INPUT	ie_order_up_to_level needs service_level (cycle service level: probability of no stock-out per cycle (decimal)). Example
Order-up-to level with average daily demand = 20, standard deviation of daily	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form:

question	outcome	result
demand = 4, review interval = 14, lead time in days = 7, cycle service level = 0.95, on hand = 150, colour = 3.		industrial_engineering_ai999x.tem
Order-up-to level with average daily demand = 20, standard deviation of daily demand = 4, review interval = 0, lead time in days = 7, cycle service level = 0.95.	INVALID_INPUT	Periodic review order-up-to level: review_period must be positive.

ABC inventory classification, `ie_abc_classification`

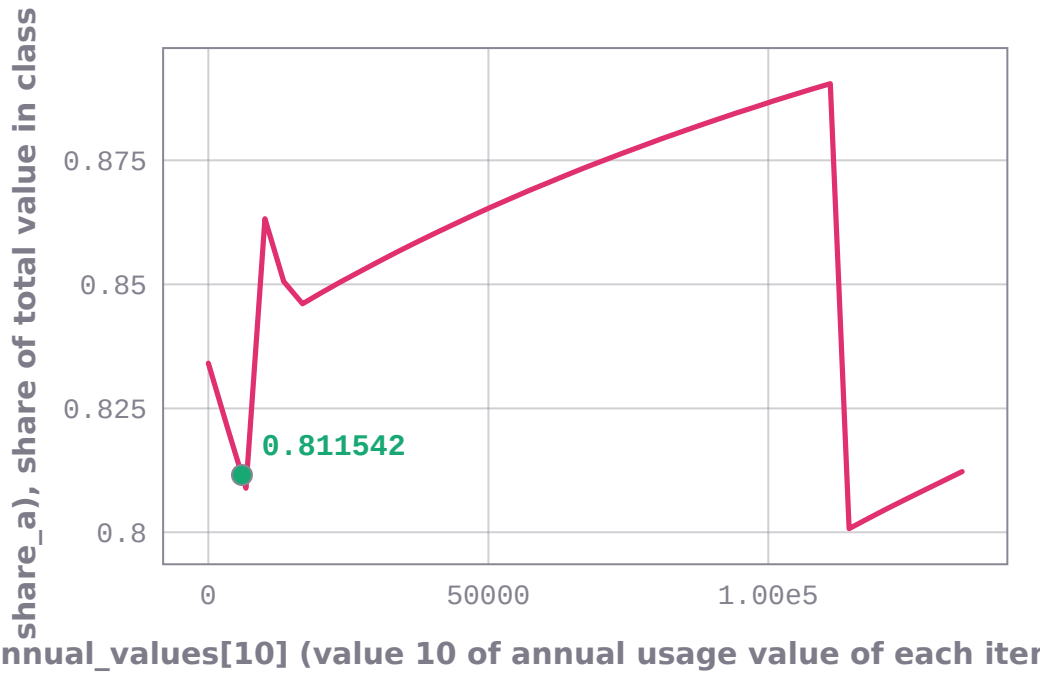
Classify items by annual usage value: A items make up the first ~80 % of value, B the next ~15 %, C the rest.

$$\text{class}(k) = A \text{ if } \frac{\sum_{j \prec k} v_j}{\sum v} < 0.8,\ B \text{ if } < 0.95,\ C$$

input	meaning	condition
`annual_values`	annual usage value of each item (annual demand x unit cost)	
`a_share`	cumulative value share closing class A (default 80%)	
`b_share`	cumulative value share closing class B (default 95%)	

output	meaning
`classes`	class of each item, in input order
`count_a`	number of A items
`count_b`	number of B items
`count_c`	number of C items
`share_a`	share of total value in class A
`share_b`	share of total value in class B

- **Method:** rank by value; cumulative share before each item decides its class
- **Assumes:** values >= 0, not all 0; 0 < a_share < b_share < 1; items ranked by value (ties by item number); an item belongs to A while the value ranked above it is below a_share of the total (so the item crossing the threshold is still A), likewise B
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



share_a (share of total value in class A) as value 10 of annual_values (annual usage value of each item (annual demand x unit cost)) moves from 0 to 134600, the other values as given ([90000, 15000, 4000, 60000, 2500, 12000, 1500, 800, 30000, 6000]); 41 runs of industrial-engineering-ai999x. Marked: the worked example, annual_values[10] = 6000, share_a = 0.811542.

```
industrial_engineering_ai999x.solve('ie_abc_classification', annual_values=[90000, 15000,
4000, 60000, 2500, 12000, 1500, 800, 30000, 6000])
# -> classes = ['A', 'B', 'C', 'A', 'C', 'B', 'C', 'C', 'A', 'B'], count_a = 3, count_b =
3, count_c = 4, share_a = 0.811542, share_b = 0.148783   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_abc_classification', annual_values=[500, 12000,
300, 8000, 200], a_share=0.7, b_share=0.9)
# -> classes = ['C', 'A', 'C', 'A', 'C'], count_a = 2, count_b = 0, count_c = 3, share_a =
0.952381, share_b = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
ABC classification with values = [90000, 15000, 4000, 60000, 2500, 12000, 1500, 800, 30000, 6000].	SOLVED	classes = ['A', 'B', 'C', 'A', 'C', 'B', 'C', 'C', 'A', 'B'], count_a = 3, count_b = 3, count_c = 4, share_a = 0.811542, share_b = 0.148783
ABC classification with values = [500, 12000, 300, 8000, 200], a cutoff = 0.7, b cutoff = 0.9.	SOLVED	classes = ['C', 'A', 'C', 'A', 'C'], count_a = 2, count_b = 0, count_c = 3, share_a = 0.952381, share_b = 0
ABC classification with values = [90000, 15000, 4000, 60000, 2500, 12000, 1500, 800, 30000, 6000], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
ABC classification with values = [0, 0, 0].	INVALID_INPUT	ABC inventory classification: the annual values must not all be 0.

Inventory turnover and days of supply, `ie_inventory_turnover`

How many times a year inventory turns over and how many days (weeks) of supply it holds.

$$\text{turns} = \frac{\text{COGS}}{\bar{I}}, \quad \text{days of supply} = \frac{365}{\text{turns}}$$

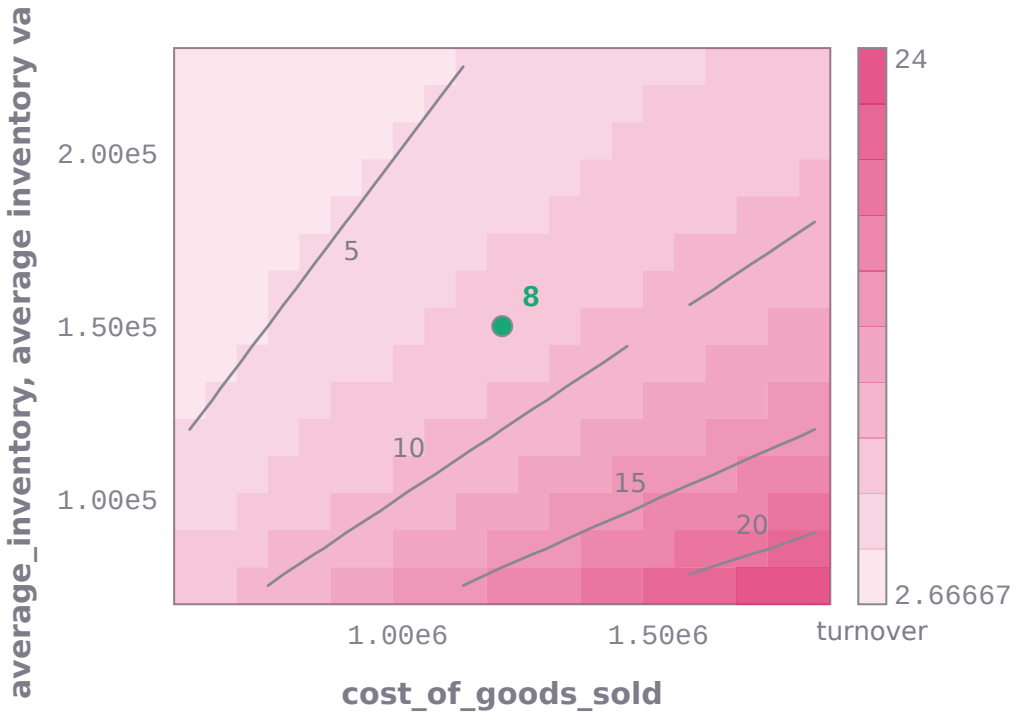
input	meaning	condition
`cost_of_goods_sold`	annual cost of goods sold	
`average_inventory`	average inventory value (at cost)	
`days_per_year`	days in the year used (default 365)	

output	meaning
`turnover`	inventory turns per year COGS / average inventory
`days_of_supply`	days of supply
`weeks_of_supply`	weeks of supply

- **Method:** turnover = COGS / average inventory; days = days per year / turnover
- **Assumes:** all inputs > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



turnover (inventory turns per year COGS / average inventory) against average_inventory (average inventory value (at cost)) from 75000 to 225000. One curve for each value of cost_of_goods_sold (annual cost of goods sold): 1.2e+06, 600000, 2.4e+06. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, average_inventory = 150000, turnover = 8.



turnover (inventory turns per year COGS / average inventory) over cost_of_goods_sold (annual cost of goods sold) from 600000 to 1.8e+06 and average_inventory (average inventory value (at cost)) from 75000 to 225000: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 2.66667 to dark 24); iso-lines at turnover = 5, 10, 15, 20. Marked: the worked example, cost_of_goods_sold = 1.2e+06, average_inventory = 150000, turnover = 8.

```
industrial_engineering_ai999x.solve('ie_inventory_turnover', cost_of_goods_sold=1200000,
average_inventory=150000)
# -> turnover = 8, days_of_supply = 45.625, weeks_of_supply = 6.5   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_inventory_turnover', cost_of_goods_sold=860000,
average_inventory=215000, days_per_year=250)
# -> turnover = 4, days_of_supply = 62.5, weeks_of_supply = 13   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inventory turnover with cogs = 1200000, inventory = 150000.	SOLVED	turnover = 8, days_of_supply = 45.625, weeks_of_supply = 6.5
Inventory turnover with cogs = 860000, inventory = 215000, days = 250.	SOLVED	turnover = 4, days_of_supply = 62.5, weeks_of_supply = 13
Inventory turnover with cogs = 1200000.	NEEDS_INPUT	ie_inventory_turnover needs average_inventory (average inventory value (at cost)). Example: solve('ie_inventory_turnover
Inventory turnover with cogs = 1200000, inventory = 150000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Inventory turnover with cogs = 1200000, inventory = 0.	INVALID_INPUT	Inventory turnover and days of supply: average_inventory must be positive.

Number of kanbans, `ie_kanban_count`

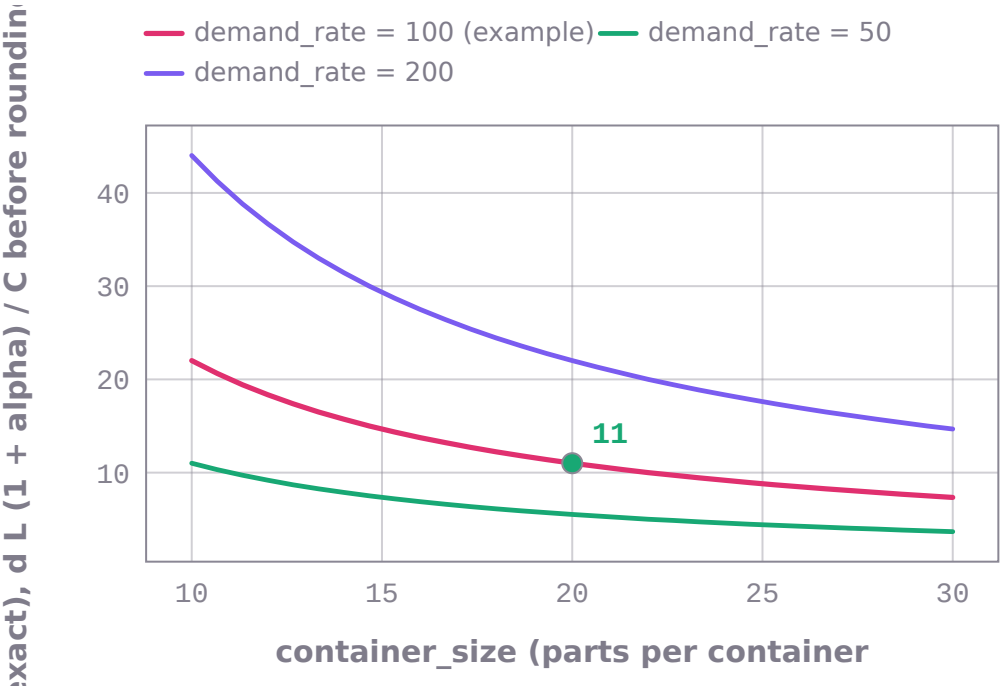
Number of kanban containers needed to cover demand during the replenishment lead time plus a safety factor.

$$N = \left\lceil \frac{d\,L\,(1+\alpha)}{C} \right\rceil$$

input	meaning	condition
`demand_rate`	demand per time unit (per hour, say)	
`lead_time`	replenishment lead time in the same time unit (waiting + processing)	
`container_size`	parts per container	
`safety_factor`	safety factor (decimal; default 0)	

output	meaning
`kanbans`	number of kanbans (rounded up)
`exact`	d L $(1+\alpha)/C$ before rounding
`max_inventory`	maximum inventory N C

- **Method:** N = ceil(d L (1 + alpha) / C)
- **Assumes:** demand_rate, lead_time, container_size > 0; safety_factor >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



exact (d L (1 + alpha) / C before rounding) against container_size (parts per container) from 10 to 30. One curve for each value of demand_rate (demand per time unit (per hour, say)): 100, 50, 200. Other inputs as in the worked example: lead_time = 2, safety_factor = 0.1. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, container_size = 20, exact = 11.

```
industrial_engineering_ai999x.solve('ie_kanban_count', demand_rate=100, lead_time=2,
container_size=20, safety_factor=0.1)
# -> kanbans = 11, exact = 11, max_inventory = 220   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_kanban_count', demand_rate=350, lead_time=0.5,
container_size=25)
# -> kanbans = 7, exact = 7, max_inventory = 175   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Number of kanbans with demand = 100, replenishment time = 2, container capacity = 20, alpha = 0.1.	SOLVED	kanbans = 11, exact = 11, max_inventory = 220
Number of kanbans with demand = 350, replenishment time = 0.5, container capacity = 25.	SOLVED	kanbans = 7, exact = 7, max_inventory = 175
Number of kanbans with demand = 100, replenishment time = 2, alpha = 0.1.	NEEDS_INPUT	ie_kanban_count needs container_size (parts per container). Example: solve('ie_kanban_count', demand_rate=100, lead_time
Number of kanbans with demand = 100, replenishment time = 2, container capacity = 20, alpha = 0.1, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Number of kanbans with demand = 100, replenishment time = 2, container capacity = 0.	INVALID_INPUT	Number of kanbans: container_size must be positive.

X-bar and R chart control limits, `ie_xbar_r_limits`

$3-\sigma$ control limits of the X-bar and R charts from the grand mean, the average range and the subgroup size.

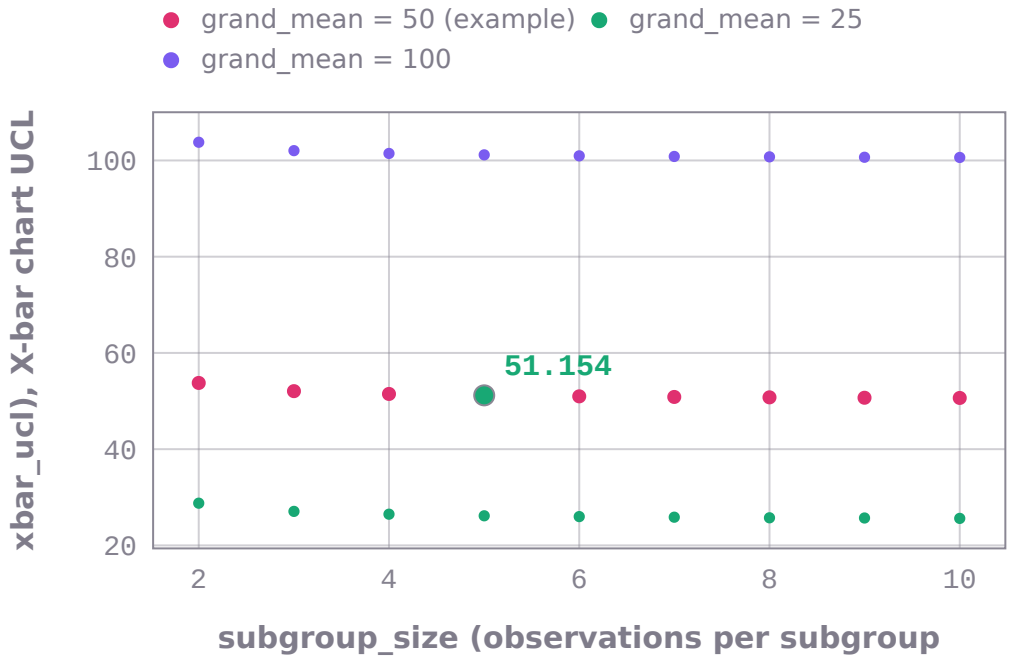
$$UCL_{\bar{x}} = \bar{\bar{x}} + A_2\bar{R}, \quad UCL_R = D_4\bar{R}, \quad LCL_R = D_3\bar{R}$$

input	meaning	condition
`grand_mean`	grand mean of the subgroup means (x-double-bar)	
`mean_range`	average subgroup range (R-bar)	
`subgroup_size`	observations per subgroup (2 to 10)	

output	meaning
`xbar_ucl`	X-bar chart UCL
`xbar_lcl`	X-bar chart LCL
`center`	X-bar chart centre line
`r_ucl`	R chart UCL D4 R-bar
`r_lcl`	R chart LCL D3 R-bar
`A2`	A2

output	meaning
`D3`	D3
`D4`	D4

- **Method:** UCL/LCL = x-double-bar +/- A2 R-bar; R chart D3 R-bar, D4 R-bar
- **Assumes:** mean_range > 0; subgroup size 2 to 10; standard 3-decimal constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



xbar_ucl (X-bar chart UCL) against subgroup_size (observations per subgroup (2 to 10)) from 1 to 13. One curve for each value of grand_mean (grand mean of the subgroup means (x-double-bar)): 50, 25, 100. Other inputs as in the worked example: mean_range = 2. 9 points per curve computed by industrial-engineering-ai999x; marked: the worked example, subgroup_size = 5, xbar_ucl = 51.154.

```
industrial_engineering_ai999x.solve('ie_xbar_r_limits', grand_mean=50, mean_range=2,
subgroup_size=5)
# -> xbar_ucl = 51.154, xbar_lcl = 48.846, center = 50, r_ucl = 4.228, r_lcl = 0, A2 =
0.577, D3 = 0, D4 = 2.114   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_xbar_r_limits', grand_mean=10.2, mean_range=0.35,
subgroup_size=4)
# -> xbar_ucl = 10.4551, xbar_lcl = 9.94485, center = 10.2, r_ucl = 0.7987, r_lcl = 0, A2 =
0.729, D3 = 0, D4 = 2.282   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
X-bar and R chart limits with x double bar = 50, r bar = 2, sample size = 5.	SOLVED	xbar_ucl = 51.154, xbar_lcl = 48.846, center = 50, r_ucl = 4.228, r_lcl = 0, A2 = 0.577, D3 = 0, D4 = 2.114
X-bar and R chart limits with x double bar = 10.2, r bar = 0.35, sample size = 4.	SOLVED	xbar_ucl = 10.4551, xbar_lcl = 9.94485, center = 10.2, r_ucl = 0.7987, r_lcl = 0, A2 = 0.729, D3 = 0, D4 = 2.282

question	outcome	result
X-bar and R chart limits with x double bar = 50, r bar = 2.	NEEDS_INPUT	ie_xbar_r_limits needs subgroup_size (observations per subgroup (2 to 10)). Example: solve('ie_xbar_r_limits', grand_mea
X-bar and R chart limits with x double bar = 50, r bar = 2, sample size = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
X-bar and R chart limits with x double bar = 50, r bar = 2, sample size = 12.	INVALID_INPUT	X-bar and R chart control limits: subgroup_size must be at most 10.

X-bar and S chart control limits, `ie_xbar_s_limits`

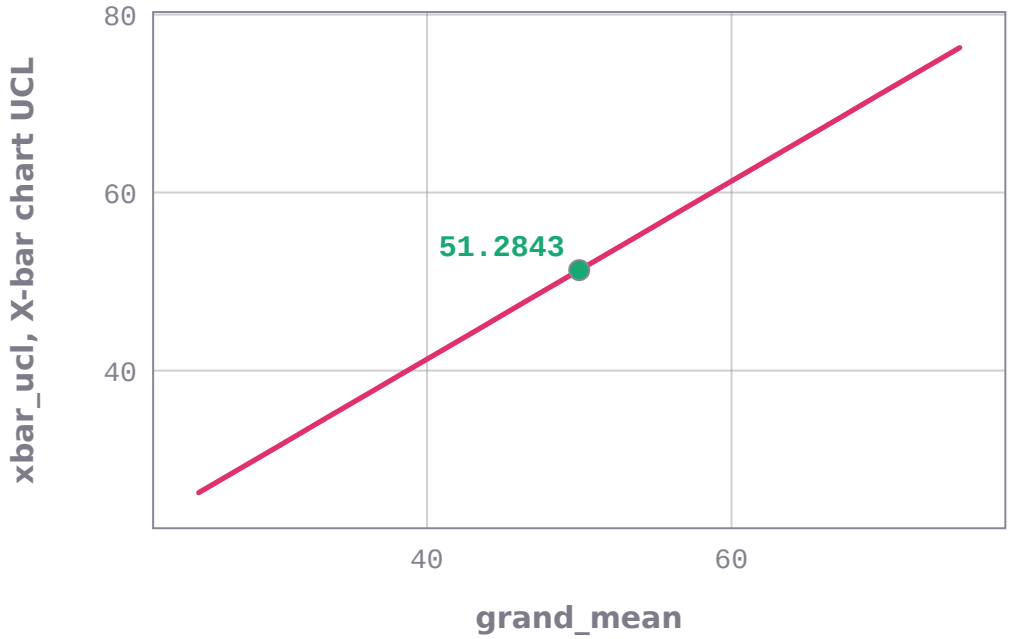
$3-\sigma$ control limits of the X-bar and S charts from the grand mean, the average subgroup standard deviation and the subgroup size.

$$UCL_{\bar{x}} = \bar{\bar{x}} + A_3\bar{s}, \quad UCL_s = B_4\bar{s}, \quad LCL_s = B_3\bar{s}$$

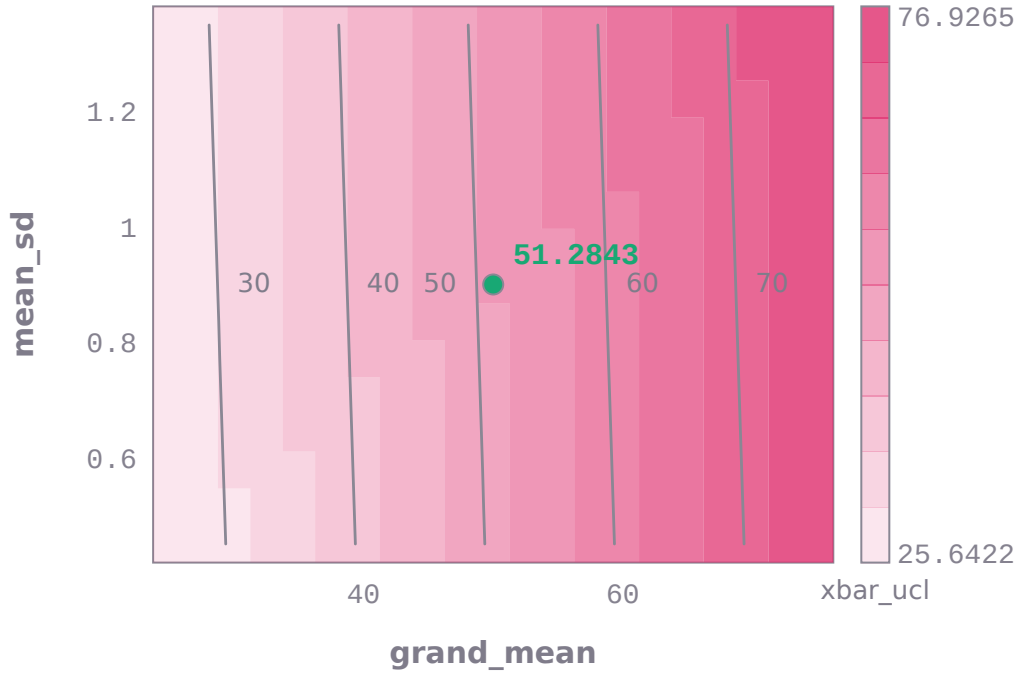
input	meaning	condition
`grand_mean`	grand mean of the subgroup means	
`mean_sd`	average subgroup standard deviation (s-bar)	
`subgroup_size`	observations per subgroup (2 to 10)	

output	meaning
`xbar_ucl`	X-bar chart UCL
`xbar_lcl`	X-bar chart LCL
`center`	centre line
`s_ucl`	S chart UCL B4 s-bar
`s_lcl`	S chart LCL B3 s-bar
`A3`	A3
`B3`	B3
`B4`	B4

- **Method:** x-double-bar +/- A3 s-bar; S chart B3 s-bar, B4 s-bar
- **Assumes:** mean_sd > 0; subgroup size 2 to 10; standard 3-decimal constants
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



xbar_ucl (X-bar chart UCL) against grand_mean (grand mean of the subgroup means) from 25 to 75. Other inputs as in the worked example: mean_sd = 0.9, subgroup_size = 5. 31 points computed by industrial-engineering-ai999x; marked: the worked example, grand_mean = 50, xbar_ucl = 51.2843.



xbar_ucl (X-bar chart UCL) over grand_mean (grand mean of the subgroup means) from 25 to 75 and mean_sd (average subgroup standard deviation (s-bar)) from 0.45 to 1.35: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 25.6422 to dark 76.9265); iso-lines at xbar_ucl = 30, 40, 50, 60, 70. Other inputs as in the worked example: subgroup_size = 5. Marked: the worked example, grand_mean = 50, mean_sd = 0.9, xbar_ucl = 51.2843.

```
industrial_engineering_ai999x.solve('ie_xbar_s_limits', grand_mean=50, mean_sd=0.9,
subgroup_size=5)
# -> xbar_ucl = 51.2843, xbar_lcl = 48.7157, center = 50, s_ucl = 1.8801, s_lcl = 0, A3 =
1.427, B3 = 0, B4 = 2.089   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_xbar_s_limits', grand_mean=74.0, mean_sd=0.0094,
subgroup_size=5)
# -> xbar_ucl = 74.0134, xbar_lcl = 73.9866, center = 74, s_ucl = 0.0196366, s_lcl = 0, A3
= 1.427, B3 = 0, B4 = 2.089   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
X-bar and S chart limits with x double bar = 50, s bar = 0.9, sample size = 5.	SOLVED	xbar_ucl = 51.2843, xbar_lcl = 48.7157, center = 50, s_ucl = 1.8801, s_lcl = 0, A3 = 1.427, B3 = 0, B4 = 2.089
X-bar and S chart limits with x double bar = 74, s bar = 0.0094, sample size = 5.	SOLVED	xbar_ucl = 74.0134, xbar_lcl = 73.9866, center = 74, s_ucl = 0.0196366, s_lcl = 0, A3 = 1.427, B3 = 0, B4 = 2.089
X-bar and S chart limits with x double bar = 50, s bar = 0.9.	NEEDS_INPUT	ie_xbar_s_limits needs subgroup_size (observations per subgroup (2 to 10)). Example: solve('ie_xbar_s_limits', grand_mea
X-bar and S chart limits with x double bar = 50, s bar = 0.9, sample size = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
X-bar and S chart limits with x double bar = 50, s bar = -1, sample size = 5.	INVALID_INPUT	X-bar and S chart control limits: mean_sd must be positive.

Process standard deviation from the average range, `ie_sigma_from_range`

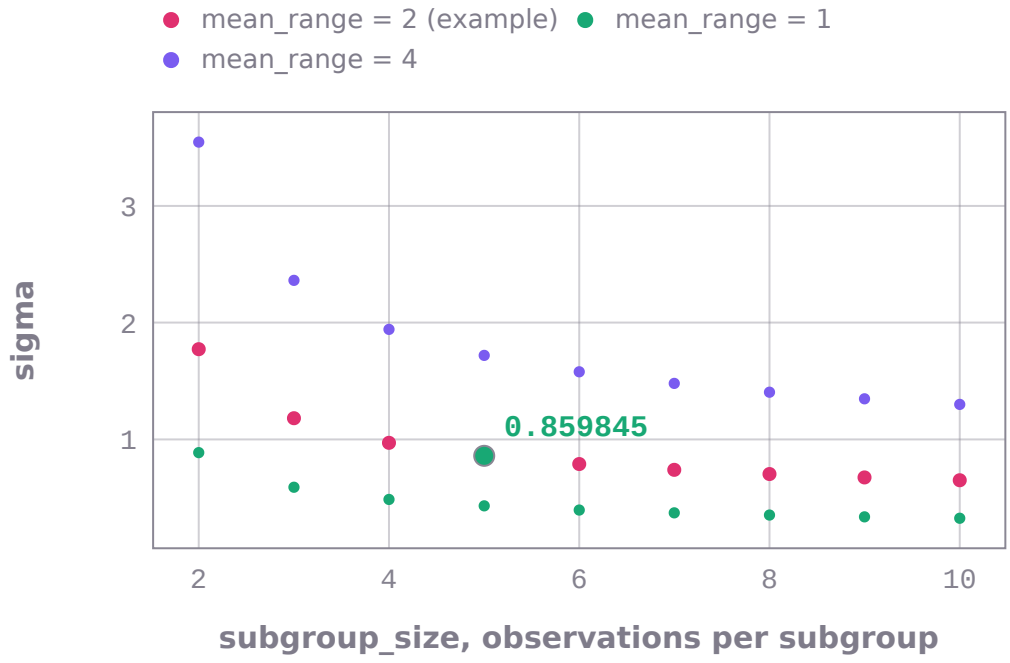
Estimate of the process standard deviation sigma = R-bar / d2 and the natural tolerance 6 sigma.

$$\hat{\sigma} = \frac{\bar{R}}{d_2}$$

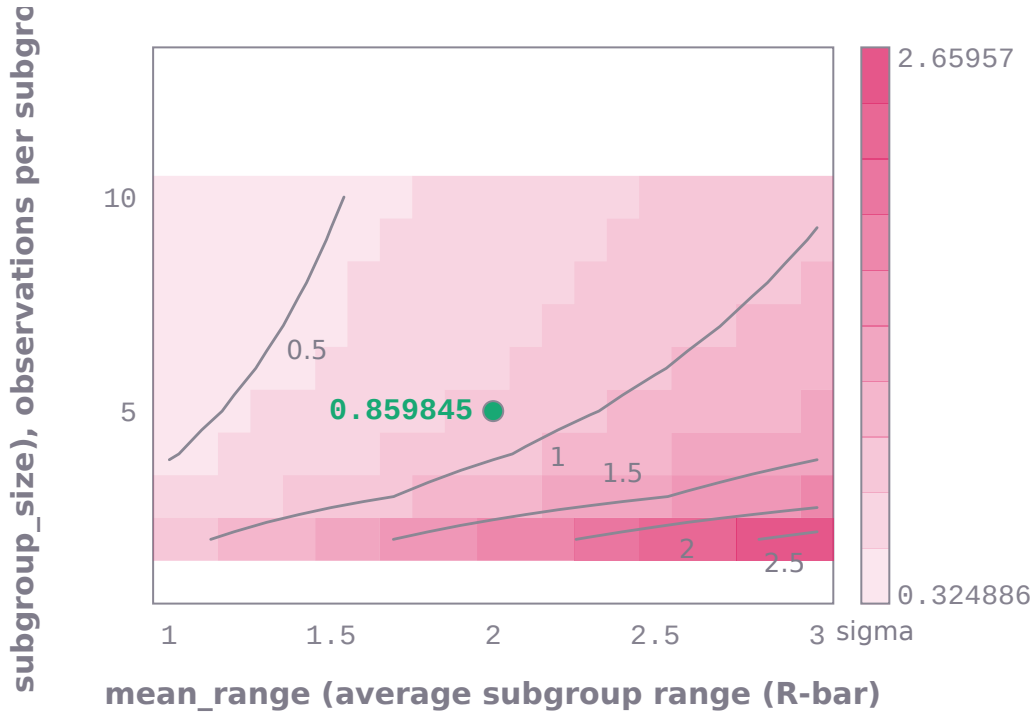
input	meaning	condition
`mean_range`	average subgroup range (R-bar)	
`subgroup_size`	observations per subgroup (2 to 10)	

output	meaning
`sigma`	estimated process standard deviation R-bar/d2
`d2`	d2 constant
`natural_tolerance`	natural tolerance 6 sigma

- **Method:** sigma = R-bar / d2
- **Assumes:** mean_range > 0; subgroup size 2 to 10
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sigma (estimated process standard deviation R-bar/d2) against subgroup_size (observations per subgroup (2 to 10)) from 1 to 13. One curve for each value of mean_range (average subgroup range (R-bar)): 2, 1, 4. 9 points per curve computed by industrial-engineering-ai999x; marked: the worked example, subgroup_size = 5, sigma = 0.859845.



sigma (estimated process standard deviation R-bar/d2) over mean_range (average subgroup range (R-bar)) from 1 to 3 and subgroup_size (observations per subgroup (2 to 10)) from 1 to 13: each of the 21 × 13 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.324886 to dark 2.65957); iso-lines at sigma = 0.5, 1, 1.5, 2, 2.5; blank cells: inputs the capability refuses. Marked: the worked example, mean_range = 2, subgroup_size = 5, sigma = 0.859845.

```
industrial_engineering_ai999x.solve('ie_sigma_from_range', mean_range=2, subgroup_size=5)
# -> sigma = 0.859845, d2 = 2.326, natural_tolerance = 5.15907   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_sigma_from_range', mean_range=0.35,
subgroup_size=4)
# -> sigma = 0.169985, d2 = 2.059, natural_tolerance = 1.01991   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Process sigma from the average range with r bar = 2, sample size = 5.	SOLVED	sigma = 0.859845, d2 = 2.326, natural_tolerance = 5.15907
Process sigma from the average range with r bar = 0.35, sample size = 4.	SOLVED	sigma = 0.169985, d2 = 2.059, natural_tolerance = 1.01991
Process sigma from the average range with r bar = 2.	NEEDS_INPUT	ie_sigma_from_range needs subgroup_size (observations per subgroup (2 to 10)). Example: solve('ie_sigma_from_range', mea
Process sigma from the average range with r bar = 2, sample size = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Process sigma from the average range with r bar = 2, sample size = 1.	INVALID_INPUT	Process standard deviation from the average range: subgroup_size must be at least 2.

p chart control limits, `ie_p_chart_limits`

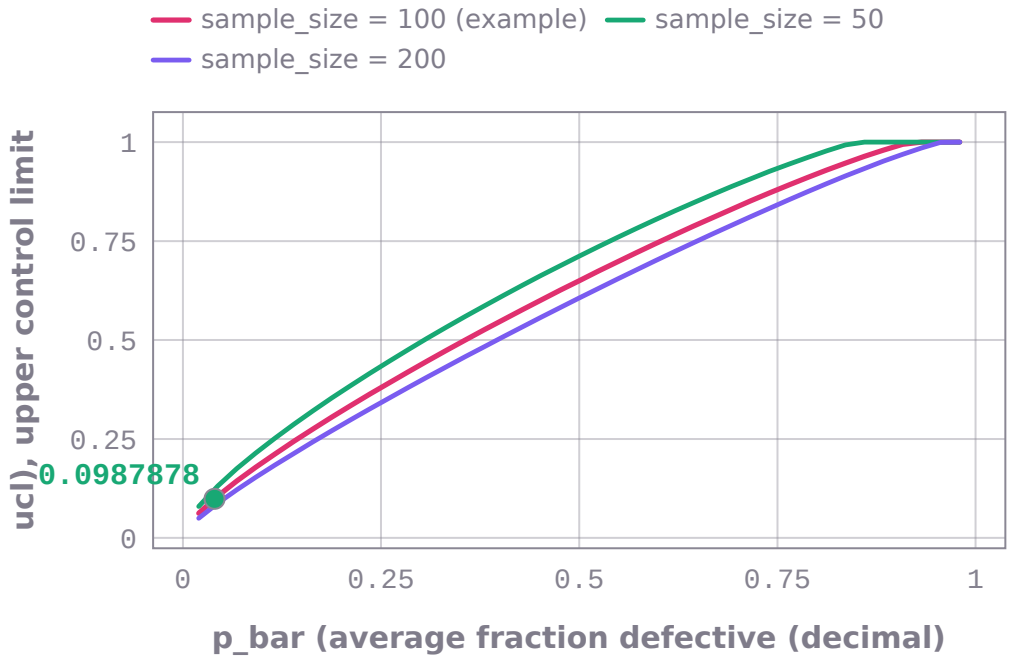
$3-\sigma$ limits of the fraction-defective (p) chart: p-bar +/- 3 sqrt(p-bar (1 - p-bar) / n).

$$UCL, LCL = \bar{p} \pm 3\sqrt{\frac{\bar{p}(1-\bar{p})}{n}}$$

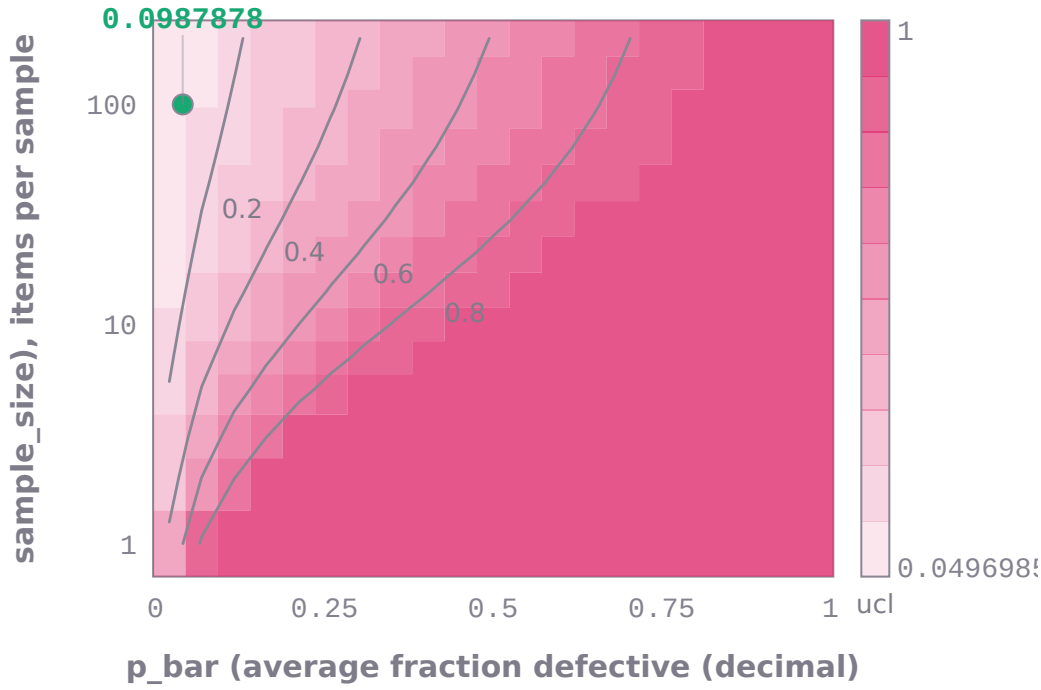
input	meaning	condition
`p_bar`	average fraction defective (decimal)	
`sample_size`	items per sample	

output	meaning
`ucl`	upper control limit
`lcl`	lower control limit (not below 0)
`center`	centre line
`sigma_p`	standard deviation of the fraction defective

- **Method:** $sigma_p = sqrt$(p (1 - p)/n); limits p +/- 3 sigma_p, clipped to [0, 1]
- **Assumes:** 0 < p_bar < 1; sample_size > 0 (whole number of items)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ucl (upper control limit) against p_bar (average fraction defective (decimal)) from 0.02 to 0.98. One curve for each value of sample_size (items per sample): 100, 50, 200. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, p_bar = 0.04, ucl = 0.0987878.



ucl (upper control limit) over p_bar (average fraction defective (decimal)) from 0.02 to 0.98 and sample_size (items per sample) from 1 to 200 (logarithmic vertical axis): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.0496985 to dark 1); iso-lines at ucl = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, p_bar = 0.04, sample_size = 100, ucl = 0.0987878.

```
industrial_engineering_ai999x.solve('ie_p_chart_limits', p_bar=0.04, sample_size=100)
# -> ucl = 0.0987878, lcl = 0, center = 0.04, sigma_p = 0.0195959   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_p_chart_limits', p_bar=0.15, sample_size=50)
# -> ucl = 0.301493, lcl = 0, center = 0.15, sigma_p = 0.0504975   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
P chart limits with average fraction defective = 0.04, sample size = 100.	SOLVED	ucl = 0.0987878, lcl = 0, center = 0.04, sigma_p = 0.0195959
P chart limits with average fraction defective = 0.15, sample size = 50.	SOLVED	ucl = 0.301493, lcl = 0, center = 0.15, sigma_p = 0.0504975
P chart limits with average fraction defective = 0.04.	NEEDS_INPUT	ie_p_chart_limits needs sample_size (items per sample). Example: solve('ie_p_chart_limits', p_bar=0.04, sample_size=100)
P chart limits with average fraction defective = 0.04, sample size = 100, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
P chart limits with average fraction defective = 1.5, sample size = 100.	INVALID_INPUT	p chart control limits: p_bar must lie strictly between 0 and 1 (write 95% or 0.95).

np chart control limits, `ie_np_chart_limits`

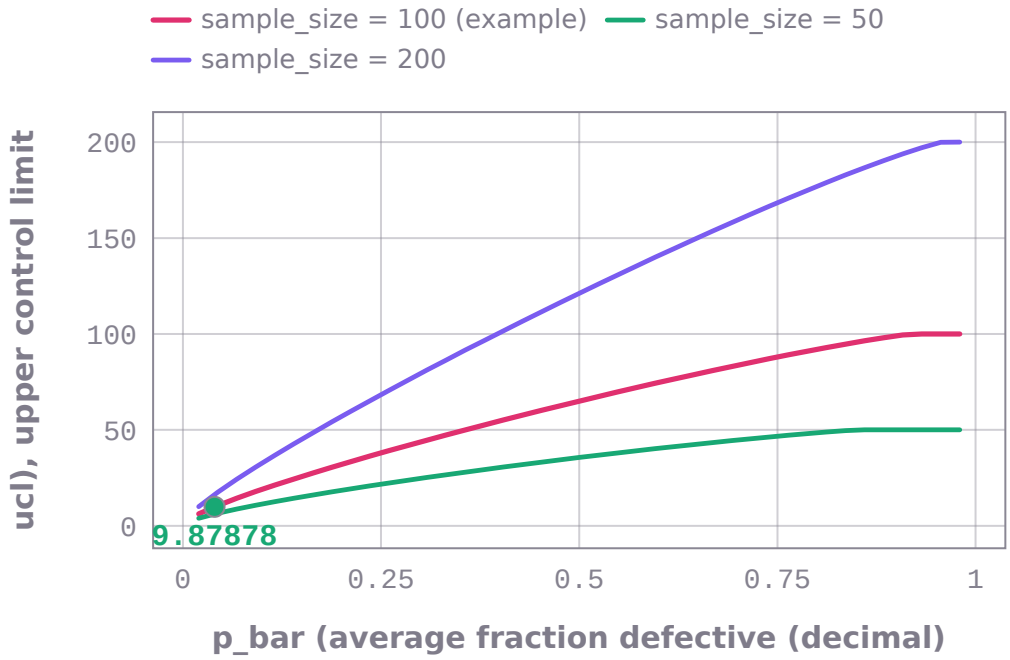
$3-\sigma$ limits of the number-defective (np) chart: n p +/- 3 sqrt(n p $(1-p)$).

$$n\bar{p} \pm 3\sqrt{n\bar{p}(1-\bar{p})}$$

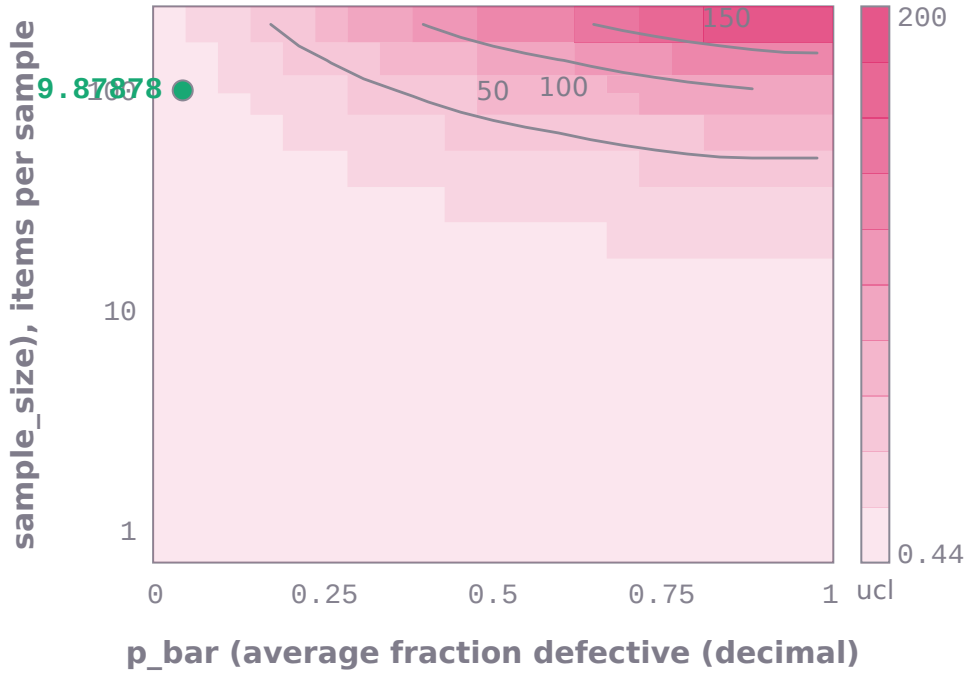
input	meaning	condition
`p_bar`	average fraction defective (decimal)	
`sample_size`	items per sample	

output	meaning
`ucl`	upper control limit
`lcl`	lower control limit (not below 0)
`center`	centre line n p
`sigma_np`	standard deviation of the number defective

- **Method:** limits n p +/- 3 sqrt(n p $(1-p)$)
- **Assumes:** 0 < p_bar < 1; constant sample size
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ucl (upper control limit) against p_bar (average fraction defective (decimal)) from 0.02 to 0.98. One curve for each value of sample_size (items per sample): 100, 50, 200. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, p_bar = 0.04, ucl = 9.87878.



ucl (upper control limit) over p_bar (average fraction defective (decimal)) from 0.02 to 0.98 and sample_size (items per sample) from 1 to 200 (logarithmic vertical axis): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.44 to dark 200); iso-lines at ucl = 50, 100, 150. Marked: the worked example, p_bar = 0.04, sample_size = 100, ucl = 9.87878.

```
industrial_engineering_ai999x.solve('ie_np_chart_limits', p_bar=0.04, sample_size=100)
# -> ucl = 9.87878, lcl = 0, center = 4, sigma_np = 1.95959   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_np_chart_limits', p_bar=0.1, sample_size=200)
# -> ucl = 32.7279, lcl = 7.27208, center = 20, sigma_np = 4.24264   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Np chart limits with average fraction defective = 0.04, sample size = 100.	SOLVED	ucl = 9.87878, lcl = 0, center = 4, sigma_np = 1.95959
Np chart limits with average fraction defective = 0.1, sample size = 200.	SOLVED	ucl = 32.7279, lcl = 7.27208, center = 20, sigma_np = 4.24264
Np chart limits with average fraction defective = 0.04.	NEEDS_INPUT	ie_np_chart_limits needs sample_size (items per sample). Example: solve('ie_np_chart_limits', p_bar=0.04, sample_size=10
Np chart limits with average fraction defective = 0.04, sample size = 100, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Np chart limits with average fraction defective = 0.04, sample size = 0.	INVALID_INPUT	np chart control limits: sample_size must be at least 1.

c chart control limits, `ie_c_chart_limits`

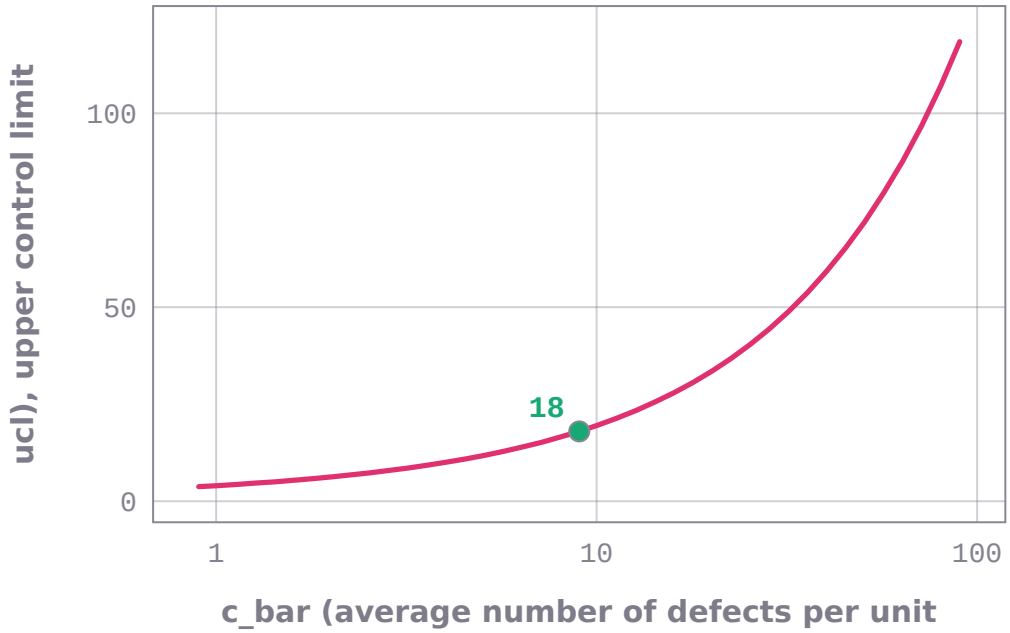
$3-\sigma$ limits of the count-of-defects (c) chart: c-bar +/- 3 sqrt(c-bar).

$$\bar{c} \pm 3\sqrt{\bar{c}}$$

input	meaning	condition
`c_bar`	average number of defects per unit (inspection unit of constant size)	

output	meaning
`ucl`	upper control limit
`lcl`	lower control limit (not below 0)
`center`	centre line
`sigma_c`	sqrt(c-bar)

- **Method:** limits c +/- 3 sqrt(c)
- **Assumes:** c_bar > 0; defects Poisson, constant inspection unit
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ucl (upper control limit) against c_bar (average number of defects per unit (inspection unit of constant size)) from 0.9 to 90 on a logarithmic axis. 41 points computed by industrial-engineering-ai999x; marked: the worked example, c_bar = 9, ucl = 18.

```
industrial_engineering_ai999x.solve('ie_c_chart_limits', c_bar=9)
# -> ucl = 18, lcl = 0, center = 9, sigma_c = 3   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_c_chart_limits', c_bar=2.5)
# -> ucl = 7.24342, lcl = 0, center = 2.5, sigma_c = 1.58114   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
C chart limits with average number of defects = 9.	SOLVED	ucl = 18, lcl = 0, center = 9, sigma_c = 3
C chart limits with average number of defects = 2.5.	SOLVED	ucl = 7.24342, lcl = 0, center = 2.5, sigma_c = 1.58114
C chart limits with average number of defects = 9, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
C chart limits with average number of defects = -3.	INVALID_INPUT	c chart control limits: c_bar must be positive.

u chart control limits, `ie_u_chart_limits`

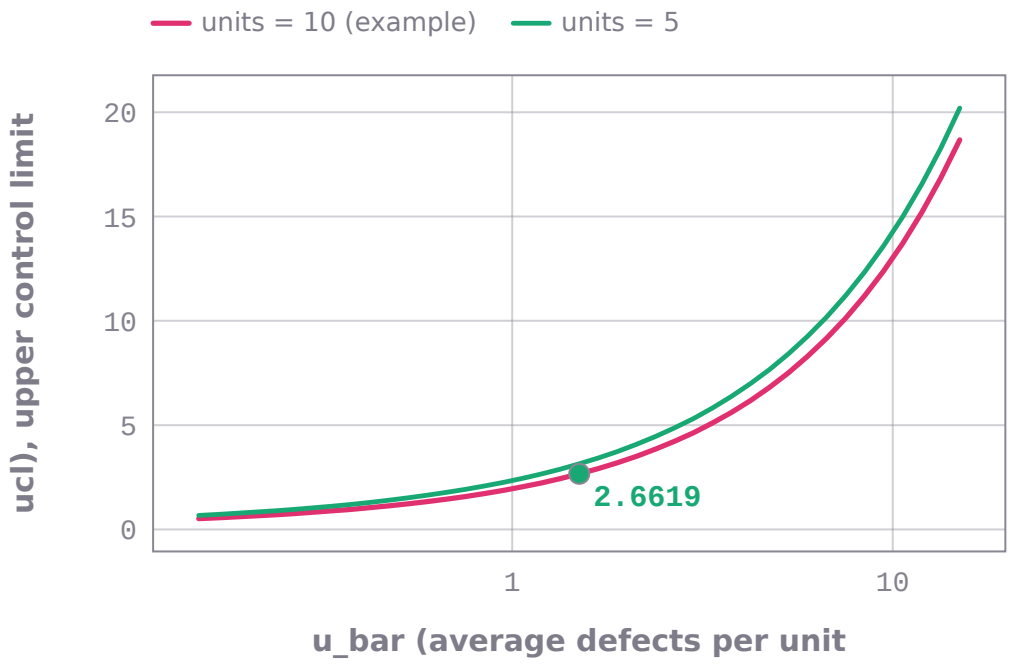
$3-\sigma$ limits of the defects-per-unit (u) chart for samples of n units: u-bar +/- 3 sqrt(u-bar / n).

$$\bar{u} \pm 3\sqrt{\bar{u}/n}$$

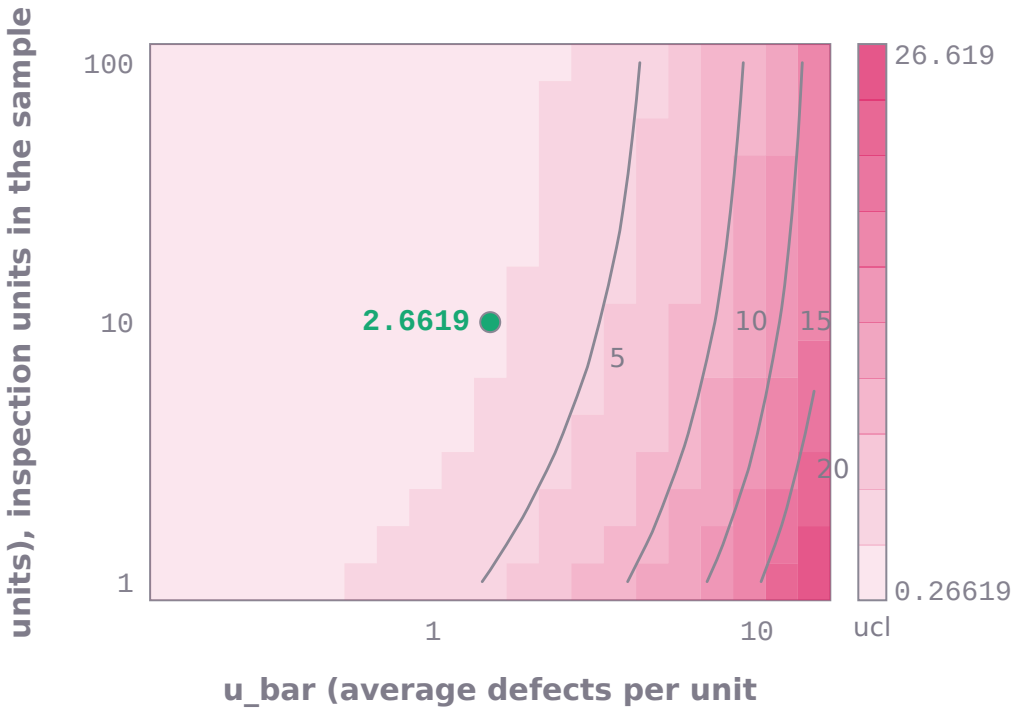
input	meaning	condition
`u_bar`	average defects per unit	
`units`	inspection units in the sample	

output	meaning
`ucl`	upper control limit
`lcl`	lower control limit (not below 0)
`center`	centre line
`sigma_u`	sqrt(u-bar / n)

- **Method:** limits u +/- 3 sqrt(u/n)
- **Assumes:** u_bar > 0; units > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ucl (upper control limit) against u_bar (average defects per unit) from 0.15 to 15 on a logarithmic axis. One curve for each value of units (inspection units in the sample): 10, 5. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, u_bar = 1.5, ucl = 2.6619.



ucl (upper control limit) over u_bar (average defects per unit) from 0.15 to 15 and units (inspection units in the sample) from 1 to 100 (logarithmic axes): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.26619 to dark 26.619); iso-lines at ucl = 5, 10, 15, 20. Marked: the worked example, u_bar = 1.5, units = 10, ucl = 2.6619.

```
industrial_engineering_ai999x.solve('ie_u_chart_limits', u_bar=1.5, units=10)
# -> ucl = 2.6619, lcl = 0.338105, center = 1.5, sigma_u = 0.387298   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_u_chart_limits', u_bar=0.32, units=25)
# -> ucl = 0.659411, lcl = 0, center = 0.32, sigma_u = 0.113137   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
U chart limits with average defects per unit = 1.5, sample size = 10.	SOLVED	ucl = 2.6619, lcl = 0.338105, center = 1.5, sigma_u = 0.387298
U chart limits with average defects per unit = 0.32, sample size = 25.	SOLVED	ucl = 0.659411, lcl = 0, center = 0.32, sigma_u = 0.113137
U chart limits with average defects per unit = 1.5.	NEEDS_INPUT	ie_u_chart_limits needs units (inspection units in the sample). Example: solve('ie_u_chart_limits', u_bar=1.5, units=10)
U chart limits with average defects per unit = 1.5, sample size = 10, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
U chart limits with average defects per unit = 1.5, sample size = 0.	INVALID_INPUT	u chart control limits: units must be positive.

p chart from sample counts, `ie_p_chart_from_samples`

Centre line and $3-\sigma$ limits of a p chart computed from the defectives found in each sample, and the samples that fall outside them.

$$\bar{p} = \frac{\sum d_i}{m\,n}, \quad \bar{p} \pm 3\sqrt{\bar{p}(1-\bar{p})/n}$$

input	meaning	condition
`defectives`	number of defective items in each sample	
`sample_size`	items in every sample	

output	meaning
`p_bar`	average fraction defective
`ucl`	upper control limit
`lcl`	lower control limit
`out_of_control`	sample numbers outside the limits
`count_out`	how many samples are outside

- **Method:** p-bar = total defectives / total inspected; limits p-bar +/- 3 sqrt(p-bar (1 - p-bar)/n); points beyond the limits flagged
- **Assumes:** counts are whole numbers <= sample size; 0 < p-bar < 1; constant sample size
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



p_bar (average fraction defective) against sample_size (items in every sample) from 1 to 200 on a logarithmic axis. Other inputs as in the worked example: defectives = [3, 5, 2, 4, 12, 3, 4, 2, 5, 3]. 23 points computed by industrial-engineering-ai999x; marked: the worked example, sample_size = 100, p_bar = 0.043.

```
industrial_engineering_ai999x.solve('ie_p_chart_from_samples', defectives=[3, 5, 2, 4, 12, 3, 4, 2, 5, 3], sample_size=100)
# -> p_bar = 0.043, ucl = 0.103857, lcl = 0, out_of_control = [5], count_out = 1   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_p_chart_from_samples', defectives=[8, 6, 9, 7, 5, 10, 6, 8], sample_size=150)
# -> p_bar = 0.0491667, ucl = 0.102129, lcl = 0, out_of_control = [], count_out = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
P chart from samples with defective counts = [3, 5, 2, 4, 12, 3, 4, 2, 5, 3], sample size = 100.	SOLVED	p_bar = 0.043, ucl = 0.103857, lcl = 0, out_of_control = [5], count_out = 1
P chart from samples with defective counts = [8, 6, 9, 7, 5, 10, 6, 8], sample size = 150.	SOLVED	p_bar = 0.0491667, ucl = 0.102129, lcl = 0, out_of_control = [], count_out = 0
P chart from samples with defective counts = [3, 5, 2, 4, 12, 3, 4, 2, 5, 3].	NEEDS_INPUT	ie_p_chart_from_samples needs sample_size (items in every sample). Example: solve('ie_p_chart_from_samples', defectives=
P chart from samples with defective counts = [3, 5, 2, 4, 12, 3, 4, 2, 5, 3], sample size = 100, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
P chart from samples with defective counts = [3, 5, 2, 400], sample size = 100.	INVALID_INPUT	p chart from sample counts: each count of defectives must be a whole number no larger than the sample size.

Process capability Cp and Cpk, `ie_process_capability`

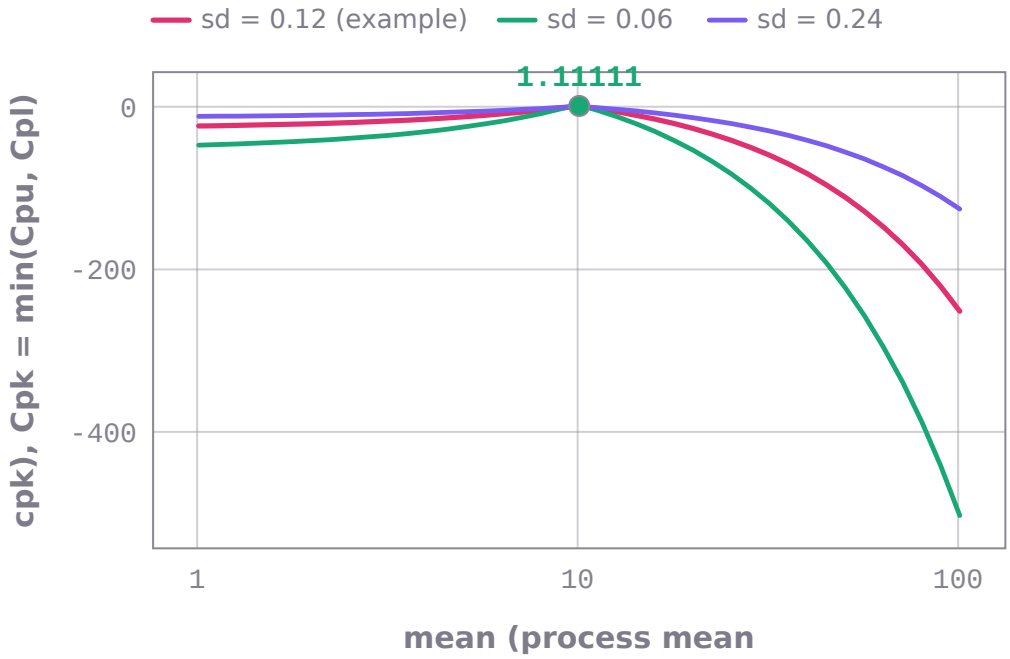
Capability indices of a process with known mean and standard deviation against two specification limits.

$$C_p = \frac{USL - LSL}{6\sigma}, \quad C_{pk} = \min\left(\frac{USL - \mu}{3\sigma}, \frac{\mu - LSL}{3\sigma}\right)$$

input	meaning	condition
`usl`	upper specification limit	
`lsl`	lower specification limit	
`mean`	process mean	
`sd`	process standard deviation	

output	meaning
`cpk`	Cpk = min(Cpu, Cpl)
`cp`	Cp = (USL - LSL)/(6 sigma)
`cpu`	(USL - mean)/(3 sigma)
`cpl`	(mean - LSL)/(3 sigma)

- **Method:** Cp = (USL - LSL)/6 sigma, Cpk = min(USL - mu, mu - LSL)/3 sigma
- **Assumes:** usl > lsl; $sd > 0$; process normal and in control
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cpk (Cpk = min(Cpu, Cpl)) against mean (process mean) from 1.01 to 101 on a logarithmic axis. One curve for each value of sd (process standard deviation): 0.12, 0.06, 0.24. Other inputs as in the worked example: usl = 10.5, lsl = 9.5. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, mean = 10.1, cpk = 1.11111.

```
industrial_engineering_ai999x.solve('ie_process_capability', usl=10.5, lsl=9.5, mean=10.1,
sd=0.12)
# -> cpk = 1.11111, cp = 1.38889, cpu = 1.11111, cpl = 1.66667   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_process_capability', usl=230, lsl=170, mean=205,
sd=8)
# -> cpk = 1.04167, cp = 1.25, cpu = 1.04167, cpl = 1.45833   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Process capability with upper spec = 10.5, lower spec = 9.5, process mean = 10.1, standard deviation = 0.12.	SOLVED	cpk = 1.11111, cp = 1.38889, cpu = 1.11111, cpl = 1.66667
Process capability with upper spec = 230, lower spec = 170, process mean = 205, standard deviation = 8.	SOLVED	cpk = 1.04167, cp = 1.25, cpu = 1.04167, cpl = 1.45833
Process capability with upper spec = 10.5, lower spec = 9.5, process mean = 10.1.	NEEDS_INPUT	ie_process_capability needs sd (process standard deviation). Example: solve('ie_process_capability', usl=10.5, lsl=9.5,
Process capability with upper spec = 10.5, lower spec = 9.5, process mean = 10.1, standard deviation = 0.12, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Process capability with upper spec = 9.5, lower spec = 10.5, process mean = 10.1, standard deviation = 0.12.	INVALID_INPUT	Process capability Cp and Cpk: the upper specification limit must exceed the lower one.

Process performance Pp and Ppk from data, `ie_process_performance`

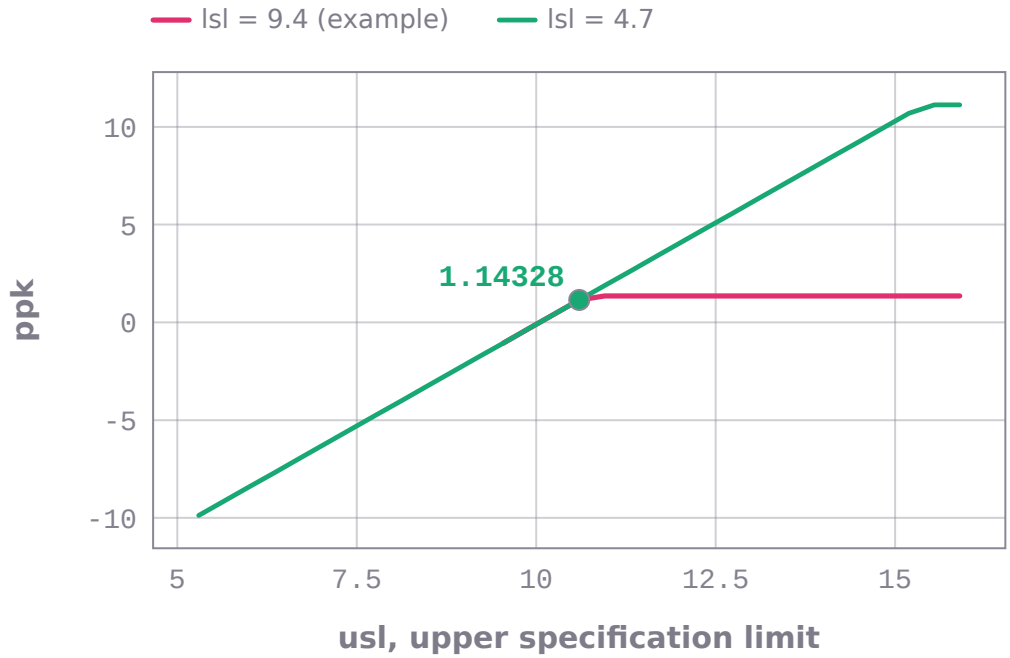
Performance indices Pp and Ppk from measured data, using the overall sample standard deviation $(n-1)$.

$$P_p = \frac{USL - LSL}{6s}, \quad P_{pk} = \min\left(\frac{USL - \bar{x}}{3s}, \frac{\bar{x} - LSL}{3s}\right)$$

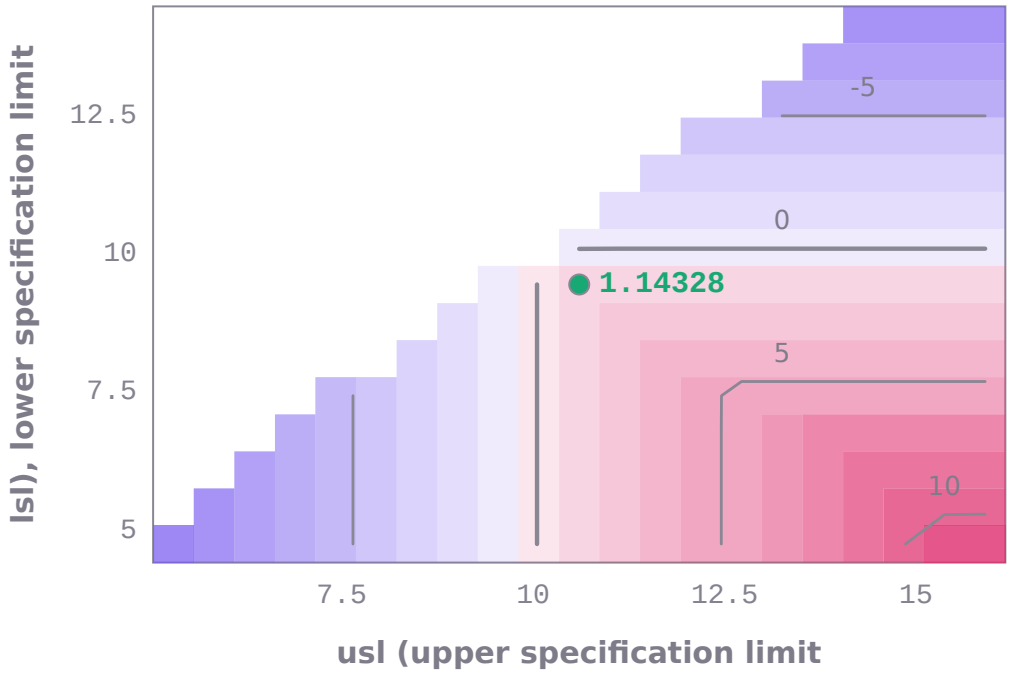
input	meaning	condition
`data`	measurements	
`usl`	upper specification limit	
`lsl`	lower specification limit	

output	meaning
`ppk`	Ppk
`pp`	Pp
`mean`	sample mean
`sd`	overall sample standard deviation

- **Method:** $s = sqrt$(sum (x - mean)^2/(n - 1)); Pp = (USL - LSL)/6s, Ppk = min(USL - mean, mean - LSL)/3s
- **Assumes:** at least 2 measurements that vary; usl > lsl
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ppk (Ppk) against usl (upper specification limit) from 5.3 to 15.9. One curve for each value of lsl (lower specification limit): 9.4, 4.7. Other inputs as in the worked example: data = [10.1, 9.9, 10.2, 10.0, 10.3, 9.8, 10.1, 10.0]. 19 points per curve computed by industrial-engineering-ai999x; marked: the worked example, usl = 10.6, ppk = 1.14328.



ppk (Ppk) over usl (upper specification limit) from 5.3 to 15.9 and lsl (lower specification limit) from 4.7 to 14.1: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its sign and magnitude (second colour negative); iso-lines at ppk = -5, 0, 5, 10; blank cells: inputs the capability refuses. Other inputs as in the worked example: data = [10.1, 9.9, 10.2, 10.0, 10.3, 9.8, 10.1, 10.0]. Marked: the worked example, usl = 10.6, lsl = 9.4, ppk = 1.14328.

```
industrial_engineering_ai999x.solve('ie_process_performance', data=[10.1, 9.9, 10.2, 10.0,
10.3, 9.8, 10.1, 10.0], usl=10.6, lsl=9.4)
# -> ppk = 1.14328, pp = 1.24722, mean = 10.05, sd = 0.160357   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_process_performance', data=[52, 48, 50, 51, 49, 53, 47, 50, 50, 52], usl=58, lsl=44)
# -> ppk = 1.10293, pp = 1.24524, mean = 50.2, sd = 1.8738   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Process performance with measurements = [10.1, 9.9, 10.2, 10, 10.3, 9.8, 10.1, 10], upper spec = 10.6, lower spec = 9.4.	SOLVED	ppk = 1.14328, pp = 1.24722, mean = 10.05, sd = 0.160357
Process performance with measurements = [52, 48, 50, 51, 49, 53, 47, 50, 50, 52], upper spec = 58, lower spec = 44.	SOLVED	ppk = 1.10293, pp = 1.24524, mean = 50.2, sd = 1.8738
Process performance with measurements = [10.1, 9.9, 10.2, 10, 10.3, 9.8, 10.1, 10], upper spec = 10.6.	NEEDS_INPUT	ie_process_performance needs lsl (lower specification limit). Example: solve('ie_process_performance', data=[10.1, 9.9,
Process performance with measurements = [10.1, 9.9, 10.2, 10, 10.3, 9.8, 10.1, 10], upper spec = 10.6, lower spec = 9.4, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Process performance with measurements = [10, 10, 10], upper spec = 10.6, lower spec = 9.4.	INVALID_INPUT	Process performance Pp and Ppk from data: the data must vary.

Taguchi capability index Cpm, `ie_cpm_taguchi`

Capability index that penalises distance from the target: Cpm = (USL - LSL) / (6 sqrt(sigma^2 + (mu - T)^2)).

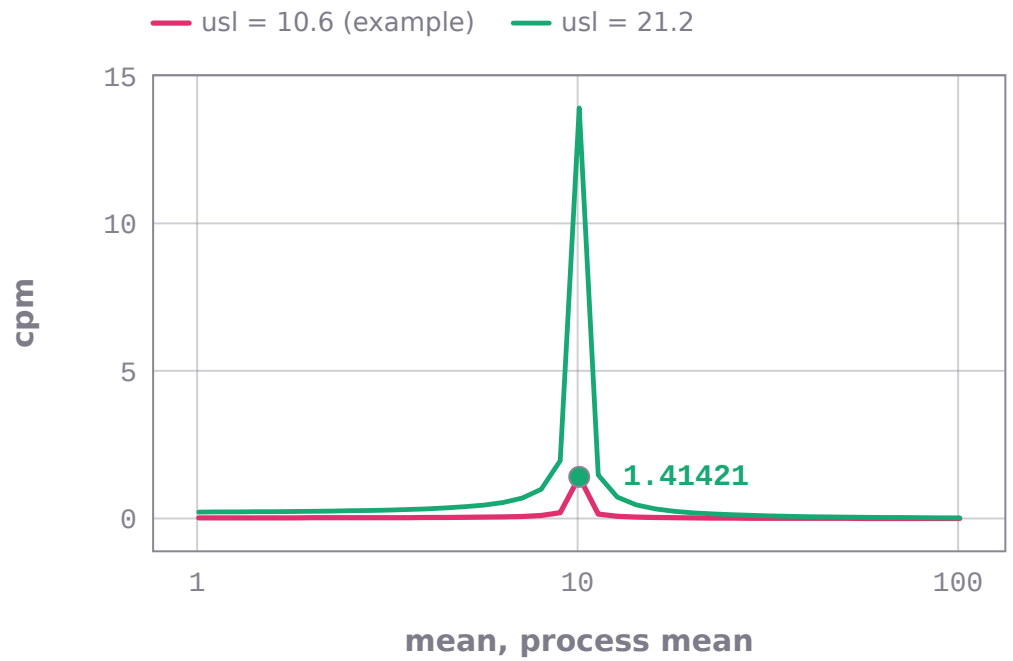
$$C_{pm} = \frac{USL - LSL}{6\sqrt{\sigma^2 + (\mu - T)^2}}$$

input	meaning	condition
`usl`	upper specification limit	
`lsl`	lower specification limit	
`mean`	process mean	
`sd`	process standard deviation	
`target`	target value T	

output	meaning
`cpm`	Cpm
`cp`	Cp
`tau`	root mean square deviation from target

- **Method:** $\tau = \sqrt{\sigma^2 + (\mu - T)^2}$, Cpm = (USL - LSL)/(6 tau)
- **Assumes:** usl > lsl; lsl < target < usl; $sd > 0$

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cpm (Cpm) against mean (process mean) from 1.01 to 101 on a logarithmic axis. One curve for each value of usl (upper specification limit): 10.6, 21.2. Other inputs as in the worked example: lsl = 9.4, sd = 0.1, target = 10.0. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, mean = 10.1, cpm = 1.41421.

```
industrial_engineering_ai999x.solve('ie_cpm_taguchi', usl=10.6, lsl=9.4, mean=10.1, sd=0.1,
target=10.0)
# -> cpm = 1.41421, cp = 2, tau = 0.141421   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_cpm_taguchi', usl=60, lsl=40, mean=52, sd=2.5,
target=50)
# -> cpm = 1.04116, cp = 1.33333, tau = 3.20156   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Taguchi capability index Cpm with upper spec = 10.6, lower spec = 9.4, process mean = 10.1, standard deviation = 0.1, nominal = 10.	SOLVED	cpm = 1.41421, cp = 2, tau = 0.141421
Taguchi capability index Cpm with upper spec = 60, lower spec = 40, process mean = 52, standard deviation = 2.5, nominal = 50.	SOLVED	cpm = 1.04116, cp = 1.33333, tau = 3.20156
Taguchi capability index Cpm with upper spec = 10.6, lower spec = 9.4, process mean = 10.1, standard deviation = 0.1.	NEEDS_INPUT	ie_cpm_taguchi needs target (target value T). Example: solve('ie_cpm_taguchi', usl=10.6, lsl=9.4, mean=10.1, sd=0.1, tar
Taguchi capability index Cpm with upper spec = 10.6, lower spec = 9.4, process mean = 10.1, standard	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
deviation = 0.1, nominal = 10, colour = 3.		
Taguchi capability index Cpm with upper spec = 10.6, lower spec = 9.4, process mean = 10.1, standard deviation = 0.1, nominal = 11.	INVALID_INPUT	Taguchi capability index Cpm: the target must lie between the specification limits.

Fraction nonconforming (normal process), `ie_fraction_nonconforming`

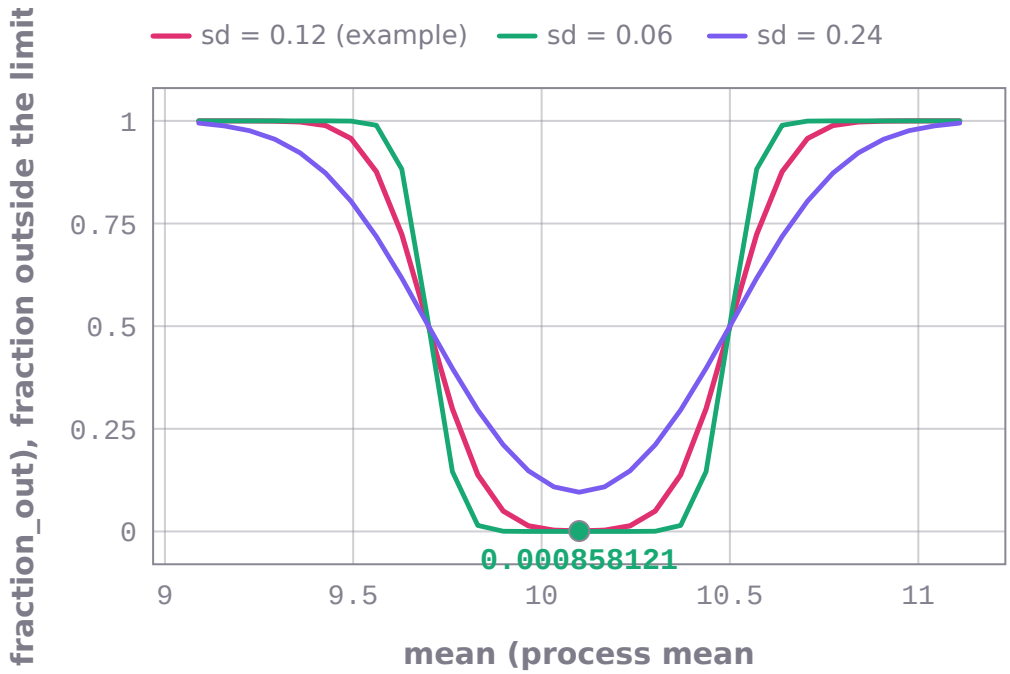
Fraction (and parts per million) of output outside the specification limits for a normal process.

$$p = \Phi\left(\frac{LSL - \mu}{\sigma}\right) + 1 - \Phi\left(\frac{USL - \mu}{\sigma}\right)$$

input	meaning	condition
`mean`	process mean	
`sd`	process standard deviation	
`lsl`	lower specification limit (optional)	
`usl`	upper specification limit (optional)	

output	meaning
`fraction_out`	fraction outside the limits
`below_lsl`	fraction below LSL
`above_usl`	fraction above USL
`ppm`	parts per million out of specification
`yield`	fraction within specification

- **Method:** P(X < LSL) + P(X > USL) for X ~ N(mu, sigma^2)
- **Assumes:** $sd > 0$; at least one limit; usl > lsl; process normal
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



fraction_out (fraction outside the limits) against mean (process mean) from 9.09 to 11.11. One curve for each value of sd (process standard deviation): 0.12, 0.06, 0.24. Other inputs as in the worked example: lsl = 9.7, usl = 10.5. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, mean = 10.1, fraction_out = 0.000858121.

```
industrial_engineering_ai999x.solve('ie_fraction_nonconforming', mean=10.1, sd=0.12,
lsl=9.7, usl=10.5)
# -> fraction_out = 0.000858121, below_lsl = 0.00042906, above_usl = 0.00042906, ppm =
858.121, yield = 0.999142   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_fraction_nonconforming', mean=205, sd=8, usl=230)
# -> fraction_out = 0.000889025, below_lsl = 0, above_usl = 0.000889025, ppm = 889.025,
yield = 0.999111   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fraction nonconforming with process mean = 10.1, standard deviation = 0.12, lower spec = 9.7, upper spec = 10.5.	SOLVED	fraction_out = 0.000858121, below_lsl = 0.00042906, above_usl = 0.00042906, ppm = 858.121, yield = 0.999142
Fraction nonconforming with process mean = 205, standard deviation = 8, upper spec = 230.	SOLVED	fraction_out = 0.000889025, below_lsl = 0, above_usl = 0.000889025, ppm = 889.025, yield = 0.999111
Fraction nonconforming with process mean = 10.1, lower spec = 9.7, upper spec = 10.5.	NEEDS_INPUT	ie_fraction_nonconforming needs sd (process standard deviation). Example: solve('ie_fraction_nonconforming', mean=10.1,
Fraction nonconforming with process mean = 10.1, standard deviation = 0.12, lower spec = 9.7, upper spec = 10.5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Fraction nonconforming with process mean = 205, standard deviation = 0, upper spec = 230.	INVALID_INPUT	Fraction nonconforming (normal process): sd must be positive.

Average run length of a Shewhart X-bar chart, `ie_arl_shewhart`

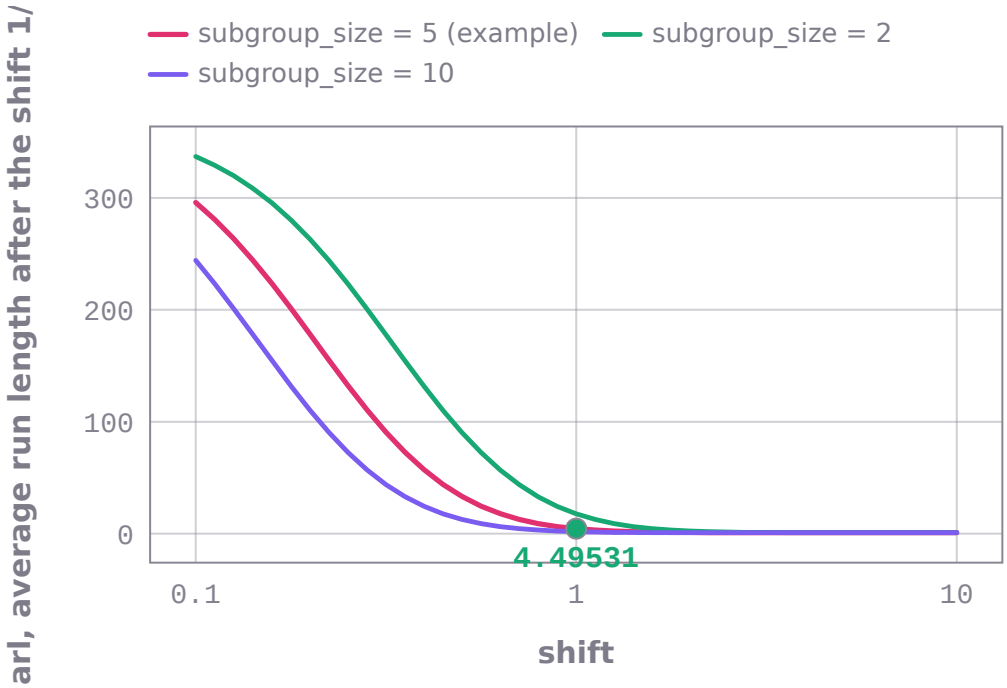
Probability that a Shewhart chart signals on a sample after the mean shifts by k sigma, and the average run length (ARL) until it does.

$$\beta = \Phi(L - k\sqrt{n}) - \Phi(-L - k\sqrt{n}), \quad ARL_1 = \frac{1}{1-\beta}$$

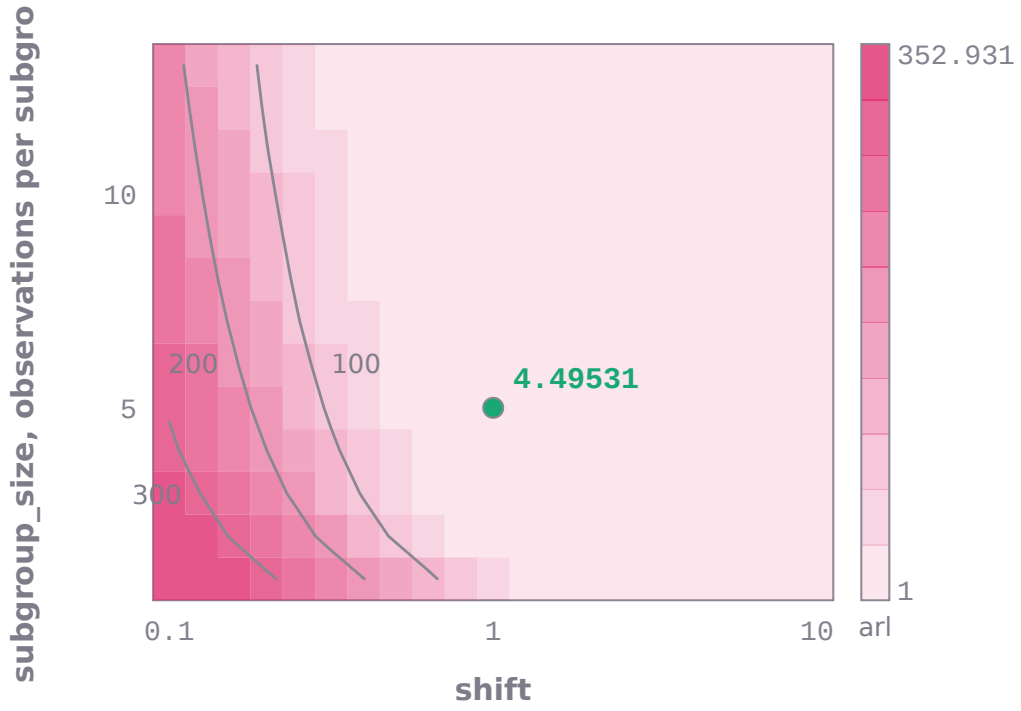
input	meaning	condition
`shift`	mean shift in units of the process standard deviation (k)	
`subgroup_size`	observations per subgroup (default 1)	
`limit_sigma`	control limits at +/- L sigma of the mean (default 3)	

output	meaning
`arl`	average run length after the shift $1/(1-\beta)$
`beta`	probability of no signal per sample
`detection_probability`	probability of a signal per sample
`arl_in_control`	in-control ARL $1/\alpha$

- **Method:** $\beta = Phi$(L - k sqrt(n)) - Phi(-L - k sqrt(n)); ARL = 1/(1 - beta)
- **Assumes:** shift >= 0; 1 <= subgroup_size <= 1000; 0 < limit_sigma <= 6
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



arl (average run length after the shift 1/(1 - beta)) against shift (mean shift in units of the process standard deviation (k)) from 0.1 to 10 on a logarithmic axis. One curve for each value of subgroup_size (observations per subgroup (default 1)): 5, 2, 10. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, shift = 1, arl = 4.49531.



arl (average run length after the shift 1/(1 - beta)) over shift (mean shift in units of the process standard deviation (k)) from 0.1 to 10 and subgroup_size (observations per subgroup (default 1)) from 1 to 13 (logarithmic horizontal axis): each of the 21 × 13 cells is one run of industrial-engineering-ai999x, shaded by its value (light 1 to dark 352.931); iso-lines at arl = 100, 200, 300. Marked: the worked example, shift = 1, subgroup_size = 5, arl = 4.49531.

```
industrial_engineering_ai999x.solve('ie_arl_shewhart', shift=1, subgroup_size=5)
# -> arl = 4.49531, beta = 0.777546, detection_probability = 0.222454, arl_in_control =
370.398   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_arl_shewhart', shift=1.5, subgroup_size=4,
limit_sigma=3)
# -> arl = 2, beta = 0.5, detection_probability = 0.5, arl_in_control = 370.398   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Average run length with mean shift = 1, sample size = 5.	SOLVED	arl = 4.49531, beta = 0.777546, detection_probability = 0.222454, arl_in_control = 370.398
Average run length with mean shift = 1.5, sample size = 4, limit width = 3.	SOLVED	arl = 2, beta = 0.5, detection_probability = 0.5, arl_in_control = 370.398
Average run length with mean shift = 1, sample size = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Average run length with mean shift = -1, sample size = 5.	INVALID_INPUT	Average run length of a Shewhart X-bar chart: shift must not be negative.

EWMA chart control limits, `ie_ewma_limits`

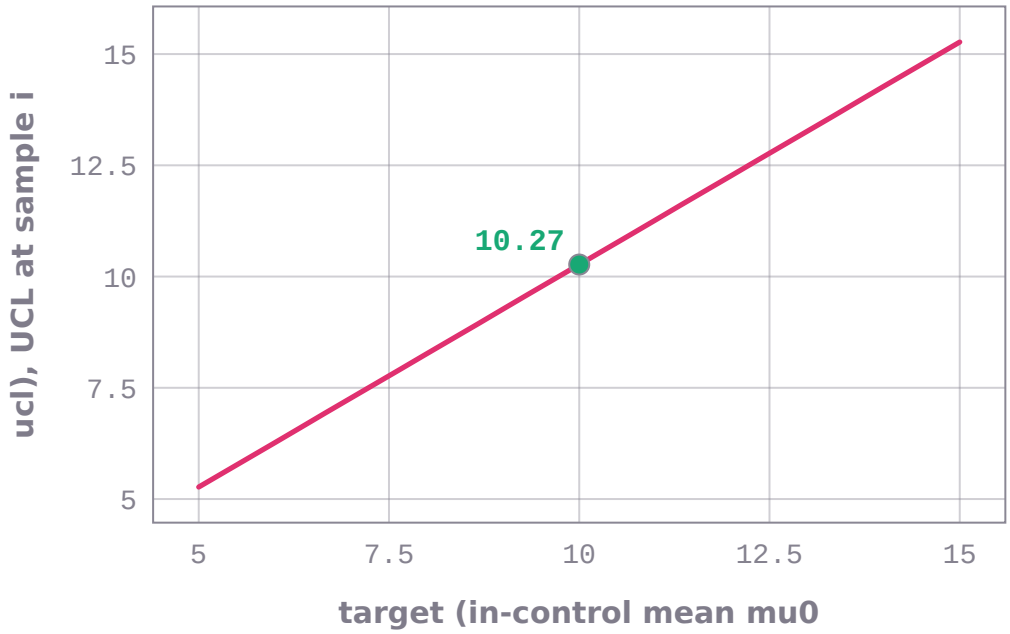
Control limits of an exponentially weighted moving average chart at sample i and in the steady state.

$$\mu_0 \pm L\sigma\sqrt{\frac{\lambda}{2-\lambda}[1-(1-\lambda)^{2i}]}$$

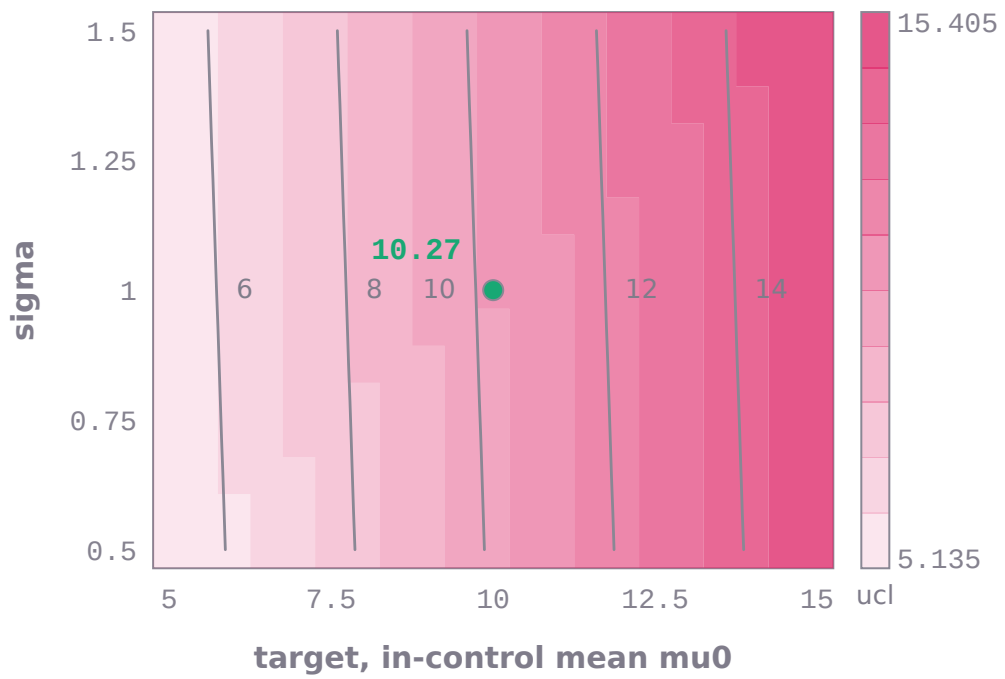
input	meaning	condition
`target`	in-control mean mu0	
`sigma`	standard deviation of the plotted observations	
`smoothing`	smoothing constant lambda $(0 < \lambda \leq 1)$	
`limit_sigma`	limit width L (often 2.7 to 3)	
`sample_number`	sample number i	

output	meaning
`ucl`	UCL at sample i
`lcl`	LCL at sample i
`ucl_steady`	steady-state UCL
`lcl_steady`	steady-state LCL
`sd_ewma`	standard deviation of the EWMA at sample i

- **Method:** limits mu0 +/- L sigma sqrt($\lambda/(2-\lambda)$ (1 - (1 - lambda)^(2i)))
- **Assumes:** $\sigma > 0$; $0 < \lambda \leq 1$; $L > 0$; sample_number >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



ucl (UCL at sample i) against target (in-control mean mu0) from 5 to 15. Other inputs as in the worked example: sigma = 1, smoothing = 0.1, limit_sigma = 2.7, sample_number = 1. 31 points computed by industrial-engineering-ai999x; marked: the worked example, target = 10, ucl = 10.27.



ucl (UCL at sample i) over target (in-control mean mu0) from 5 to 15 and sigma (standard deviation of the plotted observations) from 0.5 to 1.5: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 5.135 to dark 15.405); iso-lines at ucl = 6, 8, 10, 12, 14. Other inputs as in the worked example: smoothing = 0.1, limit_sigma = 2.7, sample_number = 1. Marked: the worked example, target = 10, sigma = 1, ucl = 10.27.

```
industrial_engineering_ai999x.solve('ie_ewma_limits', target=10, sigma=1, smoothing=0.1,
limit_sigma=2.7, sample_number=1)
# -> ucl = 10.27, lcl = 9.73, ucl_steady = 10.6194, lcl_steady = 9.38058, sd_ewma = 0.1
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_ewma_limits', target=50, sigma=2, smoothing=0.2,
limit_sigma=3, sample_number=5)
# -> ucl = 51.8896, lcl = 48.1104, ucl_steady = 52, lcl_steady = 48, sd_ewma = 0.629859
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
EWMA chart limits with centre line = 10, standard deviation = 1, lambda = 0.1, limit width = 2.7, sample = 1.	SOLVED	ucl = 10.27, lcl = 9.73, ucl_steady = 10.6194, lcl_steady = 9.38058, sd_ewma = 0.1
EWMA chart limits with centre line = 50, standard deviation = 2, lambda = 0.2, limit width = 3, sample = 5.	SOLVED	ucl = 51.8896, lcl = 48.1104, ucl_steady = 52, lcl_steady = 48, sd_ewma = 0.629859
EWMA chart limits with centre line = 10, standard deviation = 1, lambda = 0.1, limit width = 2.7.	NEEDS_INPUT	ie_ewma_limits needs sample_number (sample number i). Example: solve('ie_ewma_limits', target=10, sigma=1, smoothing=0.1
EWMA chart limits with centre line = 10, standard deviation = 1, lambda = 0.1, limit width = 2.7, sample = 1, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
EWMA chart limits with centre line = 10, standard deviation = 1, lambda = 1.5, limit width = 2.7, sample = 1.	INVALID_INPUT	EWMA chart control limits: smoothing must lie strictly between 0 and 1 (write 95% or 0.95).

Tabular CUSUM chart, `ie_tabular_cusum`

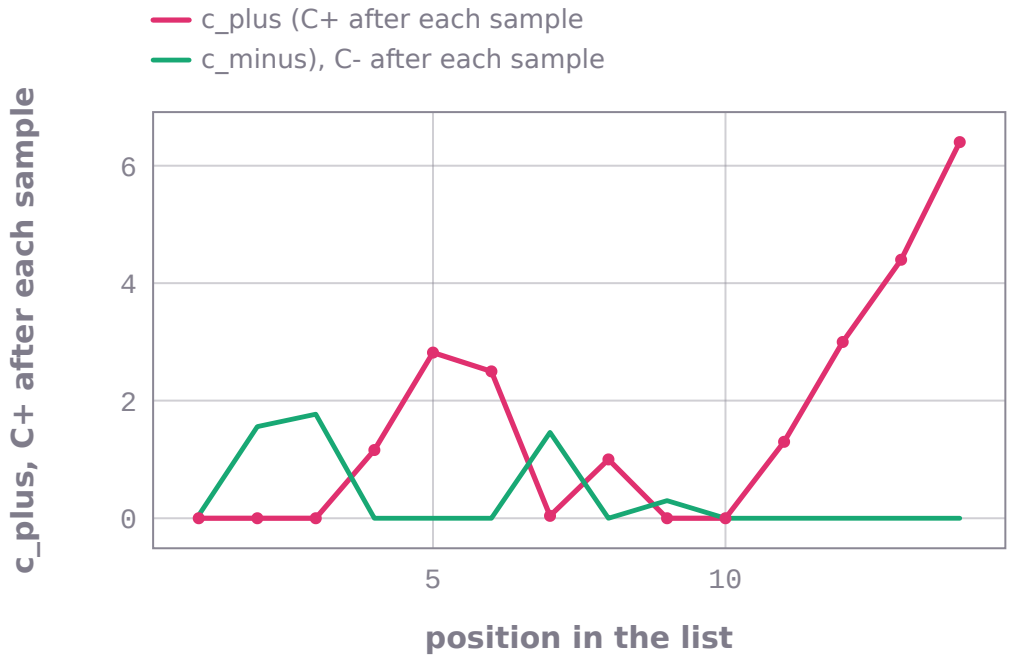
Upper and lower cumulative sums C+ and C- of a tabular CUSUM chart and the first sample that signals (C+ or C- above h).

$$C_i^+ = \max(0,\, x_i - (\mu_0 + K) + C_{i-1}^+), \quad C_i^- = \max(0,\, (\mu_0 - K) - x_i + C_{i-1}^-)$$

input	meaning	condition
`data`	observations in time order	
`target`	target mean mu0	
`k`	reference value K (often half the shift to detect)	
`h`	decision interval H	

output	meaning
`first_signal`	first sample number with C+ or C- above H (0 = none)
`c_plus`	C+ after each sample
`c_minus`	C- after each sample
`final_c_plus`	last C+
`final_c_minus`	last C-

- **Method:** C+_i = max(0, Missing superscript or subscript argument(i-1)); C-_i = max(0, Missing superscript or subscript argument(i-1)); signal when either exceeds H
- **Assumes:** $k \geq 0$; $h > 0$; C+ and C- start at 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



C+ after each sample (c_plus) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (14 values, first 0, last 6.4, largest 6.4); with c_minus (C- after each sample). Inputs: data = [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04…, target = 10, k = 0.5, h = 5.



final_c_plus (last C+) against target (target mean mu0) from 1 to 100 on a logarithmic axis. Other inputs as in the worked example: data = [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04…, k = 0.5, h = 5. 41 points computed by industrial-engineering-ai999x; marked: the worked example, target = 10, final_c_plus = 6.4.

```
industrial_engineering_ai999x.solve('ie_tabular_cusum', data=[9.45, 7.99, 9.29, 11.66,
12.16, 10.18, 8.04, 11.46, 9.2, 10.34, 11.8, 12.2, 11.9, 12.5], target=10, k=0.5, h=5)
# -> first_signal = 14, c_plus = [0.0, 0.0, 0.0, 1.16, 2.82, 2.5, 0.04, 1.0, 0.0, 0.0, 1.3,
3.0, 4.4, 6.4], c_minus = [0.05, 1.56, 1.77, 0.0, 0.0, 0.0, 1.46, 0.0, 0.3, 0.0, 0.0, 0.0,
0.0, 0.0], final_c_plus = 6.4, final_c_minus = 0   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_tabular_cusum', data=[50.2, 49.1, 51.0, 52.3, 53.1, 52.8, 54.0], target=50, k=1, h=4)
# -> first_signal = 6, c_plus = [0.0, 0.0, 0.0, 1.3, 3.4, 5.2, 8.2], c_minus = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], final_c_plus = 8.2, final_c_minus = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tabular CUSUM with observations = [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04, 11.46, 9.2, 10.34, 11.8, 12.2, 11.9, 12.5], target mean = 10, reference value = 0.5, decision interval = 5.	SOLVED	first_signal = 14, c_plus = [0.0, 0.0, 0.0, 1.16, 2.82, 2.5, 0.04, 1.0, 0.0, 0.0, 1.3, 3.0, 4.4, 6.4], c_minus = [0.05, 1.56, 1.77, 0.0, 0.0, 0.0, 1.46, 0.0, 0.3, 0.0, 0.0, 0.0, 0.0, 0.0], final_c_plus = 6.4, final_c_minus = 0
Tabular CUSUM with observations = [50.2, 49.1, 51, 52.3, 53.1, 52.8, 54], target mean = 50, reference value = 1, decision interval = 4.	SOLVED	first_signal = 6, c_plus = [0.0, 0.0, 0.0, 1.3, 3.4, 5.2, 8.2], c_minus = [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0], final_c_plus = 8.2, final_c_minus = 0
Tabular CUSUM with observations = [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04, 11.46, 9.2, 10.34, 11.8, 12.2, 11.9, 12.5], target mean = 10, reference value = 0.5.	NEEDS_INPUT	ie_tabular_cusum needs h (decision interval H). Example: solve('ie_tabular_cusum', data=[9.45, 7.99, 9.29, 11.66, 12.16,
Tabular CUSUM with observations = [9.45, 7.99, 9.29, 11.66, 12.16, 10.18, 8.04, 11.46, 9.2, 10.34, 11.8, 12.2, 11.9, 12.5], target mean = 10, reference value = 0.5, decision interval = 5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Tabular CUSUM with observations = [9.45, 7.99], target mean = 10, reference value = 0.5, decision interval = 0.	INVALID_INPUT	Tabular CUSUM chart: h must be positive.

DPMO and sigma level, `ie_dpmo_sigma_level`

Defects per unit, per opportunity and per million opportunities, and the sigma level (with the conventional 1.5 sigma shift).

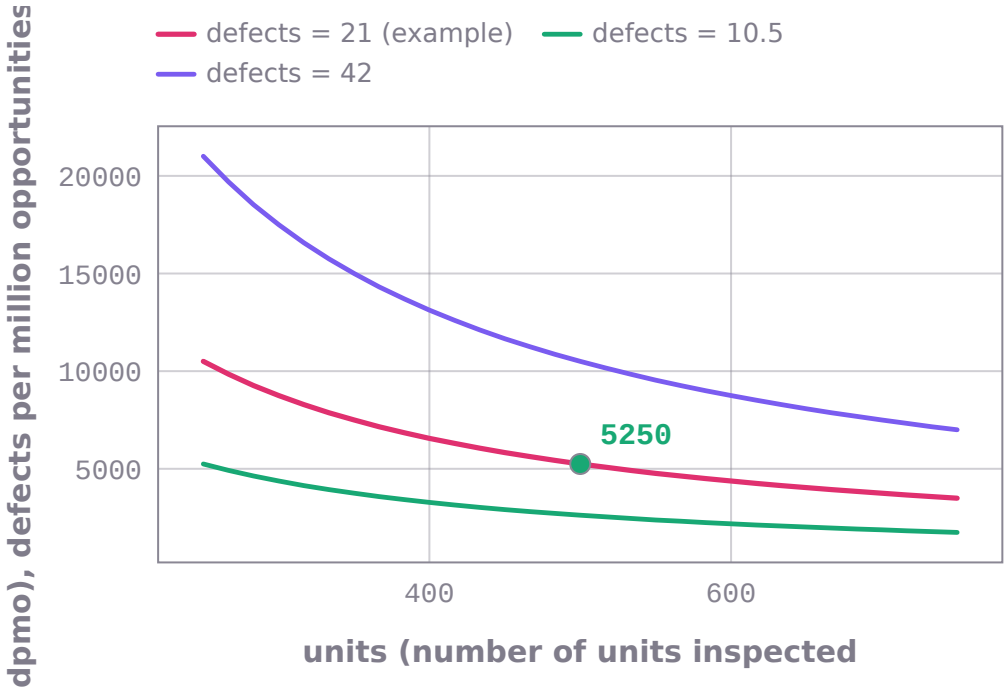
$$DPMO = \frac{10^6\, D}{U\, O}, \quad \mathrm{sigma} = \Phi^{-1}(1 - DPO) + 1.5$$

input	meaning	condition
`defects`	number of defects found	
`units`	number of units inspected	
`opportunities`	defect opportunities per unit	

output	meaning
`dpmo`	defects per million opportunities
`dpu`	defects per unit

output	meaning
`dpo`	defects per opportunity
`yield`	1 - DPO
`sigma_level`	short-term sigma level Phi^{-1}(1 - DPO) + 1.5

- **Method:** DPO = defects/(units opportunities), DPMO = 1e6 DPO, $\sigma = Phi^{-1}$(1 - DPO) + 1.5
- **Assumes:** defects >= 0; units, opportunities > 0; 0 < DPO < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dpmo (defects per million opportunities) against units (number of units inspected) from 250 to 750. One curve for each value of defects (number of defects found): 21, 10.5, 42. Other inputs as in the worked example: opportunities = 8. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, units = 500, dpmo = 5250.

```
industrial_engineering_ai999x.solve('ie_dpmo_sigma_level', defects=21, units=500,
opportunities=8)
# -> dpmo = 5250, dpu = 0.042, dpo = 0.00525, yield = 0.99475, sigma_level = 4.05891
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_dpmo_sigma_level', defects=3, units=1000,
opportunities=10)
# -> dpmo = 300, dpu = 0.003, dpo = 0.0003, yield = 0.9997, sigma_level = 4.93161
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
DPMO and sigma level with number of defects = 21, number of units = 500, opportunities per unit = 8.	SOLVED	dpmo = 5250, dpu = 0.042, dpo = 0.00525, yield = 0.99475, sigma_level = 4.05891
DPMO and sigma level with number of defects = 3, number of units = 1000, opportunities per unit = 10.	SOLVED	dpmo = 300, dpu = 0.003, dpo = 0.0003, yield = 0.9997, sigma_level = 4.93161

question	outcome	result
DPMO and sigma level with number of defects = 21, number of units = 500.	NEEDS_INPUT	ie_dpmo_sigma_level needs opportunities (defect opportunities per unit). Example: solve('ie_dpmo_sigma_level', defects=2
DPMO and sigma level with number of defects = 21, number of units = 500, opportunities per unit = 8, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
DPMO and sigma level with number of defects = 21, number of units = 0, opportunities per unit = 8.	INVALID_INPUT	DPMO and sigma level: units must be positive.

Rolled throughput yield, `ie_rolled_throughput_yield`

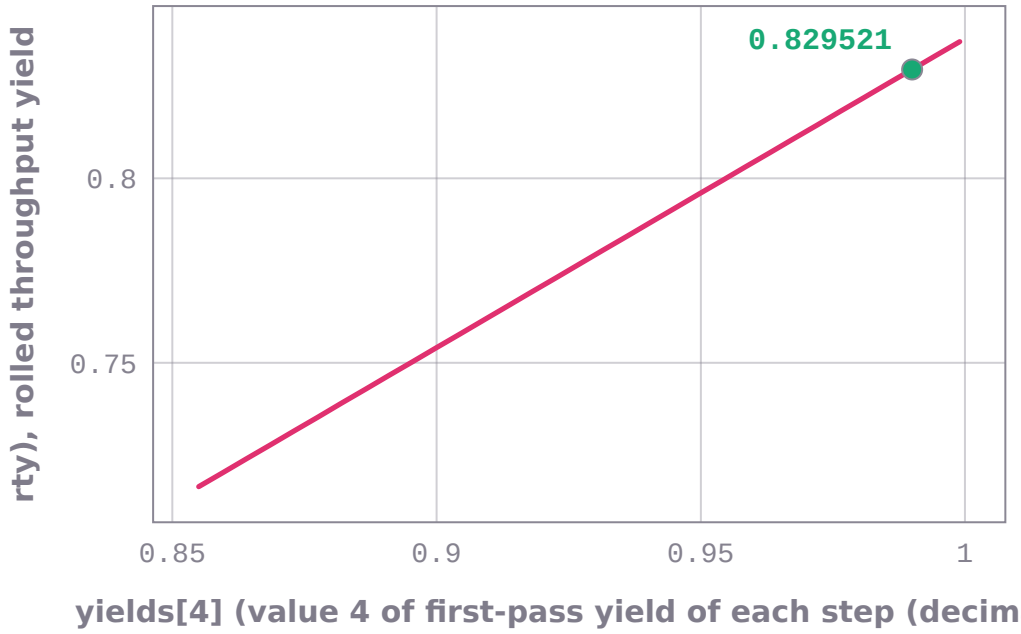
Probability that a unit passes every process step defect-free (product of step yields), its geometric mean per step and the equivalent total defects per unit -ln(RTY).

$$RTY = \prod_i Y_i, \quad Y_{norm} = RTY^{1/k}$$

input	meaning	condition
`yields`	first-pass yield of each step (decimals)	

output	meaning
`rty`	rolled throughput yield
`normalized_yield`	normalized yield RTY^(1/k)
`total_dpu`	equivalent total defects per unit -ln RTY

- **Method:** RTY = product of yields; normalized = RTY^(1/k); DPU = -ln RTY
- **Assumes:** each yield in (0, 1]
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



rty (rolled throughput yield) as value 4 of yields (first-pass yield of each step (decimals)) moves from 0.855 to 1.035, the other values as given ([0.95, 0.98, 0.9, 0.99]); 33 runs of industrial-engineering-ai999x. Marked: the worked example, yields[4] = 0.99, rty = 0.829521.

```
industrial_engineering_ai999x.solve('ie_rolled_throughput_yield', yields=[0.95, 0.98, 0.9, 0.99])
# -> rty = 0.829521, normalized_yield = 0.954348, total_dpu = 0.186907   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_rolled_throughput_yield', yields=[0.92, 0.97])
# -> rty = 0.8924, normalized_yield = 0.944669, total_dpu = 0.113841   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rolled throughput yield with step yields = [0.95, 0.98, 0.9, 0.99].	SOLVED	rty = 0.829521, normalized_yield = 0.954348, total_dpu = 0.186907
Rolled throughput yield with step yields = [0.92, 0.97].	SOLVED	rty = 0.8924, normalized_yield = 0.944669, total_dpu = 0.113841
Rolled throughput yield with step yields = [0.95, 0.98, 0.9, 0.99], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Rolled throughput yield with step yields = [0.95, 1.2].	INVALID_INPUT	Rolled throughput yield: every yield must lie in (0, 1].

Single sampling plan: OC, AOQ and ATI, `ie_acceptance_sampling`

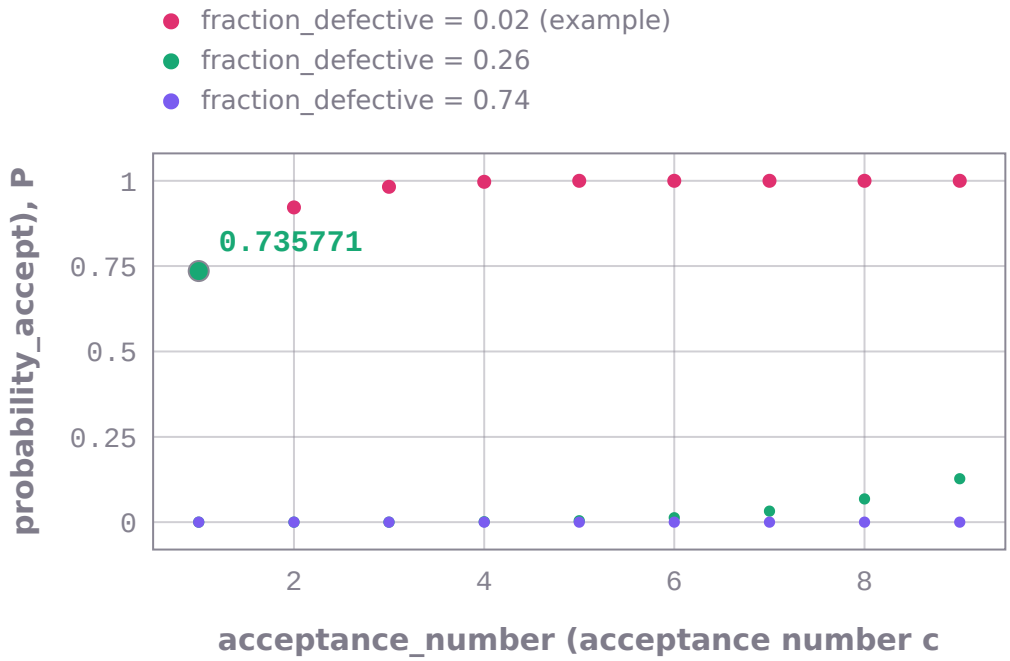
Probability of accepting a lot under a single sampling plan (n, c), the average outgoing quality with rectifying inspection and the average total inspection.

$$P_a = \sum_{k=0}^{c} \binom{n}{k} p^k (1-p)^{n-k}, \quad AOQ = \frac{P_a\, p\, (N-n)}{N}$$

input	meaning	condition
`sample_size`	items sampled n	
`acceptance_number`	acceptance number c	
`fraction_defective`	lot fraction defective p (decimal)	
`lot_size`	lot size N	

output	meaning
`probability_accept`	P(accept) = P(X <= c), X ~ Binomial(n, p)
`aoq`	average outgoing quality Pa p (N - n)/N
`ati`	average total inspection $n + (1 - Pa)$(N - n)
`probability_reject`	$1 - Pa$

- **Method:** Pa = sum_{$k \leq c$} C(n,k) $p^k\ (1-p)^{n-k}$; AOQ = Pa p (N-n)/N; ATI = n + (1 - Pa)(N - n)
- **Assumes:** $0 \leq c < n \leq N$; $0 < p < 1$; binomial sampling; $n \leq 2000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



probability_accept (P(accept) = P(X <= c), X ~ Binomial(n, p)) against acceptance_number (acceptance number c) from 1 to 9. One curve for each value of fraction_defective (lot fraction defective p (decimal)): 0.02, 0.26, 0.74. Other inputs as in the worked example: sample_size = 50, lot_size = 2000. 9 points per curve computed by industrial-engineering-ai999x; marked: the worked example, acceptance_number = 1, probability_accept = 0.735771.

```
industrial_engineering_ai999x.solve('ie_acceptance_sampling', sample_size=50,
acceptance_number=1, fraction_defective=0.02, lot_size=2000)
# -> probability_accept = 0.735771, aoq = 0.0143475, ati = 565.246, probability_reject =
0.264229   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_acceptance_sampling', sample_size=89,
acceptance_number=2, fraction_defective=0.01, lot_size=10000)
# -> probability_accept = 0.93969, aoq = 0.00931327, ati = 686.733, probability_reject =
0.0603101   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Single sampling plan with sample size = 50, acceptance number = 1, lot fraction defective = 0.02, n lot = 2000.	SOLVED	probability_accept = 0.735771, aoq = 0.0143475, ati = 565.246, probability_reject = 0.264229
Single sampling plan with sample size = 89, acceptance number = 2, lot fraction defective = 0.01, n lot = 10000.	SOLVED	probability_accept = 0.93969, aoq = 0.00931327, ati = 686.733, probability_reject = 0.0603101
Single sampling plan with sample size = 50, acceptance number = 1, lot fraction defective = 0.02.	NEEDS_INPUT	ie_acceptance_sampling needs lot_size (lot size N). Example: solve('ie_acceptance_sampling', sample_size=50, acceptance_
Single sampling plan with sample size = 50, acceptance number = 1, lot fraction defective = 0.02, n lot = 2000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Single sampling plan with sample size = 50, acceptance number = 60, lot fraction defective = 0.02, n lot = 2000.	INVALID_INPUT	Single sampling plan: OC, AOQ and ATI: the acceptance number must be below the sample size.

Reliability of components in series, `ie_series_reliability`

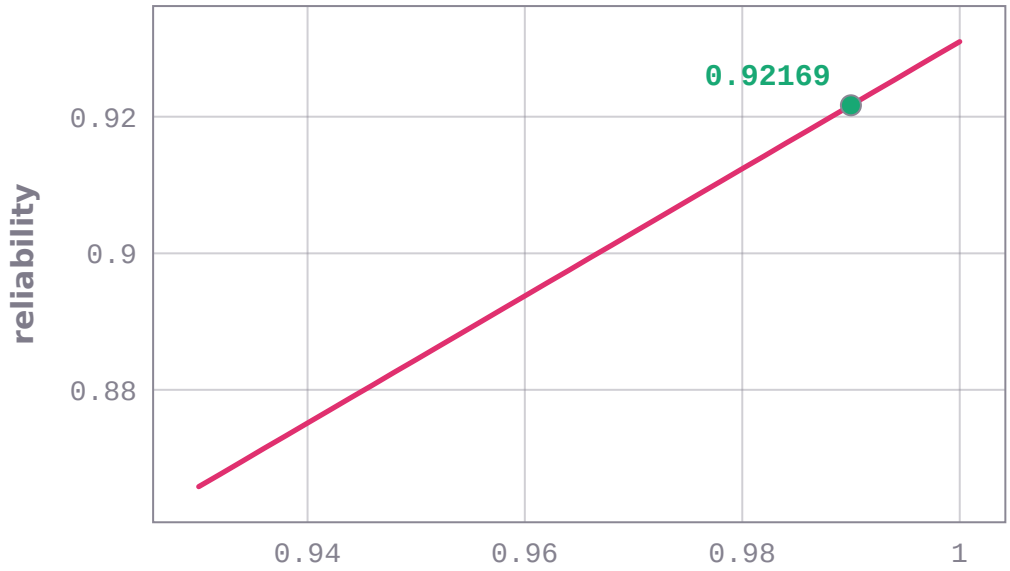
Reliability of a system that works only if every component works (independent components).

$$R_s = \prod_{i=1}^{n} R_i$$

input	meaning	condition
`reliabilities`	reliability of each component (decimals)	

output	meaning
`reliability`	system reliability product of R_i
`unreliability`	1 - system reliability

- **Method:** R = product R_i
- **Assumes:** each reliability in [0, 1]; independent failures
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reliability (system reliability product of R_i) as value 3 of reliabilities (reliability of each component (decimals)) moves from 0.93 to 1.01, the other values as given ([0.95, 0.98, 0.99]); 36 runs of industrial-engineering-ai999x. Marked: the worked example, reliabilities[3] = 0.99, reliability = 0.92169.

```
industrial_engineering_ai999x.solve('ie_series_reliability', reliabilities=[0.95, 0.98, 0.99])
# -> reliability = 0.92169, unreliability = 0.07831   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_series_reliability', reliabilities=[0.9, 0.9, 0.85, 0.97])
# -> reliability = 0.667845, unreliability = 0.332155   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Series system reliability with component reliabilities = [0.95, 0.98, 0.99].	SOLVED	reliability = 0.92169, unreliability = 0.07831
Series system reliability with component reliabilities = [0.9, 0.9, 0.85, 0.97].	SOLVED	reliability = 0.667845, unreliability = 0.332155
Series system reliability with component reliabilities = [0.95, 0.98, 0.99], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Series system reliability with component reliabilities = [0.95, 1.5].	INVALID_INPUT	Reliability of components in series: every value of reliabilities must lie between 0 and 1.

Reliability of components in parallel, `ie_parallel_reliability`

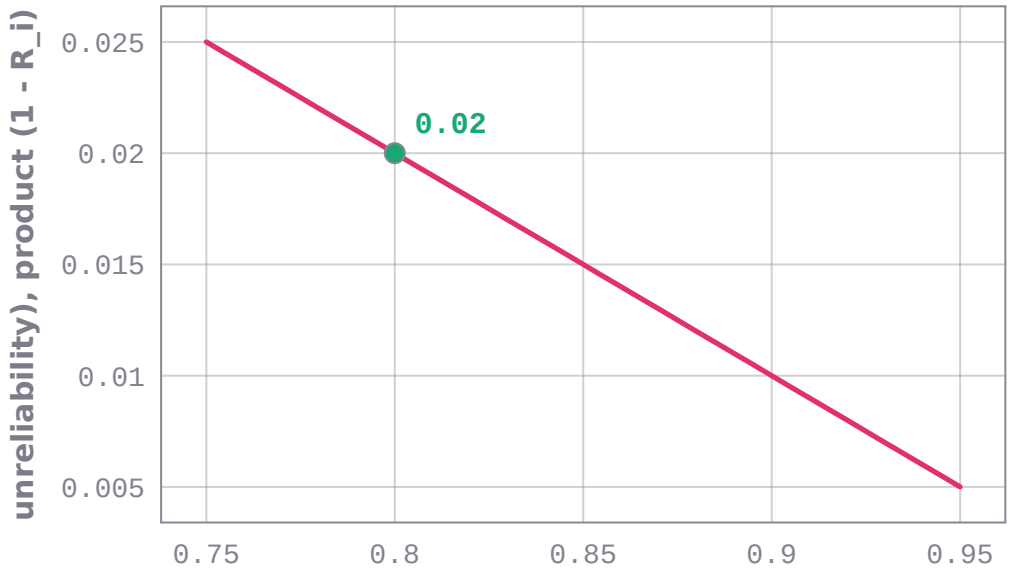
Reliability of a redundant system that works if at least one component works (independent components).

$$R_p = 1 - \prod_{i=1}^{n} (1 - R_i)$$

input	meaning	condition
`reliabilities`	reliability of each component (decimals)	

output	meaning
`reliability`	1 - product $(1 - R_i)$
`unreliability`	product $(1 - R_i)$

- **Method:** R = 1 - product $(1 - R_i)$
- **Assumes:** each reliability in [0, 1]; independent failures; active redundancy
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



unreliability (product (1 - R_i)) as value 2 of reliabilities (reliability of each component (decimals)) moves from 0.75 to 0.95, the other values as given ([0.9, 0.8]); 41 runs of industrial-engineering-ai999x. Marked: the worked example, reliabilities[2] = 0.8, unreliability = 0.02.

```
industrial_engineering_ai999x.solve('ie_parallel_reliability', reliabilities=[0.9, 0.8])
# -> reliability = 0.98, unreliability = 0.02   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_parallel_reliability', reliabilities=[0.95, 0.9,
0.85])
# -> reliability = 0.99925, unreliability = 0.00075   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallel system reliability with component reliabilities = [0.9, 0.8].	SOLVED	reliability = 0.98, unreliability = 0.02
Parallel system reliability with component reliabilities = [0.95, 0.9, 0.85].	SOLVED	reliability = 0.99925, unreliability = 0.00075
Parallel system reliability with component reliabilities = [0.9, 0.8], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Parallel system reliability with component reliabilities = [-0.1, 0.9].	INVALID_INPUT	Reliability of components in parallel: every value of reliabilities must lie between 0 and 1.

k-out-of-n system reliability, `ie_k_out_of_n_reliability`

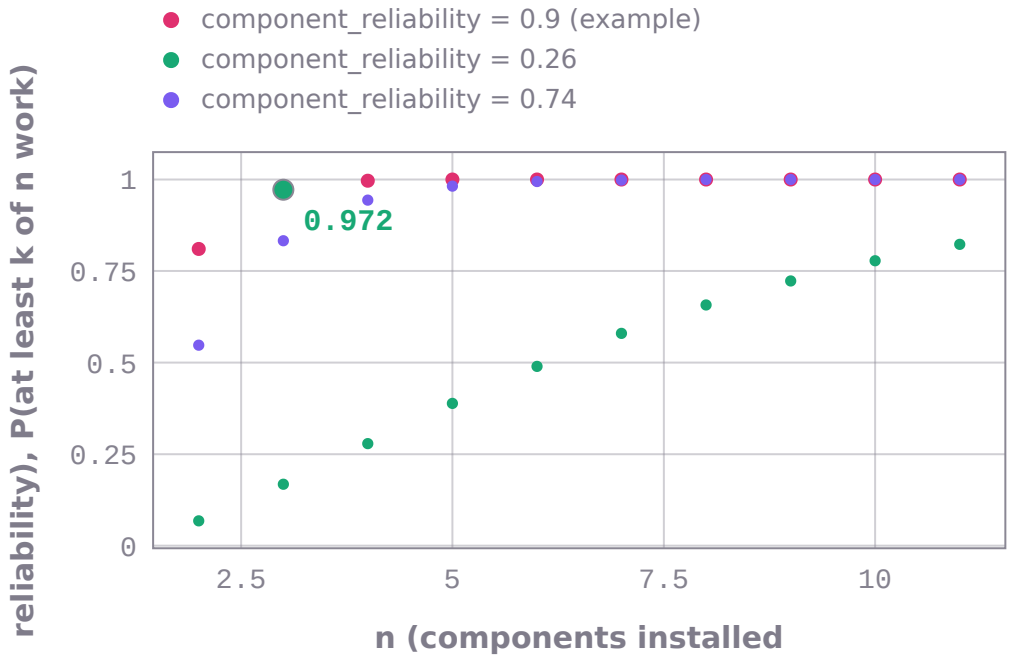
Reliability of a system of n identical independent components that works when at least k of them work.

$$R = \sum_{j=k}^{n} \binom{n}{j} r^j (1-r)^{n-j}$$

input	meaning	condition
`component_reliability`	reliability of each component	
`k`	components needed	
`n`	components installed	

output	meaning
`reliability`	P(at least k of n work)
`unreliability`	1 - reliability

- **Method:** R = sum_{$j = k$}^{n} C(n, j) $r^j \, (1 - r)^{n-j}$
- **Assumes:** $0 \le r \le 1$; $1 \le k \le n \le 1000$; identical independent components
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reliability (P(at least k of n work)) against n (components installed) from 1 to 11. One curve for each value of component_reliability (reliability of each component): 0.9, 0.26, 0.74. Other inputs as in the worked example: k = 2. 10 points per curve computed by industrial-engineering-ai999x; marked: the worked example, n = 3, reliability = 0.972.

```
industrial_engineering_ai999x.solve('ie_k_out_of_n_reliability', component_reliability=0.9,
k=2, n=3)
# -> reliability = 0.972, unreliability = 0.028   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_k_out_of_n_reliability',
component_reliability=0.95, k=3, n=5)
# -> reliability = 0.998842, unreliability = 0.00115813   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
K-out-of-n system reliability with component reliability = 0.9, required components = 2, total components = 3.	SOLVED	reliability = 0.972, unreliability = 0.028
K-out-of-n system reliability with component reliability = 0.95, required components = 3, total components = 5.	SOLVED	reliability = 0.998842, unreliability = 0.00115813
K-out-of-n system reliability with component reliability = 0.9, required components = 2.	NEEDS_INPUT	ie_k_out_of_n_reliability needs n (components installed). Example: solve('ie_k_out_of_n_reliability', component_reliabil
K-out-of-n system reliability with component reliability = 0.9, required components = 2, total components = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
K-out-of-n system reliability with component reliability = 0.9, required components = 4, total components = 3.	INVALID_INPUT	k-out-of-n system reliability: k must not exceed n.

Series system of parallel stages, `ie_series_parallel_reliability`

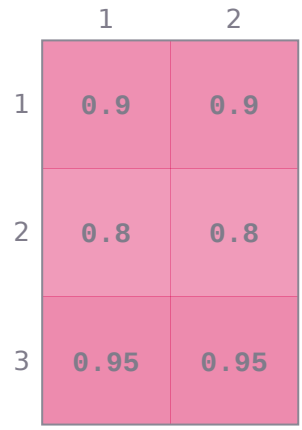
Reliability of stages in series where each stage has redundant components in parallel (each row of the matrix is one stage).

$$R = \prod_i \left[1 - \prod_j (1 - r_{ij})\right]$$

input	meaning	condition
`stage_reliabilities`	matrix: one row per stage, the reliabilities of its parallel components	

output	meaning
`reliability`	product over stages of 1 - product $(1 - r)$
`unreliability`	1 - reliability

- **Method:** R = prod_i [1 - prod_j $(1 - r_ij)$]
- **Assumes:** every value in [0, 1]; independent components; rows of equal length
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



stage_reliabilities (input)

The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): stage_reliabilities (input). Returned with them by industrial-engineering-ai999x: reliability = 0.948024, unreliability = 0.051976.

```
industrial_engineering_ai999x.solve('ie_series_parallel_reliability', stage_reliabilities=
[[0.9, 0.9], [0.8, 0.8], [0.95, 0.95]])
# -> reliability = 0.948024, unreliability = 0.051976   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_series_parallel_reliability', stage_reliabilities=
[[0.7, 0.8, 0.9], [0.85, 0.6, 0.75]])
# -> reliability = 0.97909, unreliability = 0.02091   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Series-parallel system reliability with stages = [[0.9, 0.9], [0.8, 0.8], [0.95, 0.95]].	SOLVED	reliability = 0.948024, unreliability = 0.051976
Series-parallel system reliability with stages = [[0.7, 0.8, 0.9], [0.85, 0.6, 0.75]].	SOLVED	reliability = 0.97909, unreliability = 0.02091
Series-parallel system reliability with stages = [[0.9, 0.9], [0.8, 0.8], [0.95, 0.95]], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Series-parallel system reliability with stages = [[0.9, 1.9], [0.8, 0.8]].	INVALID_INPUT	Series system of parallel stages: every reliability must lie between 0 and 1.

Exponential (constant failure rate) reliability, `ie_exponential_reliability`

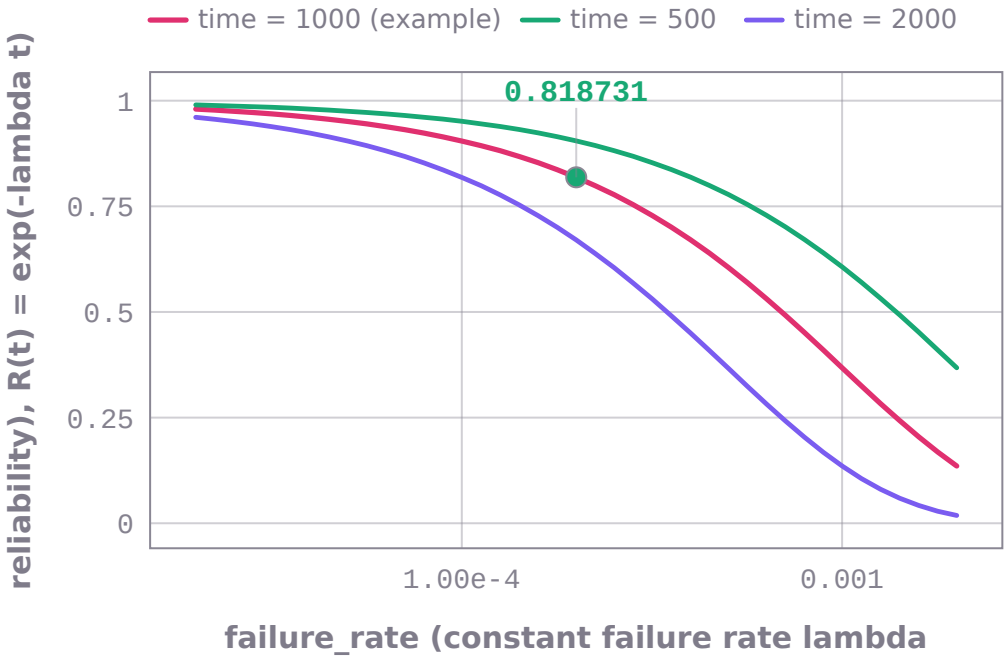
Probability of surviving a mission time with a constant failure rate: R(t) = exp(-lambda t), MTBF = 1/lambda.

$$R(t) = e^{-\lambda t}, \quad MTBF = \frac{1}{\lambda}$$

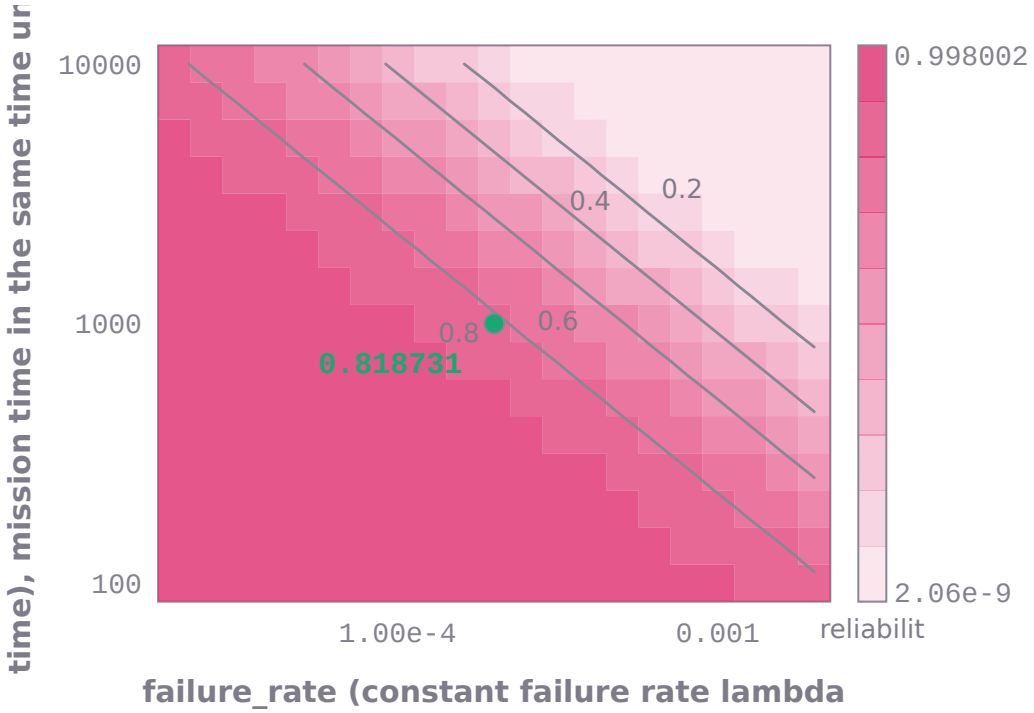
input	meaning	condition
`failure_rate`	constant failure rate lambda (failures per time unit)	
`time`	mission time in the same time unit	

output	meaning
`reliability`	R(t) = exp(-lambda t)
`unreliability`	F(t) = 1 - R(t)
`mtbf`	mean time between failures $1/\lambda$
`hazard_rate`	lambda

- **Method:** $R = exp$(-lambda t)
- **Assumes:** failure_rate > 0; time >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reliability (R(t) = exp(-lambda t)) against failure_rate (constant failure rate lambda (failures per time unit)) from 2.0000e-05 to 0.002 on a logarithmic axis. One curve for each value of time (mission time in the same time unit): 1000, 500, 2000. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, failure_rate = 0.0002, reliability = 0.818731.



reliability (R(t) = exp(-lambda t)) over failure_rate (constant failure rate lambda (failures per time unit)) from 2.0000e-05 to 0.002 and time (mission time in the same time unit) from 100 to 10000 (logarithmic axes): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 2.0612e-09 to dark 0.998002); iso-lines at reliability = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, failure_rate = 0.0002, time = 1000, reliability = 0.818731.

```
industrial_engineering_ai999x.solve('ie_exponential_reliability', failure_rate=0.0002,
time=1000)
# -> reliability = 0.818731, unreliability = 0.181269, mtbf = 5000, hazard_rate = 0.0002
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_exponential_reliability', failure_rate=0.001,
time=500)
# -> reliability = 0.606531, unreliability = 0.393469, mtbf = 1000, hazard_rate = 0.001
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential reliability with lambda = 0.0002, mission time = 1000.	SOLVED	reliability = 0.818731, unreliability = 0.181269, mtbf = 5000, hazard_rate = 0.0002
Exponential reliability with lambda = 0.001, mission time = 500.	SOLVED	reliability = 0.606531, unreliability = 0.393469, mtbf = 1000, hazard_rate = 0.001
Exponential reliability with lambda = 0.0002.	NEEDS_INPUT	ie_exponential_reliability needs time (mission time in the same time unit). Example: solve('ie_exponential_reliability',
Exponential reliability with lambda = 0.0002, mission time = 1000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Exponential reliability with lambda = -0.001, mission time = 500.	INVALID_INPUT	Exponential (constant failure rate) reliability: failure_rate must be positive.

Inherent availability from MTBF and MTTR, `ie_availability`

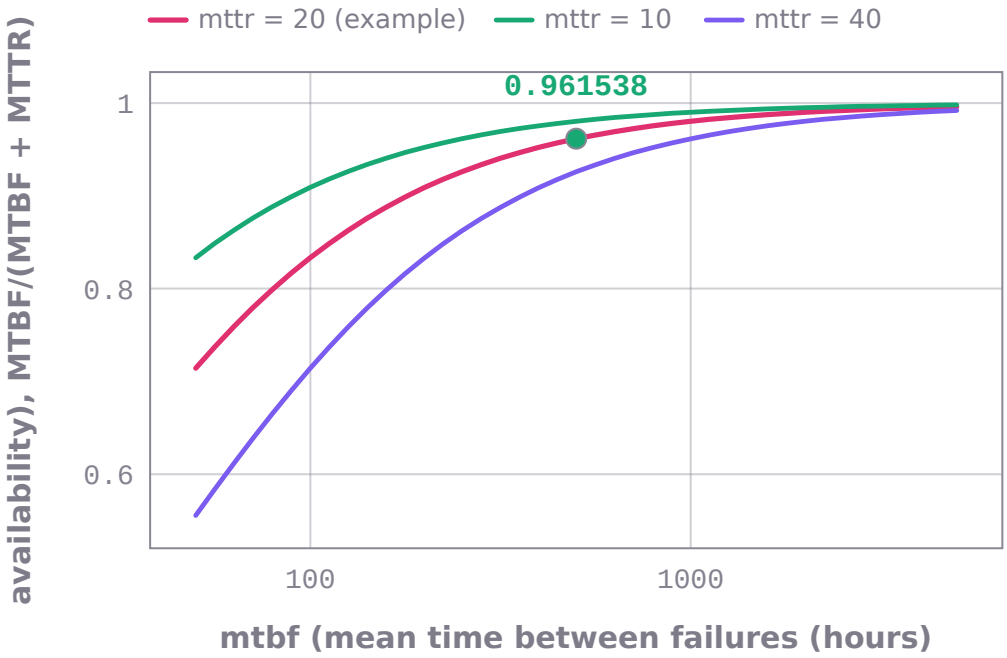
Long-run fraction of time a repairable system is up, A = MTBF / (MTBF + MTTR), and the downtime it implies.

$$A = \frac{MTBF}{MTBF + MTTR}$$

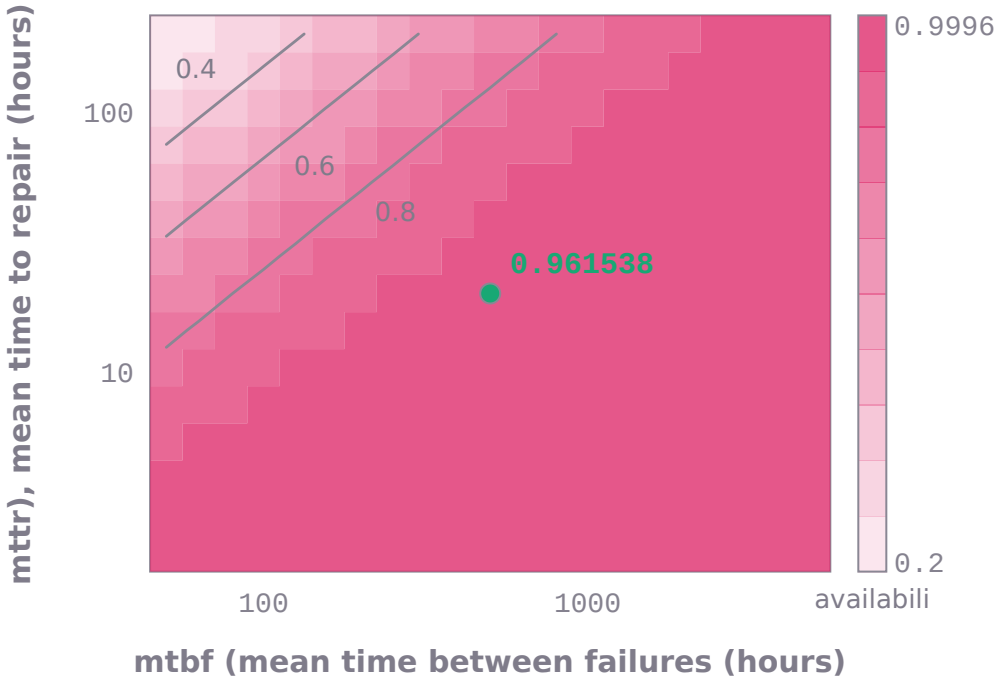
input	meaning	condition
`mtbf`	mean time between failures (hours)	
`mttr`	mean time to repair (hours)	
`hours_per_year`	operating hours in a year (default 8760)	

output	meaning
`availability`	MTBF/(MTBF + MTTR)
`unavailability`	MTTR/(MTBF + MTTR)
`downtime_per_year`	expected downtime hours per year
`failure_rate`	1/MTBF
`repair_rate`	1/MTTR

- **Method:** A = MTBF/(MTBF + MTTR)
- **Assumes:** mtbf, mttr > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



availability (MTBF/(MTBF + MTTR)) against mtbf (mean time between failures (hours)) from 50 to 5000 on a logarithmic axis. One curve for each value of mttr (mean time to repair (hours)): 20, 10, 40. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, mtbf = 500, availability = 0.961538.



availability (MTBF/(MTBF + MTTR)) over mtbf (mean time between failures (hours)) from 50 to 5000 and mttr (mean time to repair (hours)) from 2 to 200 (logarithmic axes): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.2 to dark 0.9996); iso-lines at availability = 0.4, 0.6, 0.8. Marked: the worked example, mtbf = 500, mttr = 20, availability = 0.961538.

```
industrial_engineering_ai999x.solve('ie_availability', mtbf=500, mttr=20)
# -> availability = 0.961538, unavailability = 0.0384615, downtime_per_year = 336.923,
failure_rate = 0.002, repair_rate = 0.05   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_availability', mtbf=1200, mttr=8,
hours_per_year=6000)
# -> availability = 0.993377, unavailability = 0.00662252, downtime_per_year = 39.7351,
failure_rate = 0.000833333, repair_rate = 0.125   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Inherent availability with mean time between failures = 500, mean time to repair = 20.	SOLVED	availability = 0.961538, unavailability = 0.0384615, downtime_per_year = 336.923, failure_rate = 0.002, repair_rate = 0.05
Inherent availability with mean time between failures = 1200, mean time to repair = 8, hours = 6000.	SOLVED	availability = 0.993377, unavailability = 0.00662252, downtime_per_year = 39.7351, failure_rate = 0.000833333, repair_rate = 0.125
Inherent availability with mean time between failures = 500.	NEEDS_INPUT	ie_availability needs mttr (mean time to repair (hours)). Example: solve('ie_availability', mtbf=500, mttr=20).
Inherent availability with mean time between failures = 500, mean time to repair = 20, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Inherent availability with mean time between failures = 500, mean time to repair = 0.	INVALID_INPUT	Inherent availability from MTBF and MTTR: mttr must be positive.

Weibull reliability, hazard and MTTF, `ie_weibull_reliability`

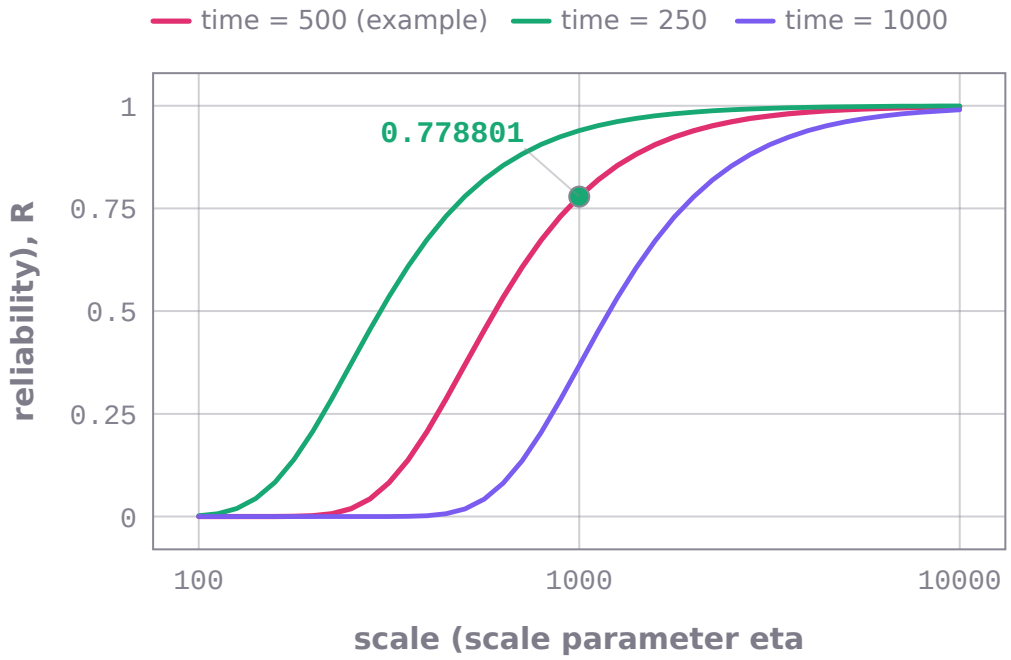
Reliability, hazard rate, mean life and B10 life of a Weibull life distribution (shape beta, scale eta, optional location gamma).

$$R(t) = \exp\left[-\left(\frac{t-\gamma}{\eta}\right)^{\beta}\right], \quad MTTF = \gamma + \eta\Gamma(1+1/\beta)$$

input	meaning	condition
`shape`	shape parameter beta	
`scale`	scale parameter eta (characteristic life)	
`time`	time t	
`location`	location gamma, failure-free time (default 0)	

output	meaning
`reliability`	R(t) = exp(-((t - gamma)/eta)^beta)
`unreliability`	$1 - R$(t)
`hazard_rate`	h(t) = beta/eta ((t - gamma)/eta)^(beta - 1)
`mttf`	gamma + eta Gamma(1 + 1/beta)
`b10_life`	time by which 10 % have failed

- **Method:** $R = e^{-\left(\frac{t-g}{\eta}\right)^b}$; MTTF = g + eta Gamma(1 + 1/b); $B10 = g + \eta$ (-ln 0.9)^(1/b)
- **Assumes:** shape, scale > 0; location >= 0; time >= location
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reliability (R(t) = exp(-((t - gamma)/eta)^beta)) against scale (scale parameter eta (characteristic life)) from 100 to 10000 on a logarithmic axis. One curve for each value of time (time t): 500, 250, 1000. Other inputs as in the worked example: shape = 2. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, scale = 1000, reliability = 0.778801.

```
industrial_engineering_ai999x.solve('ie_weibull_reliability', shape=2, scale=1000,
time=500)
# -> reliability = 0.778801, unreliability = 0.221199, hazard_rate = 0.001, mttf = 886.227,
b10_life = 324.593   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_weibull_reliability', shape=1.5, scale=5000,
time=2000, location=200)
# -> reliability = 0.805735, unreliability = 0.194265, hazard_rate = 0.00018, mttf =
4713.73, b10_life = 1315.38   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull reliability with beta = 2, scale parameter = 1000, mission time = 500.	SOLVED	reliability = 0.778801, unreliability = 0.221199, hazard_rate = 0.001, mttf = 886.227, b10_life = 324.593
Weibull reliability with beta = 1.5, scale parameter = 5000, mission time = 2000, gamma = 200.	SOLVED	reliability = 0.805735, unreliability = 0.194265, hazard_rate = 0.00018, mttf = 4713.73, b10_life = 1315.38
Weibull reliability with beta = 2, scale parameter = 1000.	NEEDS_INPUT	ie_weibull_reliability needs time (time t). Example: solve('ie_weibull_reliability', shape=2, scale=1000, time=500).

question	outcome	result
Weibull reliability with beta = 2, scale parameter = 1000, mission time = 500, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Weibull reliability with beta = 2, scale parameter = 0, mission time = 500.	INVALID_INPUT	Weibull reliability, hazard and MTTF: scale must be positive.

Weibull fit by median-rank regression, `ie_weibull_median_rank_regression`

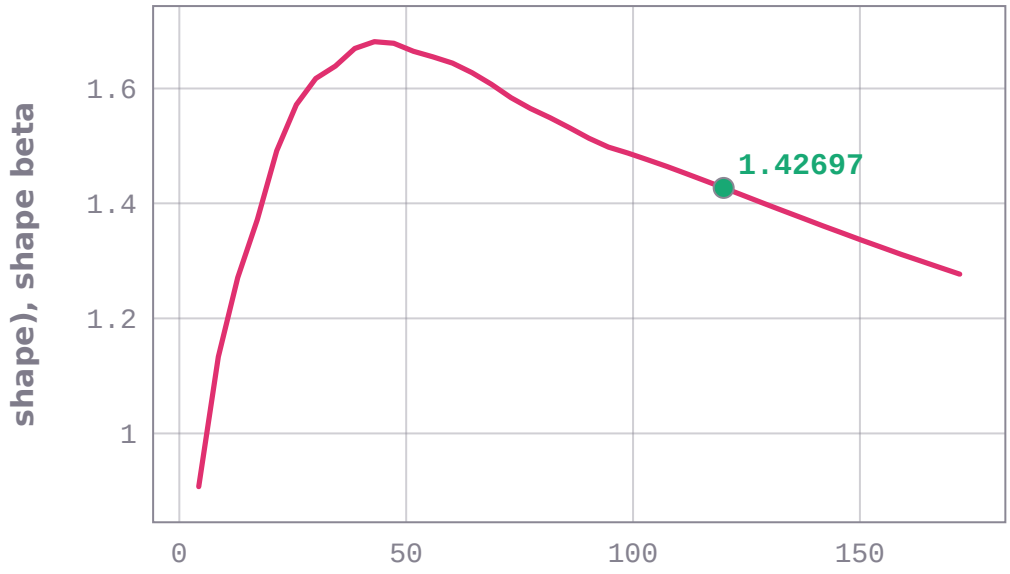
Weibull shape and scale estimated from complete failure times by regression on Bernard's median ranks (Weibull probability plot).

$$\ln\ln\frac{1}{1-F_i} = \beta\ln t_{(i)} - \beta\ln\eta, \quad F_i = \frac{i-0.3}{n+0.4}$$

input	meaning	condition
`failure_times`	failure times (complete data, any order)	

output	meaning
`shape`	shape beta (slope of ln ln$(1/(1-F))$ on ln t)
`scale`	scale eta
`r_squared`	coefficient of determination of the plot

- **Method:** $F_i = (i-0.3)/(n+0.4)$; regress $y = ln$(-ln(1 - F_i)) on $x = ln$ t_(i); beta = slope, $\eta = e^{\frac{-intercept}{\beta}}$
- **Assumes:** at least 3 positive failure times, not all equal; complete (uncensored) data
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



shape (shape beta (slope of ln ln(1/(1-F)) on ln t)) as value 6 of failure_times (failure times (complete data, any order)) moves from 0 to 172, the other values as given ([16, 34, 53, 75, 93, 120]); 40 runs of industrial-engineering-ai999x. Marked: the worked example, failure_times[6] = 120, shape = 1.42697.

```
industrial_engineering_ai999x.solve('ie_weibull_median_rank_regression', failure_times=[16, 34, 53, 75, 93, 120])
# -> shape = 1.42697, scale = 76.3454, r_squared = 0.991181   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_weibull_median_rank_regression', failure_times=[1200, 850, 2300, 1700, 3100, 640, 1950])
# -> shape = 1.90306, scale = 1930.68, r_squared = 0.982191   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weibull fit by median rank regression with failure times = [16, 34, 53, 75, 93, 120].	SOLVED	shape = 1.42697, scale = 76.3454, r_squared = 0.991181
Weibull fit by median rank regression with failure times = [1200, 850, 2300, 1700, 3100, 640, 1950].	SOLVED	shape = 1.90306, scale = 1930.68, r_squared = 0.982191
Weibull fit by median rank regression with failure times = [16, 34, 53, 75, 93, 120], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Weibull fit by median rank regression with failure times = [16, -34, 53].	INVALID_INPUT	Weibull fit by median-rank regression: every value of failure_times must be positive.

Parallel components needed for a target reliability, `ie_redundancy_needed`

Smallest number of identical components in parallel that reaches a target system reliability.

$$m = \left\lceil \frac{\ln(1 - R_{target})}{\ln(1 - r)} \right\rceil$$

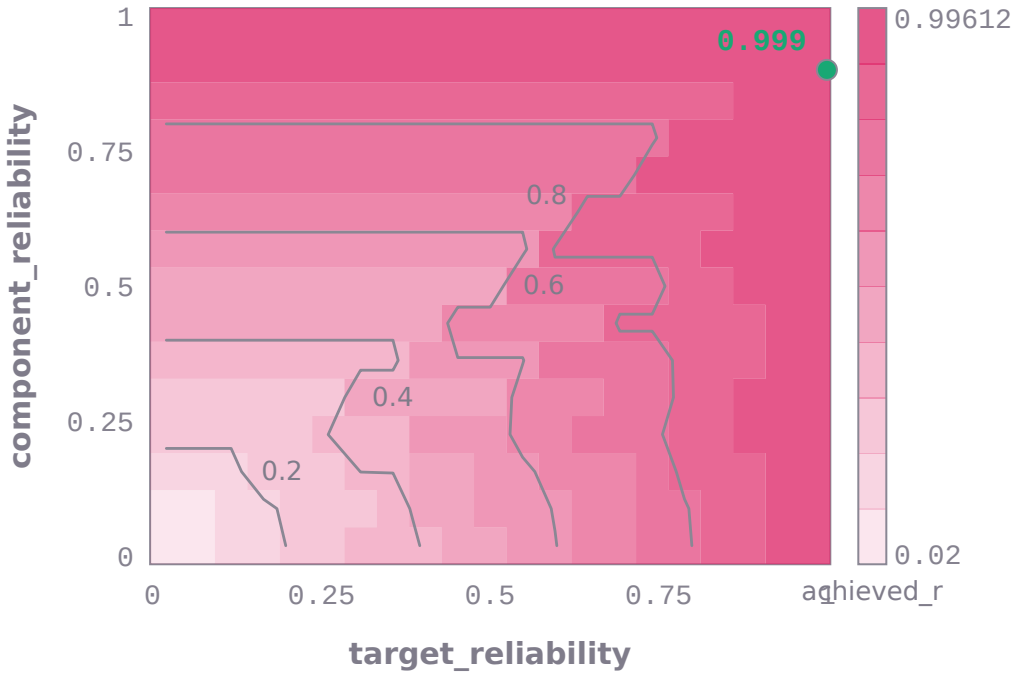
input	meaning	condition
`target_reliability`	required system reliability	
`component_reliability`	reliability of one component	

output	meaning
`components`	smallest m with 1 - (1 - r)^m >= target
`achieved_reliability`	reliability with m components

- **Method:** smallest m with 1 - (1 - r)^m >= R_target (m = ceil(ln(1 - R)/ln(1 - r)))
- **Assumes:** 0 < target < 1; $0 < r < 1$; active parallel redundancy
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



achieved_reliability (reliability with m components) against target_reliability (required system reliability) from 0.4995 to 0.999. One curve for each value of component_reliability (reliability of one component): 0.9, 0.26, 0.74. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, target_reliability = 0.999, achieved_reliability = 0.999.



achieved_reliability (reliability with m components) over target_reliability (required system reliability) from 0.02 to 0.98 and component_reliability (reliability of one component) from 0.02 to 0.98: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.02 to dark 0.99612); iso-lines at achieved_reliability = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, target_reliability = 0.999, component_reliability = 0.9, achieved_reliability = 0.999.

```
industrial_engineering_ai999x.solve('ie_redundancy_needed', target_reliability=0.999,
component_reliability=0.9)
# -> components = 3, achieved_reliability = 0.999   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_redundancy_needed', target_reliability=0.9999,
component_reliability=0.75)
# -> components = 7, achieved_reliability = 0.999939   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Parallel components needed with required reliability = 0.999, component reliability = 0.9.	SOLVED	components = 3, achieved_reliability = 0.999
Parallel components needed with required reliability = 0.9999, component reliability = 0.75.	SOLVED	components = 7, achieved_reliability = 0.999939
Parallel components needed with required reliability = 0.999.	NEEDS_INPUT	ie_redundancy_needed needs component_reliability (reliability of one component). Example: solve('ie_redundancy_needed',
Parallel components needed with required reliability = 0.999, component reliability = 0.9, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Parallel components needed with required reliability = 1.2, component reliability = 0.9.	INVALID_INPUT	Parallel components needed for a target reliability: target_reliability must lie strictly between 0 and 1 (write 95% or

Failure rate and MTBF of a series system, `ie_system_failure_rate`

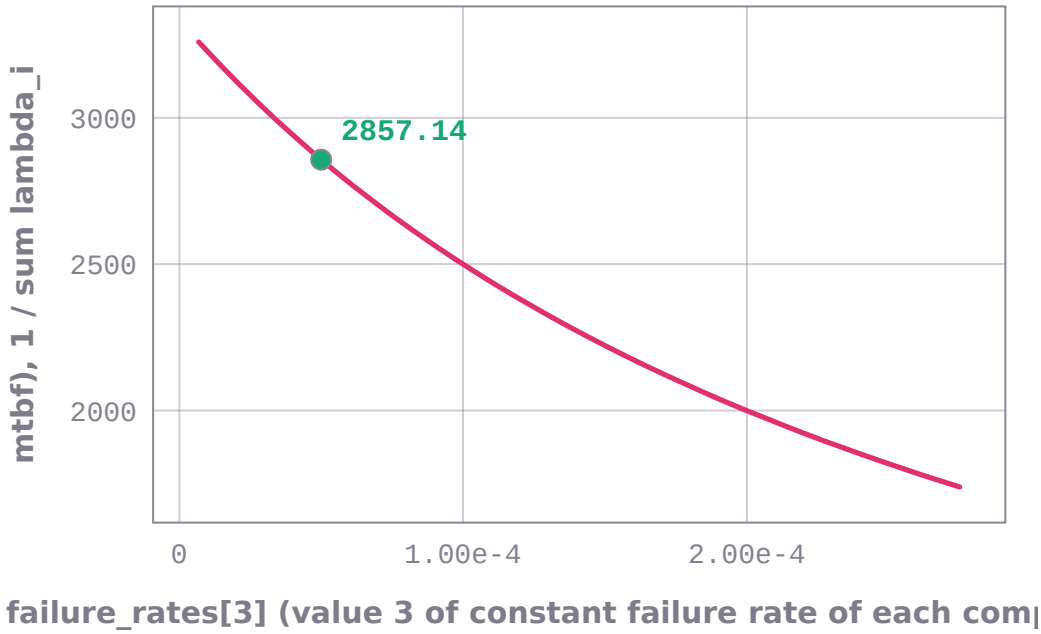
Failure rate, MTBF and mission reliability of a series system of components with constant failure rates.

$$\lambda_s = \sum_i \lambda_i, \quad MTBF_s = \frac{1}{\lambda_s}$$

input	meaning	condition
`failure_rates`	constant failure rate of each component (per hour, say)	
`time`	mission time (optional)	

output	meaning
`system_failure_rate`	sum of lambda_i
`mtbf`	1 / sum lambda_i
`reliability`	exp(-sum lambda_i t) (when a time is given)

- **Method:** lambda_s = sum lambda_i, MTBF = 1/lambda_s, R(t) = exp(-lambda_s t)
- **Assumes:** failure rates > 0; time >= 0; series system, constant failure rates
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



mtbf (1 / sum lambda_i) as value 3 of failure_rates (constant failure rate of each component (per hour, say)) moves from 0 to 0.000275, the other values as given ([0.0001, 0.0002, 5e-05]); 40 runs of industrial-engineering-ai999x. Other inputs: time = 1000. Marked: the worked example, failure_rates[3] = 5.0000e-05, mtbf = 2857.14.

```
industrial_engineering_ai999x.solve('ie_system_failure_rate', failure_rates=[0.0001,
0.0002, 5e-05], time=1000)
# -> system_failure_rate = 0.00035, mtbf = 2857.14, reliability = 0.704688   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_system_failure_rate', failure_rates=[0.002, 0.001])
# -> system_failure_rate = 0.003, mtbf = 333.333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Failure rate of a series system with component failure rates = [0.0001, 0.0002, 5e-05], mission time = 1000.	SOLVED	system_failure_rate = 0.00035, mtbf = 2857.14, reliability = 0.704688
Failure rate of a series system with component failure rates = [0.002, 0.001].	SOLVED	system_failure_rate = 0.003, mtbf = 333.333
Failure rate of a series system with component failure rates = [0.0001, 0.0002, 5e-05], mission time = 1000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Failure rate of a series system with component failure rates = [0.0001, -0.0002].	INVALID_INPUT	Failure rate and MTBF of a series system: every value of failure_rates must be positive.

MTTF of identical components in parallel, `ie_parallel_mttf`

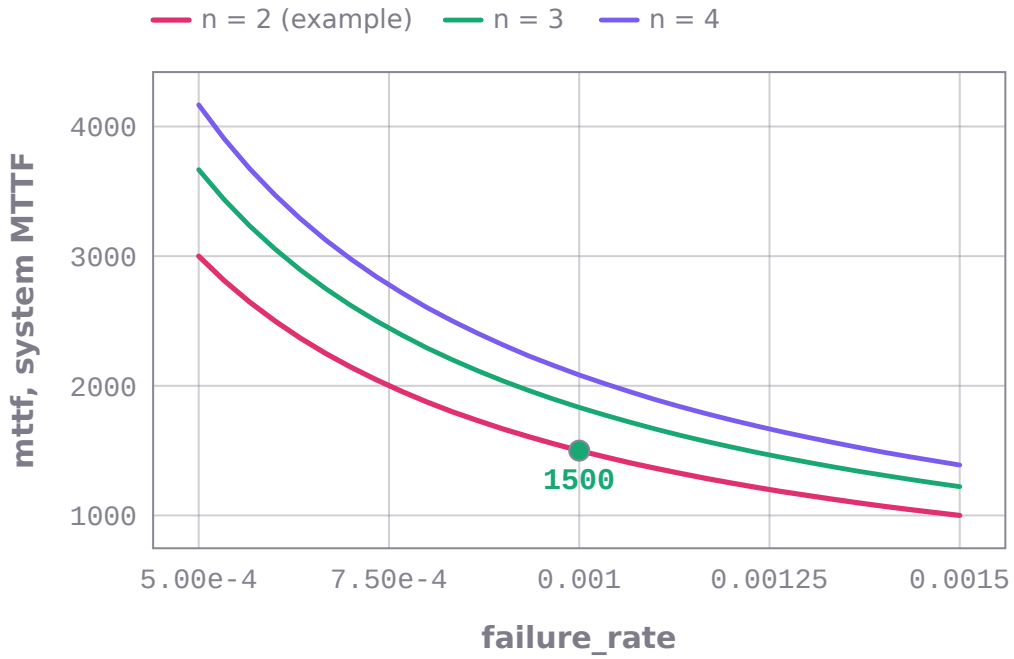
Mean time to failure of n identical exponential components in active parallel: $(1/\lambda)$ sum $1/i$.

$$MTTF = \frac{1}{\lambda} \sum_{i=1}^{n} \frac{1}{i}$$

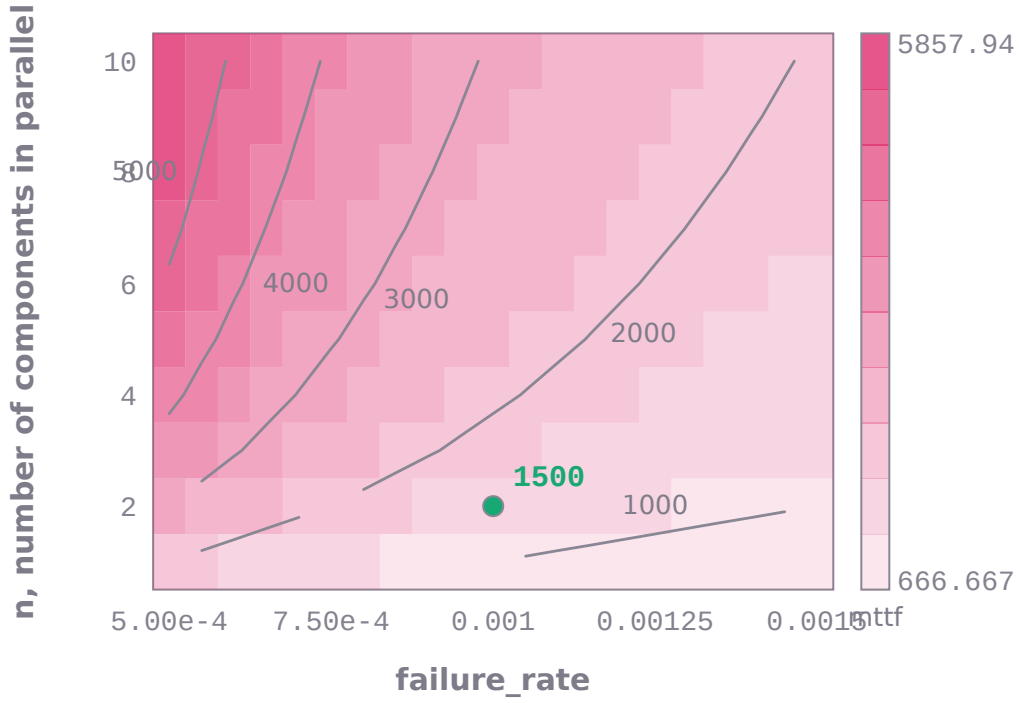
input	meaning	condition
`failure_rate`	failure rate lambda of each component	
`n`	number of components in parallel	

output	meaning
`mttf`	system MTTF $(1/\lambda)$ sum_{$i = 1$}^{n} $1/i$
`single_mttf`	$1/\lambda$
`improvement_factor`	sum $1/i$

- **Method:** MTTF = integral of 1 - (1 - e^(-lambda t))^n $dt = (1/\lambda)$ sum $1/i$
- **Assumes:** failure_rate > 0; $1 \le n \le 1000$; active redundancy, constant failure rate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



mttf (system MTTF (1/lambda) sum_{i=1}^{n} 1/i) against failure_rate (failure rate lambda of each component) from 0.0005 to 0.0015. One curve for each value of n (number of components in parallel): 2, 3, 4. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, failure_rate = 0.001, mttf = 1500.



mttf (system MTTF (1/lambda) sum_{i=1}^{n} 1/i) over failure_rate (failure rate lambda of each component) from 0.0005 to 0.0015 and n (number of components in parallel) from 1 to 10: each of the 21 × 10 cells is one run of industrial-engineering-ai999x, shaded by its value (light 666.667 to dark 5857.94); iso-lines at mttf = 1000, 2000, 3000, 4000, 5000. Marked: the worked example, failure_rate = 0.001, n = 2, mttf = 1500.

```
industrial_engineering_ai999x.solve('ie_parallel_mttf', failure_rate=0.001, n=2)
# -> mttf = 1500, single_mttf = 1000, improvement_factor = 1.5   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_parallel_mttf', failure_rate=0.0005, n=3)
# -> mttf = 3666.67, single_mttf = 2000, improvement_factor = 1.83333   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
MTTF of a parallel system with lambda = 0.001, components = 2.	SOLVED	mttf = 1500, single_mttf = 1000, improvement_factor = 1.5
MTTF of a parallel system with lambda = 0.0005, components = 3.	SOLVED	mttf = 3666.67, single_mttf = 2000, improvement_factor = 1.83333
MTTF of a parallel system with lambda = 0.001.	NEEDS_INPUT	ie_parallel_mttf needs n (number of components in parallel). Example: solve('ie_parallel_mttf', failure_rate=0.001, n=2)
MTTF of a parallel system with lambda = 0.001, components = 2, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
MTTF of a parallel system with lambda = 0.001, components = 0.	INVALID_INPUT	MTTF of identical components in parallel: n must be at least 1.

Cold standby redundancy, `ie_standby_redundancy`

Reliability and MTTF of one operating unit backed by identical cold standby units with perfect switching.

$$R(t) = e^{-\lambda t} \sum_{k=0}^{n-1} \frac{(\lambda t)^k}{k!}$$

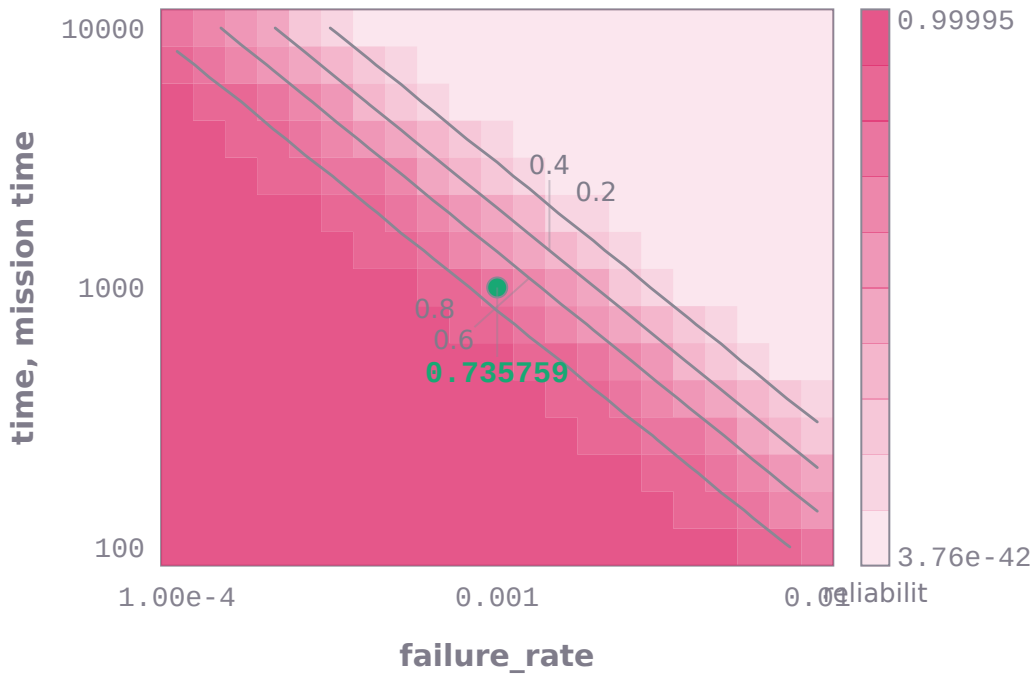
input	meaning	condition
`failure_rate`	failure rate lambda of each unit	
`time`	mission time	
`units`	total units including the operating one (default 2)	

output	meaning
`reliability`	exp(-lambda t) sum_{$k < n$} (lambda t)^k / k!
`mttf`	n/lambda

- **Method:** R(t) = $e^{(} - \lambda$ t) sum_{$k = 0$}^{$n - 1$} (lambda t)^k/k!
- **Assumes:** failure_rate > 0; time >= 0; 1 <= units <= 200; standby units do not fail, switching is perfect
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



reliability (exp(-lambda t) sum_{k<n} (lambda t)^k / k!) against failure_rate (failure rate lambda of each unit) from 0.0001 to 0.01 on a logarithmic axis. One curve for each value of time (mission time): 1000, 500, 2000. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, failure_rate = 0.001, reliability = 0.735759.



reliability (exp(-lambda t) sum_{k<n} (lambda t)^k / k!) over failure_rate (failure rate lambda of each unit) from 0.0001 to 0.01 and time (mission time) from 100 to 10000 (logarithmic axes): each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 3.7573e-42 to dark 0.99995); iso-lines at reliability = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, failure_rate = 0.001, time = 1000, reliability = 0.735759.

```
industrial_engineering_ai999x.solve('ie_standby_redundancy', failure_rate=0.001, time=1000)
# -> reliability = 0.735759, mttf = 2000   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_standby_redundancy', failure_rate=0.002, time=500,
units=3)
# -> reliability = 0.919699, mttf = 1500   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cold standby redundancy with lambda = 0.001, mission time = 1000.	SOLVED	reliability = 0.735759, mttf = 2000
Cold standby redundancy with lambda = 0.002, mission time = 500, number of units = 3.	SOLVED	reliability = 0.919699, mttf = 1500
Cold standby redundancy with lambda = 0.001.	NEEDS_INPUT	ie_standby_redundancy needs time (mission time). Example: solve('ie_standby_redundancy', failure_rate=0.001, time=1000).
Cold standby redundancy with lambda = 0.001, mission time = 1000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Cold standby redundancy with lambda = 0.001, mission time = -5.	INVALID_INPUT	Cold standby redundancy: time must not be negative.

Confidence bounds on MTBF (chi-square), `ie_mtbf_confidence`

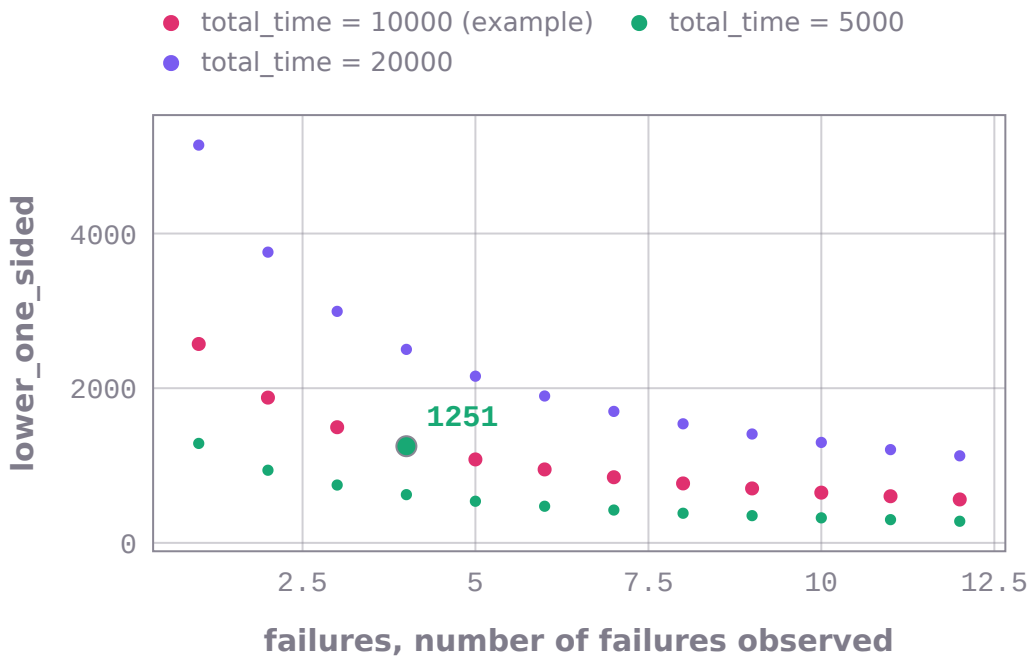
Point estimate and chi-square confidence bounds of the MTBF from a life test with constant failure rate.

$$MTBF_L = \frac{2T}{\chi^2_{C,\,2r+2}}, \quad MTBF_U = \frac{2T}{\chi^2_{\alpha/2,\,2r}}$$

input	meaning	condition
`total_time`	total accumulated test time (all units)	
`failures`	number of failures observed	
`confidence`	confidence level (decimal)	
`test`	time (test stopped at a set time) or failure (stopped at the r-th failure); default time	

output	meaning
`mtbf`	point estimate T/r
`lower_one_sided`	one-sided lower bound $2T/chi2$(C; d)
`lower_two_sided`	two-sided lower bound
`upper_two_sided`	two-sided upper bound $2T/chi2(a/2$; 2r)

- **Method:** MTBF_L = 2T / chi2_{C, d}; two-sided: Missing superscript or subscript argument{$1 - a/2$, d} and Missing superscript or subscript argument{$a/2$, 2r}
- **Assumes:** total_time > 0; failures >= 1; 0 < confidence < 1; $d = 2r + 2$ for a time-terminated test, 2r for a failure-terminated test
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



lower_one_sided (one-sided lower bound 2T/chi2(C; d)) against failures (number of failures observed) from 1 to 12. One curve for each value of total_time (total accumulated test time (all units)): 10000, 5000, 20000. Other inputs as in the worked example: confidence = 0.9, test = time. 12 points per curve computed by industrial-engineering-ai999x; marked: the worked example, failures = 4, lower_one_sided = 1251.

```
industrial_engineering_ai999x.solve('ie_mtbf_confidence', total_time=10000, failures=4,
confidence=0.9, test='time')
# -> mtbf = 2500, lower_one_sided = 1251, lower_two_sided = 1092.48, upper_two_sided =
7318.94   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_mtbf_confidence', total_time=5000, failures=2,
confidence=0.95, test='failure')
# -> mtbf = 2500, lower_one_sided = 1053.99, lower_two_sided = 897.401, upper_two_sided =
20643.3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Confidence bounds on MTBF with total test time = 10000, number of failures = 4, confidence level = 0.9, test type = time.	SOLVED	mtbf = 2500, lower_one_sided = 1251, lower_two_sided = 1092.48, upper_two_sided = 7318.94
Confidence bounds on MTBF with total test time = 5000, number of failures = 2, confidence level = 0.95, test type = failure.	SOLVED	mtbf = 2500, lower_one_sided = 1053.99, lower_two_sided = 897.401, upper_two_sided = 20643.3
Confidence bounds on MTBF with total test time = 10000, number of failures = 4, test type = time.	NEEDS_INPUT	ie_mtbf_confidence needs confidence (confidence level (decimal)). Example: solve('ie_mtbf_confidence', total_time=10000,
Confidence bounds on MTBF with total test time = 10000, number of failures = 4, confidence level = 0.9, test type = time, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Confidence bounds on MTBF with total test time = 10000, number of failures = 4, confidence level = 1.9.	INVALID_INPUT	Confidence bounds on MTBF (chi-square): confidence must lie strictly between 0 and 1 (write 95% or 0.95).

M/M/1 queue, `ie_mm1_queue`

Utilisation, mean numbers in system and in queue, and mean times of a single-server queue with Poisson arrivals and exponential service.

$$\rho = \frac{\lambda}{\mu},\ L = \frac{\rho}{1-\rho},\ W = \frac{1}{\mu-\lambda},\ L_q = \frac{\rho^2}{1-\rho}$$

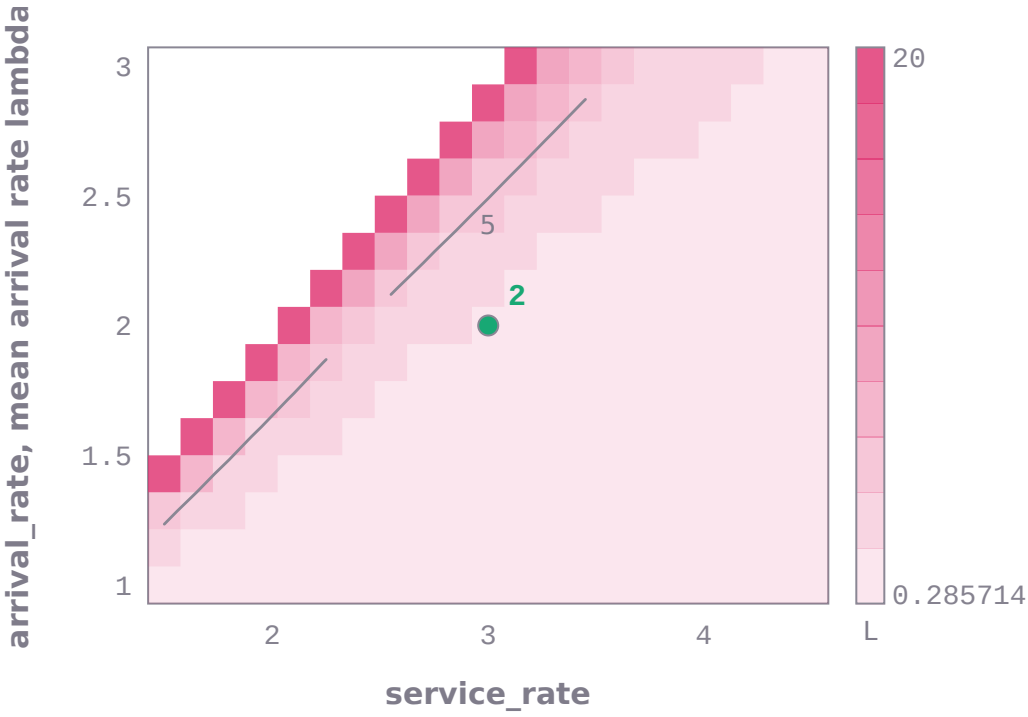
input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	

output	meaning
`utilization`	$\rho = \lambda/mu$
`L`	mean number in system $\rho/(1-\rho)$
`Lq`	mean number waiting $\rho^2/(1-\rho)$
`W`	mean time in system $1/(mu-\lambda)$
`Wq`	mean wait in queue rho/(mu - lambda)
`p0`	probability the system is empty

- **Method:** $\rho = \lambda/mu$, $L = \rho/(1-\rho)$, $W = 1/(mu-\lambda)$, $Lq = \rho$ L, $Wq = \rho$ W
- **Assumes:** arrival_rate < service_rate; Poisson arrivals, exponential service, FCFS, infinite queue
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L (mean number in system rho/(1 - rho)) against arrival_rate (mean arrival rate lambda (per time unit)) from 1 to 3. One curve for each value of service_rate (mean service rate mu of one server (per time unit)): 3, 1.5, 6. 30 points per curve computed by industrial-engineering-ai999x; marked: the worked example, arrival_rate = 2, L = 2.



L (mean number in system rho/(1 - rho)) over service_rate (mean service rate mu of one server (per time unit)) from 1.5 to 4.5 and arrival_rate (mean arrival rate lambda (per time unit)) from 1 to 3: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.285714 to dark 20); iso-lines at L = 5; blank cells: inputs the capability refuses. Marked: the worked example, service_rate = 3, arrival_rate = 2, L = 2.

```
industrial_engineering_ai999x.solve('ie_mm1_queue', arrival_rate=2, service_rate=3)
# -> utilization = 0.666667, L = 2, Lq = 1.33333, W = 1, Wq = 0.666667, p0 = 0.333333
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_mm1_queue', arrival_rate=18, service_rate=20)
# -> utilization = 0.9, L = 9, Lq = 8.1, W = 0.5, Wq = 0.45, p0 = 0.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
M/M/1 queue with lambda = 2, service rate = 3.	SOLVED	utilization = 0.666667, L = 2, Lq = 1.33333, W = 1, Wq = 0.666667, p0 = 0.333333
M/M/1 queue with lambda = 18, service rate = 20.	SOLVED	utilization = 0.9, L = 9, Lq = 8.1, W = 0.5, Wq = 0.45, p0 = 0.1
M/M/1 queue with lambda = 2.	NEEDS_INPUT	ie_mm1_queue needs service_rate (mean service rate mu of one server (per time unit)). Example: solve('ie_mm1_queue', arr
M/M/1 queue with lambda = 2, service rate = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
M/M/1 queue with lambda = 5, service rate = 3.	INVALID_INPUT	M/M/1 queue: the queue is stable only if the arrival rate is below the service rate.

M/M/c queue (Erlang C), `ie_mmc_queue`

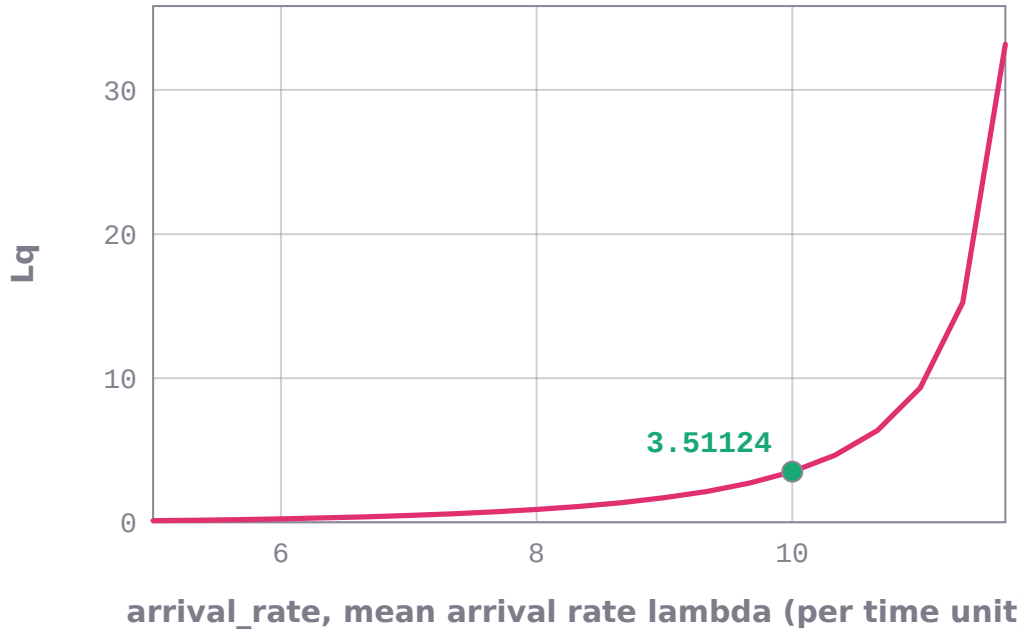
Probability of waiting (Erlang C), mean queue length and mean waits of a multi-server queue with Poisson arrivals and exponential service.

$$C(c,a) = \frac{\frac{a^c}{c!(1-\rho)}}{\sum_{k=0}^{c-1} \frac{a^k}{k!} + \frac{a^c}{c!(1-\rho)}}, \quad L_q = C\frac{\rho}{1-\rho}$$

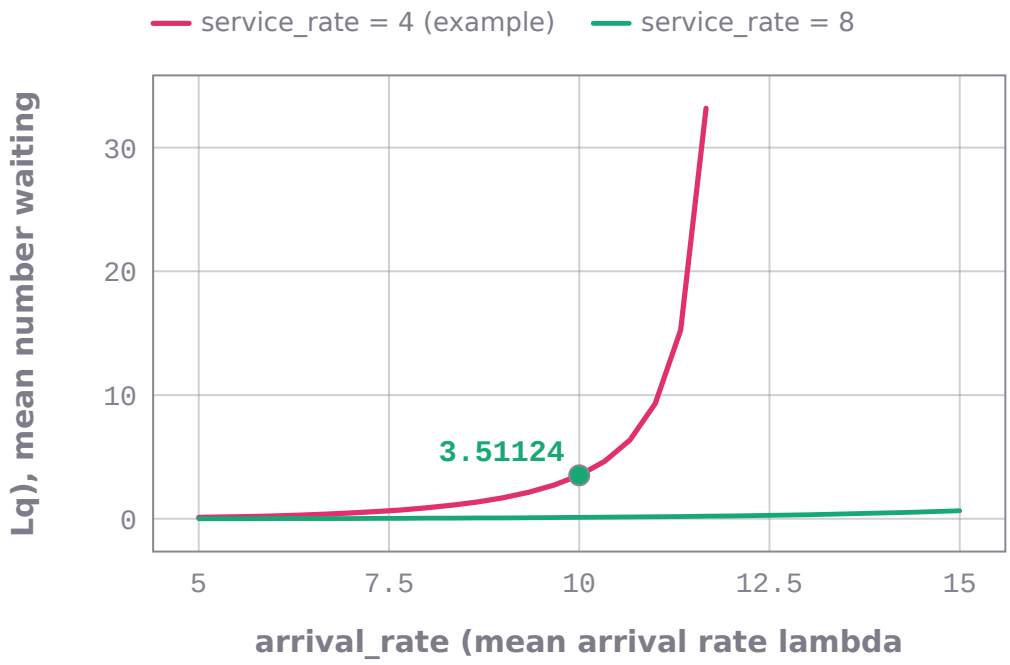
input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	
`servers`	number of servers c	

output	meaning
`utilization`	rho = lambda/(c mu)
`p_wait`	Erlang C probability of waiting
`Lq`	mean number waiting
`Wq`	mean wait in queue
`W`	mean time in system
`L`	mean number in system
`p0`	probability the system is empty

- **Method:** $a = \lambda/mu$, P0 = [sum_{$k < c$} $a^k/k!$ + a^c/(c!$(1-\rho)$)]^-1, $C = a^c$ P0/(c!$(1-\rho)$), $Lq = C\,\rho/(1-\rho)$
- **Assumes:** arrival_rate < servers x service_rate; 1 <= servers <= 500
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lq (mean number waiting) against arrival_rate (mean arrival rate lambda (per time unit)), with servers = 3, service_rate = 4. 21 points computed by industrial-engineering-ai999x; marked: arrival_rate = 10, Lq = 3.51124.



Lq (mean number waiting) against arrival_rate (mean arrival rate lambda (per time unit)) from 5 to 15. One curve for each value of service_rate (mean service rate mu of one server (per time unit)): 4, 8. Other inputs as in the worked example: servers = 3. 21 points per curve computed by industrial-engineering-ai999x; marked: the worked example, arrival_rate = 10, Lq = 3.51124.

```
industrial_engineering_ai999x.solve('ie_mmc_queue', arrival_rate=10, service_rate=4,
servers=3)
# -> utilization = 0.833333, p_wait = 0.702247, Lq = 3.51124, Wq = 0.351124, W = 0.601124,
L = 6.01124, p0 = 0.0449438   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_mmc_queue', arrival_rate=45, service_rate=6,
servers=9)
# -> utilization = 0.833333, p_wait = 0.509149, Lq = 2.54574, Wq = 0.0565721, W = 0.223239,
L = 10.0457, p0 = 0.000410114   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
M/M/c queue with lambda = 10, service rate = 4, number of servers = 3.	SOLVED	utilization = 0.833333, p_wait = 0.702247, Lq = 3.51124, Wq = 0.351124, W = 0.601124, L = 6.01124, p0 = 0.0449438
M/M/c queue with lambda = 45, service rate = 6, number of servers = 9.	SOLVED	utilization = 0.833333, p_wait = 0.509149, Lq = 2.54574, Wq = 0.0565721, W = 0.223239, L = 10.0457, p0 = 0.000410114
M/M/c queue with lambda = 10, service rate = 4.	NEEDS_INPUT	ie_mmc_queue needs servers (number of servers c). Example: solve('ie_mmc_queue', arrival_rate=10, service_rate=4, server
M/M/c queue with lambda = 10, service rate = 4, number of servers = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
M/M/c queue with lambda = 20, service rate = 4, number of servers = 3.	INVALID_INPUT	M/M/c queue (Erlang C): the queue is stable only if the arrival rate is below servers x service rate.

M/D/1 queue (constant service time), `ie_md1_queue`

Mean queue length and waits of a single-server queue with Poisson arrivals and a constant service time.

$$L_q = \frac{\rho^2}{2(1-\rho)}$$

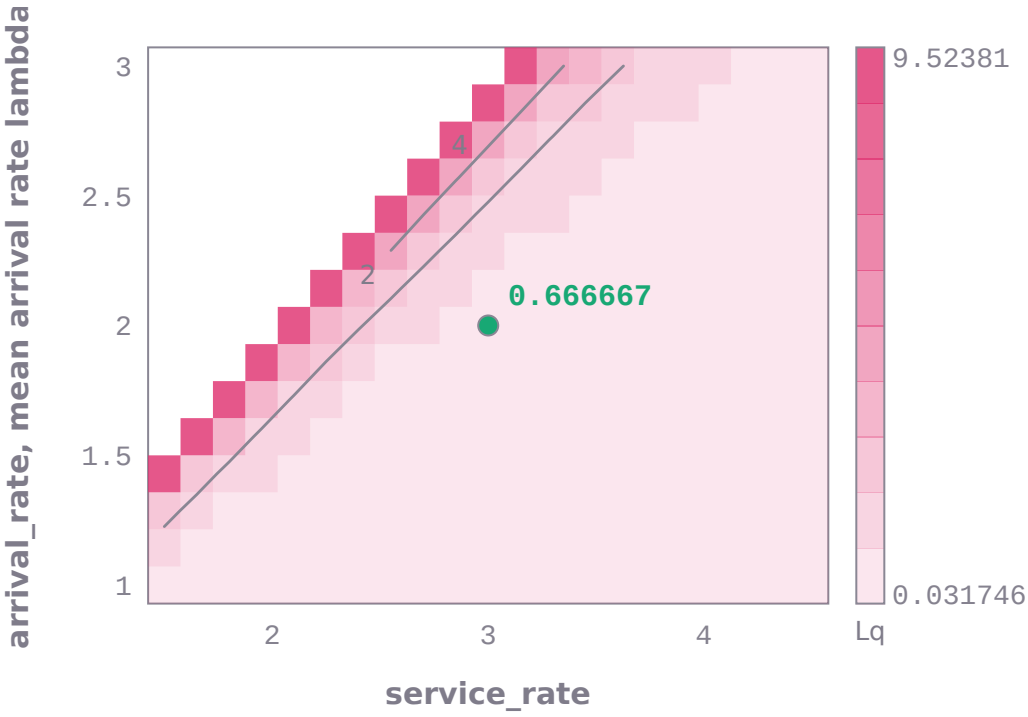
input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	

output	meaning
`utilization`	$\rho = \lambda / mu$
`Lq`	mean number waiting rho^2/(2(1 - rho))
`Wq`	mean wait in queue Lq/lambda
`W`	mean time in system $Wq + 1/mu$
`L`	mean number in system Lq + rho

- **Method:** Lq = rho^2 / (2 $(1-\rho)$), $Wq = Lq/\lambda$
- **Assumes:** arrival_rate < service_rate; constant service time $1/mu$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lq (mean number waiting rho^2/(2(1 - rho))) against arrival_rate (mean arrival rate lambda (per time unit)) from 1 to 3. One curve for each value of service_rate (mean service rate mu of one server (per time unit)): 3, 1.5, 6. 30 points per curve computed by industrial-engineering-ai999x; marked: the worked example, arrival_rate = 2, Lq = 0.666667.



Lq (mean number waiting rho^2/(2(1 - rho))) over service_rate (mean service rate mu of one server (per time unit)) from 1.5 to 4.5 and arrival_rate (mean arrival rate lambda (per time unit)) from 1 to 3: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 0.031746 to dark 9.52381); iso-lines at Lq = 2, 4; blank cells: inputs the capability refuses. Marked: the worked example, service_rate = 3, arrival_rate = 2, Lq = 0.666667.

```
industrial_engineering_ai999x.solve('ie_md1_queue', arrival_rate=2, service_rate=3)
# -> utilization = 0.666667, Lq = 0.666667, Wq = 0.333333, W = 0.666667, L = 1.33333
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_md1_queue', arrival_rate=8, service_rate=10)
# -> utilization = 0.8, Lq = 1.6, Wq = 0.2, W = 0.3, L = 2.4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
M/D/1 queue with lambda = 2, service rate = 3.	SOLVED	utilization = 0.666667, Lq = 0.666667, Wq = 0.333333, W = 0.666667, L = 1.33333
M/D/1 queue with lambda = 8, service rate = 10.	SOLVED	utilization = 0.8, Lq = 1.6, Wq = 0.2, W = 0.3, L = 2.4
M/D/1 queue with lambda = 2.	NEEDS_INPUT	ie_md1_queue needs service_rate (mean service rate mu of one server (per time unit)). Example: solve('ie_md1_queue', arr
M/D/1 queue with lambda = 2, service rate = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
M/D/1 queue with lambda = 3, service rate = 3.	INVALID_INPUT	M/D/1 queue (constant service time): the queue is stable only if arrival rate x mean service time is below 1.

M/G/1 queue (Pollaczek-Khinchine), `ie_mg1_queue`

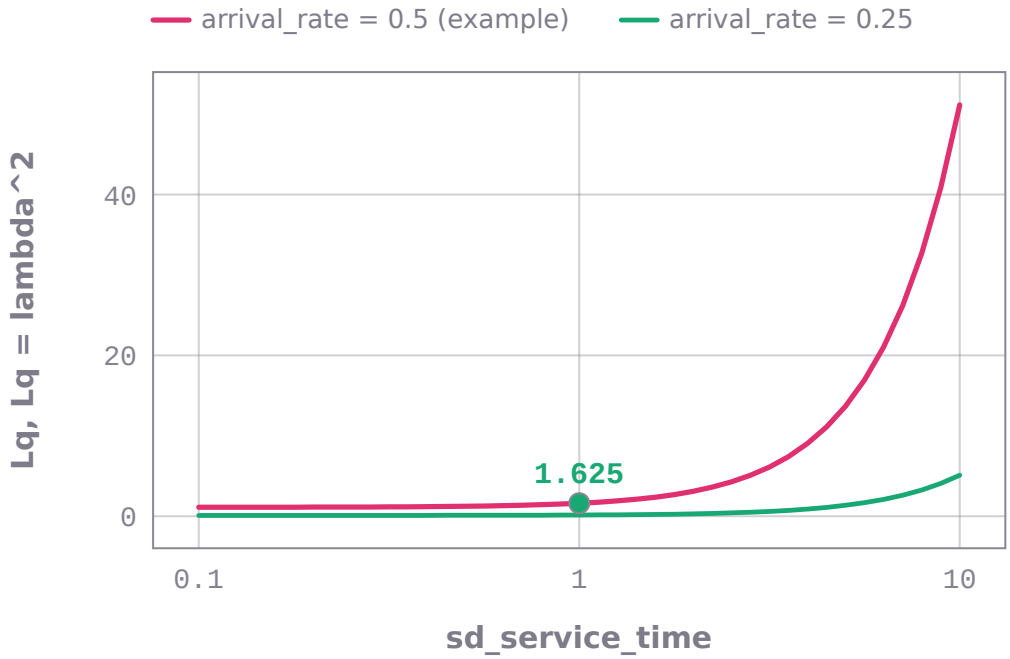
Mean queue length and waits of a single-server queue with Poisson arrivals and a general service time of given mean and standard deviation.

$$L_q = \frac{\lambda^2 \sigma^2 + \rho^2}{2(1-\rho)}$$

input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`mean_service_time`	mean service time E[S]	
`sd_service_time`	standard deviation of the service time	

output	meaning
`utilization`	$\rho = \lambda$ E[S]
`Lq`	$Lq = \lambda^2$ (sigma^2 + E[S]^$2)/(2$(1 - rho))
`Wq`	mean wait in queue
`W`	mean time in system
`L`	mean number in system

- **Method:** Pollaczek-Khinchine: Lq = (lambda^2 $\sigma^2 + \rho^2)/(2$(1 - rho)), $Wq = Lq/\lambda$, $W = Wq + E$[S]
- **Assumes:** lambda E[S] < 1; sd_service_time >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Lq (Lq = lambda^2 (sigma^2 + E[S]^2)/(2(1 - rho))) against sd_service_time (standard deviation of the service time) from 0.1 to 10 on a logarithmic axis. One curve for each value of arrival_rate (mean arrival rate lambda (per time unit)): 0.5, 0.25. Other inputs as in the worked example: mean_service_time = 1.5. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, sd_service_time = 1, Lq = 1.625.

```
industrial_engineering_ai999x.solve('ie_mg1_queue', arrival_rate=0.5,
mean_service_time=1.5, sd_service_time=1)
# -> utilization = 0.75, Lq = 1.625, Wq = 3.25, W = 4.75, L = 2.375   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_mg1_queue', arrival_rate=10,
mean_service_time=0.08, sd_service_time=0.05)
# -> utilization = 0.8, Lq = 2.225, Wq = 0.2225, W = 0.3025, L = 3.025   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
M/G/1 queue with lambda = 0.5, mean service time = 1.5, standard deviation of service time = 1.	SOLVED	utilization = 0.75, Lq = 1.625, Wq = 3.25, W = 4.75, L = 2.375
M/G/1 queue with lambda = 10, mean service time = 0.08, standard deviation of service time = 0.05.	SOLVED	utilization = 0.8, Lq = 2.225, Wq = 0.2225, W = 0.3025, L = 3.025
M/G/1 queue with lambda = 0.5, mean service time = 1.5.	NEEDS_INPUT	ie_mg1_queue needs sd_service_time (standard deviation of the service time). Example: solve('ie_mg1_queue', arrival_rate
M/G/1 queue with lambda = 0.5, mean service time = 1.5, standard deviation of service time = 1, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
M/G/1 queue with lambda = 1, mean service time = 1.5, standard deviation of service time = 1.	INVALID_INPUT	M/G/1 queue (Pollaczek-Khinchine): the queue is stable only if arrival rate x mean service time is below 1.

M/M/1/K queue (finite capacity), `ie_mm1k_queue`

Single-server queue with room for at most K customers in the system: blocking probability, effective arrival rate, mean numbers and times.

$$P_n = \frac{\rho^n}{\sum_{k=0}^{K} \rho^k}, \quad \lambda_{eff} = \lambda(1 - P_K)$$

input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	
`capacity`	maximum customers in the system K (including the one in service)	

output	meaning
`p0`	probability empty
`p_block`	probability an arrival is turned away P_K
`L`	mean number in system
`Lq`	mean number waiting
`effective_arrival_rate`	lambda $(1 - P_K)$
`W`	mean time in system L/lambda_eff
`Wq`	mean wait Lq/lambda_eff
`utilization`	$1 - P_0$

- **Method:** P_n = rho^n / sum_{$k = 0$}^{K} ρ^k; lambda_eff = lambda $(1 - P_K)$; Little's law
- **Assumes:** arrival_rate, service_rate > 0 (rho may exceed 1); 1 <= capacity <= 1000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



L (mean number in system) against arrival_rate (mean arrival rate lambda (per time unit)) from 0.4 to 40 on a logarithmic axis. One curve for each value of service_rate (mean service rate mu of one server (per time unit)): 5, 2.5, 10. Other inputs as in the worked example: capacity = 3. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, arrival_rate = 4, L = 1.22493.

```
industrial_engineering_ai999x.solve('ie_mm1k_queue', arrival_rate=4, service_rate=5,
capacity=3)
# -> p0 = 0.338753, p_block = 0.173442, L = 1.22493, Lq = 0.563686, effective_arrival_rate
= 3.30623, W = 0.370492, Wq = 0.170492, utilization = 0.661247   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_mm1k_queue', arrival_rate=12, service_rate=10,
capacity=6)
# -> p0 = 0.0774239, p_block = 0.231187, L = 3.70984, Lq = 2.78726, effective_arrival_rate
= 9.22576, W = 0.402117, Wq = 0.302117, utilization = 0.922576   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
M/M/1/K queue with lambda = 4, service rate = 5, system capacity = 3.	SOLVED	p0 = 0.338753, p_block = 0.173442, L = 1.22493, Lq = 0.563686, effective_arrival_rate = 3.30623, W = 0.370492, Wq = 0.170492, utilization = 0.661247
M/M/1/K queue with lambda = 12, service rate = 10, system capacity = 6.	SOLVED	p0 = 0.0774239, p_block = 0.231187, L = 3.70984, Lq = 2.78726, effective_arrival_rate = 9.22576, W = 0.402117, Wq = 0.302117, utilization = 0.922576
M/M/1/K queue with lambda = 4, service rate = 5.	NEEDS_INPUT	ie_mm1k_queue needs capacity (maximum customers in the system K (including the one in service)). Example: solve('ie_mm1k
M/M/1/K queue with lambda = 4, service rate = 5, system capacity = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
M/M/1/K queue with lambda = 4, service rate = 5, system capacity = 0.	INVALID_INPUT	M/M/1/K queue (finite capacity): capacity must be at least 1.

Servers needed for a service level (Erlang C), `ie_erlang_c_staffing`

Fewest servers of an M/M/c queue such that the probability of waiting no longer than a target time reaches a target service level (call-centre staffing).

$$P(W_q \le t) = 1 - C(c, a)\, e^{-(c\mu - \lambda)t}$$

input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	
`target_time`	target answer time in the same time unit	
`target_level`	required fraction answered within the target time (decimal)	

output	meaning
`servers`	fewest servers meeting the target
`service_level`	P(wait <= t) achieved
`p_wait`	Erlang C probability of waiting
`utilization`	lambda/(c mu)
`mean_wait`	mean wait in queue

- **Method:** smallest $c > \lambda/mu$ with $1 - C$(c, a) exp(-(c mu - lambda) t) >= target
- **Assumes:** arrival_rate, service_rate > 0; target_time >= 0; 0 < target_level < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



service_level (P(wait <= t) achieved) against service_rate (mean service rate mu of one server (per time unit)) from 1.2 to 120 on a logarithmic axis. One curve for each value of target_level (required fraction answered within the target time (decimal)): 0.8, 0.26, 0.74. Other inputs as in the worked example: arrival_rate = 100, target_time = 0.0333. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, service_rate = 12, service_level = 0.896775.

```
industrial_engineering_ai999x.solve('ie_erlang_c_staffing', arrival_rate=100,
service_rate=12, target_time=0.0333, target_level=0.8)
# -> servers = 11, service_level = 0.896775, p_wait = 0.299619, utilization = 0.757576,
mean_wait = 0.0093631   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_erlang_c_staffing', arrival_rate=30,
service_rate=10, target_time=0.05, target_level=0.9)
# -> servers = 5, service_level = 0.913125, p_wait = 0.236152, utilization = 0.6, mean_wait
= 0.0118076   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Servers needed for a service level with lambda = 100, service rate = 12, answer time = 0.0333, service level target = 0.8.	SOLVED	servers = 11, service_level = 0.896775, p_wait = 0.299619, utilization = 0.757576, mean_wait = 0.0093631
Servers needed for a service level with lambda = 30, service rate = 10, answer time = 0.05, service level target = 0.9.	SOLVED	servers = 5, service_level = 0.913125, p_wait = 0.236152, utilization = 0.6, mean_wait = 0.0118076
Servers needed for a service level with lambda = 100, service rate = 12, answer time = 0.0333.	NEEDS_INPUT	ie_erlang_c_staffing needs target_level (required fraction answered within the target time (decimal)). Example: solve('i
Servers needed for a service level with lambda = 100, service rate = 12, answer time = 0.0333, service level target = 0.8, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Servers needed for a service level with lambda = 100, service rate = 12, answer time = 0.0333, service level target = 1.8.	INVALID_INPUT	Servers needed for a service level (Erlang C): target_level must lie strictly between 0 and 1 (write 95% or 0.95).

Cost-optimal number of servers, `ie_queue_server_cost`

Number of servers of an M/M/c queue that minimises server cost plus the cost of customers waiting in the system per time unit.

$$TC(c) = c\,C_s + C_w\,L(c)$$

input	meaning	condition
`arrival_rate`	mean arrival rate lambda (per time unit)	
`service_rate`	mean service rate mu of one server (per time unit)	
`server_cost_rate`	cost of one server per time unit	
`waiting_cost_rate`	cost of one customer in the system per time unit	
`max_servers`	largest number of servers considered (default 50)	

output	meaning
`servers`	cost-minimising number of servers
`total_cost`	c C_s + C_w L at that number
`L`	mean number in system at that number

- **Method:** for each stable c: L from Erlang C; total cost c C_s + C_w L; the smallest wins
- **Assumes:** arrival_rate, service_rate > 0; costs >= 0; ties go to fewer servers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



total_cost (c C_s + C_w L at that number) against waiting_cost_rate (cost of one customer in the system per time unit) from 4 to 400 on a logarithmic axis. One curve for each value of arrival_rate (mean arrival rate lambda (per time unit)): 20, 10, 40. Other inputs as in the worked example: service_rate = 6, server_cost_rate = 25. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, waiting_cost_rate = 40, total_cost = 284.467.

```
industrial_engineering_ai999x.solve('ie_queue_server_cost', arrival_rate=20,
service_rate=6, server_cost_rate=25, waiting_cost_rate=40)
# -> servers = 5, total_cost = 284.467, L = 3.98667   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_queue_server_cost', arrival_rate=8, service_rate=3,
server_cost_rate=15, waiting_cost_rate=20)
# -> servers = 4, total_cost = 128.47, L = 3.4235   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Optimal number of servers with lambda = 20, service rate = 6, server cost = 25, waiting cost = 40.	SOLVED	servers = 5, total_cost = 284.467, L = 3.98667
Optimal number of servers with lambda = 8, service rate = 3,	SOLVED	servers = 4, total_cost = 128.47, L = 3.4235

question	outcome	result
server cost = 15, waiting cost = 20.		
Optimal number of servers with lambda = 20, service rate = 6, server cost = 25.	NEEDS_INPUT	ie_queue_server_cost needs waiting_cost_rate (cost of one customer in the system per time unit). Example: solve('ie_queu
Optimal number of servers with lambda = 20, service rate = 6, server cost = 25, waiting cost = 40, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Optimal number of servers with lambda = 20, service rate = 6, server cost = -25, waiting cost = 40.	INVALID_INPUT	Cost-optimal number of servers: server_cost_rate must not be negative.

Little's law, `ie_littles_law`

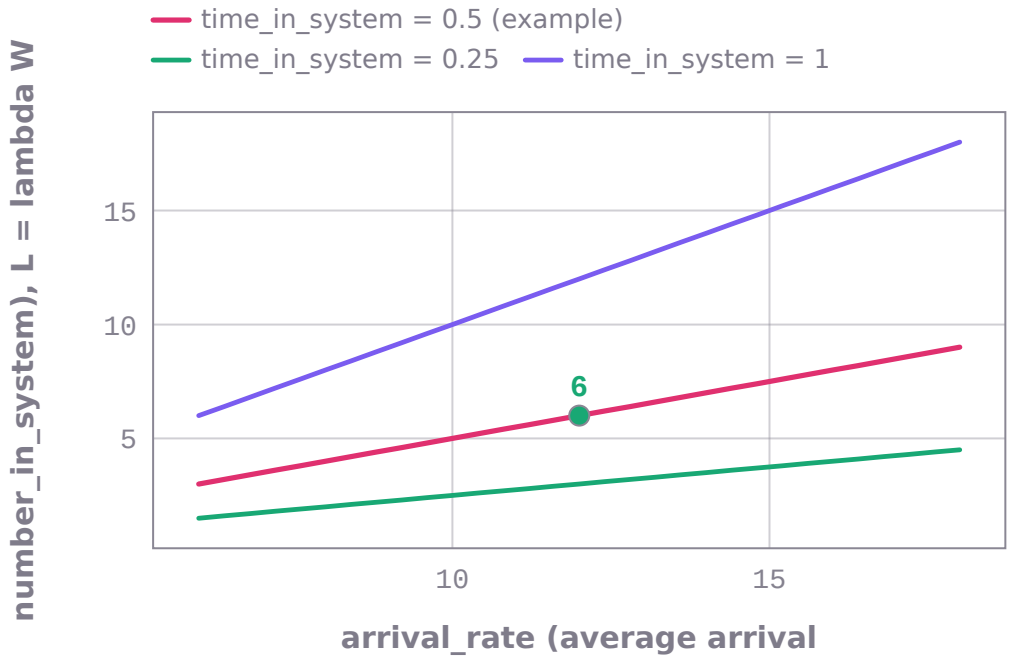
$L = \lambda$ W: the average number in a system equals the arrival rate times the average time spent; give any two to get the third.

$$L = \lambda W$$

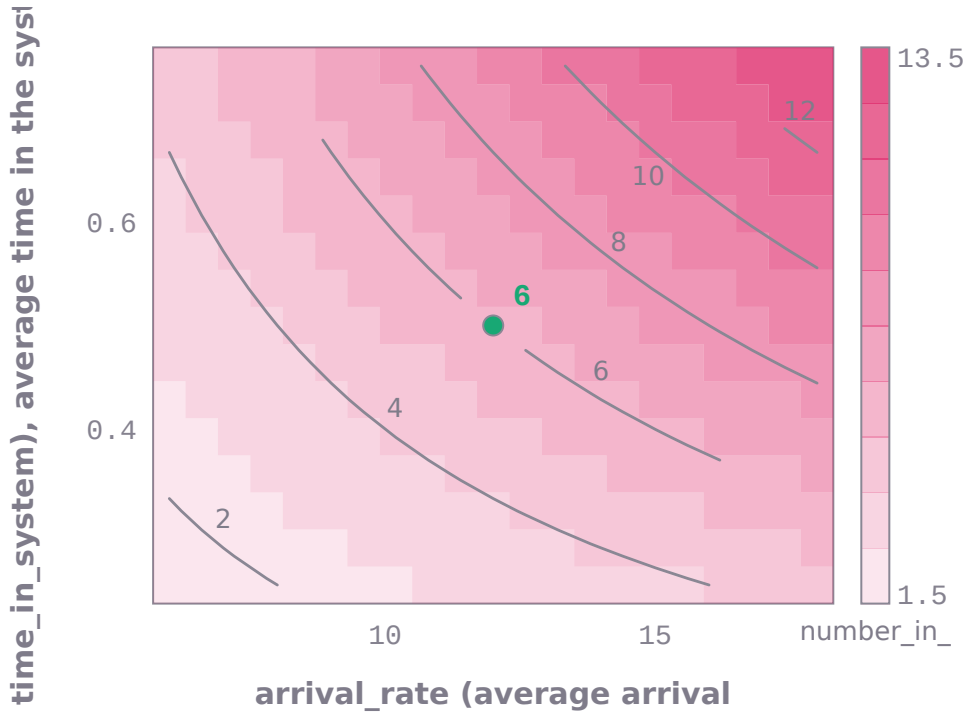
input	meaning	condition
`arrival_rate`	average arrival (throughput) rate	
`number_in_system`	average number in the system (WIP)	
`time_in_system`	average time in the system (flow time)	

output	meaning
`arrival_rate`	lambda
`number_in_system`	$L = \lambda$ W
`time_in_system`	$W = L/\lambda$

- **Method:** $L = \lambda$ W
- **Assumes:** exactly two of the three given; consistent time units; steady state
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



number_in_system (L = lambda W) against arrival_rate (average arrival (throughput) rate) from 6 to 18. One curve for each value of time_in_system (average time in the system (flow time)): 0.5, 0.25, 1. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, arrival_rate = 12, number_in_system = 6.



number_in_system (L = lambda W) over arrival_rate (average arrival (throughput) rate) from 6 to 18 and time_in_system (average time in the system (flow time)) from 0.25 to 0.75: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 1.5 to dark 13.5); iso-lines at number_in_system = 2, 4, 6, 8, 10, 12. Marked: the worked example, arrival_rate = 12, time_in_system = 0.5, number_in_system = 6.

```
industrial_engineering_ai999x.solve('ie_littles_law', arrival_rate=12, time_in_system=0.5)
# -> arrival_rate = 12, number_in_system = 6, time_in_system = 0.5   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_littles_law', number_in_system=300,
time_in_system=2.5)
# -> arrival_rate = 120, number_in_system = 300, time_in_system = 2.5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Little's law with lambda = 12, average time in system = 0.5.	SOLVED	arrival_rate = 12, number_in_system = 6, time_in_system = 0.5
Little's law with average number in system = 300, average time in system = 2.5.	SOLVED	arrival_rate = 120, number_in_system = 300, time_in_system = 2.5
Little's law with lambda = 12, average time in system = 0.5, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Little's law with lambda = 12, average time in system = 0.5, average number in system = 6.	INVALID_INPUT	Little's law: give exactly two of arrival rate, number in system and time in system.

Shortest processing time (SPT) sequence, `ie_spt_sequence`

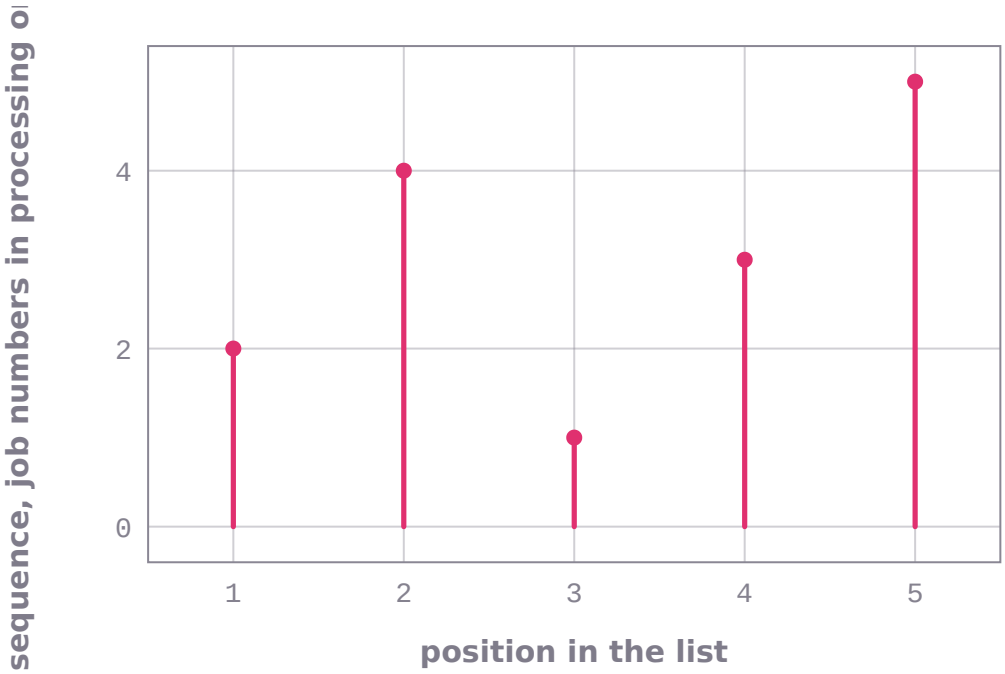
Order jobs on one machine by shortest processing time first (minimises mean flow time) and report the usual measures.

$$p_{[1]} \le p_{[2]} \le \dots, \quad \bar{F} = \frac{1}{n}\sum_{k} C_{[k]}$$

input	meaning	condition
`processing_times`	processing time of each job (jobs numbered 1, 2, ... in this order)	
`due_dates`	due date of each job (same order)	

output	meaning
`sequence`	job numbers in processing order
`mean_flow_time`	average completion (flow) time
`makespan`	completion time of the last job
`average_jobs_in_system`	sum of completion times / makespan
`utilization`	sum of processing times / sum of completion times
`total_tardiness`	sum of tardiness
`mean_tardiness`	average tardiness (job lateness)
`max_tardiness`	largest tardiness
`max_lateness`	largest lateness C - d
`tardy_jobs`	number of late jobs

- **Method:** sort by processing time; completion C_k = sum of the first k times; flow = mean C
- **Assumes:** processing times > 0; all jobs available at time 0; ties go to the lower job number; tardiness measures only when due dates are given
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



job numbers in processing order (sequence) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 2, last 5, largest 5). Inputs: processing_times = [6, 2, 8, 3, 9].

```
industrial_engineering_ai999x.solve('ie_spt_sequence', processing_times=[6, 2, 8, 3, 9])
# -> sequence = [2, 4, 1, 3, 5], mean_flow_time = 13, makespan = 28, average_jobs_in_system
= 2.32143, utilization = 0.430769   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_spt_sequence', processing_times=[4, 7, 2, 5],
due_dates=[10, 15, 5, 12])
# -> sequence = [3, 1, 4, 2], mean_flow_time = 9.25, makespan = 18, average_jobs_in_system
= 2.05556, utilization = 0.486486, total_tardiness = 3, mean_tardiness = 0.75,
max_tardiness = 3, max_lateness = 3, tardy_jobs = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
SPT sequence with processing times = [6, 2, 8, 3, 9].	SOLVED	sequence = [2, 4, 1, 3, 5], mean_flow_time = 13, makespan = 28, average_jobs_in_system = 2.32143, utilization = 0.430769
SPT sequence with processing times = [4, 7, 2, 5], due dates = [10, 15, 5, 12].	SOLVED	sequence = [3, 1, 4, 2], mean_flow_time = 9.25, makespan = 18, average_jobs_in_system = 2.05556, utilization = 0.486486, total_tardiness = 3, mean_tardiness = 0.75, max_tardiness = 3, max_lateness = 3, tardy_jobs = 1
SPT sequence with processing times = [6, 2, 8, 3, 9], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
SPT sequence with processing times = [6, -2, 8].	INVALID_INPUT	Shortest processing time (SPT) sequence: every value of processing_times must be positive.

Earliest due date (EDD) sequence, `ie_edd_sequence`

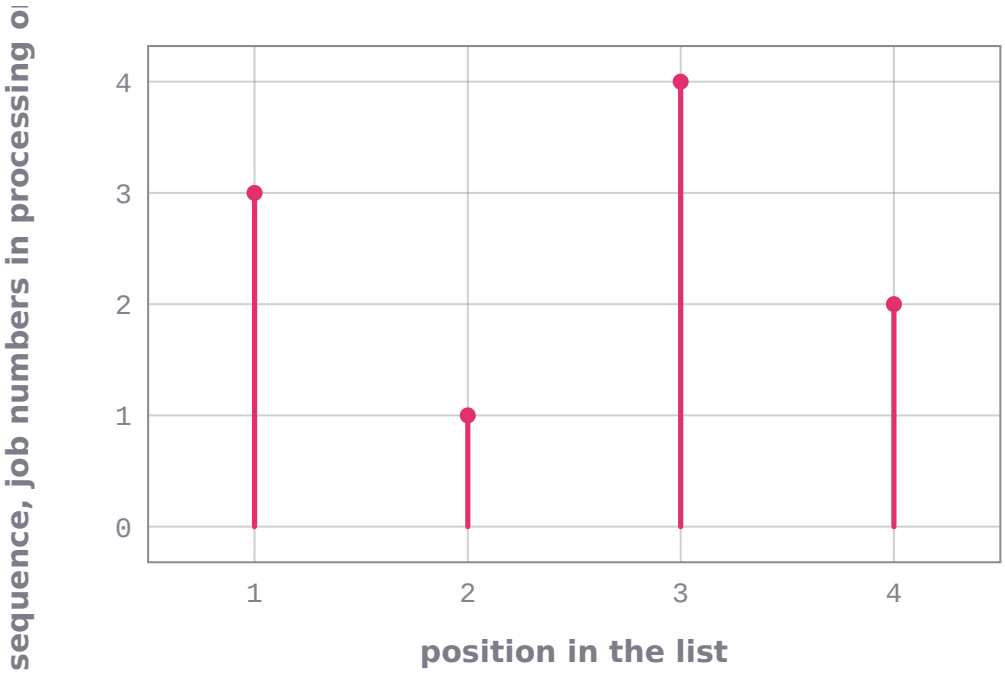
Order jobs on one machine by earliest due date (minimises the maximum lateness) and report tardiness measures.

$$d_{[1]} \le d_{[2]} \le \ldots, \quad L_{max} = \max_{k}(C_{[k]} - d_{[k]})$$

input	meaning	condition
`processing_times`	processing time of each job (jobs numbered 1, 2, ... in this order)	
`due_dates`	due date of each job (same order)	

output	meaning
`sequence`	job numbers in processing order
`mean_flow_time`	average completion (flow) time
`makespan`	completion time of the last job
`average_jobs_in_system`	sum of completion times / makespan
`utilization`	sum of processing times / sum of completion times
`total_tardiness`	sum of tardiness
`mean_tardiness`	average tardiness (job lateness)
`max_tardiness`	largest tardiness
`max_lateness`	largest lateness C - d
`tardy_jobs`	number of late jobs

- **Method:** sort by due date; lateness C - d; tardiness max(0, C - d)
- **Assumes:** processing times > 0; due dates >= 0; ties go to the lower job number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



job numbers in processing order (sequence) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (4 values, first 3, last 2, largest 4). Inputs: processing_times = [4, 7, 2, 5], due_dates = [10, 15, 5, 12].

```
industrial_engineering_ai999x.solve('ie_edd_sequence', processing_times=[4, 7, 2, 5],
due_dates=[10, 15, 5, 12])
# -> sequence = [3, 1, 4, 2], mean_flow_time = 9.25, makespan = 18, average_jobs_in_system
= 2.05556, utilization = 0.486486, total_tardiness = 3, mean_tardiness = 0.75,
max_tardiness = 3, max_lateness = 3, tardy_jobs = 1   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_edd_sequence', processing_times=[3, 6, 2, 8, 4],
due_dates=[8, 20, 5, 18, 12])
# -> sequence = [3, 1, 5, 4, 2], mean_flow_time = 11.2, makespan = 23,
average_jobs_in_system = 2.43478, utilization = 0.410714, total_tardiness = 3,
mean_tardiness = 0.6, max_tardiness = 3, max_lateness = 3, tardy_jobs = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
EDD sequence with processing times = [4, 7, 2, 5], due dates = [10, 15, 5, 12].	SOLVED	sequence = [3, 1, 4, 2], mean_flow_time = 9.25, makespan = 18, average_jobs_in_system = 2.05556, utilization = 0.486486, total_tardiness = 3, mean_tardiness = 0.75, max_tardiness = 3, max_lateness = 3, tardy_jobs = 1
EDD sequence with processing times = [3, 6, 2, 8, 4], due dates = [8, 20, 5, 18, 12].	SOLVED	sequence = [3, 1, 5, 4, 2], mean_flow_time = 11.2, makespan = 23, average_jobs_in_system = 2.43478, utilization = 0.410714, total_tardiness = 3, mean_tardiness = 0.6, max_tardiness = 3, max_lateness = 3, tardy_jobs = 1
EDD sequence with processing times = [4, 7, 2, 5].	NEEDS_INPUT	ie_edd_sequence needs due_dates (due date of each job (same order)). Example: solve('ie_edd_sequence', processing_times=
EDD sequence with processing times = [4, 7, 2, 5], due dates = [10, 15, 5, 12], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
EDD sequence with processing times = [4, 7, 2], due dates = [10, 15].	INVALID_INPUT	Earliest due date (EDD) sequence: processing_times and due_dates must have the same number of values.

Sequencing by a dispatching rule (FCFS, SPT, LPT, EDD, CR), `ie_dispatching_rule`

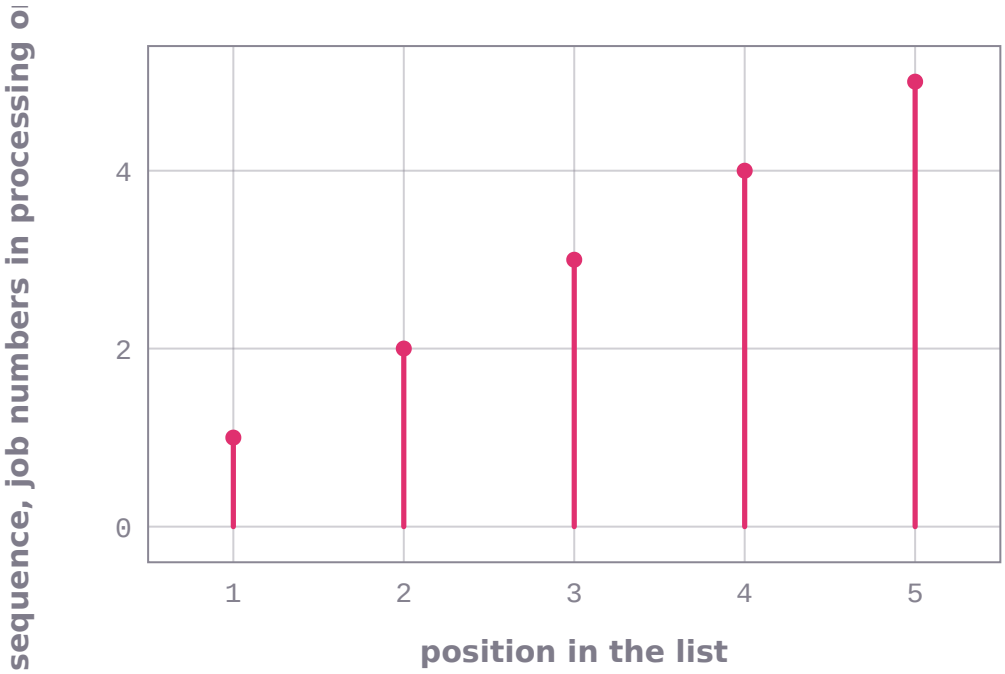
Sequence jobs at one work centre by a priority rule and compare average completion time, utilisation, average number of jobs and tardiness.

$$\bar{F} = \frac{\sum C_j}{n}, \quad \text{utilization} = \frac{\sum p_j}{\sum C_j}, \quad \bar{N} = \frac{\sum C_j}{C_{max}}$$

input	meaning	condition
`processing_times`	processing time of each job (jobs numbered 1, 2, ... in this order)	
`due_dates`	due date of each job (same order)	
`rule`	fcfs (given order), spt, lpt (longest first), edd or cr (critical ratio due date / processing time at time 0, smallest first)	

output	meaning
`sequence`	job numbers in processing order
`mean_flow_time`	average completion (flow) time
`makespan`	completion time of the last job
`average_jobs_in_system`	sum of completion times / makespan
`utilization`	sum of processing times / sum of completion times
`total_tardiness`	sum of tardiness
`mean_tardiness`	average tardiness (job lateness)
`max_tardiness`	largest tardiness
`max_lateness`	largest lateness C - d
`tardy_jobs`	number of late jobs

- **Method:** order the jobs by the rule; completion times are running sums; tardiness max(0, C - d)
- **Assumes:** processing times > 0; due dates >= 0; ties go to the lower job number; jobs available at time 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



job numbers in processing order (sequence) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 1, last 5, largest 5). Inputs: processing_times = [6, 2, 8, 3, 9], due_dates = [8, 6, 18, 15, 23], rule = fcfs.

```
industrial_engineering_ai999x.solve('ie_dispatching_rule', processing_times=[6, 2, 8, 3,
9], due_dates=[8, 6, 18, 15, 23], rule='fcfs')
# -> sequence = [1, 2, 3, 4, 5], mean_flow_time = 15.4, makespan = 28,
average_jobs_in_system = 2.75, utilization = 0.363636, total_tardiness = 11, mean_tardiness
= 2.2, max_tardiness = 5, max_lateness = 5, tardy_jobs = 3   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_dispatching_rule', processing_times=[6, 2, 8, 3,
9], due_dates=[8, 6, 18, 15, 23], rule='cr')
# -> sequence = [1, 3, 5, 2, 4], mean_flow_time = 19.2, makespan = 28,
average_jobs_in_system = 3.42857, utilization = 0.291667, total_tardiness = 32,
mean_tardiness = 6.4, max_tardiness = 19, max_lateness = 19, tardy_jobs = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Sequence by dispatching rule with processing times = [6, 2, 8, 3, 9], due dates = [8, 6, 18, 15, 23], priority rule = fcfs.	SOLVED	sequence = [1, 2, 3, 4, 5], mean_flow_time = 15.4, makespan = 28, average_jobs_in_system = 2.75, utilization = 0.363636, total_tardiness = 11, mean_tardiness = 2.2, max_tardiness = 5, max_lateness = 5, tardy_jobs = 3
Sequence by dispatching rule with processing times = [6, 2, 8, 3, 9], due dates = [8, 6, 18, 15, 23], priority rule = cr.	SOLVED	sequence = [1, 3, 5, 2, 4], mean_flow_time = 19.2, makespan = 28, average_jobs_in_system = 3.42857, utilization = 0.291667, total_tardiness = 32, mean_tardiness = 6.4, max_tardiness = 19, max_lateness = 19, tardy_jobs = 2
Sequence by dispatching rule with processing times = [6, 2, 8, 3, 9], due dates = [8, 6, 18, 15, 23].	NEEDS_INPUT	ie_dispatching_rule needs rule (fcfs (given order), spt, lpt (longest first), edd or cr (critical ratio due date / proce

question	outcome	result
Sequence by dispatching rule with processing times = [6, 2, 8, 3, 9], due dates = [8, 6, 18, 15, 23], priority rule = fcfs, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Sequence by dispatching rule with processing times = [6, 2, 8], due dates = [8, 6, 18], priority rule = random.	INVALID_INPUT	rule: one of fcfs, spt, lpt, edd, cr; got 'random'.

Johnson's rule (two machines), `ie_johnson_two_machine`

Sequence jobs through two machines in the same order so that the makespan is minimal (Johnson's rule).

$$\min(a_{1i}, a_{2j}) \le \min(a_{1j}, a_{2i}) \Rightarrow i \text{ before } j$$

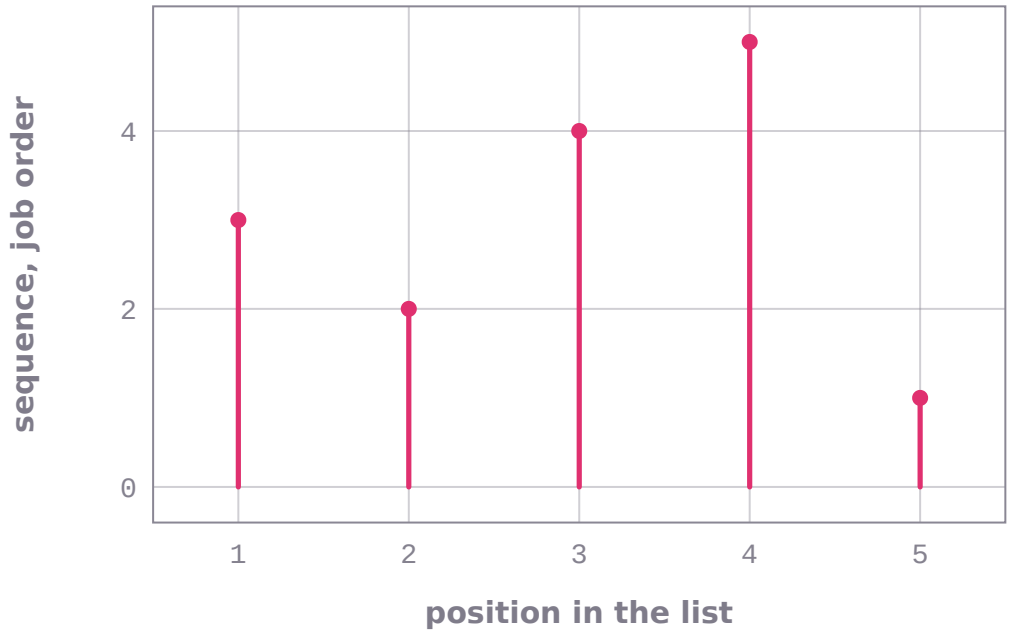
input	meaning	condition
`machine1_times`	time of each job on machine 1 (first)	
`machine2_times`	time of each job on machine 2 (second)	

output	meaning
`sequence`	job order
`makespan`	time all jobs finish on machine 2
`idle_time_machine2`	idle time of machine 2 (makespan - its work)

- **Method:** Johnson: front set $(a_1 \le a_2)$ by a1 ascending, back set by a2 descending
- **Assumes:** times >= 0; jobs with $a_1 \le a_2$ go first by increasing a1, the others last by decreasing a2; ties by the lower job number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



sequence (job order) for each index, as returned by industrial-engineering-ai999x for its example (5 values).



job order (sequence) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 3, last 1, largest 5). Inputs: machine1_times = [5, 2, 1, 7, 6], machine2_times = [2, 6, 2, 5, 3].

```
industrial_engineering_ai999x.solve('ie_johnson_two_machine', machine1_times=[5, 2, 1, 7, 6], machine2_times=[2, 6, 2, 5, 3])
# -> sequence = [3, 2, 4, 5, 1], makespan = 23, idle_time_machine2 = 5   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_johnson_two_machine', machine1_times=[3, 6, 4, 8], machine2_times=[7, 2, 5, 4])
# -> sequence = [1, 3, 4, 2], makespan = 23, idle_time_machine2 = 5   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Johnson's rule with machine 1 times = [5, 2, 1, 7, 6], machine 2 times = [2, 6, 2, 5, 3].	SOLVED	sequence = [3, 2, 4, 5, 1], makespan = 23, idle_time_machine2 = 5
Johnson's rule with machine 1 times = [3, 6, 4, 8], machine 2 times = [7, 2, 5, 4].	SOLVED	sequence = [1, 3, 4, 2], makespan = 23, idle_time_machine2 = 5
Johnson's rule with machine 1 times = [5, 2, 1, 7, 6].	NEEDS_INPUT	ie_johnson_two_machine needs machine2_times (time of each job on machine 2 (second)). Example: solve('ie_johnson_two_mac
Johnson's rule with machine 1 times = [5, 2, 1, 7, 6], machine 2 times = [2, 6, 2, 5, 3], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Johnson's rule with machine 1 times = [5, 2, 1], machine 2 times = [2, 6].	INVALID_INPUT	Johnson's rule (two machines): both machines need a time for every job.

Johnson's rule extended to three machines, `ie_johnson_three_machine`

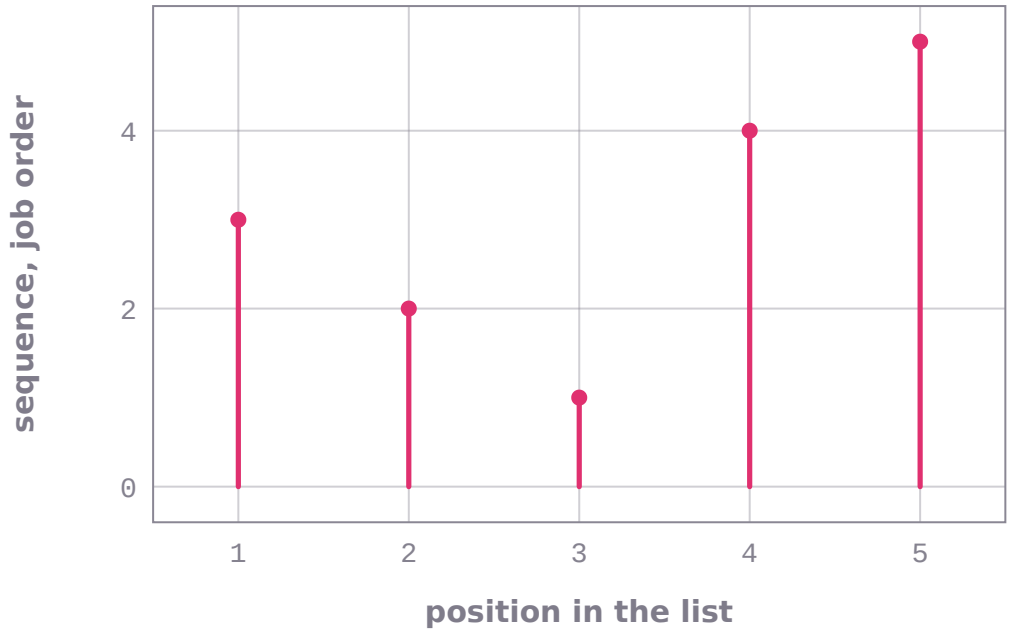
Optimal job order through three machines when machine 2 is dominated (Johnson's extension with pseudo times $a_1 + a_2$ and a2 + a3).

$$a'_i = a_{1i} + a_{2i}, \quad a''_i = a_{2i} + a_{3i}$$

input	meaning	condition
`machine1_times`	time of each job on machine 1	
`machine2_times`	time of each job on machine 2	
`machine3_times`	time of each job on machine 3	

output	meaning
`sequence`	job order
`makespan`	time all jobs finish on machine 3

- **Method:** Johnson's rule on pseudo machines (a1 + a2, a2 + a3); makespan by the three-machine recursion
- **Assumes:** min(machine 1) >= max(machine 2) or min(machine 3) >= max(machine 2) (else refused); ties by the lower job number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



job order (sequence) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 3, last 5, largest 5). Inputs: machine1_times = [8, 10, 6, 7, 11], machine2_times = [5, 6, 2, 3, 4], machine3_times = [4, 9, 8, 6, 5].

```
industrial_engineering_ai999x.solve('ie_johnson_three_machine', machine1_times=[8, 10, 6, 7, 11], machine2_times=[5, 6, 2, 3, 4], machine3_times=[4, 9, 8, 6, 5])
# -> sequence = [3, 2, 1, 4, 5], makespan = 51   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_johnson_three_machine', machine1_times=[3, 8, 7, 4], machine2_times=[4, 3, 2, 5], machine3_times=[6, 7, 5, 11])
# -> sequence = [1, 4, 2, 3], makespan = 36   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Johnson's rule for three machines with machine 1 times = [8, 10, 6, 7, 11], machine 2 times = [5, 6, 2, 3, 4], machine 3 times = [4, 9, 8, 6, 5].	SOLVED	sequence = [3, 2, 1, 4, 5], makespan = 51
Johnson's rule for three machines with machine 1 times = [3, 8, 7, 4], machine 2 times = [4, 3, 2, 5], machine 3 times = [6, 7, 5, 11].	SOLVED	sequence = [1, 4, 2, 3], makespan = 36
Johnson's rule for three machines with machine 1 times = [8, 10, 6, 7, 11], machine 2 times = [5, 6, 2, 3, 4].	NEEDS_INPUT	ie_johnson_three_machine needs machine3_times (time of each job on machine 3). Example: solve('ie_johnson_three_machine'
Johnson's rule for three machines with machine 1 times = [8, 10, 6, 7, 11], machine 2 times = [5, 6, 2, 3, 4], machine 3 times = [4, 9, 8, 6, 5], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Johnson's rule for three machines with machine 1 times = [1, 2],	INVALID_INPUT	Johnson's rule extended to three machines: Johnson's extension needs min(machine 1) >= max(machine 2) or min(machine 3)

question	outcome	result
machine 2 times = [5, 6], machine 3 times = [1, 2].		

Minimise the number of late jobs (Moore-Hodgson), `ie_moore_hodgson`

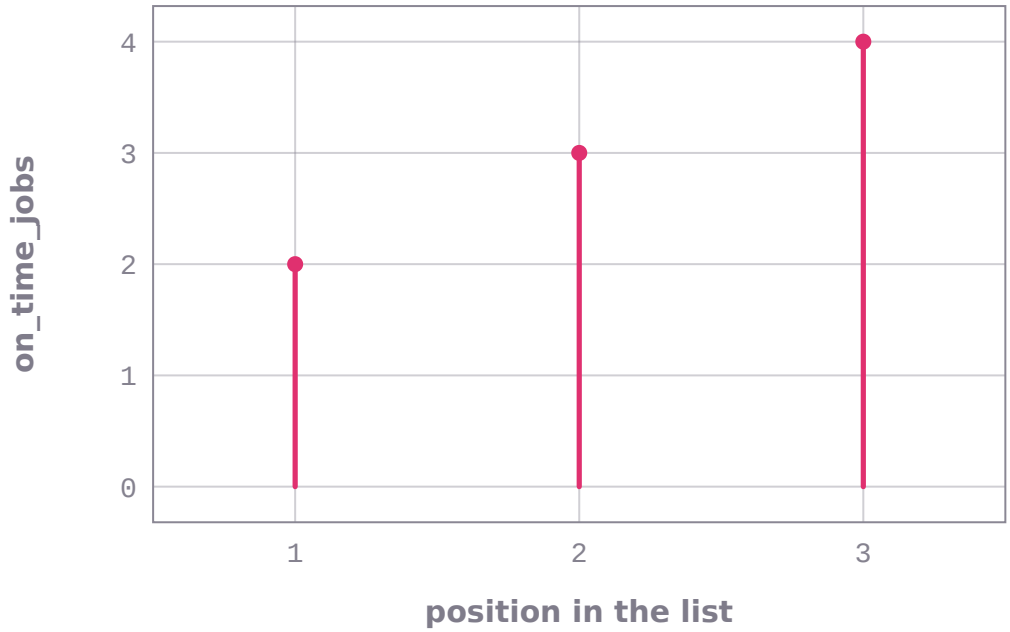
Sequence on one machine that minimises the number of late jobs: on-time jobs in EDD order, then the late ones (Moore-Hodgson algorithm).

$$\min \sum_j U_j, \quad U_j = [C_j > d_j]$$

input	meaning	condition
`processing_times`	processing time of each job (jobs numbered 1, 2, ... in this order)	
`due_dates`	due date of each job (same order)	

output	meaning
`number_late`	least possible number of late jobs
`late_jobs`	the jobs scheduled late
`on_time_jobs`	the on-time jobs in processing (EDD) order
`sequence`	full sequence

- **Method:** add jobs in EDD order; whenever the current job finishes late, drop the longest job scheduled so far
- **Assumes:** processing times > 0; due dates >= 0; when a job is late, the longest scheduled job is removed (ties: the lower job number)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



the on-time jobs in processing (EDD) order (on_time_jobs) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 2, last 4, largest 4). Inputs: processing_times = [4, 3, 2, 5, 6], due_dates = [5, 6, 8, 10, 12].

```
industrial_engineering_ai999x.solve('ie_moore_hodgson', processing_times=[4, 3, 2, 5, 6], due_dates=[5, 6, 8, 10, 12])
# -> number_late = 2, late_jobs = [1, 5], on_time_jobs = [2, 3, 4], sequence = [2, 3, 4, 1, 5]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_moore_hodgson', processing_times=[2, 6, 3, 4, 5, 1], due_dates=[3, 8, 9, 10, 12, 14])
# -> number_late = 2, late_jobs = [2, 5], on_time_jobs = [1, 3, 4, 6], sequence = [1, 3, 4, 6, 2, 5]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moore-Hodgson with processing times = [4, 3, 2, 5, 6], due dates = [5, 6, 8, 10, 12].	SOLVED	number_late = 2, late_jobs = [1, 5], on_time_jobs = [2, 3, 4], sequence = [2, 3, 4, 1, 5]
Moore-Hodgson with processing times = [2, 6, 3, 4, 5, 1], due dates = [3, 8, 9, 10, 12, 14].	SOLVED	number_late = 2, late_jobs = [2, 5], on_time_jobs = [1, 3, 4, 6], sequence = [1, 3, 4, 6, 2, 5]
Moore-Hodgson with processing times = [4, 3, 2, 5, 6].	NEEDS_INPUT	ie_moore_hodgson needs due_dates (due date of each job (same order)). Example: solve('ie_moore_hodgson', processing_time
Moore-Hodgson with processing times = [4, 3, 2, 5, 6], due dates = [5, 6, 8, 10, 12], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Moore-Hodgson with processing times = [4, 3, 0], due dates = [5, 6, 8].	INVALID_INPUT	Minimise the number of late jobs (Moore-Hodgson): every value of processing_times must be positive.

Permutation flow shop makespan, `ie_flow_shop_makespan`

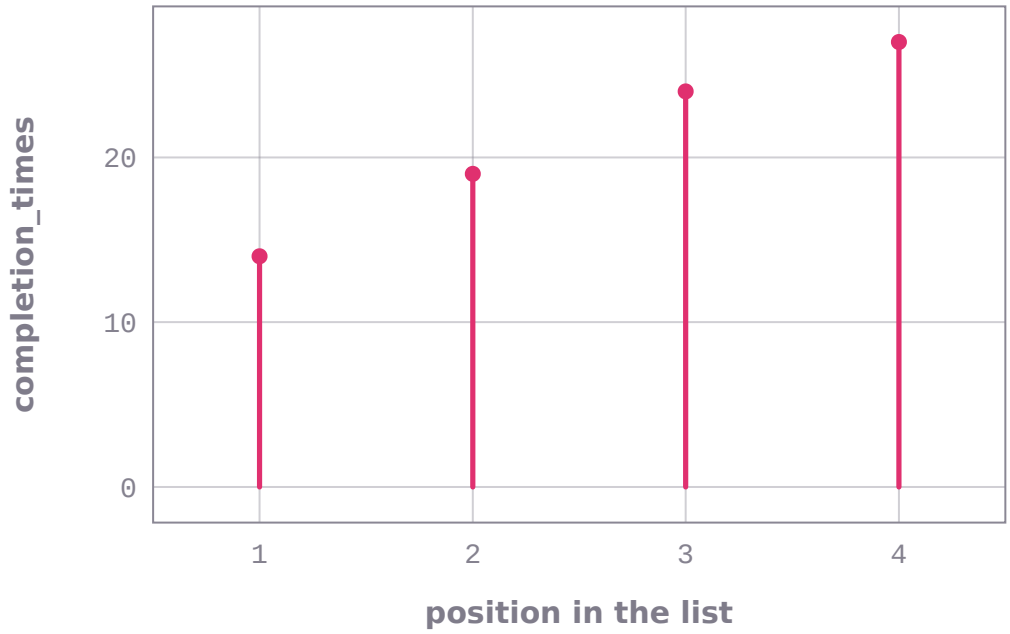
Makespan and completion times of jobs processed in a given order through machines in series (each row a job in processing order, each column a machine).

$$C_{i,j} = \max(C_{i-1,j},\, C_{i,j-1}) + t_{i,j}$$

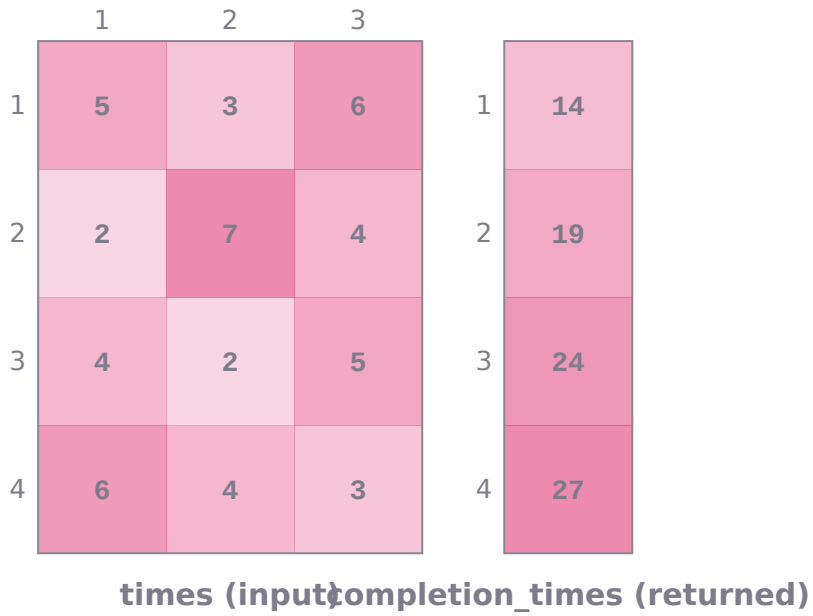
input	meaning	condition
`times`	matrix: row = job (in processing order), column = machine (in route order)	

output	meaning
`makespan`	completion time of the last job on the last machine
`completion_times`	completion time of each job on the last machine
`idle_time_last_machine`	idle time of the last machine

- **Method:** C(i, j) = max(C($i - 1$, j), C(i, j-1)) + t(i, j)
- **Assumes:** times >= 0; the same job order on every machine, no passing
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



completion time of each job on the last machine (completion_times) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (4 values, first 14, last 27, largest 27). Inputs: times = [[5, 3, 6], [2, 7, 4], [4, 2, 5], [6, 4, 3]].



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): times (input); completion_times (returned). Returned with them by industrial-engineering-ai999x: makespan = 27, idle_time_last_machine = 9.

```
industrial_engineering_ai999x.solve('ie_flow_shop_makespan', times=[[5, 3, 6], [2, 7, 4], [4, 2, 5], [6, 4, 3]])
# -> makespan = 27, completion_times = [14.0, 19.0, 24.0, 27.0], idle_time_last_machine = 9 (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_flow_shop_makespan', times=[[3, 6], [5, 2], [1, 2]])
# -> makespan = 13, completion_times = [9.0, 11.0, 13.0], idle_time_last_machine = 3 (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Flow shop makespan with processing time matrix = [[5, 3, 6], [2, 7, 4], [4, 2, 5], [6, 4, 3]].	SOLVED	makespan = 27, completion_times = [14.0, 19.0, 24.0, 27.0], idle_time_last_machine = 9
Flow shop makespan with processing time matrix = [[3, 6], [5, 2], [1, 2]].	SOLVED	makespan = 13, completion_times = [9.0, 11.0, 13.0], idle_time_last_machine = 3
Flow shop makespan with processing time matrix = [[5, 3, 6], [2, 7, 4], [4, 2, 5], [6, 4, 3]], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Flow shop makespan with processing time matrix = [[5, -3], [2, 7]].	INVALID_INPUT	Permutation flow shop makespan: processing times must not be negative.

Moving average forecast, `ie_moving_average_forecast`

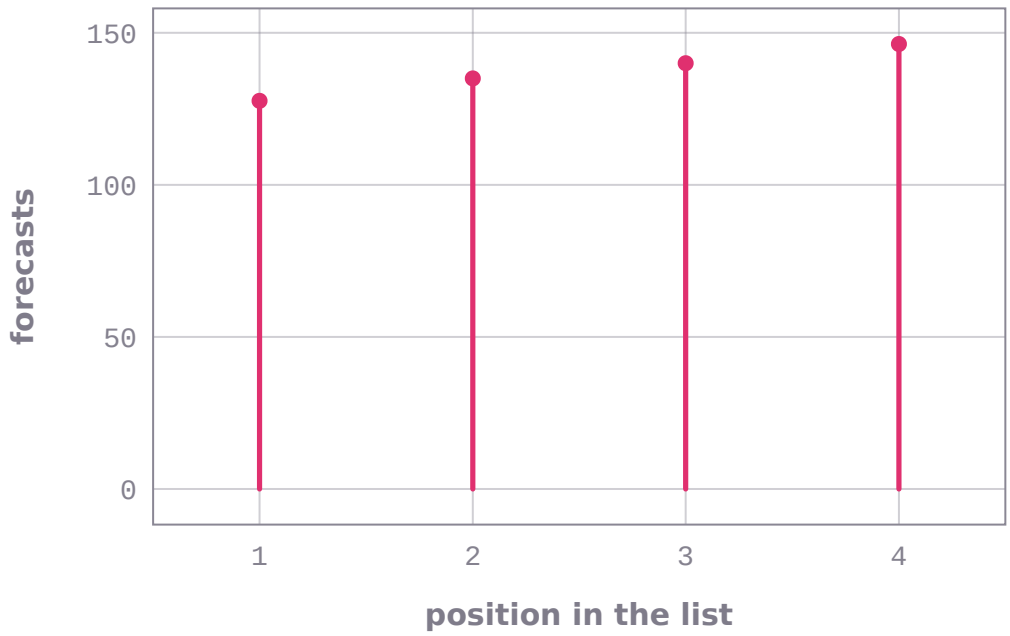
Next-period forecast as the average of the last n observations (and the forecast each period would have had).

$$F_{t+1} = \frac{1}{n} \sum_{i=0}^{n-1} A_{t-i}$$

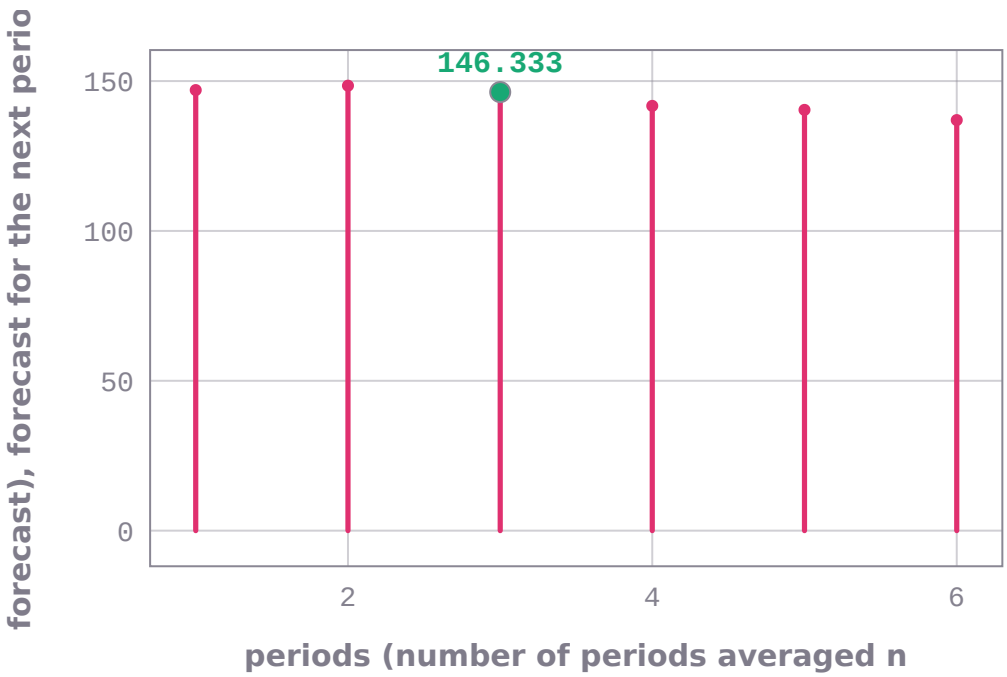
input	meaning	condition
`data`	observations in time order (oldest first)	
`periods`	number of periods averaged n	

output	meaning
`forecast`	forecast for the next period
`forecasts`	moving averages: forecast for periods $n+1$, ..., (last period + 1)

- **Method:** F_(Missing superscript or subscript argument(t-n+1)) / n
- **Assumes:** 1 <= periods <= number of observations
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



moving averages: forecast for periods n+1, ..., (last period + 1) (forecasts) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (4 values, first 127.667, last 146.333, largest 146.333). Inputs: data = [120, 135, 128, 142, 150, 147], periods = 3.



forecast (forecast for the next period) against periods (number of periods averaged n) from 1 to 11. Other inputs as in the worked example: data = [120, 135, 128, 142, 150, 147]. 6 points computed by industrial-engineering-ai999x; marked: the worked example, periods = 3, forecast = 146.333.

```
industrial_engineering_ai999x.solve('ie_moving_average_forecast', data=[120, 135, 128, 142,
150, 147], periods=3)
# -> forecast = 146.333, forecasts = [127.666666667, 135.0, 140.0, 146.333333333]
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_moving_average_forecast', data=[40, 44, 39, 47],
periods=2)
# -> forecast = 43, forecasts = [42.0, 41.5, 43.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Moving average forecast with demand = [120, 135, 128, 142, 150, 147], window = 3.	SOLVED	forecast = 146.333, forecasts = [127.666666667, 135.0, 140.0, 146.333333333]
Moving average forecast with demand = [40, 44, 39, 47], window = 2.	SOLVED	forecast = 43, forecasts = [42.0, 41.5, 43.0]
Moving average forecast with demand = [120, 135, 128, 142, 150, 147].	NEEDS_INPUT	ie_moving_average_forecast needs periods (number of periods averaged n). Example: solve('ie_moving_average_forecast', da
Moving average forecast with demand = [120, 135, 128, 142, 150, 147], window = 3, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Moving average forecast with demand = [40, 44, 39], window = 5.	INVALID_INPUT	Moving average forecast: periods must not exceed the number of observations.

Weighted moving average forecast, `ie_weighted_moving_average`

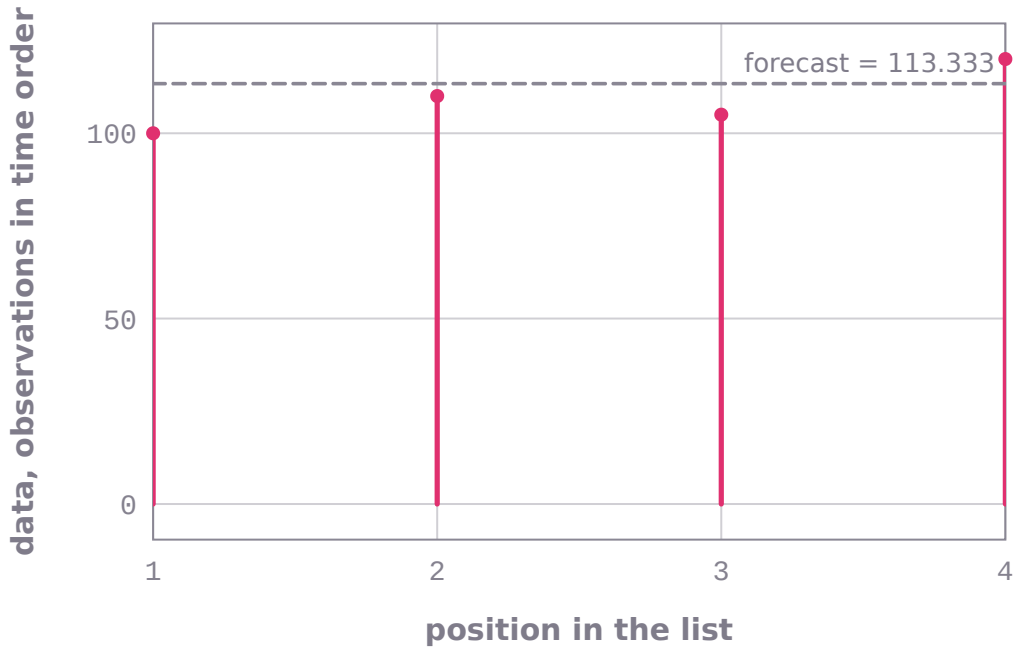
Next-period forecast as a weighted average of the most recent observations (weights listed oldest to newest, divided by their sum).

$$F_{t+1} = \frac{\sum_{i=1}^{k} w_i A_{t-k+i}}{\sum w_i}$$

input	meaning	condition
`data`	observations in time order (oldest first)	
`weights`	weights for the last k observations, oldest first	

output	meaning
`forecast`	sum w_i A_i / sum w_i
`weight_sum`	sum of the weights

- **Method:** F = sum w_i A_i / sum w_i over the last k observations
- **Assumes:** weights >= 0, not all 0; at least as many observations as weights; the last weight applies to the most recent observation
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input data (observations in time order (oldest first)) as given in the worked example, one value per position (4 values); marked from what industrial-engineering-ai999x returned: forecast = 113.333. Also returned: weight_sum = 6. Other inputs: weights = [1, 2, 3].

```
industrial_engineering_ai999x.solve('ie_weighted_moving_average', data=[100, 110, 105,
120], weights=[1, 2, 3])
# -> forecast = 113.333, weight_sum = 6   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_weighted_moving_average', data=[52, 48, 55, 60,
58], weights=[0.2, 0.3, 0.5])
# -> forecast = 58, weight_sum = 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weighted moving average forecast with demand = [100, 110, 105, 120], weights = [1, 2, 3].	SOLVED	forecast = 113.333, weight_sum = 6
Weighted moving average forecast with demand = [52, 48, 55, 60, 58], weights = [0.2, 0.3, 0.5].	SOLVED	forecast = 58, weight_sum = 1
Weighted moving average forecast with demand = [100, 110, 105, 120].	NEEDS_INPUT	ie_weighted_moving_average needs weights (weights for the last k observations, oldest first). Example: solve('ie_weighte
Weighted moving average forecast with demand = [100, 110, 105, 120], weights = [1, 2, 3], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Weighted moving average forecast with demand = [100, 110], weights = [1, 2, 3].	INVALID_INPUT	Weighted moving average forecast: there must be at least as many observations as weights.

Simple exponential smoothing forecast, `ie_exponential_smoothing`

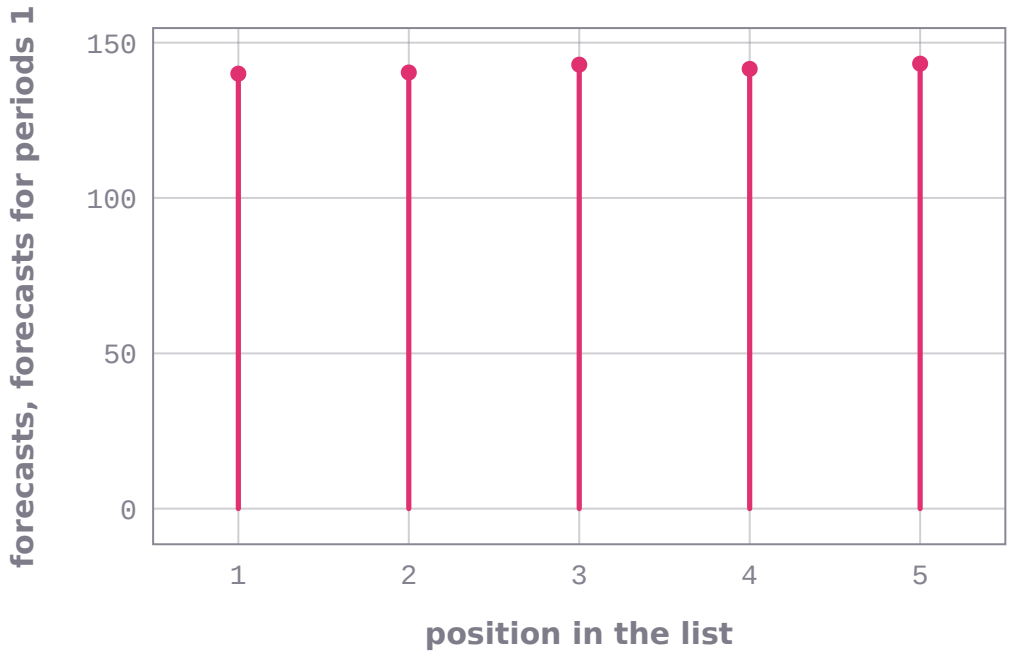
Exponentially smoothed forecasts F_(t+1) = F_t + alpha (A_t - F_t), starting from a given first forecast (default: the first observation).

$$F_{t+1} = F_t + \alpha\,(A_t - F_t)$$

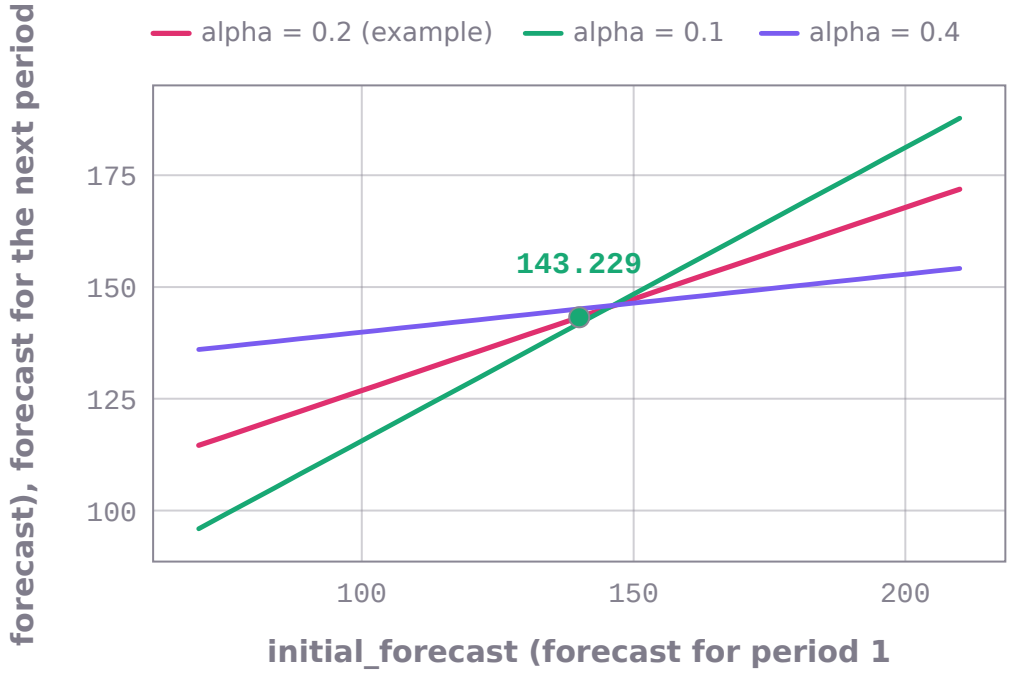
input	meaning	condition
`data`	observations in time order (oldest first)	
`alpha`	smoothing constant $(0 < \alpha \leq 1)$	
`initial_forecast`	forecast for period 1 (default: the first observation)	

output	meaning
`forecast`	forecast for the next period
`forecasts`	forecasts for periods 1, ..., $n + 1$

- **Method:** F_1 given (or A_1); F_(t+1) = alpha $A_t + (1 - \alpha)$ F_t
- **Assumes:** $0 < \alpha \leq 1$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



forecasts for periods 1, ..., n + 1 (forecasts) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 140, last 143.229, largest 143.229). Inputs: data = [142, 153, 136, 150], alpha = 0.2, initial_forecast = 140.



forecast (forecast for the next period) against initial_forecast (forecast for period 1 (default: the first observation)) from 70 to 210. One curve for each value of alpha (smoothing constant (0 < alpha <= 1)): 0.2, 0.1, 0.4. Other inputs as in the worked example: data = [142, 153, 136, 150]. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, initial_forecast = 140, forecast = 143.229.

```
industrial_engineering_ai999x.solve('ie_exponential_smoothing', data=[142, 153, 136, 150], alpha=0.2, initial_forecast=140)
# -> forecast = 143.229, forecasts = [140.0, 140.4, 142.92, 141.536, 143.2288]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_exponential_smoothing', data=[20, 24, 22, 27, 25], alpha=0.4)
# -> forecast = 24.3136, forecasts = [20.0, 20.0, 21.6, 21.76, 23.856, 24.3136]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Exponential smoothing forecast with demand = [142, 153, 136, 150], smoothing constant = 0.2, first forecast = 140.	SOLVED	forecast = 143.229, forecasts = [140.0, 140.4, 142.92, 141.536, 143.2288]
Exponential smoothing forecast with demand = [20, 24, 22, 27, 25], smoothing constant = 0.4.	SOLVED	forecast = 24.3136, forecasts = [20.0, 20.0, 21.6, 21.76, 23.856, 24.3136]
Exponential smoothing forecast with demand = [142, 153, 136, 150], first forecast = 140.	NEEDS_INPUT	ie_exponential_smoothing needs alpha (smoothing constant (0 < alpha <= 1)). Example: solve('ie_exponential_smoothing', d
Exponential smoothing forecast with demand = [142, 153, 136, 150], smoothing constant = 0.2, first forecast = 140, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Exponential smoothing forecast with demand = [142, 153], smoothing constant = 1.4.	INVALID_INPUT	Simple exponential smoothing forecast: alpha must lie strictly between 0 and 1 (write 95% or 0.95).

Holt's linear trend (double exponential smoothing), `ie_holt_trend`

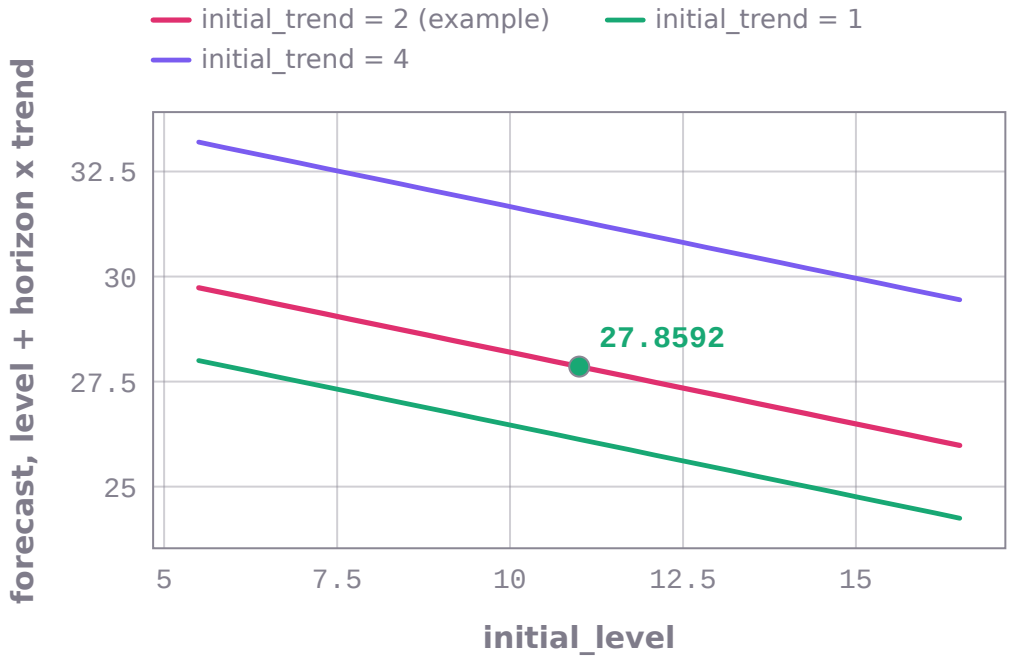
Trend-adjusted exponential smoothing: level and trend updated each period; the forecast h periods ahead is level + h x trend.

$$L_t = \alpha A_t + (1-\alpha)(L_{t-1} + T_{t-1}), \quad T_t = \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1}$$

input	meaning	condition
`data`	observations in time order (oldest first)	
`alpha`	level smoothing constant	
`beta`	trend smoothing constant	
`initial_level`	level before the first observation	
`initial_trend`	trend before the first observation	
`horizon`	periods ahead (default 1)	

output	meaning
`forecast`	level + horizon x trend
`level`	final level
`trend`	final trend

- **Method:** $L_t = \alpha\, A_t + (1-\alpha)$(L_(t-1) + T_(t-1)); $T_t = \beta$ (L_t - L_(t-1)) + $(1-\beta)$ T_(t-1); F_(t+h) = L_t + h T_t
- **Assumes:** $0 < \alpha, \beta \leq 1$; 1 <= horizon <= 1000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



forecast (level + horizon x trend) against initial_level (level before the first observation) from 5.5 to 16.5. One curve for each value of initial_trend (trend before the first observation): 2, 1, 4. Other inputs as in the worked example: data = [12, 17, 20, 19, 24, 27], alpha = 0.2, beta = 0.4. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, initial_level = 11, forecast = 27.8592.

```
industrial_engineering_ai999x.solve('ie_holt_trend', data=[12, 17, 20, 19, 24, 27],
alpha=0.2, beta=0.4, initial_level=11, initial_trend=2)
# -> forecast = 27.8592, level = 25.3133, trend = 2.54593   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_holt_trend', data=[100, 108, 115, 118, 126],
alpha=0.5, beta=0.3, initial_level=95, initial_trend=5, horizon=3)
# -> forecast = 143.02, level = 125.196, trend = 5.94142   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Holt's linear trend forecast with demand = [12, 17, 20, 19, 24, 27], level smoothing = 0.2, trend smoothing = 0.4, starting level = 11, starting trend = 2.	SOLVED	forecast = 27.8592, level = 25.3133, trend = 2.54593
Holt's linear trend forecast with demand = [100, 108, 115, 118, 126], level smoothing = 0.5, trend smoothing = 0.3, starting level = 95, starting trend = 5, periods ahead = 3.	SOLVED	forecast = 143.02, level = 125.196, trend = 5.94142
Holt's linear trend forecast with demand = [12, 17, 20, 19, 24, 27], level smoothing = 0.2, trend smoothing = 0.4, starting level = 11.	NEEDS_INPUT	ie_holt_trend needs initial_trend (trend before the first observation). Example: solve('ie_holt_trend', data=[12, 17, 20
Holt's linear trend forecast with demand = [12, 17, 20, 19, 24, 27], level smoothing = 0.2, trend smoothing = 0.4, starting level = 11, starting trend = 2, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Holt's linear trend forecast with demand = [12, 17], level smoothing = 0.2, trend smoothing = 1.4, starting level = 11, starting trend = 2.	INVALID_INPUT	Holt's linear trend (double exponential smoothing): beta must lie strictly between 0 and 1 (write 95% or 0.95).

Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal, `ie_forecast_errors`

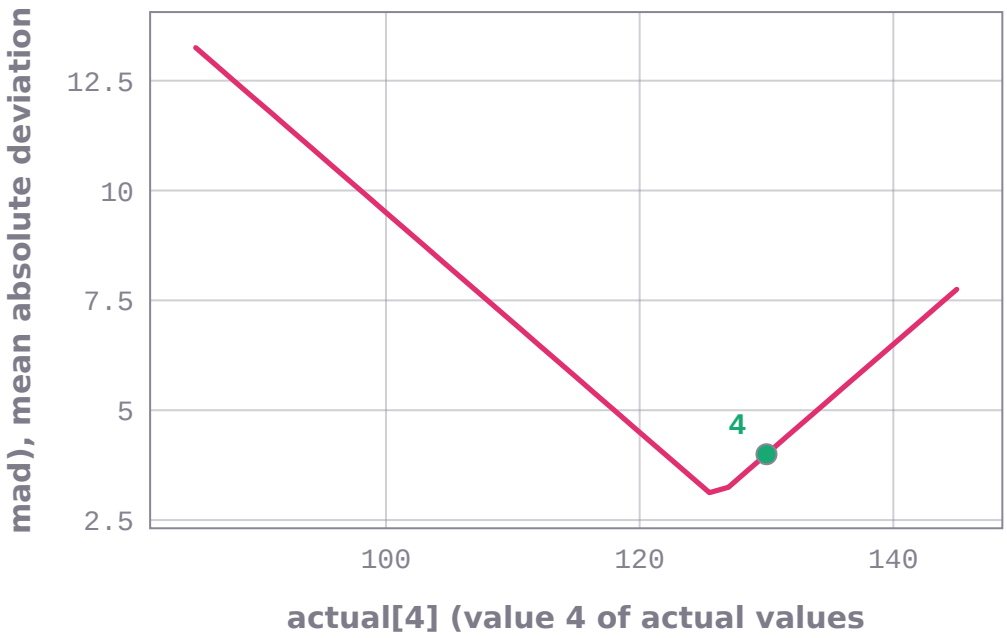
Standard accuracy measures of a forecast against actual values (error = actual - forecast).

$$MAD = \frac{\sum |e_t|}{n},\ MSE = \frac{\sum e_t^2}{n},\ MAPE = \frac{100}{n} \sum \left| \frac{e_t}{A_t} \right|,\ TS = \frac{\sum e_t}{MAD}$$

input	meaning	condition
`actual`	actual values	
`forecast`	forecast values (same periods)	

output	meaning
`mad`	mean absolute deviation
`mse`	mean squared error
`rmse`	root mean squared error
`mape`	mean absolute percentage error, in percent (when no actual is 0)
`bias`	mean error (positive: forecasts too low)
`running_sum_of_errors`	sum of errors
`tracking_signal`	running sum of errors / MAD

- **Method:** $e_t = A_t - F_t$; MAD = mean |e|; MSE = mean e^2; MAPE = 100 mean |e/A|; TS = sum e / MAD
- **Assumes:** equal lengths; error = actual - forecast; MAPE only if no actual value is 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



mad (mean absolute deviation) as value 4 of actual (actual values) moves from 85 to 145, the other values as given ([100, 110, 120, 130]); 41 runs of industrial-engineering-ai999x. Other inputs: forecast = [105, 108, 125, 126]. Marked: the worked example, actual[4] = 130, mad = 4.

```
industrial_engineering_ai999x.solve('ie_forecast_errors', actual=[100, 110, 120, 130],
forecast=[105, 108, 125, 126])
# -> mad = 4, mse = 17.5, rmse = 4.1833, bias = -1, running_sum_of_errors = -4, mape =
3.51544, tracking_signal = -1   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_forecast_errors', actual=[52, 48, 60, 55, 62],
forecast=[50, 50, 54, 58, 58])
# -> mad = 3.4, mse = 13.8, rmse = 3.71484, bias = 1.4, running_sum_of_errors = 7, mape =
5.9838, tracking_signal = 2.05882   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Forecast accuracy with actual = [100, 110, 120, 130], forecasts = [105, 108, 125, 126].	SOLVED	mad = 4, mse = 17.5, rmse = 4.1833, bias = -1, running_sum_of_errors = -4, mape = 3.51544, tracking_signal = -1
Forecast accuracy with actual = [52, 48, 60, 55, 62], forecasts = [50, 50, 54, 58, 58].	SOLVED	mad = 3.4, mse = 13.8, rmse = 3.71484, bias = 1.4, running_sum_of_errors = 7, mape = 5.9838, tracking_signal = 2.05882
Forecast accuracy with actual = [100, 110, 120, 130].	NEEDS_INPUT	ie_forecast_errors needs forecast (forecast values (same periods)). Example: solve('ie_forecast_errors', actual=[100, 11
Forecast accuracy with actual = [100, 110, 120, 130], forecasts = [105, 108, 125, 126], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Forecast accuracy with actual = [100, 110, 120], forecasts = [105, 108].	INVALID_INPUT	Forecast accuracy: MAD, MSE, RMSE, MAPE, bias, tracking signal: actual and forecast must have the same number of values.

Tracking signal over time, `ie_tracking_signal`

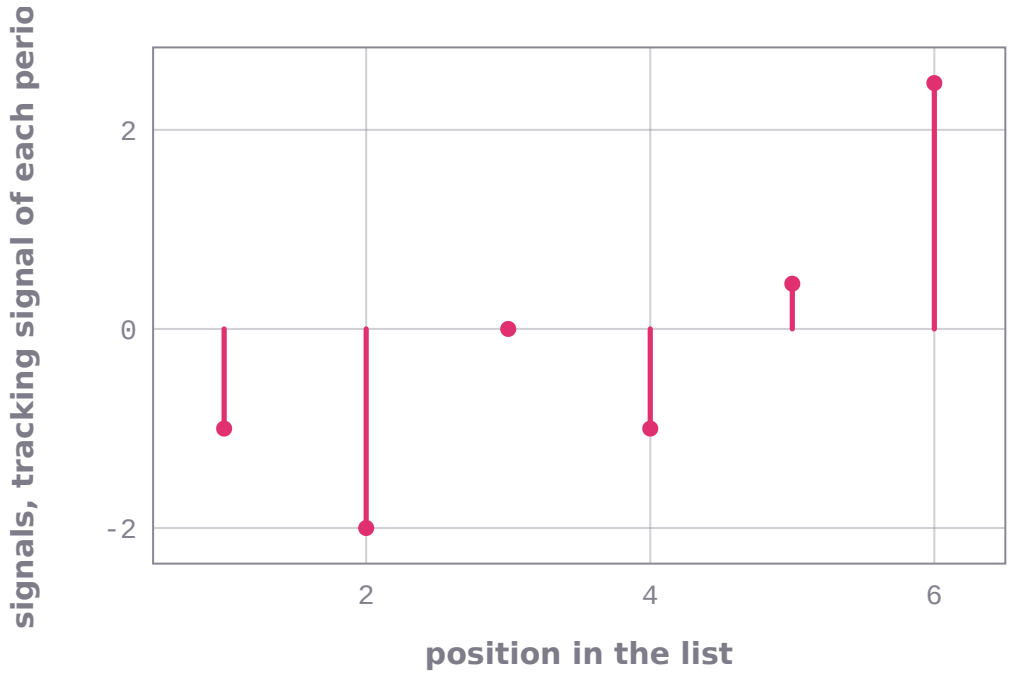
Tracking signal each period (running sum of errors / running MAD) and the first period it leaves the +/- limit.

$$TS_t = \frac{\sum_{i \le t} e_i}{MAD_t}$$

input	meaning	condition
`actual`	actual values	
`forecast`	forecast values (same periods)	
`limit`	control limit in MADs (default 4)	

output	meaning
`final_signal`	tracking signal in the last period
`signals`	tracking signal of each period
`first_out_of_control`	first period with |TS| > limit (0 = never)

- **Method:** TS_t = (sum_{$i \le t$} e_i) / (sum_{$i \le t$} |e_i| / t)
- **Assumes:** equal lengths; running MAD uses all periods so far
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



tracking signal of each period (signals) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (6 values, first -1, last 2.47059, largest 2.47059). Inputs: actual = [90, 95, 115, 100, 125, 140], forecast = [100, 100, 100, 110, 110, 110].

```
industrial_engineering_ai999x.solve('ie_tracking_signal', actual=[90, 95, 115, 100, 125,
140], forecast=[100, 100, 100, 110, 110, 110])
# -> final_signal = 2.47059, signals = [-1.0, -2.0, 0.0, -1.0, 0.454545454545,
2.47058823529], first_out_of_control = 0   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_tracking_signal', actual=[52, 48, 60, 55, 62],
forecast=[50, 50, 54, 58, 58], limit=3)
# -> final_signal = 2.05882, signals = [1.0, 0.0, 1.8, 0.923076923077, 2.05882352941],
first_out_of_control = 0   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Tracking signal with actual = [90, 95, 115, 100, 125, 140], forecasts = [100, 100, 100, 110, 110, 110].	SOLVED	final_signal = 2.47059, signals = [-1.0, -2.0, 0.0, -1.0, 0.454545454545, 2.47058823529], first_out_of_control = 0
Tracking signal with actual = [52, 48, 60, 55, 62], forecasts = [50, 50, 54, 58, 58], control limit = 3.	SOLVED	final_signal = 2.05882, signals = [1.0, 0.0, 1.8, 0.923076923077, 2.05882352941], first_out_of_control = 0
Tracking signal with actual = [90, 95, 115, 100, 125, 140].	NEEDS_INPUT	ie_tracking_signal needs forecast (forecast values (same periods)). Example: solve('ie_tracking_signal', actual=[90, 95,
Tracking signal with actual = [90, 95, 115, 100, 125, 140], forecasts = [100, 100, 100, 110, 110, 110], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Tracking signal with actual = [90, 95], forecasts = [100, 100], control limit = -4.	INVALID_INPUT	Tracking signal over time: limit must be positive.

Linear trend projection (least squares), `ie_linear_trend_forecast`

Least-squares trend line $y = a + b\,t$ through the data at periods $t = 1, \ldots, n$, projected h periods ahead.

$$b = \frac{\sum (t - \bar{t})(y - \bar{y})}{\sum (t - \bar{t})^2}, \quad a = \bar{y} - b\bar{t}$$

input	meaning	condition
`data`	observations in time order (oldest first)	
`horizon`	periods beyond the last observation (default 1)	

output	meaning
`forecast`	a + b (n + h)
`intercept`	a
`slope`	b, change per period

- **Method:** b = sum (t - t_bar)(y - y_bar) / sum (t - t_bar)^2, a = y_bar - b t_bar
- **Assumes:** at least 2 observations; periods numbered 1, 2, ..., n
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The input data (observations in time order (oldest first)) as given in the worked example, one value per position (7 values); marked from what industrial-engineering-ai999x returned: forecast = 141. Also returned: intercept = 56.7143, slope = 10.5357.

```
industrial_engineering_ai999x.solve('ie_linear_trend_forecast', data=[74, 79, 80, 90, 105, 142, 122])
# -> forecast = 141, intercept = 56.7143, slope = 10.5357   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_linear_trend_forecast', data=[20, 24, 27, 31], horizon=3)
# -> forecast = 41.7, intercept = 16.5, slope = 3.6   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Linear trend projection with demand = [74, 79, 80, 90, 105, 142, 122].	SOLVED	forecast = 141, intercept = 56.7143, slope = 10.5357
Linear trend projection with demand = [20, 24, 27, 31], periods ahead = 3.	SOLVED	forecast = 41.7, intercept = 16.5, slope = 3.6
Linear trend projection with demand = [74, 79, 80, 90, 105, 142, 122], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Linear trend projection with demand = [20], periods ahead = 1.	INVALID_INPUT	Linear trend projection (least squares): data needs at least 2 values.

Seasonal indices (average method), `ie_seasonal_index`

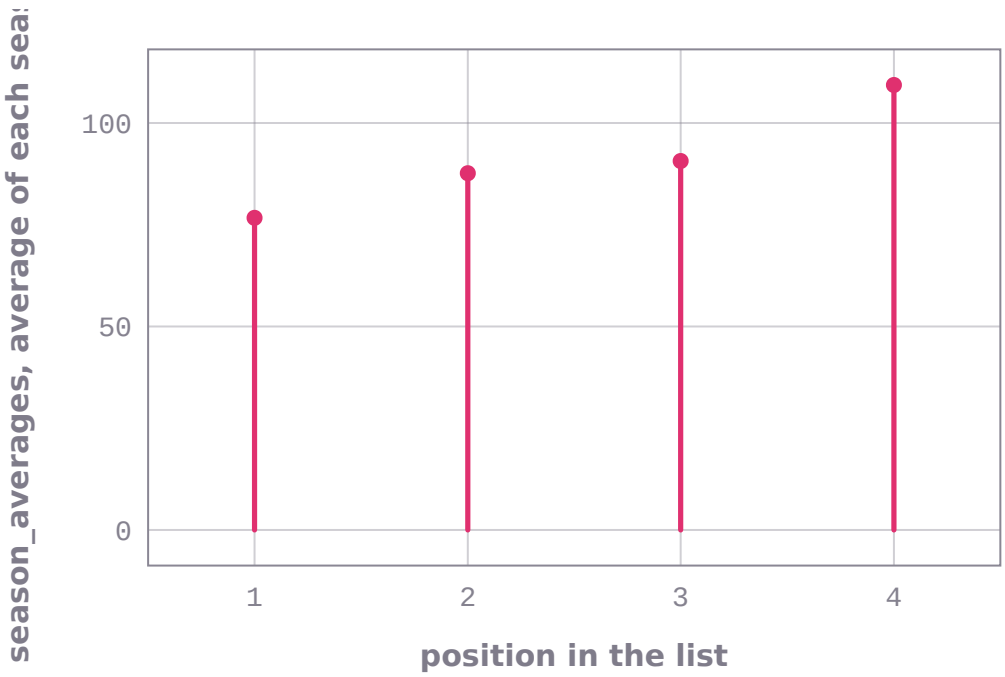
Seasonal index of each season = average demand in that season / average demand over all seasons; with an expected annual total, the seasonal forecasts.

$$S_k = \frac{\bar{A}_k}{\bar{A}}, \quad F_k = \frac{T}{s} S_k$$

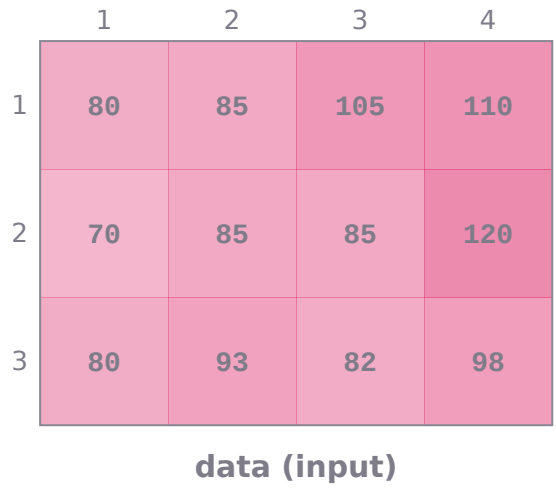
input	meaning	condition
`data`	matrix: one row per year, one column per season (month, quarter)	
`next_year_total`	expected total demand next year (optional)	

output	meaning
`indices`	seasonal index of each season
`season_averages`	average of each season
`overall_average`	average over all seasons and years
`forecasts`	next-year forecast of each season (annual total / seasons x index)

- **Method:** index_k = mean of season k / overall mean; forecast_k = (annual total / s) index_k
- **Assumes:** rows of equal length (at least 2 seasons); positive average
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



average of each season (season_averages) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (4 values, first 76.6667, last 109.333, largest 109.333). Inputs: data = [[80, 85, 105, 110], [70, 85, 85, 120], [80…, next_year_total = 1200.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): data (input). Returned with them by industrial-engineering-ai999x: overall_average = 91.0833. Other inputs: next_year_total = 1200.

```
industrial_engineering_ai999x.solve('ie_seasonal_index', data=[[80, 85, 105, 110], [70, 85, 85, 120], [80, 93, 82, 98]], next_year_total=1200)
# -> indices = [0.841720036597, 0.962488563586, 0.995425434584, 1.20036596523], season_averages = [76.6666666667, 87.6666666667, 90.6666666667, 109.333333333], overall_average = 91.0833, forecasts = [252.516010979, 288.746569076, 298.627630375, 360.10978957]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_seasonal_index', data=[[200, 350, 150], [220, 330, 170]])
# -> indices = [0.887323943662, 1.43661971831, 0.676056338028], season_averages = [210.0, 340.0, 160.0], overall_average = 236.667   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Seasonal indices with demand table = [[80, 85, 105, 110], [70, 85, 85, 120], [80, 93, 82, 98]], annual forecast = 1200.	SOLVED	indices = [0.841720036597, 0.962488563586, 0.995425434584, 1.20036596523], season_averages = [76.6666666667, 87.6666666667, 90.6666666667, 109.333333333], overall_average = 91.0833, forecasts = [252.516010979, 288.746569076, 298.627630375, 360.10978957]
Seasonal indices with demand table = [[200, 350, 150], [220, 330, 170]].	SOLVED	indices = [0.887323943662, 1.43661971831, 0.676056338028], season_averages = [210.0, 340.0, 160.0], overall_average = 236.667
Seasonal indices with demand table = [[80, 85, 105, 110], [70, 85, 85, 120], [80, 93, 82, 98]], annual forecast = 1200, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Seasonal indices with demand table = [[0, 0], [0, 0]].	INVALID_INPUT	Seasonal indices (average method): the average demand must be positive.

Associative forecast by simple linear regression, `ie_regression_forecast`

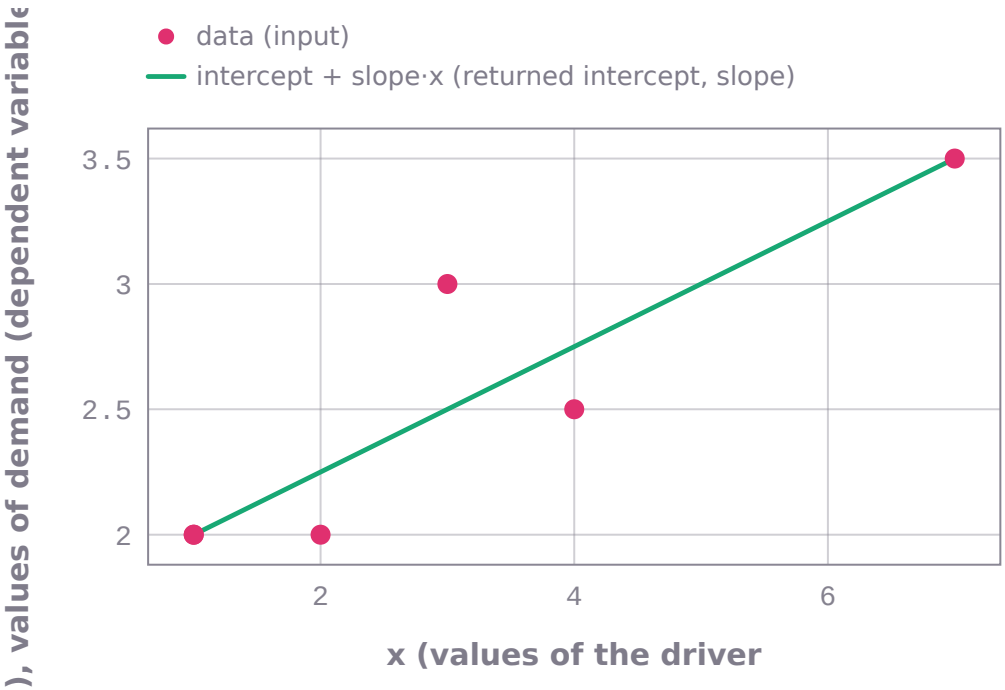
Least-squares line $y = a + b$ x of demand on a driver, its correlation and standard error of estimate, and the forecast at a new x.

$$\hat{y} = a + bx, \quad S_{y,x} = \sqrt{\frac{\sum (y - \hat{y})^2}{n - 2}}$$

input	meaning	condition
`x`	values of the driver (independent variable)	
`y`	values of demand (dependent variable)	
`x_new`	driver value to forecast at (optional)	

output	meaning
`intercept`	a
`slope`	b
`r`	correlation coefficient
`r_squared`	coefficient of determination
`standard_error`	standard error of the estimate sqrt(SSE/(n - 2))
`forecast`	a + b x_new

- **Method:** ordinary least squares; $S_y x = sqrt$(sum (y - y_hat)^2 / (n - 2))
- **Assumes:** at least 3 pairs; x and y vary
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (values of demand (dependent variable)) against x (values of the driver (independent variable)) as given in the worked example (6 points); drawn on it from what industrial-engineering-ai999x returned: the line y = 1.75 + 0.25·x from the returned intercept and slope. Also returned: r_squared = 0.8125, standard_error = 0.306186, forecast = 3.25. Other inputs: x_new = 6.

```
industrial_engineering_ai999x.solve('ie_regression_forecast', x=[1, 3, 4, 2, 1, 7], y=[2.0, 3.0, 2.5, 2.0, 2.0, 3.5], x_new=6)
# -> intercept = 1.75, slope = 0.25, r = 0.901388, r_squared = 0.8125, standard_error = 0.306186, forecast = 3.25   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_regression_forecast', x=[10, 12, 15, 18, 20], y=[110, 125, 140, 170, 182])
# -> intercept = 36.2088, slope = 7.27941, r = 0.99451, r_squared = 0.989051, standard_error = 3.64651   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Regression forecast with driver = [1, 3, 4, 2, 1, 7], dependent variable = [2, 3, 2.5, 2, 2, 3.5], new x = 6.	SOLVED	intercept = 1.75, slope = 0.25, r = 0.901388, r_squared = 0.8125, standard_error = 0.306186, forecast = 3.25
Regression forecast with driver = [10, 12, 15, 18, 20], dependent variable = [110, 125, 140, 170, 182].	SOLVED	intercept = 36.2088, slope = 7.27941, r = 0.99451, r_squared = 0.989051, standard_error = 3.64651
Regression forecast with driver = [1, 3, 4, 2, 1, 7], new x = 6.	NEEDS_INPUT	ie_regression_forecast needs y (values of demand (dependent variable)). Example: solve('ie_regression_forecast', x=[1, 3
Regression forecast with driver = [1, 3, 4, 2, 1, 7], dependent variable = [2, 3, 2.5, 2, 2, 3.5], new x = 6, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Regression forecast with driver = [1, 1, 1], dependent variable = [2, 3, 4].	INVALID_INPUT	Associative forecast by simple linear regression: x and y must both vary.

Takt (cycle) time and minimum stations, `ie_takt_time`

Cycle time that meets demand (available production time / demand) and, with the total task time, the theoretical minimum number of stations.

$$C = \frac{\text{available time}}{\text{demand}}, \quad N_{min} = \left\lceil \frac{\sum t_i}{C} \right\rceil$$

input	meaning	condition
`available_time`	production time available per period (minutes, say)	
`demand`	units required per period	
`total_task_time`	sum of all task times (same time unit), optional	

output	meaning
`cycle_time`	takt time = available time / demand
`output_rate`	units per time unit
`minimum_stations`	theoretical minimum stations ceil(total task time / cycle time)
`exact_stations`	total task time / cycle time

- **Method:** C = available time / demand; N_min = ceil(sum t / C)
- **Assumes:** inputs > 0; consistent time units
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cycle_time (takt time = available time / demand) against demand (units required per period) from 30 to 90. One curve for each value of available_time (production time available per period (minutes, say)): 480, 240, 960. Other inputs as in the worked example: total_task_time = 30. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, demand = 60, cycle_time = 8.

```
industrial_engineering_ai999x.solve('ie_takt_time', available_time=480, demand=60,
total_task_time=30)
# -> cycle_time = 8, output_rate = 0.125, minimum_stations = 4, exact_stations = 3.75
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_takt_time', available_time=450, demand=900)
# -> cycle_time = 0.5, output_rate = 2   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Takt time with available production time = 480, required output = 60, sum of task times = 30.	SOLVED	cycle_time = 8, output_rate = 0.125, minimum_stations = 4, exact_stations = 3.75
Takt time with available production time = 450, required output = 900.	SOLVED	cycle_time = 0.5, output_rate = 2
Takt time with available production time = 480, sum of task times = 30.	NEEDS_INPUT	ie_takt_time needs demand (units required per period). Example: solve('ie_takt_time', available_time=480, demand=60, tot
Takt time with available production time = 480, required output = 60, sum of task times = 30, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Takt time with available production time = 480, required output = 0.	INVALID_INPUT	Takt (cycle) time and minimum stations: demand must be positive.

Line balance efficiency, balance delay and idle time, `ie_line_efficiency`

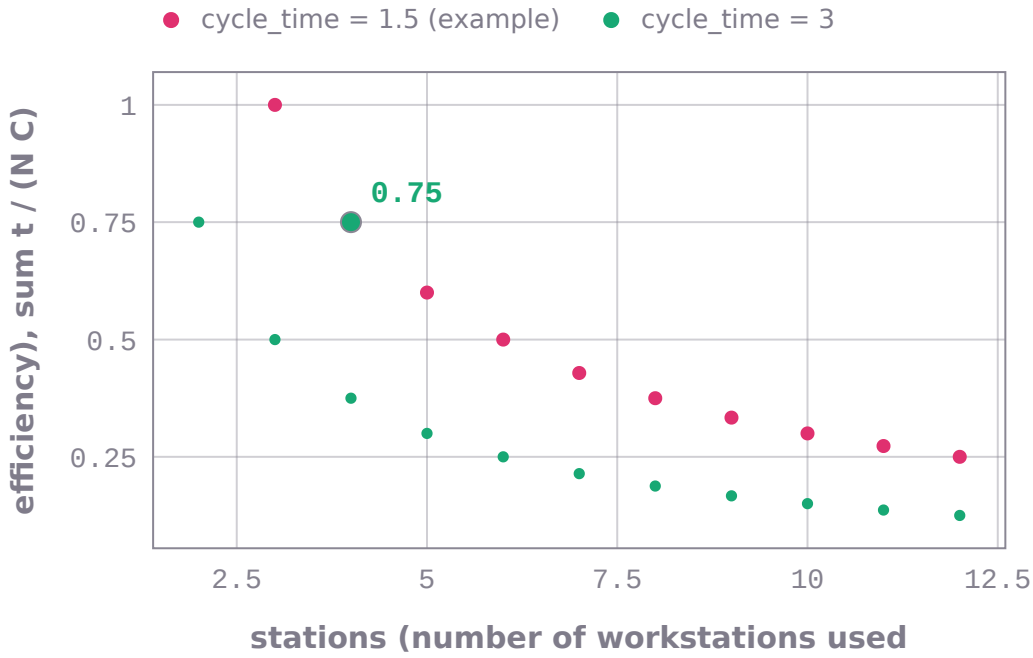
Efficiency of a balanced line: total task time / (stations x cycle time), the balance delay and the idle time per cycle.

$$E = \frac{\sum t_i}{N\,C}, \quad \text{balance delay} = 1 - E$$

input	meaning	condition
`task_times`	time of each task	
`cycle_time`	cycle time	
`stations`	number of workstations used	

output	meaning
`efficiency`	sum t / (N C)
`balance_delay`	1 - efficiency
`idle_time`	N C - sum t per cycle
`total_task_time`	sum of task times
`minimum_stations`	theoretical minimum stations

- **Method:** E = sum t / (N C); idle = N C - sum t
- **Assumes:** task times > 0; each task <= cycle time; sum $t \leq N$ C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



efficiency (sum t / (N C)) against stations (number of workstations used) from 1 to 12. One curve for each value of cycle_time (cycle time): 1.5, 3. Other inputs as in the worked example: task_times = [0.5, 1.2, 0.8, 0.4, 1.0, 0.6]. 10 points per curve computed by industrial-engineering-ai999x; marked: the worked example, stations = 4, efficiency = 0.75.

```
industrial_engineering_ai999x.solve('ie_line_efficiency', task_times=[0.5, 1.2, 0.8, 0.4,
1.0, 0.6], cycle_time=1.5, stations=4)
# -> efficiency = 0.75, balance_delay = 0.25, idle_time = 1.5, total_task_time = 4.5,
minimum_stations = 3   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_line_efficiency', task_times=[45, 30, 50, 20, 35,
40], cycle_time=60, stations=5)
# -> efficiency = 0.733333, balance_delay = 0.266667, idle_time = 80, total_task_time =
220, minimum_stations = 4   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line efficiency with task times = [0.5, 1.2, 0.8, 0.4, 1, 0.6], cycle time = 1.5, workstations = 4.	SOLVED	efficiency = 0.75, balance_delay = 0.25, idle_time = 1.5, total_task_time = 4.5, minimum_stations = 3
Line efficiency with task times = [45, 30, 50, 20, 35, 40], cycle time = 60, workstations = 5.	SOLVED	efficiency = 0.733333, balance_delay = 0.266667, idle_time = 80, total_task_time = 220, minimum_stations = 4
Line efficiency with task times = [0.5, 1.2, 0.8, 0.4, 1, 0.6], cycle time = 1.5.	NEEDS_INPUT	ie_line_efficiency needs stations (number of workstations used). Example: solve('ie_line_efficiency', task_times=[0.5, 1
Line efficiency with task times = [0.5, 1.2, 0.8, 0.4, 1, 0.6], cycle time = 1.5, workstations = 4, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Line efficiency with task times = [0.5, 2.2, 0.8], cycle time = 1.5, workstations = 4.	INVALID_INPUT	Line balance efficiency, balance delay and idle time: every task must fit within the cycle time.

Line balancing by the largest candidate rule, `ie_largest_candidate_rule`

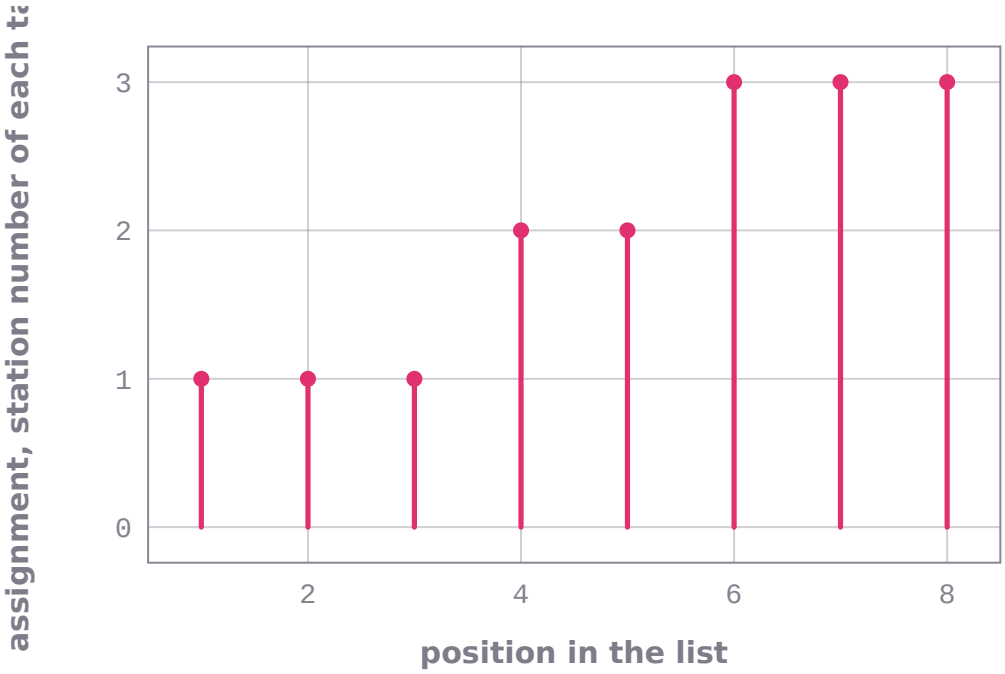
Assign tasks to stations: fill each station with the longest eligible task that still fits (all predecessors assigned), then open the next station.

$$E = \frac{\sum t_i}{N\,C}$$

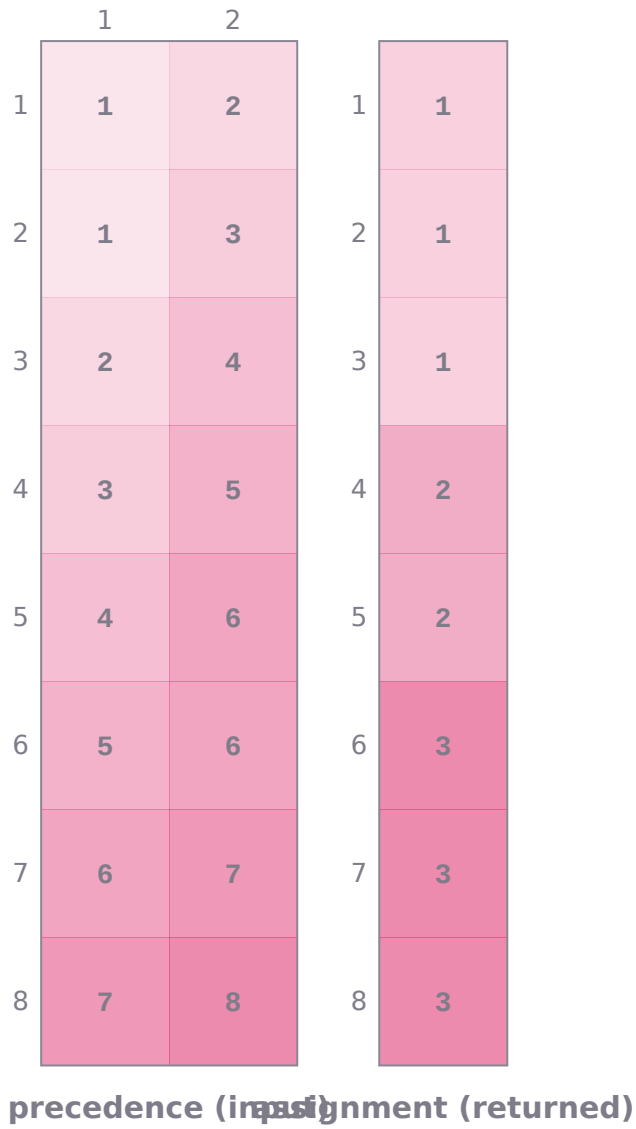
input	meaning	condition
`task_times`	time of each task (tasks numbered 1, 2, ...)	
`precedence`	precedence rows [a, b]: task a must be done before task b	
`cycle_time`	cycle time	

output	meaning
`stations`	number of stations
`assignment`	station number of each task
`station_times`	work assigned to each station
`efficiency`	sum t / (N C)
`idle_time`	N C - sum t
`minimum_stations`	theoretical minimum stations

- **Method:** largest candidate rule: at each station repeatedly add the eligible task with the largest time that fits
- **Assumes:** task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



station number of each task (assignment) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (8 values, first 1, last 3, largest 3). Inputs: task_times = [2, 3, 4, 3, 6, 4, 2, 3], precedence = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5…, cycle_time = 10.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): precedence (input); assignment (returned). Returned with them by industrial-engineering-ai999x: stations = 3, efficiency = 0.9, idle_time = 3, minimum_stations = 3. Other inputs: task_times = [2, 3, 4, 3, 6, 4, 2, 3], cycle_time = 10.

```
industrial_engineering_ai999x.solve('ie_largest_candidate_rule', task_times=[2, 3, 4, 3, 6, 4, 2, 3], precedence=[[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], cycle_time=10)
# -> stations = 3, assignment = [1, 1, 1, 2, 2, 3, 3, 3], station_times = [9.0, 9.0, 9.0], efficiency = 0.9, idle_time = 3, minimum_stations = 3   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_largest_candidate_rule', task_times=[5, 3, 4, 3, 6, 1], precedence=[[1, 3], [2, 3], [3, 4], [3, 5], [4, 6], [5, 6]], cycle_time=8)
# -> stations = 3, assignment = [1, 1, 2, 2, 3, 3], station_times = [8.0, 7.0, 7.0], efficiency = 0.916667, idle_time = 2, minimum_stations = 3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line balancing by the largest candidate rule with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], takt time = 10.	SOLVED	stations = 3, assignment = [1, 1, 1, 2, 2, 3, 3, 3], station_times = [9.0, 9.0, 9.0], efficiency = 0.9, idle_time = 3, minimum_stations = 3
Line balancing by the largest candidate rule with task times = [5, 3, 4, 3, 6, 1], precedence relations = [[1, 3], [2, 3], [3, 4], [3, 5], [4, 6], [5, 6]], takt time = 8.	SOLVED	stations = 3, assignment = [1, 1, 2, 2, 3, 3], station_times = [8.0, 7.0, 7.0], efficiency = 0.916667, idle_time = 2, minimum_stations = 3
Line balancing by the largest candidate rule with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]].	NEEDS_INPUT	ie_largest_candidate_rule needs cycle_time (cycle time). Example: solve('ie_largest_candidate_rule', task_times=[2, 3, 4
Line balancing by the largest candidate rule with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], takt time = 10, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Line balancing by the largest candidate rule with task times = [5, 3, 4], precedence relations = [[1, 2], [2, 3], [3, 1]], takt time = 8.	INVALID_INPUT	Line balancing by the largest candidate rule: the precedence relations contain a cycle.

Line balancing by ranked positional weight, `ie_ranked_positional_weight`

Helgeson-Birnie method: rank tasks by positional weight (own time + times of all following tasks) and fill stations in that order.

$$RPW_j = t_j + \sum_{k \in S(j)} t_k$$

input	meaning	condition
`task_times`	time of each task (tasks numbered 1, 2, ...)	
`precedence`	precedence rows [a, b]: task a must be done before task b	
`cycle_time`	cycle time	

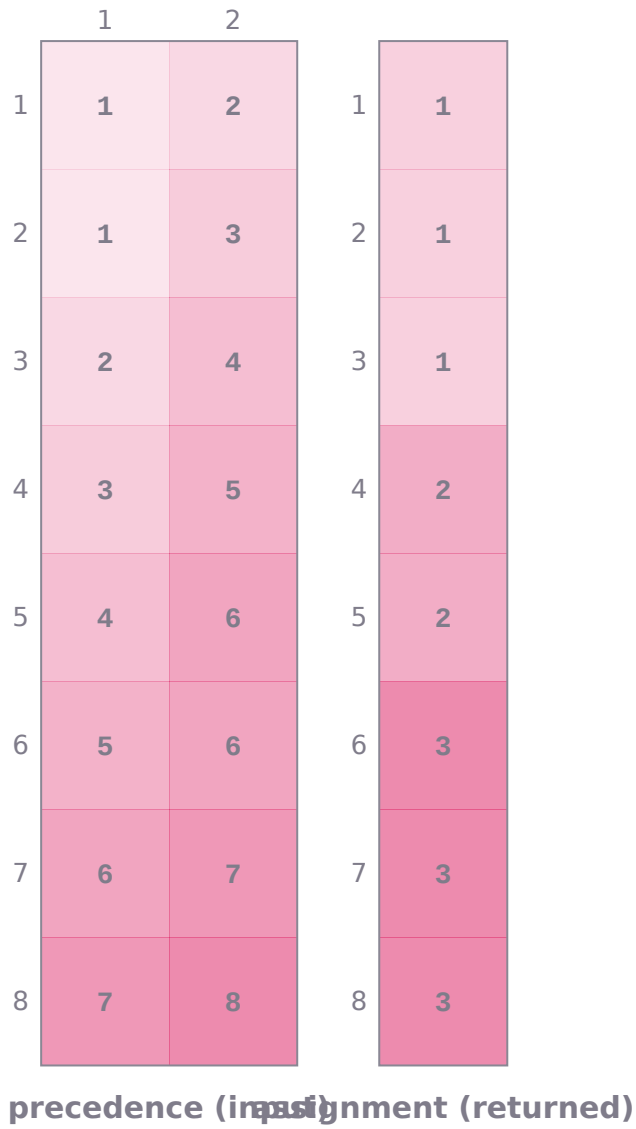
output	meaning
`stations`	number of stations
`assignment`	station number of each task
`station_times`	work assigned to each station
`efficiency`	sum t / (N C)
`idle_time`	N C - sum t
`minimum_stations`	theoretical minimum stations
`positional_weights`	positional weight of each task

- **Method:** RPW_j = t_j + sum of t over all successors of j; assign the eligible task with the highest RPW that fits

- **Assumes:** task times > 0 and <= cycle time; precedence acyclic; ties go to the lower task number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



station number of each task (assignment) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (8 values, first 1, last 3, largest 3); with positional_weights (positional weight of each task). Inputs: task_times = [2, 3, 4, 3, 6, 4, 2, 3], precedence = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5…, cycle_time = 10.



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): precedence (input); assignment (returned). Returned with them by industrial-engineering-ai999x: stations = 3, efficiency = 0.9, idle_time = 3, minimum_stations = 3. Other inputs: task_times = [2, 3, 4, 3, 6, 4, 2, 3], cycle_time = 10.

```
industrial_engineering_ai999x.solve('ie_ranked_positional_weight', task_times=[2, 3, 4, 3, 6, 4, 2, 3], precedence=[[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], cycle_time=10)
# -> stations = 3, assignment = [1, 1, 1, 2, 2, 3, 3, 3], station_times = [9.0, 9.0, 9.0], efficiency = 0.9, idle_time = 3, minimum_stations = 3, positional_weights = [27.0, 15.0, 19.0, 12.0, 15.0, 9.0, 5.0, 3.0]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_ranked_positional_weight', task_times=[5, 3, 4, 3, 6, 1], precedence=[[1, 3], [2, 3], [3, 4], [3, 5], [4, 6], [5, 6]], cycle_time=8)
# -> stations = 3, assignment = [1, 1, 2, 2, 3, 3], station_times = [8.0, 7.0, 7.0], efficiency = 0.916667, idle_time = 2, minimum_stations = 3, positional_weights = [19.0, 17.0, 14.0, 4.0, 7.0, 1.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Line balancing by ranked positional weight with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], takt time = 10.	SOLVED	stations = 3, assignment = [1, 1, 1, 2, 2, 3, 3, 3], station_times = [9.0, 9.0, 9.0], efficiency = 0.9, idle_time = 3, minimum_stations = 3, positional_weights = [27.0, 15.0, 19.0, 12.0, 15.0, 9.0, 5.0, 3.0]
Line balancing by ranked positional weight with task times = [5, 3, 4, 3, 6, 1], precedence relations = [[1, 3], [2, 3], [3, 4], [3, 5], [4, 6], [5, 6]], takt time = 8.	SOLVED	stations = 3, assignment = [1, 1, 2, 2, 3, 3], station_times = [8.0, 7.0, 7.0], efficiency = 0.916667, idle_time = 2, minimum_stations = 3, positional_weights = [19.0, 17.0, 14.0, 4.0, 7.0, 1.0]
Line balancing by ranked positional weight with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]].	NEEDS_INPUT	ie_ranked_positional_weight needs cycle_time (cycle time). Example: solve('ie_ranked_positional_weight', task_times=[2,
Line balancing by ranked positional weight with task times = [2, 3, 4, 3, 6, 4, 2, 3], precedence relations = [[1, 2], [1, 3], [2, 4], [3, 5], [4, 6], [5, 6], [6, 7], [7, 8]], takt time = 10, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Line balancing by ranked positional weight with task times = [5, 3, 9], precedence relations = [[1, 2], [2, 3]], takt time = 8.	INVALID_INPUT	Line balancing by ranked positional weight: every task must fit within the cycle time.

Learning curve: time of the n-th unit, `ie_learning_curve_unit`

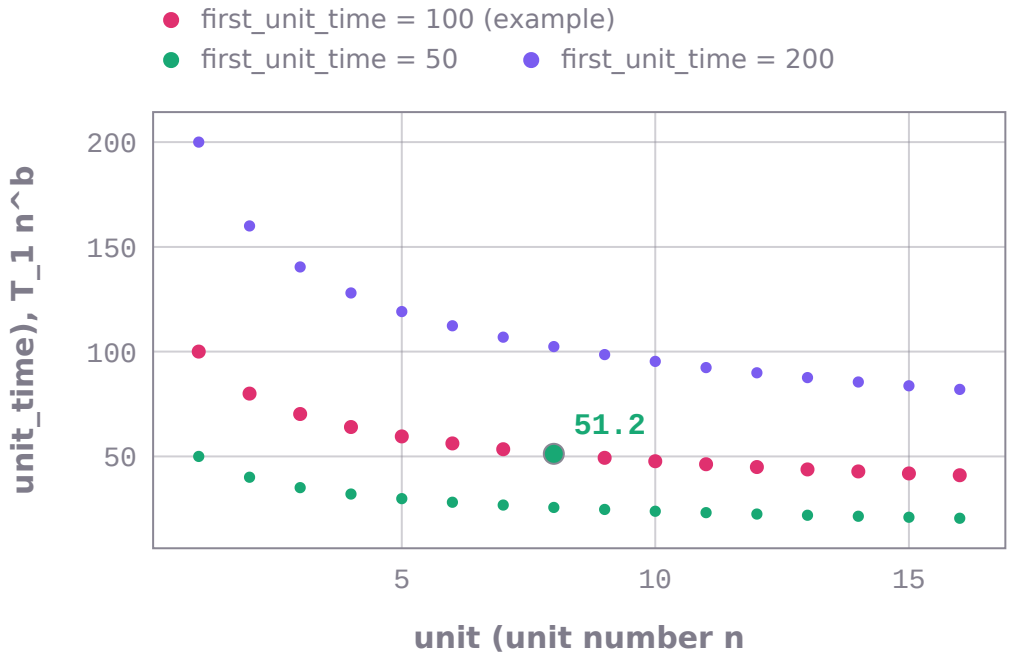
Time of the n-th unit on a learning curve: each doubling of output multiplies the unit time by the learning rate (T_n = T_1 n^b, $b = log2$ r).

$$T_n = T_1\, n^b, \quad b = \frac{\ln r}{\ln 2}$$

input	meaning	condition
`first_unit_time`	time (or cost) of the first unit	
`learning_rate`	learning rate (decimal; 80% = 0.8)	
`unit`	unit number n	

output	meaning
`unit_time`	T_1 n^b
`exponent`	$b = log2$(learning rate)

- **Method:** $T_n = T_1$ n^(log2 r)
- **Assumes:** first_unit_time > 0; 0 < learning_rate <= 1; unit >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



unit_time (T_1 n^b) against unit (unit number n) from 1 to 16. One curve for each value of first_unit_time (time (or cost) of the first unit): 100, 50, 200. Other inputs as in the worked example: learning_rate = 0.8. 16 points per curve computed by industrial-engineering-ai999x; marked: the worked example, unit = 8, unit_time = 51.2.

```
industrial_engineering_ai999x.solve('ie_learning_curve_unit', first_unit_time=100,
learning_rate=0.8, unit=8)
# -> unit_time = 51.2, exponent = -0.321928   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_learning_curve_unit', first_unit_time=50,
learning_rate=0.85, unit=10)
# -> unit_time = 29.141, exponent = -0.234465   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Learning curve unit time with first unit time = 100, learning curve rate = 0.8, unit number = 8.	SOLVED	unit_time = 51.2, exponent = -0.321928
Learning curve unit time with first unit time = 50, learning curve rate = 0.85, unit number = 10.	SOLVED	unit_time = 29.141, exponent = -0.234465
Learning curve unit time with first unit time = 100, learning curve rate = 0.8.	NEEDS_INPUT	ie_learning_curve_unit needs unit (unit number n). Example: solve('ie_learning_curve_unit', first_unit_time=100, learnin
Learning curve unit time with first unit time = 100, learning curve rate = 0.8, unit number = 8, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Learning curve unit time with first unit time = 100, learning curve rate = 1.8, unit number = 8.	INVALID_INPUT	Learning curve: time of the n-th unit: learning_rate must lie strictly between 0 and 1 (write 95% or 0.95).

Learning curve: total and average time of N units, `ie_learning_curve_total`

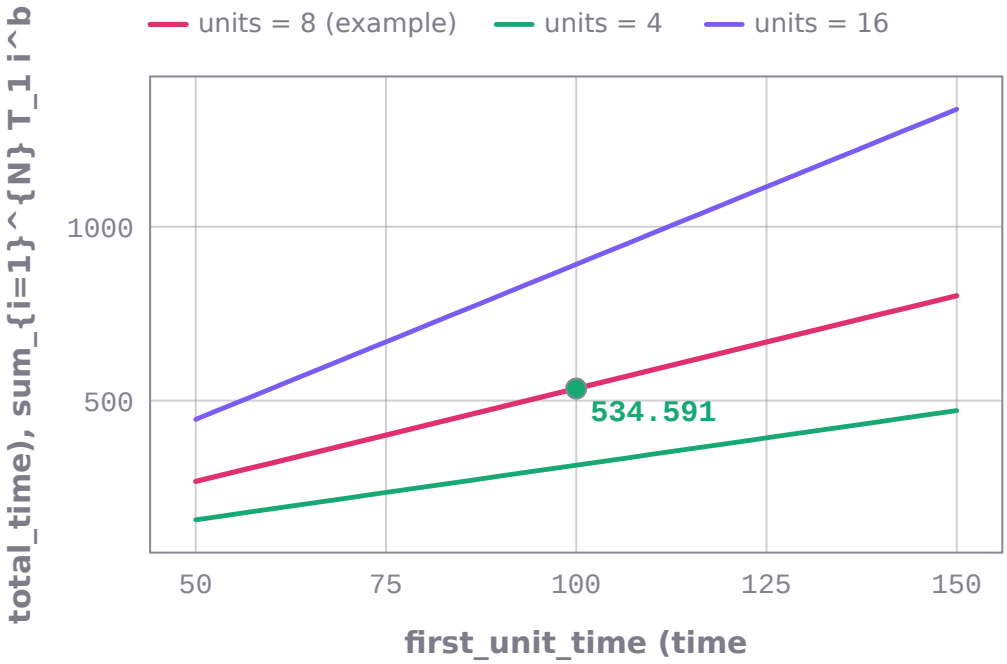
Total and average time to make units 1 to N on a unit learning curve (sum of T_1 i^b).

$$\sum_{i=1}^{N} T_1 \, i^b$$

input	meaning	condition
`first_unit_time`	time (or cost) of the first unit	
`learning_rate`	learning rate (decimal; 80% = 0.8)	
`units`	number of units N	

output	meaning
`total_time`	sum_{$i = 1$}^{N} T_1 i^b
`average_time`	total / N
`last_unit_time`	T_1 N^b

- **Method:** total = T_1 sum_{$i = 1$}^{N} i^b
- **Assumes:** first_unit_time > 0; 0 < learning_rate <= 1; 1 <= units <= 200000
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



total_time (sum_{i=1}^{N} T_1 i^b) against first_unit_time (time (or cost) of the first unit) from 50 to 150. One curve for each value of units (number of units N): 8, 4, 16. Other inputs as in the worked example: learning_rate = 0.8. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, first_unit_time = 100, total_time = 534.591.

```
industrial_engineering_ai999x.solve('ie_learning_curve_total', first_unit_time=100,
learning_rate=0.8, units=8)
# -> total_time = 534.591, average_time = 66.8239, last_unit_time = 51.2   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_learning_curve_total', first_unit_time=60,
learning_rate=0.9, units=25)
# -> total_time = 1062.79, average_time = 42.5118, last_unit_time = 36.7841   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Learning curve total time with first unit time = 100, learning curve rate = 0.8, number of units = 8.	SOLVED	total_time = 534.591, average_time = 66.8239, last_unit_time = 51.2
Learning curve total time with first unit time = 60, learning curve rate = 0.9, number of units = 25.	SOLVED	total_time = 1062.79, average_time = 42.5118, last_unit_time = 36.7841
Learning curve total time with first unit time = 100, learning curve rate = 0.8.	NEEDS_INPUT	ie_learning_curve_total needs units (number of units N). Example: solve('ie_learning_curve_total', first_unit_time=100,
Learning curve total time with first unit time = 100, learning curve rate = 0.8, number of units = 8, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Learning curve total time with first unit time = 100, learning curve rate = 0.8, number of units = 0.	INVALID_INPUT	Learning curve: total and average time of N units: units must be at least 1.

Learning rate from two unit times, `ie_learning_rate_from_two_units`

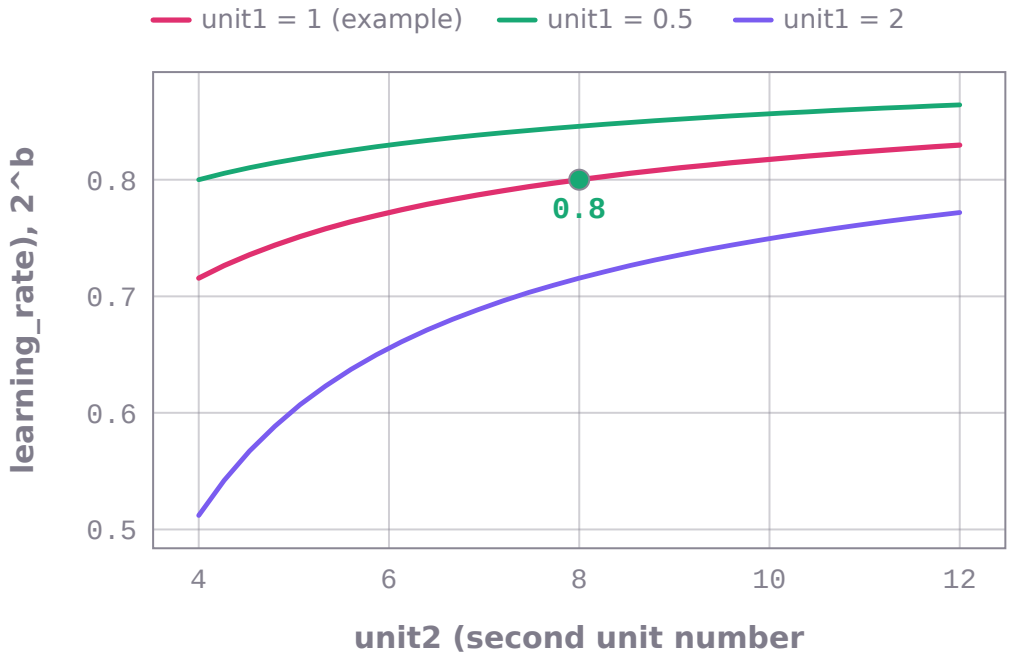
Learning rate, exponent and first-unit time of a unit learning curve through two observed (unit, time) points.

$$b = \frac{\ln(t_2/t_1)}{\ln(n_2/n_1)}, \quad r = 2^b$$

input	meaning	condition
`unit1`	first unit number	
`time1`	time of that unit	
`unit2`	second unit number	
`time2`	time of that unit	

output	meaning
`learning_rate`	2^b
`exponent`	$b = ln$(t2/t1)/ln(n2/n1)
`first_unit_time`	t_1/n_1^b

- **Method:** $b = ln$(t2/t1) / ln(n2/n1), $r = 2^b$, $T_1 = t_1/n_1^b$
- **Assumes:** units and times > 0; unit numbers differ
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



learning_rate (2^b) against unit2 (second unit number) from 4 to 12. One curve for each value of unit1 (first unit number): 1, 0.5, 2. Other inputs as in the worked example: time1 = 100, time2 = 51.2. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, unit2 = 8, learning_rate = 0.8.

```
industrial_engineering_ai999x.solve('ie_learning_rate_from_two_units', unit1=1, time1=100,
unit2=8, time2=51.2)
# -> learning_rate = 0.8, exponent = -0.321928, first_unit_time = 100   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_learning_rate_from_two_units', unit1=10, time1=40,
unit2=40, time2=27)
# -> learning_rate = 0.821584, exponent = -0.28352, first_unit_time = 76.8387   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Learning rate from two units with first unit = 1, time a = 100, second unit = 8, time b = 51.2.	SOLVED	learning_rate = 0.8, exponent = -0.321928, first_unit_time = 100
Learning rate from two units with first unit = 10, time a = 40, second unit = 40, time b = 27.	SOLVED	learning_rate = 0.821584, exponent = -0.28352, first_unit_time = 76.8387
Learning rate from two units with first unit = 1, time a = 100, second unit = 8.	NEEDS_INPUT	ie_learning_rate_from_two_units needs time2 (time of that unit). Example: solve('ie_learning_rate_from_two_units', unit1
Learning rate from two units with first unit = 1, time a = 100, second unit = 8, time b = 51.2, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Learning rate from two units with first unit = 4, time a = 100, second unit = 4, time b = 51.2.	INVALID_INPUT	Learning rate from two unit times: the two unit numbers must differ.

Fit a learning curve to data (log-log regression), `ie_learning_curve_fit`

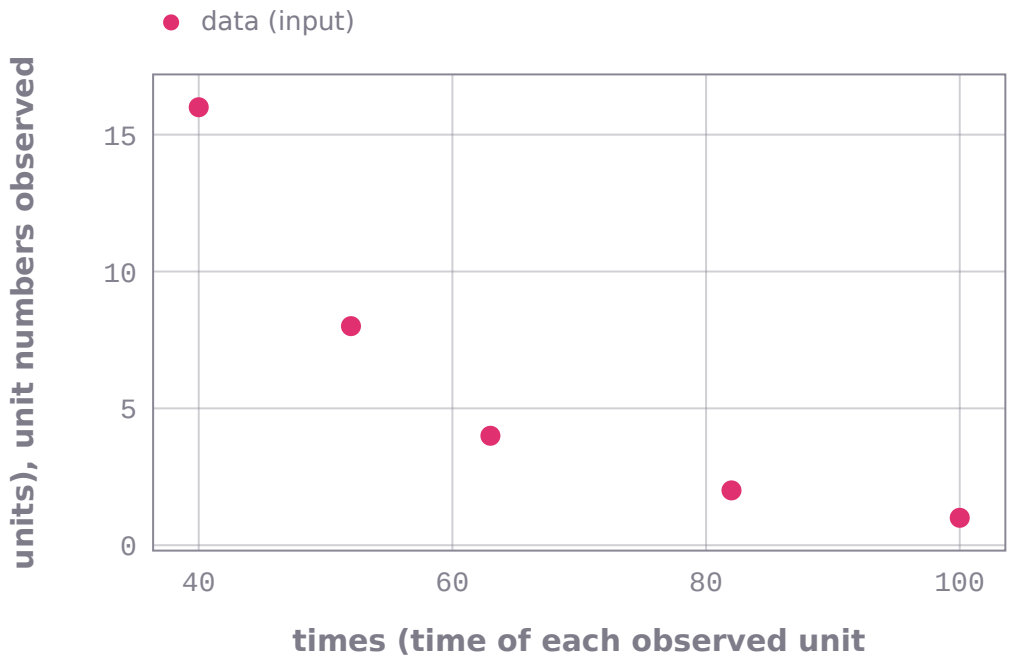
Learning rate and first-unit time estimated by least squares on log(time) against log(unit number).

$$\ln T_n = \ln T_1 + b \ln n$$

input	meaning	condition
`units`	unit numbers observed	
`times`	time of each observed unit	

output	meaning
`learning_rate`	2^b
`exponent`	slope b of the log-log fit
`first_unit_time`	exp(intercept)
`r_squared`	coefficient of determination

- **Method:** regress ln T on ln n: ln $T = ln\, T_1 + b$ ln n; $r = 2^b$
- **Assumes:** positive values; at least 2 different unit numbers
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data units (unit numbers observed) against times (time of each observed unit) as given in the worked example (5 points). Also returned: learning_rate = 0.795483, exponent = -0.330097, first_unit_time = 101.154, r_squared = 0.997447.

```
industrial_engineering_ai999x.solve('ie_learning_curve_fit', units=[1, 2, 4, 8, 16], times=
[100, 82, 63, 52, 40])
# -> learning_rate = 0.795483, exponent = -0.330097, first_unit_time = 101.154, r_squared =
0.997447   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_learning_curve_fit', units=[5, 10, 20, 40], times=
[60, 51, 44, 38])
# -> learning_rate = 0.859168, exponent = -0.218988, first_unit_time = 84.955, r_squared =
0.999367   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Fit a learning curve with unit numbers = [1, 2, 4, 8, 16], unit times = [100, 82, 63, 52, 40].	SOLVED	learning_rate = 0.795483, exponent = -0.330097, first_unit_time = 101.154, r_squared = 0.997447
Fit a learning curve with unit numbers = [5, 10, 20, 40], unit times = [60, 51, 44, 38].	SOLVED	learning_rate = 0.859168, exponent = -0.218988, first_unit_time = 84.955, r_squared = 0.999367
Fit a learning curve with unit numbers = [1, 2, 4, 8, 16].	NEEDS_INPUT	ie_learning_curve_fit needs times (time of each observed unit). Example: solve('ie_learning_curve_fit', units=[1, 2, 4,
Fit a learning curve with unit numbers = [1, 2, 4, 8, 16], unit times = [100, 82, 63, 52, 40], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Fit a learning curve with unit numbers = [3, 3], unit times = [60, 51].	INVALID_INPUT	Fit a learning curve to data (log-log regression): the unit numbers must not all be equal.

Cumulative-average learning curve (Wright), `ie_learning_curve_cumulative_average`

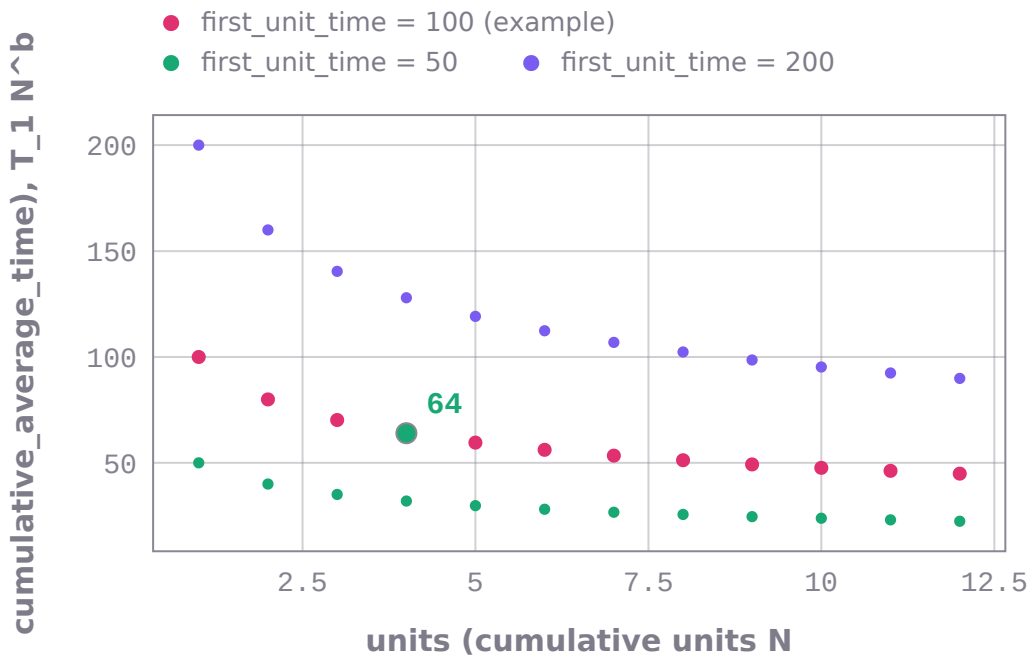
Wright's model, where the cumulative AVERAGE time falls by the learning rate at each doubling: average, total and N-th unit time.

$$\bar{T}_N = T_1 N^b, \quad \text{total}_N = T_1 N^{b+1}$$

input	meaning	condition
`first_unit_time`	time (or cost) of the first unit	
`learning_rate`	learning rate (decimal; 80% = 0.8)	
`units`	cumulative units N	

output	meaning
`cumulative_average_time`	T_1 N^b
`total_time`	T_1 N^{b+1}
`unit_time`	time of unit N: total(N) - total(N - 1)

- **Method:** average(N) = T_1 N^b; total = N average; unit N = total(N) - total(N - 1)
- **Assumes:** first_unit_time > 0; 0 < learning_rate <= 1; units >= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cumulative_average_time (T_1 N^b) against units (cumulative units N) from 1 to 12. One curve for each value of first_unit_time (time (or cost) of the first unit): 100, 50, 200. Other inputs as in the worked example: learning_rate = 0.8. 12 points per curve computed by industrial-engineering-ai999x; marked: the worked example, units = 4, cumulative_average_time = 64.

```
industrial_engineering_ai999x.solve('ie_learning_curve_cumulative_average',
first_unit_time=100, learning_rate=0.8, units=4)
# -> cumulative_average_time = 64, total_time = 256, unit_time = 45.3689   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_learning_curve_cumulative_average',
first_unit_time=2000, learning_rate=0.85, units=50)
# -> cumulative_average_time = 799.246, total_time = 39962.3, unit_time = 613.297
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Cumulative average learning curve with first unit time = 100, learning curve rate = 0.8, number of units = 4.	SOLVED	cumulative_average_time = 64, total_time = 256, unit_time = 45.3689
Cumulative average learning curve with first unit time = 2000, learning curve rate = 0.85, number of units = 50.	SOLVED	cumulative_average_time = 799.246, total_time = 39962.3, unit_time = 613.297
Cumulative average learning curve with first unit time = 100, learning curve rate = 0.8.	NEEDS_INPUT	ie_learning_curve_cumulative_average needs units (cumulative units N). Example: solve('ie_learning_curve_cumulative_aver
Cumulative average learning curve with first unit time = 100, learning curve rate = 0.8, number of units = 4, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Cumulative average learning curve with first unit time = 100,	INVALID_INPUT	Cumulative-average learning curve (Wright): learning_rate must lie strictly between 0 and 1 (write 95% or 0.95).

question	outcome	result
learning curve rate = 0, number of units = 4.		

Centre-of-gravity location, `ie_center_of_gravity`

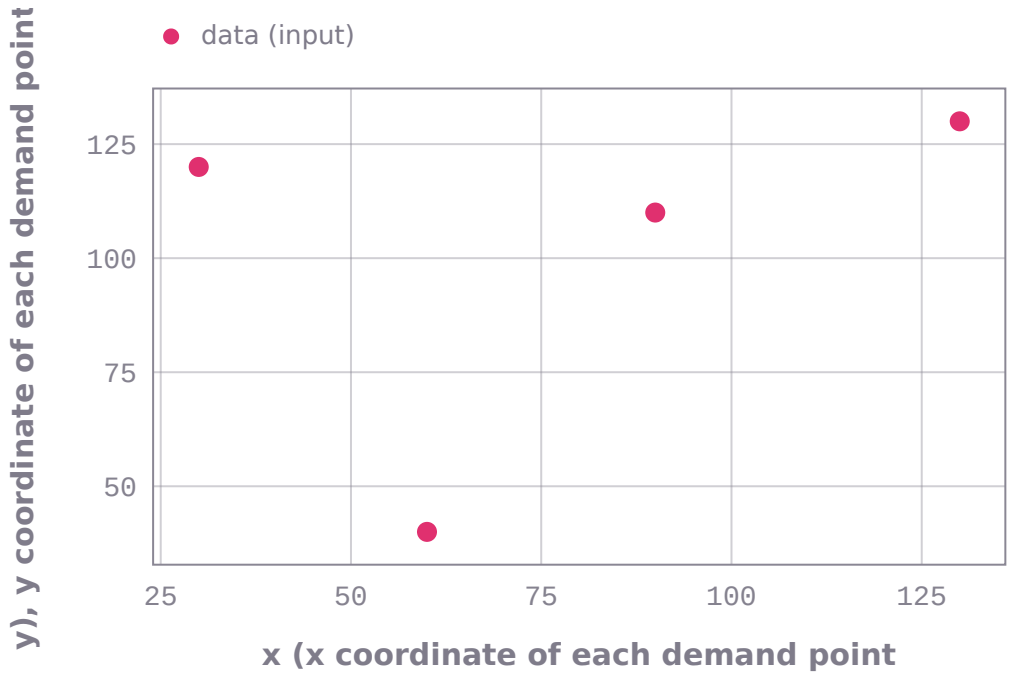
Location of a facility at the load-weighted average of the demand points' coordinates.

$$x^* = \frac{\sum w_i x_i}{\sum w_i}, \quad y^* = \frac{\sum w_i y_i}{\sum w_i}$$

input	meaning	condition
`x`	x coordinate of each demand point	
`y`	y coordinate of each demand point	
`weights`	load or volume of each point	

output	meaning
`x`	sum w x / sum w
`y`	sum w y / sum w
`total_weight`	sum of loads

- **Method:** x* = sum w_i x_i / sum w_i, y* = sum w_i y_i / sum w_i
- **Assumes:** equal lengths; weights >= 0, not all 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y coordinate of each demand point) against x (x coordinate of each demand point) as given in the worked example (4 points). Also returned: x = 66.6667, y = 93.3333, total_weight = 6000. Other inputs: weights = [2000, 1000, 1000, 2000].

```
industrial_engineering_ai999x.solve('ie_center_of_gravity', x=[30, 90, 130, 60], y=[120,
110, 130, 40], weights=[2000, 1000, 1000, 2000])
# -> x = 66.6667, y = 93.3333, total_weight = 6000   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_center_of_gravity', x=[2, 6, 8], y=[3, 9, 4],
weights=[400, 300, 200])
# -> x = 4.66667, y = 5.22222, total_weight = 900   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Centre-of-gravity location with x coordinates = [30, 90, 130, 60], y coordinates = [120, 110, 130, 40], loads = [2000, 1000, 1000, 2000].	SOLVED	x = 66.6667, y = 93.3333, total_weight = 6000
Centre-of-gravity location with x coordinates = [2, 6, 8], y coordinates = [3, 9, 4], loads = [400, 300, 200].	SOLVED	x = 4.66667, y = 5.22222, total_weight = 900
Centre-of-gravity location with x coordinates = [30, 90, 130, 60], y coordinates = [120, 110, 130, 40].	NEEDS_INPUT	ie_center_of_gravity needs weights (load or volume of each point). Example: solve('ie_center_of_gravity', x=[30, 90, 130
Centre-of-gravity location with x coordinates = [30, 90, 130, 60], y coordinates = [120, 110, 130, 40], loads = [2000, 1000, 1000, 2000], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Centre-of-gravity location with x coordinates = [30, 90], y coordinates = [120, 110, 130], loads = [2000, 1000].	INVALID_INPUT	Centre-of-gravity location: x, y and weights must have the same number of values.

Minimum weighted-distance location (Weber problem), `ie_weber_location`

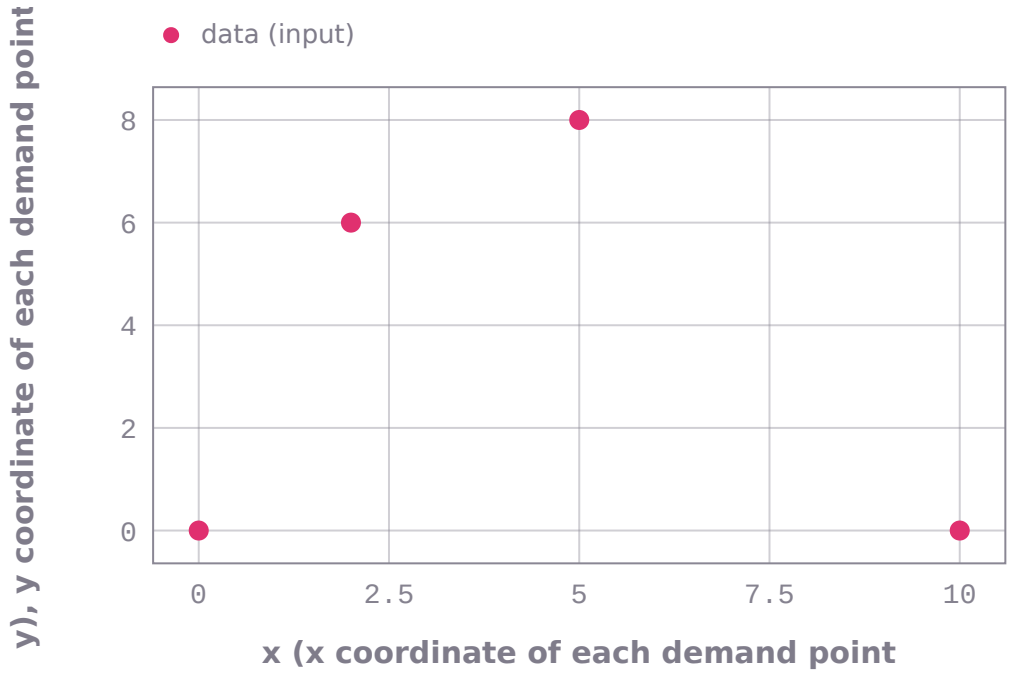
Location that minimises the sum of load x straight-line distance to the demand points (Weiszfeld's algorithm; a demand point when it is optimal).

$$\min_{x,y} \sum_i w_i \sqrt{(x - x_i)^2 + (y - y_i)^2}$$

input	meaning	condition
`x`	x coordinate of each demand point	
`y`	y coordinate of each demand point	
`weights`	load or volume of each point	

output	meaning
`x`	optimal x
`y`	optimal y
`total_weighted_distance`	sum w_i distance_i at the optimum

- **Method:** Weiszfeld: x <- sum (w_i x_i / d_i) / sum (w_i / d_i) until it settles; a demand point i is optimal when |sum_{$j \neq i$} w_j (p_i - p_j)/d_ij| <= w_i
- **Assumes:** at least 2 distinct points; weights >= 0, not all 0; Euclidean distance
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y coordinate of each demand point) against x (x coordinate of each demand point) as given in the worked example (4 points). Also returned: x = 3.19149, y = 5.10638, total_weighted_distance = 19.434. Other inputs: weights = [1, 1, 1, 1].

```
industrial_engineering_ai999x.solve('ie_weber_location', x=[0, 10, 5, 2], y=[0, 0, 8, 6],
weights=[1, 1, 1, 1])
# -> x = 3.19149, y = 5.10638, total_weighted_distance = 19.434   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_weber_location', x=[30, 90, 130, 60], y=[120, 110,
130, 40], weights=[2000, 1000, 1000, 2000])
# -> x = 63.8645, y = 97.2782, total_weighted_distance = 299234   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Weber problem location with x coordinates = [0, 10, 5, 2], y coordinates = [0, 0, 8, 6], loads = [1, 1, 1, 1].	SOLVED	x = 3.19149, y = 5.10638, total_weighted_distance = 19.434
Weber problem location with x coordinates = [30, 90, 130, 60], y coordinates = [120, 110, 130, 40], loads = [2000, 1000, 1000, 2000].	SOLVED	x = 63.8645, y = 97.2782, total_weighted_distance = 299234
Weber problem location with x coordinates = [0, 10, 5, 2], y coordinates = [0, 0, 8, 6].	NEEDS_INPUT	ie_weber_location needs weights (load or volume of each point). Example: solve('ie_weber_location', x=[0, 10, 5, 2], y=[

question	outcome	result
Weber problem location with x coordinates = [0, 10, 5, 2], y coordinates = [0, 0, 8, 6], loads = [1, 1, 1, 1], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Weber problem location with x coordinates = [1, 1], y coordinates = [2, 2], loads = [1, 1].	INVALID_INPUT	Minimum weighted-distance location (Weber problem): the points must be distinct.

Minimum rectilinear-distance location (weighted median), `ie_rectilinear_location`

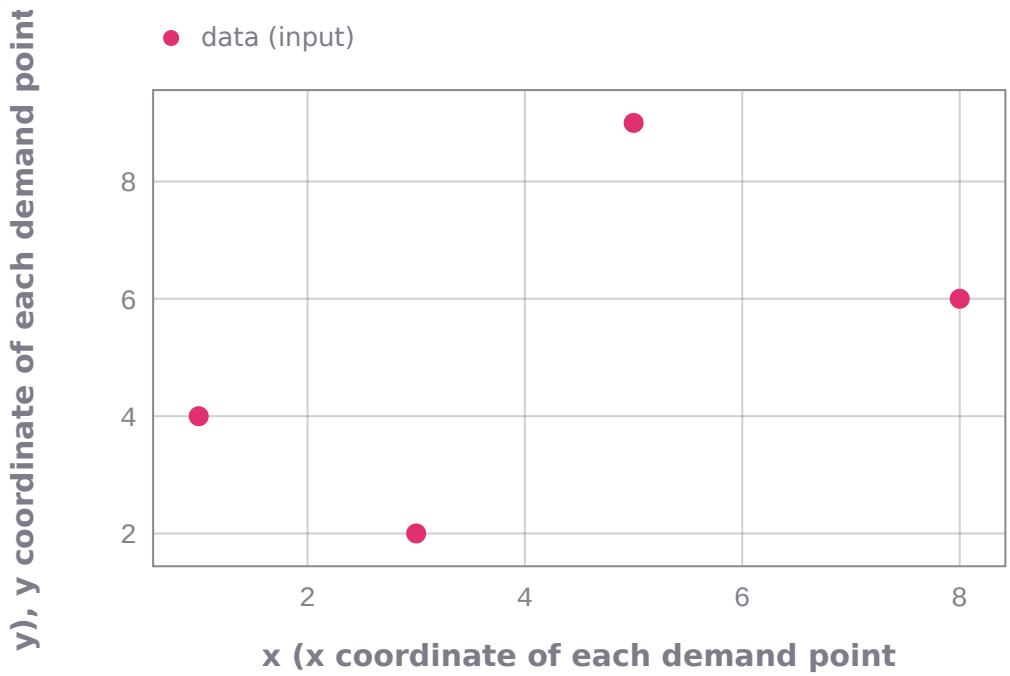
Location minimising the sum of load x rectilinear (city-block) distance: the weighted median of x and of y.

$$\min_{x,y} \sum_i w_i \left(|x - x_i| + |y - y_i| \right)$$

input	meaning	condition
`x`	x coordinate of each demand point	
`y`	y coordinate of each demand point	
`weights`	load or volume of each point	

output	meaning
`x`	weighted median of x
`y`	weighted median of y
`total_weighted_distance`	sum w_i (|x - x_i| + |y - y_i|)

- **Method:** sort by coordinate; the first point where cumulative weight reaches half the total
- **Assumes:** weights >= 0, not all 0; refused when the optimum is a whole interval (cumulative weight exactly half)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The data y (y coordinate of each demand point) against x (x coordinate of each demand point) as given in the worked example (4 points). Also returned: x = 3, y = 4, total_weighted_distance = 47. Other inputs: weights = [5, 2, 3, 1].

```
industrial_engineering_ai999x.solve('ie_rectilinear_location', x=[3, 8, 5, 1], y=[2, 6, 9, 4], weights=[5, 2, 3, 1])
# -> x = 3, y = 4, total_weighted_distance = 47   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_rectilinear_location', x=[30, 90, 130, 60], y=[120, 110, 130, 40], weights=[2000, 1000, 1500, 2000])
# -> x = 60, y = 120, total_weighted_distance = 380000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Rectilinear distance location with x coordinates = [3, 8, 5, 1], y coordinates = [2, 6, 9, 4], loads = [5, 2, 3, 1].	SOLVED	x = 3, y = 4, total_weighted_distance = 47
Rectilinear distance location with x coordinates = [30, 90, 130, 60], y coordinates = [120, 110, 130, 40], loads = [2000, 1000, 1500, 2000].	SOLVED	x = 60, y = 120, total_weighted_distance = 380000
Rectilinear distance location with x coordinates = [3, 8, 5, 1], y coordinates = [2, 6, 9, 4].	NEEDS_INPUT	ie_rectilinear_location needs weights (load or volume of each point). Example: solve('ie_rectilinear_location', x=[3, 8,
Rectilinear distance location with x coordinates = [3, 8, 5, 1], y coordinates = [2, 6, 9, 4], loads = [5, 2, 3, 1], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Rectilinear distance location with x coordinates = [3, 8], y coordinates = [2, 6], loads = [0, 0].	INVALID_INPUT	Minimum rectilinear-distance location (weighted median): the weights must not all be 0.

Factor-rating method for site selection, `ie_factor_rating`

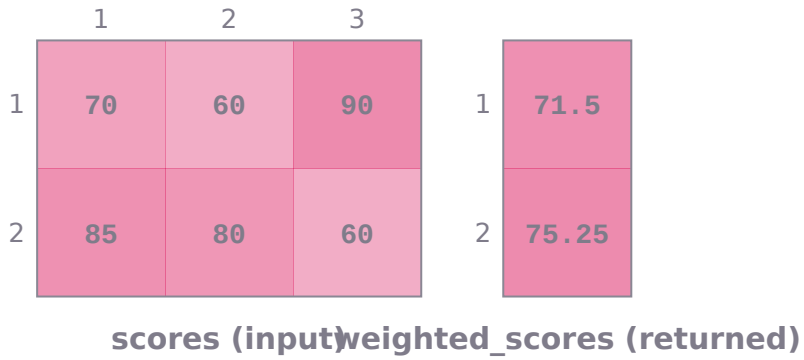
Weighted score of each candidate site (sum of factor weight x score) and the best site.

$$S_i = \sum_j w_j \, s_{ij}$$

input	meaning	condition
`weights`	weight of each factor	
`scores`	matrix: one row per site, one score per factor	

output	meaning
`weighted_scores`	weighted score of each site
`best_site`	site with the highest score
`best_score`	highest weighted score

- **Method:** score_i = sum_j w_j s_ij
- **Assumes:** weights >= 0; one score per factor in every row; ties go to the lower site number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



The matrices of the worked example as heatmaps (rows and columns numbered; each cell shaded by its magnitude, negative values in the second colour; each panel shaded by its own largest magnitude): scores (input); weighted_scores (returned). Returned with them by industrial-engineering-ai999x: best_site = 2, best_score = 75.25. Other inputs: weights = [0.25, 0.45, 0.3].

```
industrial_engineering_ai999x.solve('ie_factor_rating', weights=[0.25, 0.45, 0.3], scores=
[[70, 60, 90], [85, 80, 60]])
# -> weighted_scores = [71.5, 75.25], best_site = 2, best_score = 75.25   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_factor_rating', weights=[0.1, 0.2, 0.3, 0.4],
scores=[[80, 70, 60, 50], [60, 70, 80, 90], [75, 75, 75, 75]])
# -> weighted_scores = [60.0, 80.0, 75.0], best_site = 2, best_score = 80   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Factor rating with factor weights = [0.25, 0.45, 0.3], site scores = [[70, 60, 90], [85, 80, 60]].	SOLVED	weighted_scores = [71.5, 75.25], best_site = 2, best_score = 75.25
Factor rating with factor weights = [0.1, 0.2, 0.3, 0.4], site scores = [[80, 70, 60, 50], [60, 70, 80, 90], [75, 75, 75, 75]].	SOLVED	weighted_scores = [60.0, 80.0, 75.0], best_site = 2, best_score = 80
Factor rating with factor weights = [0.25, 0.45, 0.3].	NEEDS_INPUT	ie_factor_rating needs scores (matrix: one row per site, one score per factor). Example: solve('ie_factor_rating', weigh
Factor rating with factor weights = [0.25, 0.45, 0.3], site scores = [[70, 60, 90], [85, 80, 60]], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Factor rating with factor weights = [0.25, 0.45], site scores = [[70, 60, 90], [85, 80, 60]].	INVALID_INPUT	Factor-rating method for site selection: scores must have 2 columns.

Locational cost-volume (break-even) analysis, `ie_location_cost_volume`

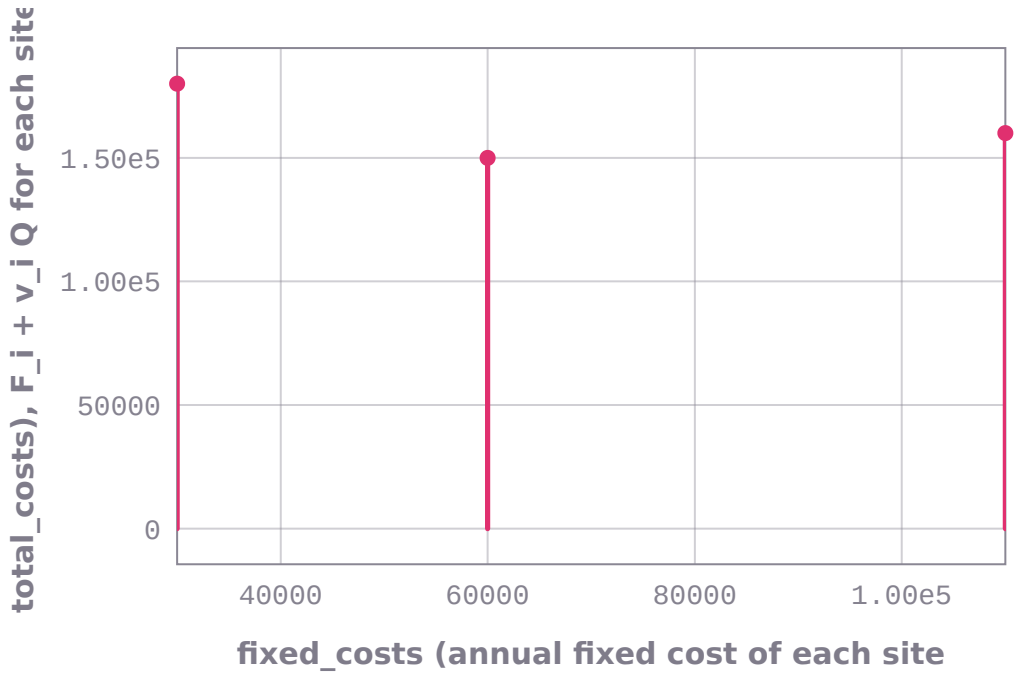
Total annual cost of each candidate site at an expected volume (fixed + variable x volume) and the cheapest site.

$$TC_i(Q) = F_i + v_i\,Q$$

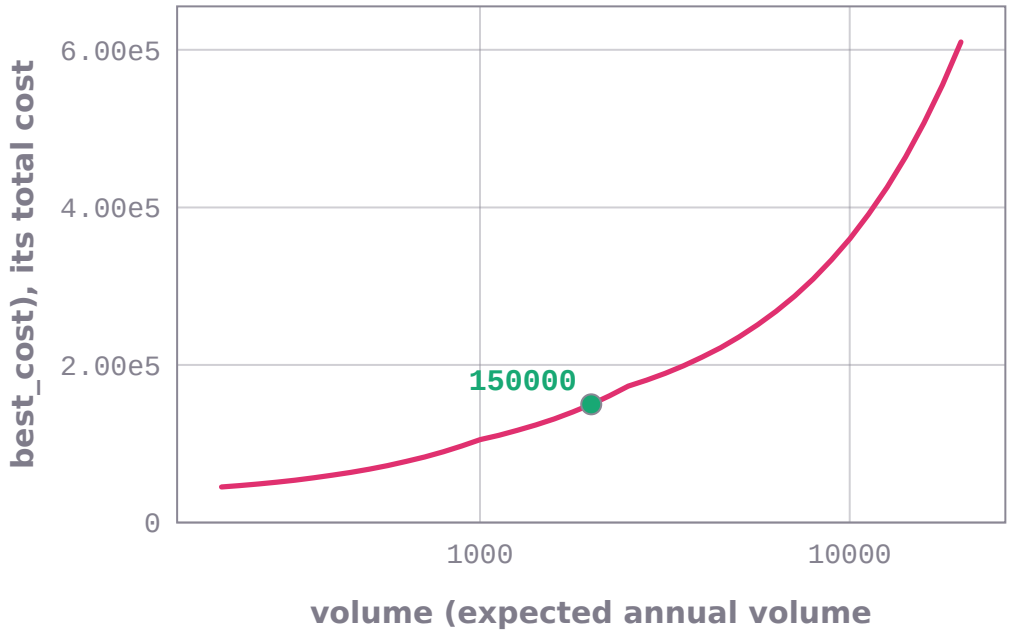
input	meaning	condition
`fixed_costs`	annual fixed cost of each site	
`variable_costs`	variable cost per unit at each site	
`volume`	expected annual volume	

output	meaning
`total_costs`	F_i + v_i Q for each site
`best_site`	cheapest site
`best_cost`	its total cost

- **Method:** $TC_i = F_i + v_i$ Q; the minimum wins
- **Assumes:** costs >= 0; volume >= 0; ties go to the lower site number
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F_i + v_i Q for each site (total_costs) returned by industrial-engineering-ai999x for the worked example, one value for each fixed_costs (annual fixed cost of each site) (3 values, first 180000, last 160000, largest 180000). Inputs: fixed_costs = [30000, 60000, 110000], variable_costs = [75, 45, 25], volume = 2000.



best_cost (its total cost) against volume (expected annual volume) from 200 to 20000 on a logarithmic axis. Other inputs as in the worked example: fixed_costs = [30000, 60000, 110000], variable_costs = [75, 45, 25]. 41 points computed by industrial-engineering-ai999x; marked: the worked example, volume = 2000, best_cost = 150000.

```
industrial_engineering_ai999x.solve('ie_location_cost_volume', fixed_costs=[30000, 60000, 110000], variable_costs=[75, 45, 25], volume=2000)
# -> total_costs = [180000.0, 150000.0, 160000.0], best_site = 2, best_cost = 150000 (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_location_cost_volume', fixed_costs=[150000, 250000], variable_costs=[62, 38], volume=3500)
# -> total_costs = [367000.0, 383000.0], best_site = 1, best_cost = 367000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Locational cost-volume analysis with fixed cost = [30000, 60000, 110000], variable cost = [75, 45, 25], expected volume = 2000.	SOLVED	total_costs = [180000.0, 150000.0, 160000.0], best_site = 2, best_cost = 150000
Locational cost-volume analysis with fixed cost = [150000, 250000], variable cost = [62, 38], expected volume = 3500.	SOLVED	total_costs = [367000.0, 383000.0], best_site = 1, best_cost = 367000
Locational cost-volume analysis with fixed cost = [30000, 60000, 110000], variable cost = [75, 45, 25].	NEEDS_INPUT	ie_location_cost_volume needs volume (expected annual volume). Example: solve('ie_location_cost_volume', fixed_costs=[30
Locational cost-volume analysis with fixed cost = [30000, 60000, 110000], variable cost = [75, 45, 25], expected volume = 2000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Locational cost-volume analysis with fixed cost = [30000, 60000], variable cost = [75], expected volume = 2000.	INVALID_INPUT	Locational cost-volume (break-even) analysis: variable_costs needs at least 2 values.

Break-even point, `ie_break_even`

Volume (and revenue) at which revenue equals fixed plus variable cost, and the volume for a target profit.

$$Q_{BE} = \frac{F}{p - v}, \quad R_{BE} = p\,Q_{BE}$$

input	meaning	condition
`fixed_cost`	fixed cost per period	
`price`	selling price per unit	
`variable_cost`	variable cost per unit	
`target_profit`	target profit (optional)	

output	meaning
`break_even_units`	F / (p - v)
`break_even_revenue`	p F / (p - v)
`contribution_margin`	p - v
`contribution_margin_ratio`	(p - v)/p
`units_for_target_profit`	(F + target)/(p - v)

- **Method:** $Q_BE = F/(p - v)$
- **Assumes:** fixed_cost >= 0; price > variable_cost >= 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



break_even_units (F / (p - v)) against price (selling price per unit) from 12.5 to 37.5. One curve for each value of fixed_cost (fixed cost per period): 10000, 5000, 20000. Other inputs as in the worked example: variable_cost = 15. 27 points per curve computed by industrial-engineering-ai999x; marked: the worked example, price = 25, break_even_units = 1000.

```
industrial_engineering_ai999x.solve('ie_break_even', fixed_cost=10000, price=25,
variable_cost=15)
# -> break_even_units = 1000, break_even_revenue = 25000, contribution_margin = 10,
contribution_margin_ratio = 0.4   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_break_even', fixed_cost=50000, price=8,
variable_cost=3, target_profit=20000)
# -> break_even_units = 10000, break_even_revenue = 80000, contribution_margin = 5,
contribution_margin_ratio = 0.625, units_for_target_profit = 14000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Break-even point with fixed costs = 10000, selling price = 25, variable cost per unit = 15.	SOLVED	break_even_units = 1000, break_even_revenue = 25000, contribution_margin = 10, contribution_margin_ratio = 0.4
Break-even point with fixed costs = 50000, selling price = 8, variable cost per unit = 3, desired profit = 20000.	SOLVED	break_even_units = 10000, break_even_revenue = 80000, contribution_margin = 5, contribution_margin_ratio = 0.625, units_for_target_profit = 14000
Break-even point with fixed costs = 10000, selling price = 25.	NEEDS_INPUT	ie_break_even needs variable_cost (variable cost per unit). Example: solve('ie_break_even', fixed_cost=10000, price=25,
Break-even point with fixed costs = 10000, selling price = 25, variable cost per unit = 15, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Break-even point with fixed costs = 10000, selling price = 15, variable cost per unit = 25.	INVALID_INPUT	Break-even point: the price must exceed the variable cost per unit.

Multiproduct break-even (sales-mix weighted), `ie_multiproduct_break_even`

Break-even revenue of several products sold in a fixed revenue mix: F / sum $(1 - v_i/p_i)$ w_i.

$$BEP_{\$} = \frac{F}{\sum_i \left(1 - \frac{V_i}{P_i}\right) W_i}$$

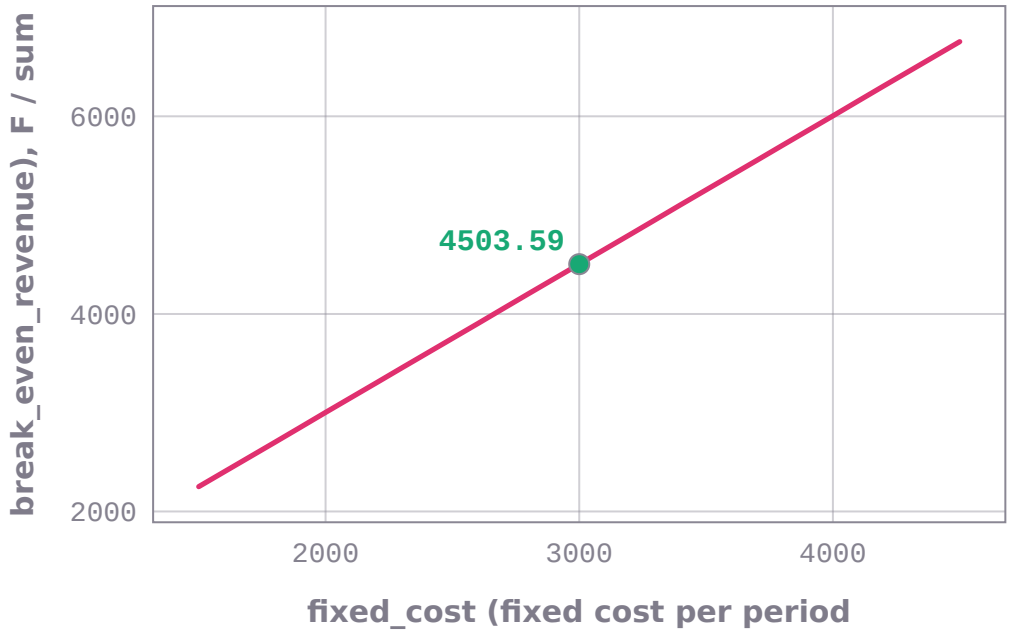
input	meaning	condition
`fixed_cost`	fixed cost per period	
`prices`	selling price of each product	
`variable_costs`	variable cost per unit of each product	
`sales_mix`	share of revenue of each product (sum 1)	

output	meaning
`break_even_revenue`	F / sum $(1 - v_i/p_i)$ w_i
`weighted_contribution_ratio`	sum $(1 - v_i/p_i)$ w_i
`revenue_by_product`	break-even revenue of each product
`units_by_product`	break-even units of each product

- **Method:** BEP() $= F/sum[$(1 - V_i/P_i)$ W_i]
- **Assumes:** fixed_cost >= 0; prices > 0; mix fractions of revenue adding to 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



break-even revenue of each product (revenue_by_product) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 2251.79, last 900.718, largest 2251.79); with units_by_product (break-even units of each product). Inputs: fixed_cost = 3000, prices = [2.95, 1.25, 0.8], variable_costs = [1.25, 0.3, 0.2], sales_mix = [0.5, 0.3, 0.2].



break_even_revenue (F / sum (1 - v_i/p_i) w_i) against fixed_cost (fixed cost per period) from 1500 to 4500. Other inputs as in the worked example: prices = [2.95, 1.25, 0.8], variable_costs = [1.25, 0.3, 0.2], sales_mix = [0.5, 0.3, 0.2]. 31 points computed by industrial-engineering-ai999x; marked: the worked example, fixed_cost = 3000, break_even_revenue = 4503.59.

```
industrial_engineering_ai999x.solve('ie_multiproduct_break_even', fixed_cost=3000, prices=
[2.95, 1.25, 0.8], variable_costs=[1.25, 0.3, 0.2], sales_mix=[0.5, 0.3, 0.2])
# -> break_even_revenue = 4503.59, weighted_contribution_ratio = 0.666136,
revenue_by_product = [2251.79380184, 1351.07628111, 900.717520737], units_by_product =
[763.319932828, 1080.86102488, 1125.89690092]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_multiproduct_break_even', fixed_cost=120000,
prices=[50, 80], variable_costs=[30, 40], sales_mix=[0.6, 0.4])
# -> break_even_revenue = 272727, weighted_contribution_ratio = 0.44, revenue_by_product =
[163636.363636, 109090.909091], units_by_product = [3272.72727273, 1363.63636364]
(SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Multiproduct break-even with fixed costs = 3000, selling prices = [2.95, 1.25, 0.8], variable cost per unit = [1.25, 0.3, 0.2], revenue mix = [0.5, 0.3, 0.2].	SOLVED	break_even_revenue = 4503.59, weighted_contribution_ratio = 0.666136, revenue_by_product = [2251.79380184, 1351.07628111, 900.717520737], units_by_product = [763.319932828, 1080.86102488, 1125.89690092]
Multiproduct break-even with fixed costs = 120000, selling prices = [50, 80], variable cost per unit = [30, 40], revenue mix = [0.6, 0.4].	SOLVED	break_even_revenue = 272727, weighted_contribution_ratio = 0.44, revenue_by_product = [163636.363636, 109090.909091], units_by_product = [3272.72727273, 1363.63636364]

question	outcome	result
Multiproduct break-even with fixed costs = 3000, selling prices = [2.95, 1.25, 0.8], variable cost per unit = [1.25, 0.3, 0.2].	NEEDS_INPUT	ie_multiproduct_break_even needs sales_mix (share of revenue of each product (sum 1)). Example: solve('ie_multiproduct_b
Multiproduct break-even with fixed costs = 3000, selling prices = [2.95, 1.25, 0.8], variable cost per unit = [1.25, 0.3, 0.2], revenue mix = [0.5, 0.3, 0.2], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Multiproduct break-even with fixed costs = 3000, selling prices = [2.95, 1.25], variable cost per unit = [1.25, 0.3], revenue mix = [0.5, 0.3].	INVALID_INPUT	Multiproduct break-even (sales-mix weighted): the sales mix fractions must add up to 1.

Standard time from a time study, `ie_standard_time`

Normal time (mean observed time x performance rating) and standard time with a personal, fatigue and delay allowance.

$$NT = \bar{t} \times PR, \quad ST = NT(1 + A) \text{ or } \frac{NT}{1 - A}$$

input	meaning	condition
`observed_times`	observed cycle times	
`rating`	performance rating (decimal; 110% = 1.1)	
`allowance`	allowance fraction (decimal; 15% = 0.15)	
`basis`	job: $ST = NT\ (1 + A)$; shift: $ST = NT/(1 - A)$, A a share of the workday	

output	meaning
`standard_time`	standard time
`normal_time`	mean x rating
`mean_observed_time`	mean observed time

- **Method:** NT = mean x rating; $ST = NT\ (1 + A)$ on a job basis or $NT/(1 - A)$ on a workday basis
- **Assumes:** observed times > 0; rating > 0; 0 <= allowance < 1; the allowance basis must be stated
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



standard_time (standard time) against rating (performance rating (decimal; 110% = 1.1)) from 0.55 to 1.65. One curve for each value of allowance (allowance fraction (decimal; 15% = 0.15)): 0.15, 0.26, 0.74. Other inputs as in the worked example: observed_times = [4.2, 4.5, 4.0, 4.3, 4.5], basis = job. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, rating = 1.1, standard_time = 5.4395.



standard_time (standard time) over rating (performance rating (decimal; 110% = 1.1)) from 0.55 to 1.65 and allowance (allowance fraction (decimal; 15% = 0.15)) from 0.02 to 0.98: each of the 21 × 15 cells is one run of industrial-engineering-ai999x, shaded by its value (light 2.4123 to dark 14.0481); iso-lines at standard_time = 4, 6, 8, 10, 12. Other inputs as in the worked example: observed_times = [4.2, 4.5, 4.0, 4.3, 4.5], basis = job. Marked: the worked example, rating = 1.1, allowance = 0.15, standard_time = 5.4395.

```
industrial_engineering_ai999x.solve('ie_standard_time', observed_times=[4.2, 4.5, 4.0, 4.3, 4.5], rating=1.1, allowance=0.15, basis='job')
# -> standard_time = 5.4395, normal_time = 4.73, mean_observed_time = 4.3   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_standard_time', observed_times=[12, 11, 13, 12],
rating=0.95, allowance=0.12, basis='shift')
# -> standard_time = 12.9545, normal_time = 11.4, mean_observed_time = 12   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Standard time with observed times = [4.2, 4.5, 4, 4.3, 4.5], performance rating = 1.1, allowance factor = 0.15, allowance basis = job.	SOLVED	standard_time = 5.4395, normal_time = 4.73, mean_observed_time = 4.3
Standard time with observed times = [12, 11, 13, 12], performance rating = 0.95, allowance factor = 0.12, allowance basis = shift.	SOLVED	standard_time = 12.9545, normal_time = 11.4, mean_observed_time = 12
Standard time with observed times = [4.2, 4.5, 4, 4.3, 4.5], performance rating = 1.1, allowance factor = 0.15.	NEEDS_INPUT	ie_standard_time needs basis (job: ST = NT (1 + A); shift: ST = NT / (1 - A), A a share of the workday). Example: solve(
Standard time with observed times = [4.2, 4.5, 4, 4.3, 4.5], performance rating = 1.1, allowance factor = 0.15, allowance basis = job, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Standard time with observed times = [4.2, 4.5], performance rating = 1.1, allowance factor = 1.5, allowance basis = job.	INVALID_INPUT	Standard time from a time study: allowance must lie strictly between -1e-300 and 1 (write 95% or 0.95).

Number of cycles to time in a time study, `ie_time_study_sample_size`

Observations needed so the mean cycle time is within +/- h (a fraction of the mean) at a confidence level: n = (z s / (h x-bar))^2.

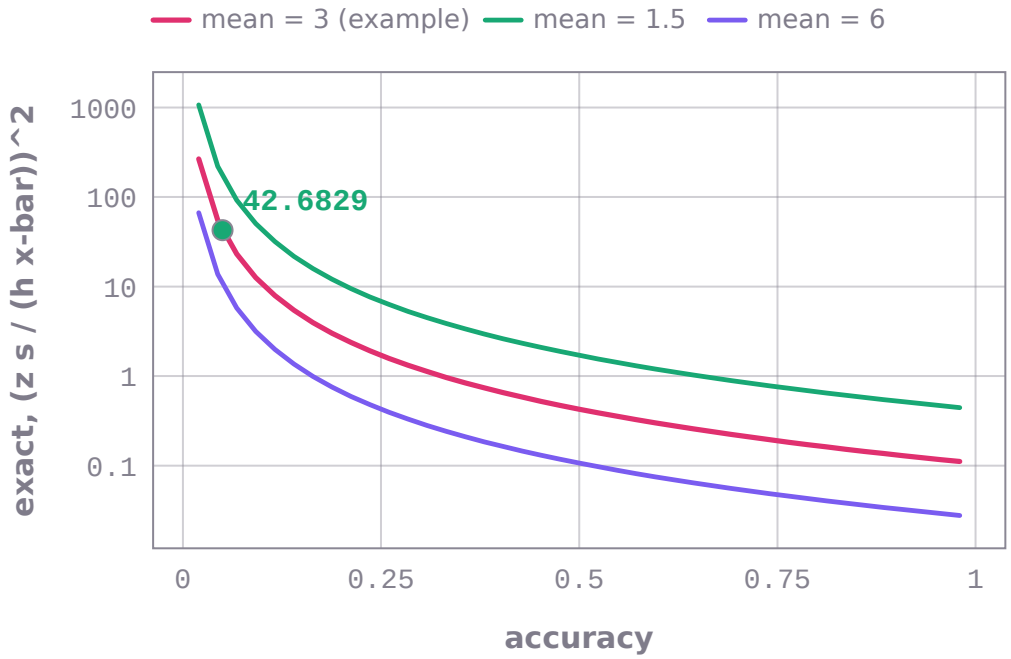
$$n = \left(\frac{z\,s}{h\,\bar{x}}\right)^2$$

input	meaning	condition
`mean`	mean of the pilot observations	
`sd`	standard deviation of the pilot observations	
`accuracy`	desired accuracy as a fraction of the mean (decimal; 5% = 0.05)	
`confidence`	confidence level (decimal)	

output	meaning
`sample_size`	observations needed (rounded up)
`exact`	(z s / (h x-bar))^2
`z`	two-sided z

- **Method:** n = (z s / (h x-bar))^2, z for a two-sided confidence
- **Assumes:** mean, $sd > 0$; 0 < accuracy < 1; 0 < confidence < 1; normal approximation

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



exact ((z s / (h x-bar))^2) against accuracy (desired accuracy as a fraction of the mean (decimal; 5% = 0.05)) from 0.02 to 0.98, logarithmic output axis. One curve for each value of mean (mean of the pilot observations): 3, 1.5, 6. Other inputs as in the worked example: sd = 0.5, confidence = 0.95. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, accuracy = 0.05, exact = 42.6829.

```
industrial_engineering_ai999x.solve('ie_time_study_sample_size', mean=3.0, sd=0.5,
accuracy=0.05, confidence=0.95)
# -> sample_size = 43, exact = 42.6829, z = 1.95996   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_time_study_sample_size', mean=12, sd=2.2,
accuracy=0.1, confidence=0.99)
# -> sample_size = 23, exact = 22.3006, z = 2.57583   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Time study sample size with sample mean = 3, standard deviation = 0.5, desired accuracy = 0.05, confidence level = 0.95.	SOLVED	sample_size = 43, exact = 42.6829, z = 1.95996
Time study sample size with sample mean = 12, standard deviation = 2.2, desired accuracy = 0.1, confidence level = 0.99.	SOLVED	sample_size = 23, exact = 22.3006, z = 2.57583
Time study sample size with sample mean = 3, standard deviation = 0.5, desired accuracy = 0.05.	NEEDS_INPUT	ie_time_study_sample_size needs confidence (confidence level (decimal)). Example: solve('ie_time_study_sample_size', mea
Time study sample size with sample mean = 3, standard deviation = 0.5, desired accuracy = 0.05, confidence level = 0.95, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
Time study sample size with sample mean = 3, standard deviation = 0.5, desired accuracy = 1.5, confidence level = 0.95.	INVALID_INPUT	Number of cycles to time in a time study: accuracy must lie strictly between 0 and 1 (write 95% or 0.95).

Work sampling sample size, `ie_work_sampling_sample_size`

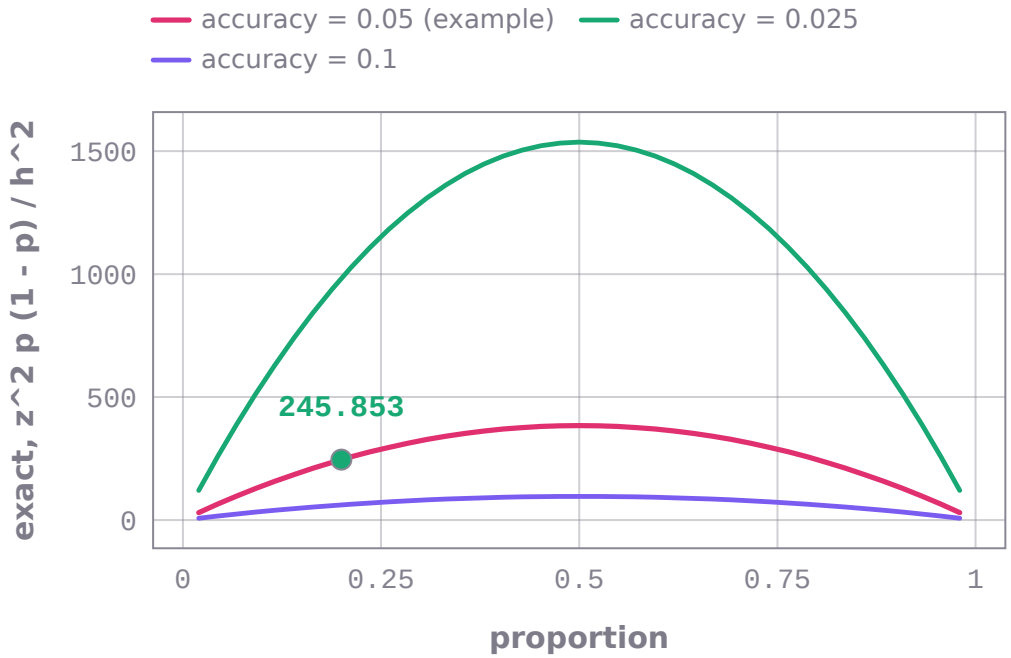
Random observations needed to estimate the fraction of time in an activity within +/- h (absolute) at a confidence level: $n = z^2$ p $(1-p)/h^2$.

$$n = \frac{z^2\,p(1-p)}{h^2}$$

input	meaning	condition
`proportion`	estimated fraction of time in the activity (decimal)	
`accuracy`	absolute accuracy (decimal; 0.05 = +/- 5 points)	
`confidence`	confidence level (decimal)	

output	meaning
`sample_size`	observations needed (rounded up)
`exact`	z^2 p $(1-p)/h^2$
`z`	two-sided z

- **Method:** $n = z^2$ p $(1-p)/h^2$
- **Assumes:** $0 < p < 1$; $0 < h < 1$; 0 < confidence < 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



exact (z^2 p (1 - p) / h^2) against proportion (estimated fraction of time in the activity (decimal)) from 0.02 to 0.98. One curve for each value of accuracy (absolute accuracy (decimal; 0.05 = +/- 5 points)): 0.05, 0.025, 0.1. Other inputs as in the worked example: confidence = 0.95. 41 points per curve computed by industrial-engineering-ai999x; marked: the worked example, proportion = 0.2, exact = 245.853.

```
industrial_engineering_ai999x.solve('ie_work_sampling_sample_size', proportion=0.2,
accuracy=0.05, confidence=0.95)
# -> sample_size = 246, exact = 245.853, z = 1.95996   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_work_sampling_sample_size', proportion=0.1,
accuracy=0.02, confidence=0.9)
# -> sample_size = 609, exact = 608.747, z = 1.64485   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Work sampling sample size with estimated proportion = 0.2, desired accuracy = 0.05, confidence level = 0.95.	SOLVED	sample_size = 246, exact = 245.853, z = 1.95996
Work sampling sample size with estimated proportion = 0.1, desired accuracy = 0.02, confidence level = 0.9.	SOLVED	sample_size = 609, exact = 608.747, z = 1.64485
Work sampling sample size with estimated proportion = 0.2, desired accuracy = 0.05.	NEEDS_INPUT	ie_work_sampling_sample_size needs confidence (confidence level (decimal)). Example: solve('ie_work_sampling_sample_size
Work sampling sample size with estimated proportion = 0.2, desired accuracy = 0.05, confidence level = 0.95, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Work sampling sample size with estimated proportion = 1.2,	INVALID_INPUT	Work sampling sample size: proportion must lie strictly between 0 and 1 (write 95% or 0.95).

question	outcome	result
desired accuracy = 0.05, confidence level = 0.95.		

Overall equipment effectiveness (OEE), `ie_oee`

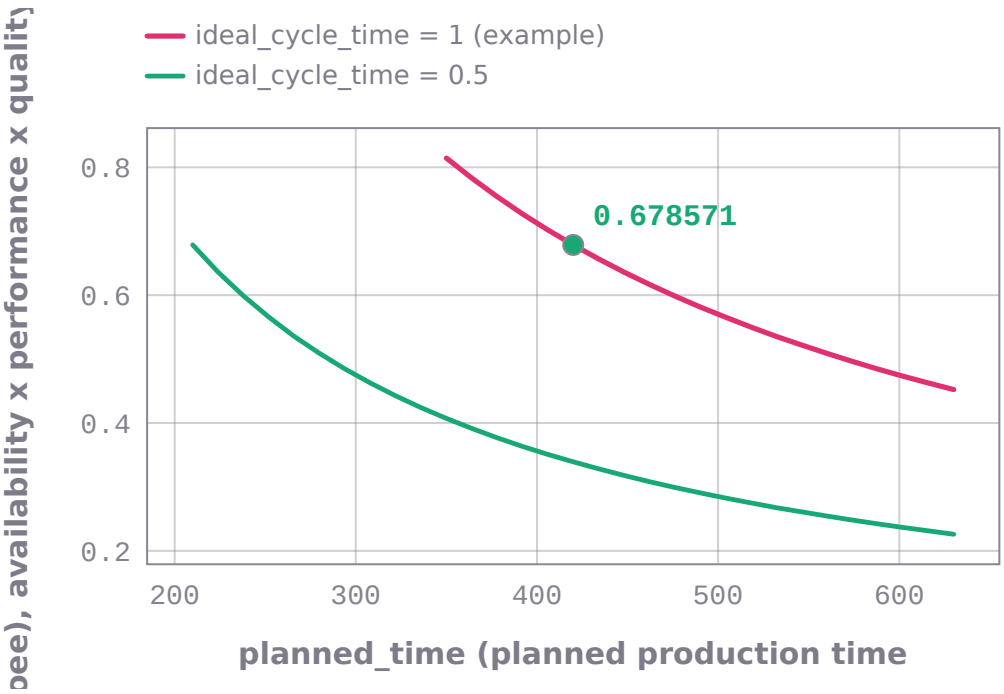
OEE = availability x performance x quality from planned time, downtime, ideal cycle time and counts.

$$OEE = A \times P \times Q = \frac{\text{good} \times t_{ideal}}{T_{planned}}$$

input	meaning	condition
`planned_time`	planned production time	
`downtime`	stop time within the planned time	
`ideal_cycle_time`	ideal time per unit (same time unit)	
`total_count`	units produced	
`good_count`	good units	

output	meaning
`oee`	availability x performance x quality
`availability`	run time / planned time
`performance`	ideal cycle time x total count / run time
`quality`	good / total
`run_time`	planned time - downtime

- **Method:** $A = (T - D)/T$, $P = ct$ n / (T - D), Q = good/n, OEE = A P Q (= good x ct / T)
- **Assumes:** downtime < planned time; good <= total; performance <= 1
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



oee (availability x performance x quality) against planned_time (planned production time) from 210 to 630. One curve for each value of ideal_cycle_time (ideal time per unit (same time unit)): 1, 0.5. Other inputs as in the worked example: downtime = 47, total_count = 300, good_count = 285. 21 points per curve computed by industrial-engineering-ai999x; marked: the worked example, planned_time = 420, oee = 0.678571.

```
industrial_engineering_ai999x.solve('ie_oee', planned_time=420, downtime=47,
ideal_cycle_time=1.0, total_count=300, good_count=285)
# -> oee = 0.678571, availability = 0.888095, performance = 0.80429, quality = 0.95,
run_time = 373   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_oee', planned_time=480, downtime=60,
ideal_cycle_time=0.5, total_count=700, good_count=680)
# -> oee = 0.708333, availability = 0.875, performance = 0.833333, quality = 0.971429,
run_time = 420   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Overall equipment effectiveness with planned production time = 420, stop time = 47, ideal cycle time = 1, total units = 300, good units = 285.	SOLVED	oee = 0.678571, availability = 0.888095, performance = 0.80429, quality = 0.95, run_time = 373
Overall equipment effectiveness with planned production time = 480, stop time = 60, ideal cycle time = 0.5, total units = 700, good units = 680.	SOLVED	oee = 0.708333, availability = 0.875, performance = 0.833333, quality = 0.971429, run_time = 420
Overall equipment effectiveness with planned production time = 420, stop time = 47, ideal cycle time = 1, total units = 300.	NEEDS_INPUT	ie_oee needs good_count (good units). Example: solve('ie_oee', planned_time=420, downtime=47, ideal_cycle_time=1, total_
Overall equipment effectiveness with planned production time = 420, stop time = 47, ideal cycle time = 1, total	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem

question	outcome	result
units = 300, good units = 285, colour = 3.		
Overall equipment effectiveness with planned production time = 420, stop time = 47, ideal cycle time = 1, total units = 300, good units = 385.	INVALID_INPUT	Overall equipment effectiveness (OEE): good count cannot exceed the total count.

Capacity utilisation and efficiency, `ie_capacity_utilization`

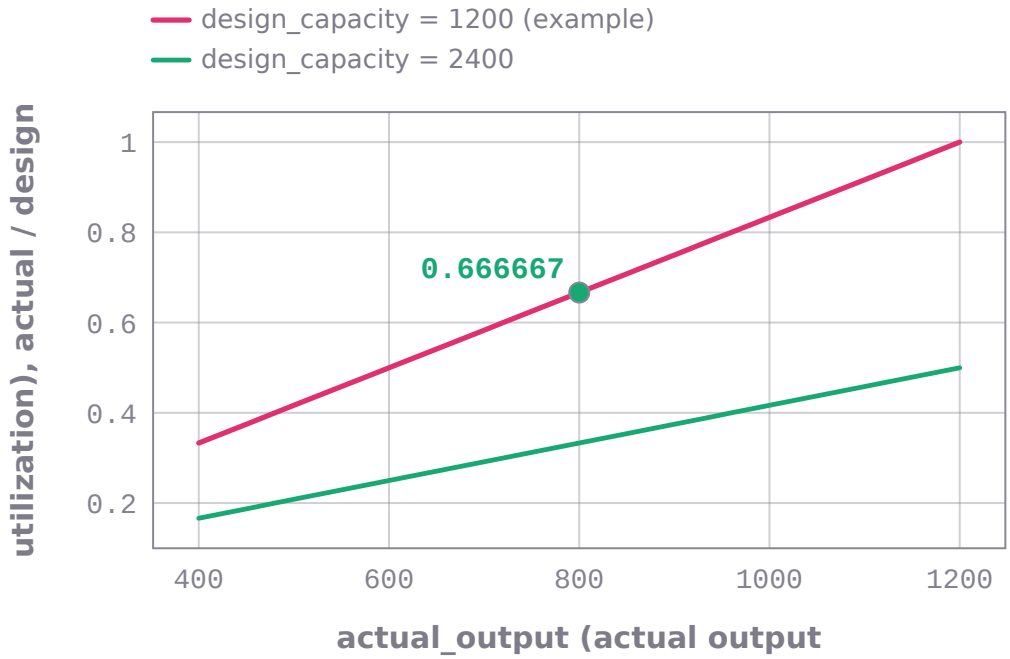
Utilisation (actual output / design capacity) and efficiency (actual output / effective capacity).

$$U = \frac{\text{actual}}{\text{design}}, \quad E = \frac{\text{actual}}{\text{effective}}$$

input	meaning	condition
`design_capacity`	design capacity	
`effective_capacity`	effective capacity	
`actual_output`	actual output	

output	meaning
`utilization`	actual / design
`efficiency`	actual / effective
`effective_share`	effective / design

- **Method:** utilisation = actual/design, efficiency = actual/effective
- **Assumes:** capacities > 0; effective <= design; same units and period
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



utilization (actual / design) against actual_output (actual output) from 400 to 1200. One curve for each value of design_capacity (design capacity): 1200, 2400. Other inputs as in the worked example: effective_capacity = 1000. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, actual_output = 800, utilization = 0.666667.

```
industrial_engineering_ai999x.solve('ie_capacity_utilization', design_capacity=1200,
effective_capacity=1000, actual_output=800)
# -> utilization = 0.666667, efficiency = 0.8, effective_share = 0.833333   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_capacity_utilization', design_capacity=50,
effective_capacity=40, actual_output=36)
# -> utilization = 0.72, efficiency = 0.9, effective_share = 0.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Capacity utilization and efficiency with design capacity = 1200, effective capacity = 1000, actual output = 800.	SOLVED	utilization = 0.666667, efficiency = 0.8, effective_share = 0.833333
Capacity utilization and efficiency with design capacity = 50, effective capacity = 40, actual output = 36.	SOLVED	utilization = 0.72, efficiency = 0.9, effective_share = 0.8
Capacity utilization and efficiency with design capacity = 1200, effective capacity = 1000.	NEEDS_INPUT	ie_capacity_utilization needs actual_output (actual output). Example: solve('ie_capacity_utilization', design_capacity=1
Capacity utilization and efficiency with design capacity = 1200, effective capacity = 1000, actual output = 800, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Capacity utilization and efficiency with design capacity = 1000, effective capacity = 1200, actual output = 800.	INVALID_INPUT	Capacity utilisation and efficiency: effective capacity cannot exceed design capacity.

Depreciation schedule (straight line, declining balance, sum of years' digits, MACRS), `ie_depreciation_schedule`

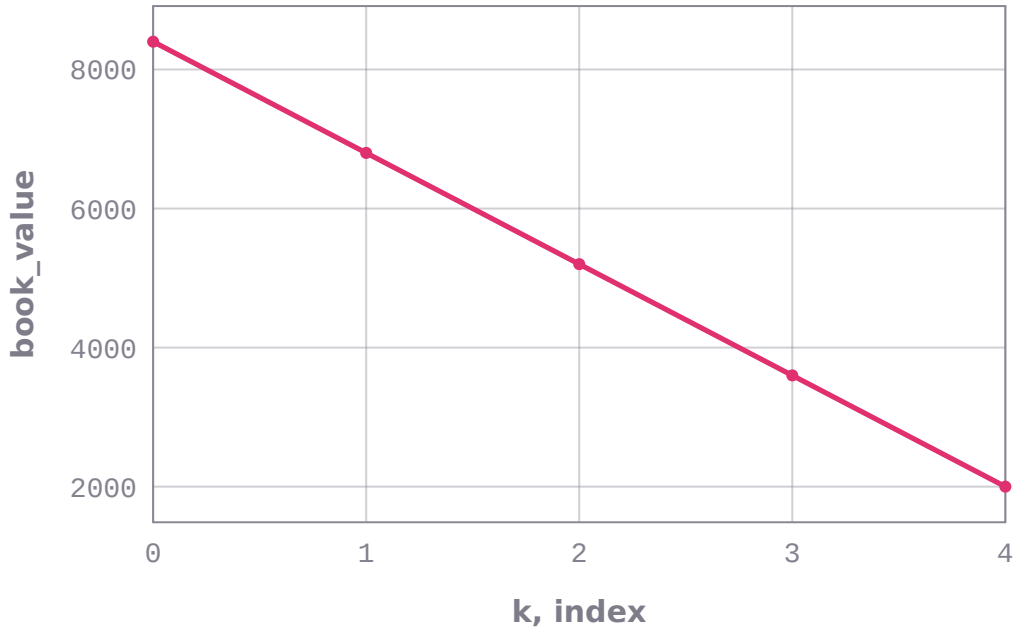
Depreciation charge and end-of-year book value for every year of an asset's life by straight line, declining balance (factor/n of the book value, never below salvage), sum of years' digits or MACRS (half-year convention, salvage ignored), with one year picked out on request.

$$d^{SL} = \frac{P - S}{n}, \quad d_t^{DB} = \frac{f}{n} BV_{t-1}, \quad d_t^{SYD} = (P - S)\frac{n - t + 1}{n(n+1)/2}, \quad d_t^{MACRS} = r_t P$$

input	meaning	condition
`cost`	first (installed) cost of the asset	
`life`	useful life or MACRS recovery period in years	
`method`	straight-line, declining-balance, sum-of-years-digits or macrs	
`salvage`	estimated salvage value at the end of the life (default 0; not used by MACRS)	
`factor`	declining-balance factor: 2 for double declining (default), 1.5 for 150 %	
`year`	the year to report (optional)	

output	meaning
`depreciation`	depreciation charge of each year
`book_value`	book value at the end of each year
`total_depreciation`	cost minus the final book value
`depreciation_in_year`	the charge in the given year
`book_value_end_of_year`	book value at the end of the given year
`accumulated_depreciation`	depreciation up to the end of the given year

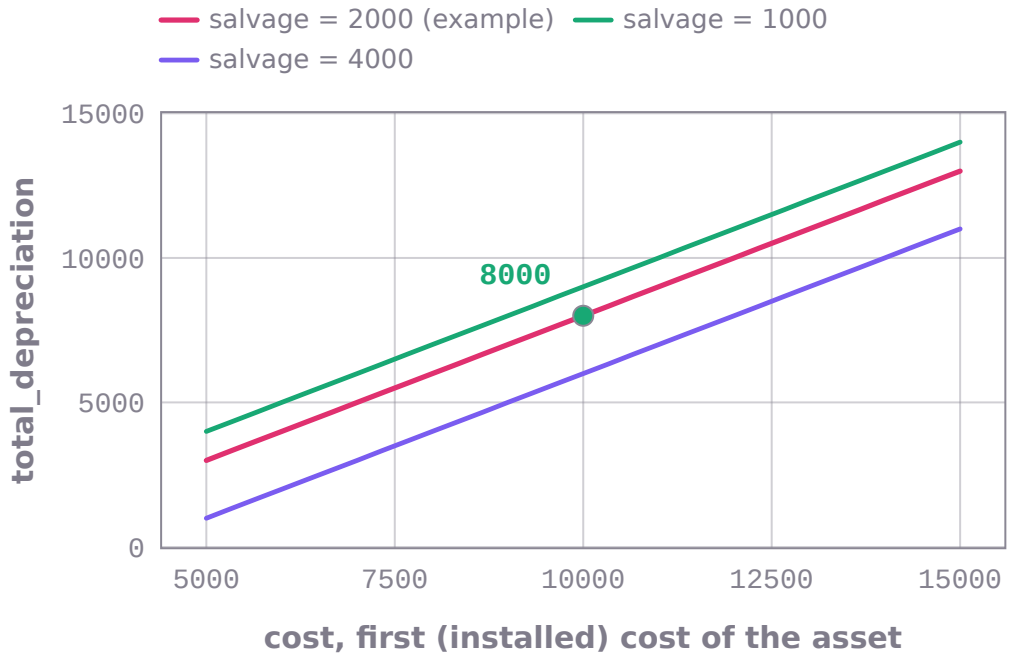
- **Method:** SL: (P - S)/n; DB: d_t = min(f/n BV_{$t-1$}, BV_{$t-1$} - S); SYD: (P - S)(n - t + 1)/(n(n + 1)/2); MACRS: P x table rate
- **Assumes:** cost > 0; 0 <= salvage < cost; life a whole number of years; MACRS: recovery period 3, 5, 7, 10, 15 or 20, no salvage; declining balance: no switch to straight line
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



book_value (book value at the end of each year) for each index, as returned by industrial-engineering-ai999x for its example (5 values).



depreciation charge of each year (depreciation) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (5 values, first 1600, last 1600, largest 1600); with book_value (book value at the end of each year). Inputs: cost = 10000, life = 5, method = straight-line, salvage = 2000.



total_depreciation (cost minus the final book value) against cost (first (installed) cost of the asset) from 5000 to 15000. One curve for each value of salvage (estimated salvage value at the end of the life (default 0; not used by MACRS)): 2000, 1000, 4000. Other inputs as in the worked example: life = 5, method = straight-line. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, cost = 10000, total_depreciation = 8000.

```
industrial_engineering_ai999x.solve('ie_depreciation_schedule', cost=10000, salvage=2000,
life=5, method='straight-line')
# -> depreciation = [1600.0, 1600.0, 1600.0, 1600.0, 1600.0], book_value = [8400.0, 6800.0,
5200.0, 3600.0, 2000.0], total_depreciation = 8000   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_depreciation_schedule', cost=10000, salvage=1000,
life=5, method='sum-of-years-digits', year=2)
# -> depreciation = [3000.0, 2400.0, 1800.0, 1200.0, 600.0], book_value = [7000.0, 4600.0,
2800.0, 1600.0, 1000.0], total_depreciation = 9000, depreciation_in_year = 2400,
book_value_end_of_year = 4600, accumulated_depreciation = 5400   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_depreciation_schedule', cost=10000, salvage=1000,
life=5, method='declining-balance')
# -> depreciation = [4000.0, 2400.0, 1440.0, 864.0, 296.0], book_value = [6000.0, 3600.0,
2160.0, 1296.0, 1000.0], total_depreciation = 9000   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_depreciation_schedule', cost=10000, life=5,
method='macrs')
# -> depreciation = [2000.0, 3200.0, 1920.0, 1152.0, 1152.0, 576.0], book_value = [8000.0,
4800.0, 2880.0, 1728.0, 576.0, 0.0], total_depreciation = 10000   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Depreciation schedule with first cost = 10000, salvage value = 2000, useful life = 5, depreciation method = straight-line.	SOLVED	depreciation = [1600.0, 1600.0, 1600.0, 1600.0, 1600.0], book_value = [8400.0, 6800.0, 5200.0, 3600.0, 2000.0], total_depreciation = 8000

question	outcome	result
Depreciation schedule with first cost = 10000, salvage value = 1000, useful life = 5, depreciation method = sum-of-years-digits, year number = 2.	SOLVED	depreciation = [3000.0, 2400.0, 1800.0, 1200.0, 600.0], book_value = [7000.0, 4600.0, 2800.0, 1600.0, 1000.0], total_depreciation = 9000, depreciation_in_year = 2400, book_value_end_of_year = 4600, accumulated_depreciation = 5400
Depreciation schedule with first cost = 10000, salvage value = 2000, useful life = 5.	NEEDS_INPUT	ie_depreciation_schedule needs method (straight-line, declining-balance, sum-of-years-digits or macrs). Example: solve('
Depreciation schedule with first cost = 10000, salvage value = 2000, useful life = 5, depreciation method = straight-line, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Depreciation schedule with first cost = 10000, salvage value = 12000, useful life = 5, depreciation method = straight-line.	INVALID_INPUT	Depreciation schedule (straight line, declining balance, sum of years' digits, MACRS): the salvage value must be less th
Depreciation schedule with first cost = 10000, useful life = 6, depreciation method = macrs.	INVALID_INPUT	Depreciation schedule (straight line, declining balance, sum of years' digits, MACRS): a MACRS recovery period is 3, 5,

After-tax cash flow with depreciation and recapture, `ie_after_tax_cash_flow`

Taxable income, taxes and after-tax cash flow of each year from the before-tax cash flows and the depreciation charges, and with the first cost and a sale price the book value, the gain (depreciation recapture) or loss on the sale and its tax, and the full after-tax cash-flow series.

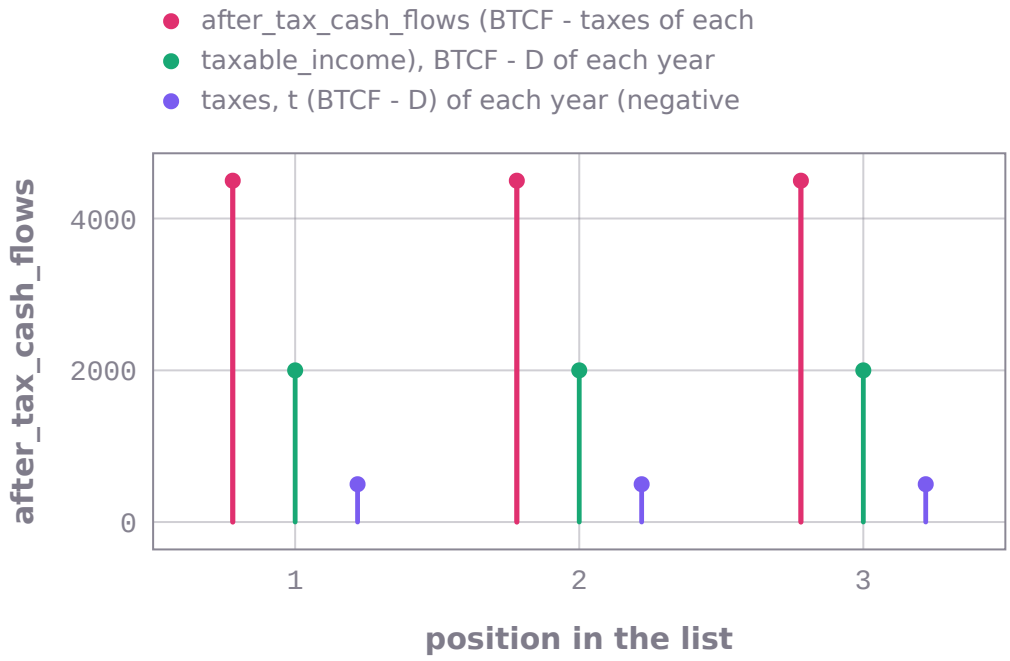
$$ATCF_k = BTCF_k - t\,(BTCF_k - D_k), \quad ATCF_n \mathrel{+}= S - t\,(S - BV_n)$$

input	meaning	condition
`before_tax_cash_flows`	before-tax cash flow of each year 1..n	
`depreciation`	depreciation charge of each year 1..n	
`tax_rate`	income tax rate (decimal; 25% = 0.25)	
`investment`	first cost at year 0 (optional)	
`sale_price`	market (salvage) value received at the end of year n (optional; needs the investment)	

output	meaning
`taxable_income`	BTCF - D of each year
`taxes`	t (BTCF - D) of each year (negative = a tax credit)
`after_tax_cash_flows`	BTCF - taxes of each year (without the sale)
`total_tax`	sum of the yearly taxes
`book_value_end`	first cost minus all depreciation
`gain_on_sale`	sale price - book value (recapture; negative = loss)

output	meaning
`tax_on_sale`	t x gain on the sale
`cash_flows_with_investment`	after-tax series from year 0, the sale in year n

- **Method:** TI = BTCF - D; taxes = t TI; ATCF = BTCF - taxes; at the sale: + S - t (S - BV)
- **Assumes:** 0 <= tax rate < 1; one depreciation value per year, each >= 0; sale price <= first cost (no capital gain); a negative taxable income gives a tax credit; the gain or loss on the sale is taxed at the same rate
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



BTCF - taxes of each year (without the sale) (after_tax_cash_flows) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 4500, last 4500, largest 4500); with taxable_income (BTCF - D of each year), taxes (t (BTCF - D) of each year (negative = a tax credit)). Inputs: before_tax_cash_flows = [5000, 5000, 5000], depreciation = [3000, 3000, 3000], tax_rate = 0.25, investment = 9000, sale_price = 1000.



total_tax (sum of the yearly taxes) against tax_rate (income tax rate (decimal; 25% = 0.25)) from 0.125 to 0.375. Other inputs as in the worked example: before_tax_cash_flows = [5000, 5000, 5000], depreciation = [3000, 3000, 3000], investment = 9000, sale_price = 1000. 31 points computed by industrial-engineering-ai999x; marked: the worked example, tax_rate = 0.25, total_tax = 1500.

```
industrial_engineering_ai999x.solve('ie_after_tax_cash_flow', before_tax_cash_flows=[5000, 5000, 5000], depreciation=[3000, 3000, 3000], tax_rate=0.25, investment=9000, sale_price=1000)
# -> taxable_income = [2000.0, 2000.0, 2000.0], taxes = [500.0, 500.0, 500.0], after_tax_cash_flows = [4500.0, 4500.0, 4500.0], total_tax = 1500, book_value_end = 0, gain_on_sale = 1000, tax_on_sale = 250, cash_flows_with_investment = [-9000.0, 4500.0, 4500.0, 5250.0]   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_after_tax_cash_flow', before_tax_cash_flows=[4000, 6000], depreciation=[5000, 3000], tax_rate=0.3)
# -> taxable_income = [-1000.0, 3000.0], taxes = [-300.0, 900.0], after_tax_cash_flows = [4300.0, 5100.0], total_tax = 600   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
After-tax cash flow with before tax cash flows = [5000, 5000, 5000], depreciation charges = [3000, 3000, 3000], tax rate = 0.25, first cost = 9000, sale price = 1000.	SOLVED	taxable_income = [2000.0, 2000.0, 2000.0], taxes = [500.0, 500.0, 500.0], after_tax_cash_flows = [4500.0, 4500.0, 4500.0], total_tax = 1500, book_value_end = 0, gain_on_sale = 1000, tax_on_sale = 250, cash_flows_with_investment = [-9000.0, 4500.0, 4500.0, 5250.0]
After-tax cash flow with before tax cash flows = [4000, 6000], depreciation charges = [5000, 3000], tax rate = 0.3.	SOLVED	taxable_income = [-1000.0, 3000.0], taxes = [-300.0, 900.0], after_tax_cash_flows = [4300.0, 5100.0], total_tax = 600
After-tax cash flow with before tax cash flows = [5000, 5000, 5000], depreciation charges =	NEEDS_INPUT	ie_after_tax_cash_flow needs tax_rate (income tax rate (decimal; 25% = 0.25)). Example: solve('ie_after_tax_cash_flow',

question	outcome	result
[3000, 3000, 3000], first cost = 9000, sale price = 1000.		
After-tax cash flow with before tax cash flows = [5000, 5000, 5000], depreciation charges = [3000, 3000, 3000], tax rate = 0.25, first cost = 9000, sale price = 1000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
After-tax cash flow with before tax cash flows = [4000, 6000], depreciation charges = [5000, 3000], tax rate = 1.2.	INVALID_INPUT	After-tax cash flow with depreciation and recapture: tax_rate must lie strictly between -1e-300 and 1 (write 95% or 0.95
After-tax cash flow with before tax cash flows = [5000, 5000], depreciation charges = [3000, 3000], tax rate = 0.25, first cost = 9000, sale price = 12000.	INVALID_INPUT	After-tax cash flow with depreciation and recapture: a sale above the first cost (a capital gain) is outside this capabi

Earned value analysis (CV, SV, CPI, SPI, EAC), `ie_earned_value`

Cost and schedule variances and performance indices of a project from planned value, earned value and actual cost; with the budget at completion the estimates at completion (CPI method, atypical, CPI x SPI), estimate to complete, variance at completion, TCPI and per cent complete.

$$CV = EV - AC,\ SV = EV - PV,\ CPI = \frac{EV}{AC},\ SPI = \frac{EV}{PV},\ EAC = \frac{BAC}{CPI},\ TCPI = \frac{BAC}$$

input	meaning	condition
`planned_value`	planned value (budgeted cost of work scheduled) to date	
`earned_value`	earned value (budgeted cost of work performed) to date	
`actual_cost`	actual cost (of work performed) to date	
`budget_at_completion`	budget at completion (optional)	

output	meaning
`cost_variance`	$CV = EV - AC$ (negative = over budget)
`schedule_variance`	$SV = EV - PV$ (negative = behind schedule)
`cpi`	CPI = EV/AC
`spi`	SPI = EV/PV
`eac`	EAC = BAC/CPI (current cost efficiency continues)
`etc`	ETC = EAC - AC
`vac`	VAC = BAC - EAC
`eac_atypical`	EAC = AC + BAC - EV (remaining work at budget)
`eac_cpi_spi`	EAC = AC + (BAC - EV)/(CPI SPI)
`tcpi`	TCPI = (BAC - EV)/(BAC - AC), when BAC > AC

output	meaning
`percent_complete`	EV/BAC

- **Method:** $CV = EV - AC$, $SV = EV - PV$, CPI = EV/AC, SPI = EV/PV, EAC = BAC/CPI, ETC = EAC - AC, VAC = BAC - EAC
- **Assumes:** PV, $AC > 0$; 0 <= EV <= BAC; estimates at completion need $EV > 0$; TCPI only while the budget is not used up (BAC > AC)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



cpi (CPI = EV/AC) against actual_cost (actual cost (of work performed) to date) from 225 to 675. One curve for each value of earned_value (earned value (budgeted cost of work performed) to date): 400, 200, 800. Other inputs as in the worked example: planned_value = 500, budget_at_completion = 1000. 31 points per curve computed by industrial-engineering-ai999x; marked: the worked example, actual_cost = 450, cpi = 0.888889.

```
industrial_engineering_ai999x.solve('ie_earned_value', planned_value=500, earned_value=400,
actual_cost=450, budget_at_completion=1000)
# -> cost_variance = -50, schedule_variance = -100, cpi = 0.888889, spi = 0.8, eac = 1125,
etc = 675, vac = -125, eac_atypical = 1050, eac_cpi_spi = 1293.75, percent_complete = 0.4,
tcpi = 1.09091   (SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_earned_value', planned_value=200000,
earned_value=220000, actual_cost=200000)
# -> cost_variance = 20000, schedule_variance = 20000, cpi = 1.1, spi = 1.1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Earned value analysis with planned value = 500, earned value = 400, actual cost = 450, budget at completion = 1000.	SOLVED	cost_variance = -50, schedule_variance = -100, cpi = 0.888889, spi = 0.8, eac = 1125, etc = 675, vac = -125, eac_atypical = 1050, eac_cpi_spi = 1293.75, percent_complete = 0.4, tcpi = 1.09091

question	outcome	result
Earned value analysis with planned value = 200000, earned value = 220000, actual cost = 200000.	SOLVED	cost_variance = 20000, schedule_variance = 20000, cpi = 1.1, spi = 1.1
Earned value analysis with planned value = 500, earned value = 400, budget at completion = 1000.	NEEDS_INPUT	ie_earned_value needs actual_cost (actual cost (of work performed) to date). Example: solve('ie_earned_value', planned_v
Earned value analysis with planned value = 500, earned value = 400, actual cost = 450, budget at completion = 1000, colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Earned value analysis with planned value = 500, earned value = 1200, actual cost = 450, budget at completion = 1000.	INVALID_INPUT	Earned value analysis (CV, SV, CPI, SPI, EAC): the earned value cannot exceed the budget at completion.
Earned value analysis with planned value = 500, earned value = 400, actual cost = 0.	INVALID_INPUT	Earned value analysis (CV, SV, CPI, SPI, EAC): actual_cost must be positive.

Single-factor and multifactor productivity, `ie_productivity`

Partial (single-factor) productivity of output over each input, multifactor productivity over the sum of the inputs (all in one unit, e.g. money), and with a previous period the change in per cent.

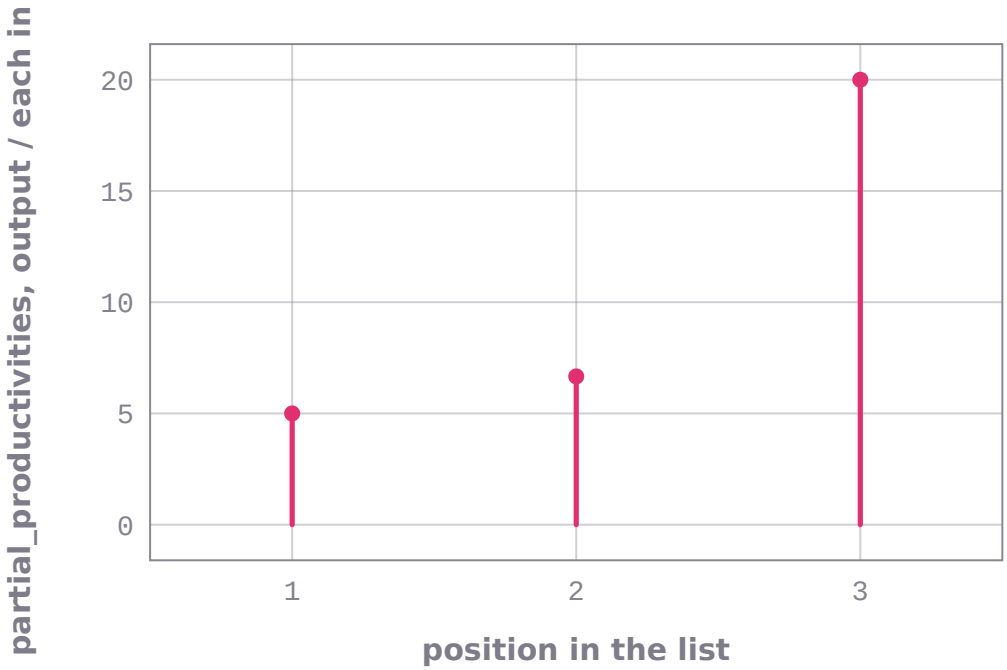
$$P_k = \frac{O}{I_k}, \quad MFP = \frac{O}{\sum_k I_k}, \quad \Delta\% = 100\,\frac{MFP_{now} - MFP_{prev}}{MFP_{prev}}$$

input	meaning	condition
`output`	output of the period (units or value)	
`input_amounts`	amount of each input (labour, material, capital, energy ...; one unit, e.g. cost, for the multifactor ratio)	
`previous_output`	output of the previous period (optional)	
`previous_inputs`	inputs of the previous period, in the same order (optional)	

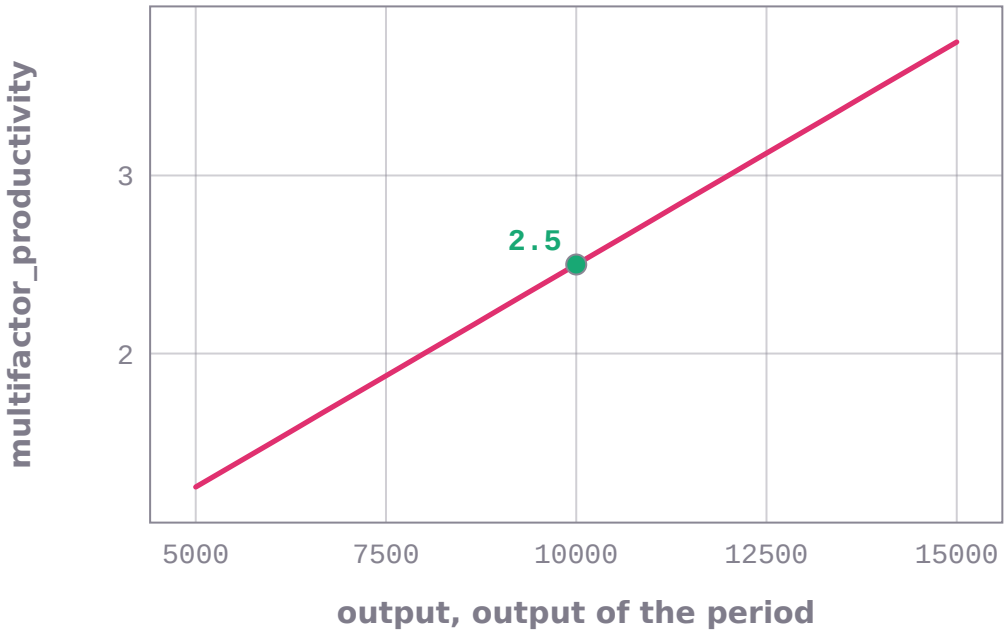
output	meaning
`partial_productivities`	output / each input
`multifactor_productivity`	output / sum of inputs
`previous_multifactor_productivity`	previous output / sum of previous inputs
`change_percent`	change in multifactor productivity, %
`partial_change_percent`	change of each partial productivity, %

- **Method:** partial = O / I_k; multifactor = O / sum I_k; change = (P_now - P_prev)/P_prev x 100
- **Assumes:** output > 0; every input > 0; inputs in one unit for the multifactor ratio; the previous period lists the same inputs

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



output / each input (partial_productivities) returned by industrial-engineering-ai999x for the worked example, one value for each position in the list (3 values, first 5, last 20, largest 20). Inputs: output = 10000, input_amounts = [2000, 1500, 500].



multifactor_productivity (output / sum of inputs) against output (output of the period (units or value)) from 5000 to 15000. Other inputs as in the worked example: input_amounts = [2000, 1500, 500]. 31 points computed by industrial-engineering-ai999x; marked: the worked example, output = 10000, multifactor_productivity = 2.5.

```
industrial_engineering_ai999x.solve('ie_productivity', output=10000, input_amounts=[2000,
1500, 500])
# -> partial_productivities = [5.0, 6.66666666667, 20.0], multifactor_productivity = 2.5
(SOLVED)
```

```
industrial_engineering_ai999x.solve('ie_productivity', output=12000, input_amounts=[2500, 1500, 500], previous_output=10000, previous_inputs=[2000, 1500, 500])
# -> partial_productivities = [4.8, 8.0, 24.0], multifactor_productivity = 2.66667, previous_multifactor_productivity = 2.5, change_percent = 6.66667, partial_change_percent = [-4.0, 20.0, 20.0]   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Productivity with output value = 10000, input amounts = [2000, 1500, 500].	SOLVED	partial_productivities = [5.0, 6.66666666667, 20.0], multifactor_productivity = 2.5
Productivity with output value = 12000, input amounts = [2500, 1500, 500], previous output = 10000, previous inputs = [2000, 1500, 500].	SOLVED	partial_productivities = [4.8, 8.0, 24.0], multifactor_productivity = 2.66667, previous_multifactor_productivity = 2.5, change_percent = 6.66667, partial_change_percent = [-4.0, 20.0, 20.0]
Productivity with output value = 10000.	NEEDS_INPUT	ie_productivity needs input_amounts (amount of each input (labour, material, capital, energy ...; one unit, e.g. cost, f
Productivity with output value = 10000, input amounts = [2000, 1500, 500], colour = 3.	UNRECOGNIZED_FORMAT	I could not match this question to a documented form, so I will not guess. Every form: industrial_engineering_ai999x.tem
Productivity with output value = 10000, input amounts = [2000, -5].	INVALID_INPUT	Single-factor and multifactor productivity: every value of input_amounts must be positive.
Productivity with output value = 12000, input amounts = [2500, 1500], previous output = 10000, previous inputs = [2000, 1500, 500].	INVALID_INPUT	Single-factor and multifactor productivity: previous_inputs must list the same inputs as input_amounts.